Analyse de séries temporelles avec R

Alexis Gabadinho
alexis.gabadinho@protonmail.com

4 novembre 2026





Sommaire I

1 Introduction

2 Environnement de travail et données

3 Librairies spécialisées et structures de séries temporelles dans R

4 Concepts théoriques

5 Analyse descriptive et représentations graphiques

6 Régression Linéaire

7 Décomposition, tendances, transformation des données

Sommaire II

8 Séries temporelles non-stationnaires

9 Modélisation de séries temporelles stationnaires

10 Modèlisation de séries non-stationnaires

11 Modèles multivariés pour l'analyse de séries temporelles stationnaires

12 Cointégration et modèle à correction d'erreur

13 Modèles VECM et SVECM

14 Bibliographie

1 Introduction

Support de cours et exercices

- Ce support est écrit avec LaTeX et `knitr` (document incluant du code R exécuté dynamiquement)
- L'environnement RStudio sera utilisé lors de la formation
- Le fichier source du support sera fourni aux participant-e-s, et leur permettra d'exécuter le code contenu dans les diapositives sur leur ordinateur
- Plusieurs jeux de données contenant des séries macroéconomiques sont utilisées pour les exemples. Ces données sont disponibles sous la forme de fichiers 'csv' ou fournies par certaines des librairies R utilisées
- De nombreuses formules mathématiques sont présentes dans les diapositives. Il n'est pas nécessaire de les comprendre, elles seront expliquées en détail lors de la formation

Librairies R requises

- Pour reproduire les exemples, les librairies suivantes doivent être installées :

`BVAR`	Modèles VAR bayésiens
`feasts`	Description, décomposition, représentations graphiques de séries ten
`fable`	
`GGally`	ajout de fonctions à `ggplot2`
`ggfortify`	ajout de fonctions à `ggplot2`
`insee`	Accès aux données INSEE
`MTS`	Séries multivariées, modèles VAR et VEC
`structchange`	Tests de changement structurel
`tidyverse`	Collection de librairies dont : `tibble`, `tidyr`, `ggplot2`
`tsibble`	Objets de type 'tidy' pour le stockage de données temporelle
`urca`	Tests de racine unitaire et de cointégration
`vars`	Modèles VAR, SVAR, VEC et SVEC

2 Environnement de travail et données

Le tidyverse et ggplot

- Pour importer et manipuler les données, on utilisera principalement le tidyverse une collection de librairies pour la science des données (data science)
- Les librairies partagent des structures de données, une philosophie

```
library(tidyverse)
```

```
## - Attaching core tidyverse packages ------------ tidyverse 2.0.0 -
## v dplyr     1.2.1     v readr     2.2.0
## v forcats   1.0.1     v stringr   1.6.0
## v ggplot2   4.0.3     v tibble    3.3.1
## v lubridate 1.9.5     v tidyr     1.3.2
## v purrr     1.2.2
## - Conflicts ---------------------- tidyverse_conflicts() -
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

- Pour les graphiques nous utiliserons principalement la librairie ggplot2, avec le theme ‘minimal‘

```
library(ggplot2)
theme_set(theme_minimal())
```

Données Nelson-Plosser

Introduction

- Les données 'Nelson-Plosser' contiennent 14 séries temporelles macroécononiques
- L'article original [26], Trends and Random Walks in Macroeconomic Time Series est paru dans *Journal of Monetary Economics*
- Dans leur article, Nelson et Plosser font l'hypothèse que la plupart des séries temporelles macroéconomiques sont mieux décrites par une nonstationnarité de type stochastique (unit-root stationarity) que par une tendance déterministe
- La longueur des séries est variable, mais elles se terminent toutes en 1988
- Le jeu de données est décrit ici
- NB : les données disponibles contiennent le log des séries décrites

Données Nelson-Plosser

Liste des séries

- Les 14 séries temporelles :

cpi	consumer price index
ip	industrial production
gnp.nom	nominal GNP (millions de dollars 1988)
vel	velocity
emp	employment
int.rate	interest rate
nom.wages	nominal wages
gnp.def	GNP deflator
money.stock	money stock
gnp.real	real GNP (milliards de dollars 1958)
stock.prices	stock prices (S&P500)
gnp.capita	GNP per capita (dollars 1958)
real.wages	real wages
unemp	unemployment

Données Nelson-Plosser

Importation des données au format csv

- La fonction `read.csv2` permet d'importer les données à partir du fichier csv
- A noter : dans le fichier csv, le séparateur de champ est une virgule et le séparateur décimal est un point (format européen)

```
Nelson_Plosser <- read.csv2("../data/Nelson_Plosser.csv",
                            header=TRUE, sep=",", dec = ".")
head(Nelson_Plosser)
```

```
##   year      cpi         ip gnp.nom vel emp int.rate nom.wages gnp.def
## 1 1860 3.295837 -0.1053605      NA  NA  NA       NA        NA      NA
## 2 1861 3.295837 -0.1053605      NA  NA  NA       NA        NA      NA
## 3 1862 3.401197 -0.1053605      NA  NA  NA       NA        NA      NA
## 4 1863 3.610918  0.0000000      NA  NA  NA       NA        NA      NA
## 5 1864 3.871201  0.0000000      NA  NA  NA       NA        NA      NA
## 6 1865 3.850148  0.0000000      NA  NA  NA       NA        NA      NA
##   money.stock gnp.real stock.prices gnp.capita real.wages unemp
## 1          NA       NA           NA         NA         NA    NA
## 2          NA       NA           NA         NA         NA    NA
## 3          NA       NA           NA         NA         NA    NA
## 4          NA       NA           NA         NA         NA    NA
## 5          NA       NA           NA         NA         NA    NA
## 6          NA       NA           NA         NA         NA    NA
```

Données Nelson-Plosser

Statistiques descriptives (1)

- Pour les statistiques descriptives de chaque variable on peut utiliser la fonction `descr()` (librairie `summarytools`)

```
library(summarytools)
descr(Nelson_Plosser[,1:7])
```

```
## Descriptive Statistics
## Nelson_Plosser
## N: 129
##
##                      cpi      emp   gnp.nom   int.rate       ip      vel      year
## ----------------- -------- -------- --------- ---------- -------- -------- ---------
##              Mean     3.99    10.85     12.59       4.92     2.73     0.78   1924.00
##           Std.Dev     0.71     0.45      1.47       2.50     1.56     0.38     37.38
##               Min     3.22     9.96     10.42       2.43    -0.11     0.15   1860.00
##                Q1     3.39    10.53     11.35       3.30     1.48     0.53   1892.00
##            Median     3.78    10.78     12.46       4.10     2.77     0.68   1924.00
##                Q3     4.40    11.19     13.71       5.10     4.07     0.92   1956.00
##               Max     5.87    11.67     15.40      13.23     5.23     1.72   1988.00
##               MAD     0.67     0.49      1.67       1.26     1.92     0.24     47.44
##               IQR     1.01     0.63      2.34       1.80     2.59     0.38     64.00
##                CV     0.18     0.04      0.12       0.51     0.57     0.48      0.02
##          Skewness     1.06    -0.09      0.34       1.66    -0.12     0.97      0.00
##       SE.Skewness     0.21     0.24      0.27       0.26     0.21     0.22      0.21
##          Kurtosis     0.24    -0.94     -1.13       2.11    -1.13     0.13     -1.23
##           N.Valid   129.00    99.00     80.00      89.00   129.00   120.00    129.00
##                 N   129.00   129.00    129.00     129.00   129.00   129.00    129.00
##         Pct.Valid   100.00    76.74     62.02      68.99   100.00    93.02    100.00
```

Données Nelson-Plosser

Matrice de corrélation

- La matrice de corrélation contient les coefficients de corrélation linéaire entre les variables prises 2 à 2

```
datanum <- Nelson_Plosser %>% filter(year>1909) %>% select(2:8)
cormat <- round(cor(datanum),2)
cormat
```

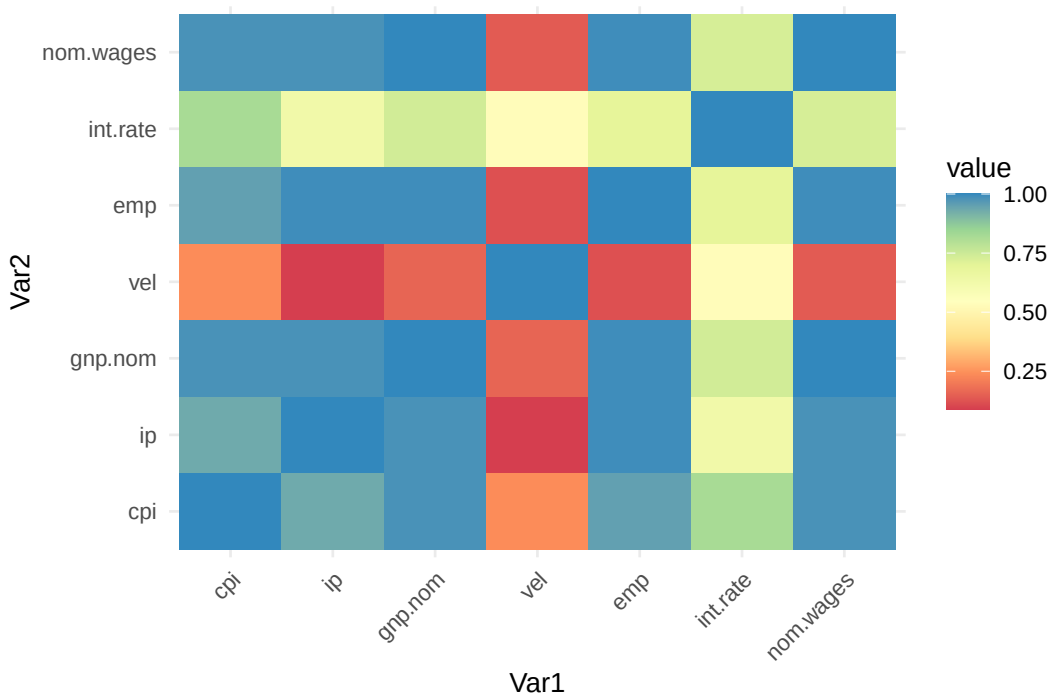
```
##             cpi   ip gnp.nom  vel  emp int.rate nom.wages
## cpi        1.00 0.93    0.98 0.24 0.95     0.82      0.98
## ip         0.93 1.00    0.98 0.09 0.99     0.63      0.98
## gnp.nom    0.98 0.98    1.00 0.16 0.99     0.74      1.00
## vel        0.24 0.09    0.16 1.00 0.12     0.53      0.14
## emp        0.95 0.99    0.99 0.12 1.00     0.69      0.99
## int.rate   0.82 0.63    0.74 0.53 0.69     1.00      0.73
## nom.wages  0.98 0.98    1.00 0.14 0.99     0.73      1.00
```

Données Nelson-Plosser

Matrice de corrélation : Heatmap

- On peut reprédenter la matrice de corrélation sous la forme d'une *heatmap*

```
library(reshape2)
cormat %>% melt() %>% ggplot(aes(x=Var1, y=Var2, fill=value)) +
  geom_tile() +
  theme(axis.text.x = element_text(angle = 45, vjust = 1, hjust=1)) +
  scale_fill_distiller(palette = "Spectral", direction = 1)
```



Données Nelson-Plosser

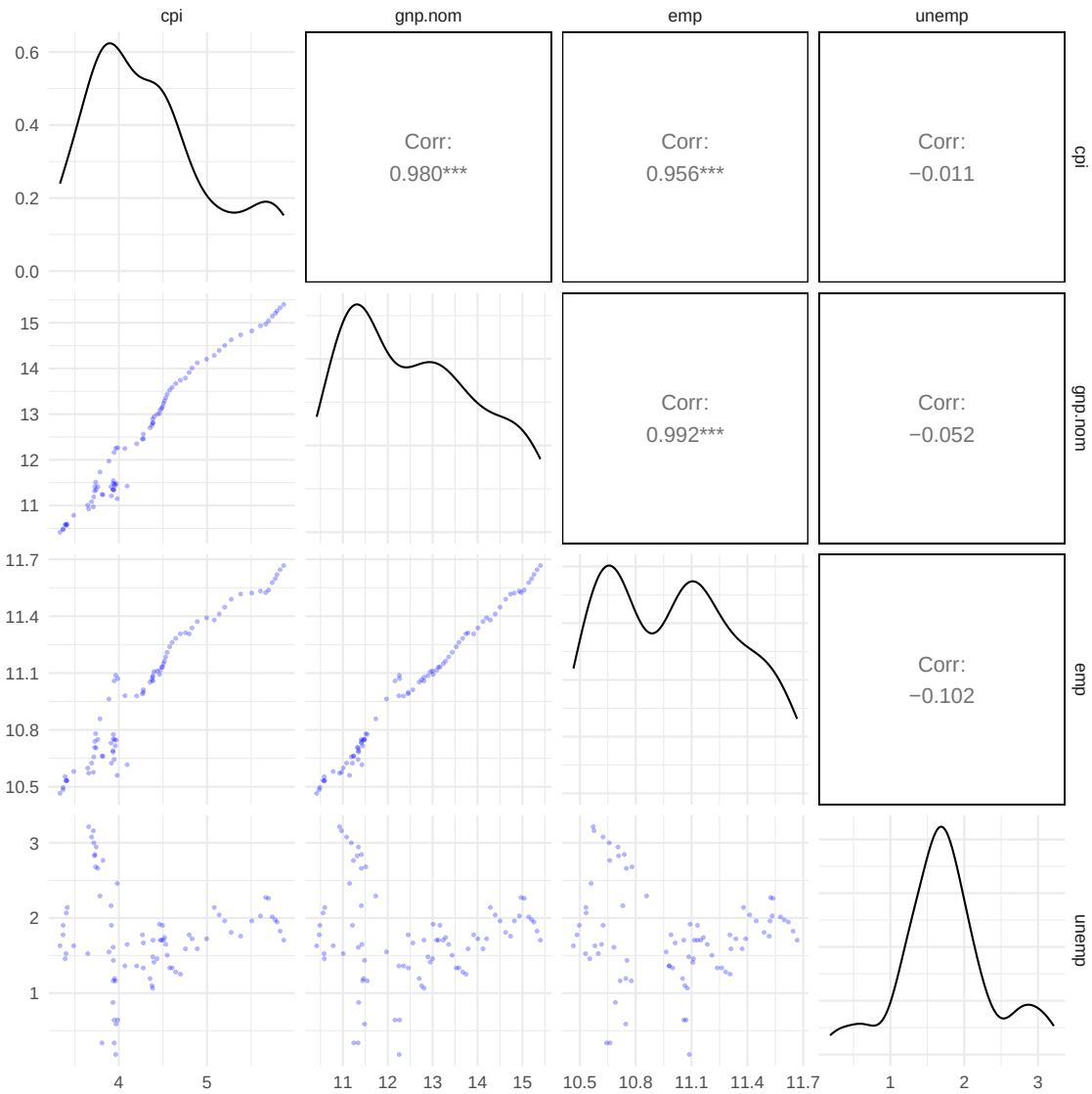
Pairplot (1)

- On peut visualiser la distribution et la corrélation entre variables avec la fonction `ggpairs()` (librairie `GGally`)
- On sélectionne ici un sous-ensemble de variables pour limiter la taille du graphique

```
Nelson_Plosser %>%
  filter(year>=1909) %>%
  select(cpi, gnp.nom, emp, unemp) %>%
  ggpairs(upper = list(continuous = wrap("cor", size = 4)),
          lower = list(continuous = wrap("points", colour="blue", alpha=0.3, size=0.5)))
```

Données Nelson-Plosser

Pairplot (2)



Données retail

Importation des données au format csv

- Série des ventes du commerce de détail et nourriture (Advance retail sales)
- Données mensuelles, en millions de dollars, non corrigées des variations saisonnières
- Source : FRED (Federal Reserve Bank Economic Data)
- On importe les données à partir du fichier csv
- A noter : dans le fichier csv, le séparateur décimal est un point

```
retail <- read.csv2("../data/RSXFSN.csv", header=TRUE, sep=",", dec = ".")
head(retail)
```

```
##         DATE RSXFSN
## 1 1992-01-01 130683
## 2 1992-02-01 131244
## 3 1992-03-01 142488
## 4 1992-04-01 147175
## 5 1992-05-01 152420
## 6 1992-06-01 151849
```

Données retail

Conversion de la date

- La date est au format `character`, il faut la convertir

```
summary(retail)
```

```
##         DATE         RSXFSN
##  Length   :381   Min.   :130683
##  N.unique :381   1st Qu.:236830
##  N.blank  :  0   Median :315540
##  Min.nchar: 10   Mean   :326550
##  Max.nchar: 10   3rd Qu.:394764
##                  Max.   :654825
```

- On utilise la fonction `as.Date()`

```
retail$DATE <- as.Date(retail$DATE,format="%Y-%m-%d")
summary(retail)
```

```
##       DATE                RSXFSN
##  Min.   :1992-01-01   Min.   :130683
##  1st Qu.:1999-12-01   1st Qu.:236830
##  Median :2007-11-01   Median :315540
##  Mean   :2007-10-31   Mean   :326550
##  3rd Qu.:2015-10-01   3rd Qu.:394764
##  Max.   :2023-09-01   Max.   :654825
```

La librairie insee

Taux de chômage (1)

- La librairie insee permet d'accéder directement à de nombreuses séries temporelles macro-économiques
- Ici on récupère la série des taux de chômage

```
library(insee)
df_idbank_list_selected =
  get_idbank_list("CHOMAGE-TRIM-NATIONAL") %>% #Unemployment dataset
  add_insee_title() %>%
  filter(INDICATEUR == "CTTXC") %>% #unemployment rate based on ILO standards
  filter(REF_AREA == "FM") %>%  # all France excluding overseas departements
  filter(SEXE == 0) # men and women

list_idbank = df_idbank_list_selected %>% pull(idbank)

fr_unemp = get_insee_idbank(list_idbank, startPeriod = "1950-01") %>% split_title()
```

La librairie insee

Taux de chômage (2)

- La série est sauvegardée dans le fichier `fr_unemp.RData`, elle peut être importée avec la fonction `load()`

```
load("../data/fr_unemp.RData")
head(fr_unemp)
```

```
## # A tibble: 6 x 24
##   DATE       TIME_PERIOD OBS_VALUE OBS_STATUS OBS_QUAL OBS_TYPE IDBANK    FREQ
##   <date>     <chr>           <dbl> <chr>      <chr>    <chr>    <chr>     <chr>
## 1 2024-04-01 2024-Q2           7.1 A          DEF      A        001688526 T
## 2 2024-01-01 2024-Q1           7.2 A          DEF      A        001688526 T
## 3 2023-10-01 2023-Q4           7.3 A          DEF      A        001688526 T
## 4 2023-07-01 2023-Q3           7.2 A          DEF      A        001688526 T
## 5 2023-04-01 2023-Q2           7   A          DEF      A        001688526 T
## 6 2023-01-01 2023-Q1           6.9 A          DEF      A        001688526 T
## # i 16 more variables: TITLE_FR <chr>, TITLE_FR1 <chr>, TITLE_FR2 <chr>,
## #   TITLE_FR3 <chr>, TITLE_FR4 <chr>, TITLE_EN <chr>, TITLE_EN1 <chr>,
## #   TITLE_EN2 <chr>, TITLE_EN3 <chr>, TITLE_EN4 <chr>, LAST_UPDATE <chr>,
## #   UNIT_MEASURE <chr>, UNIT_MULT <chr>, REF_AREA <chr>, DECIMALS <chr>,
## #   OBS_REV <chr>
```

La librairie insee

Taux de chômage (3)

- Les colonnes TITLE_xxx contiennent l'intitulé des séries

```
fr_unemp %>% distinct(TITLE_FR)
```

```
## # A tibble: 4 x 1
##   TITLE_FR
##   <chr>
## 1 Taux de chômage au sens du BIT - Ensemble - France métropolitaine - Données C~
## 2 Taux de chômage au sens du BIT - Ensemble des 25 à 49 ans - France métropolit~
## 3 Taux de chômage au sens du BIT - Ensemble des 50 ans ou plus - France métropo~
## 4 Taux de chômage au sens du BIT - Ensemble des moins de 25 ans - France métrop~
```

- Début et fin de la série

```
fr_unemp %>% summarise(début=min(DATE), fin=max(DATE))
```

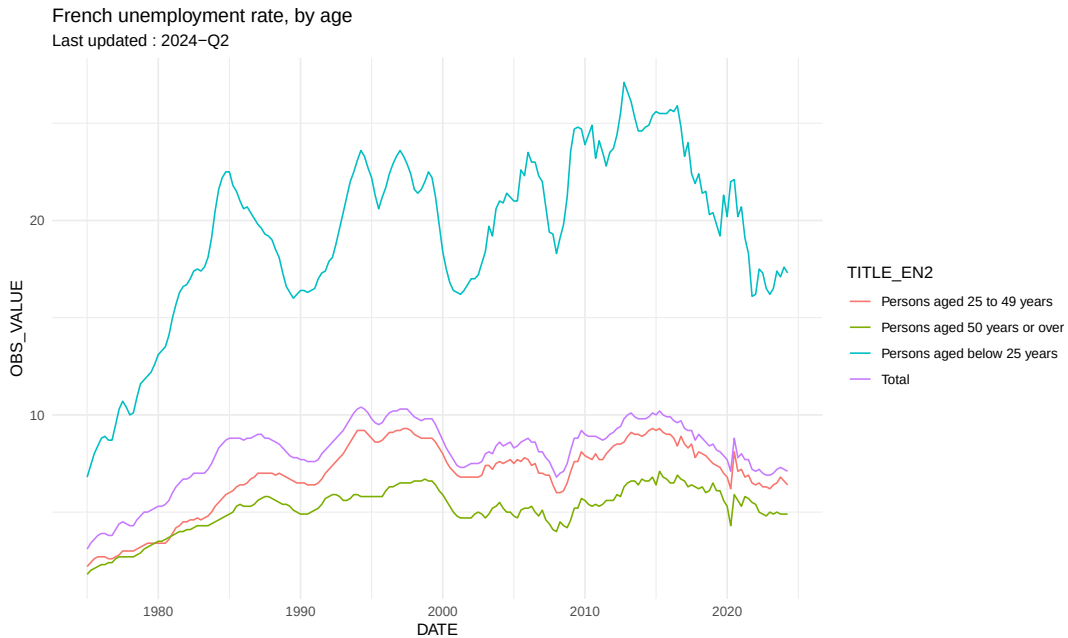
```
## # A tibble: 1 x 2
##   début      fin
##   <date>     <date>
## 1 1975-01-01 2024-04-01
```

La librairie insee

Taux de chômage (4)

- On visualise les séries avec ggplot()

```
ggplot(fr_unemp, aes(x = DATE, y = OBS_VALUE, colour = TITLE_EN2)) +
  geom_line() +
  ggtitle("French unemployment rate, by age") +
  labs(subtitle = sprintf("Last updated : %s", fr_unemp$TIME_PERIOD[1]))
```



La librairie insee

PIB (1)

- Téléchargement de la série trimestrielle du PIB, corrigé des variations saisonnières (CVS)

```
df_idbank_list_selected =
  get_idbank_list("CNT-2014-PIB-EQB-RF") %>% # Gross domestic product balance
  filter(FREQ == "T") %>% #quarter
  add_insee_title() %>% #add titles
  filter(OPERATION == "PIB") %>% #GDP
  filter(NATURE == "VALEUR_ABSOLUE") %>% #rate
  filter(CORRECTION == "CVS-CJO") #SA-WDA, seasonally adjusted, working day adjusted

idbank = df_idbank_list_selected %>% pull(idbank)

fr_pib = get_insee_idbank(idbank)
```

La librairie insee

PIB (2)

- La série est sauvegardée dans le fichier `fr_pib.RData`, on l'importe avec la fonction `load()`

```
load("../data/fr_pib.RData")
head(fr_pib)
```

```
## # A tibble: 6 x 16
##   DATE       TIME_PERIOD OBS_VALUE OBS_STATUS OBS_QUAL OBS_TYPE IDBANK    FREQ
##   <date>     <chr>           <dbl> <chr>      <chr>    <chr>    <chr>     <chr>
## 1 2024-01-01 2024-Q1        595694 A          DEF      A        010565708 T
## 2 2023-10-01 2023-Q4        594336 A          DEF      A        010565708 T
## 3 2023-07-01 2023-Q3        593602 A          DEF      A        010565708 T
## 4 2023-04-01 2023-Q2        593156 A          DEF      A        010565708 T
## 5 2023-01-01 2023-Q1        589375 A          DEF      A        010565708 T
## 6 2022-10-01 2022-Q4        589499 A          DEF      A        010565708 T
## # i 8 more variables: TITLE_FR <chr>, TITLE_EN <chr>, LAST_UPDATE <chr>,
## #   UNIT_MEASURE <chr>, UNIT_MULT <chr>, REF_AREA <chr>, DECIMALS <chr>,
## #   OBS_REV <chr>
```

- Date de début de la série

```
min(fr_pib$DATE)
```

```
## [1] "1949-01-01"
```

- Les colonnes `TITLE_xxx` contiennent l'intitulé des séries

```
fr_pib %>% distinct(TITLE_EN)
```

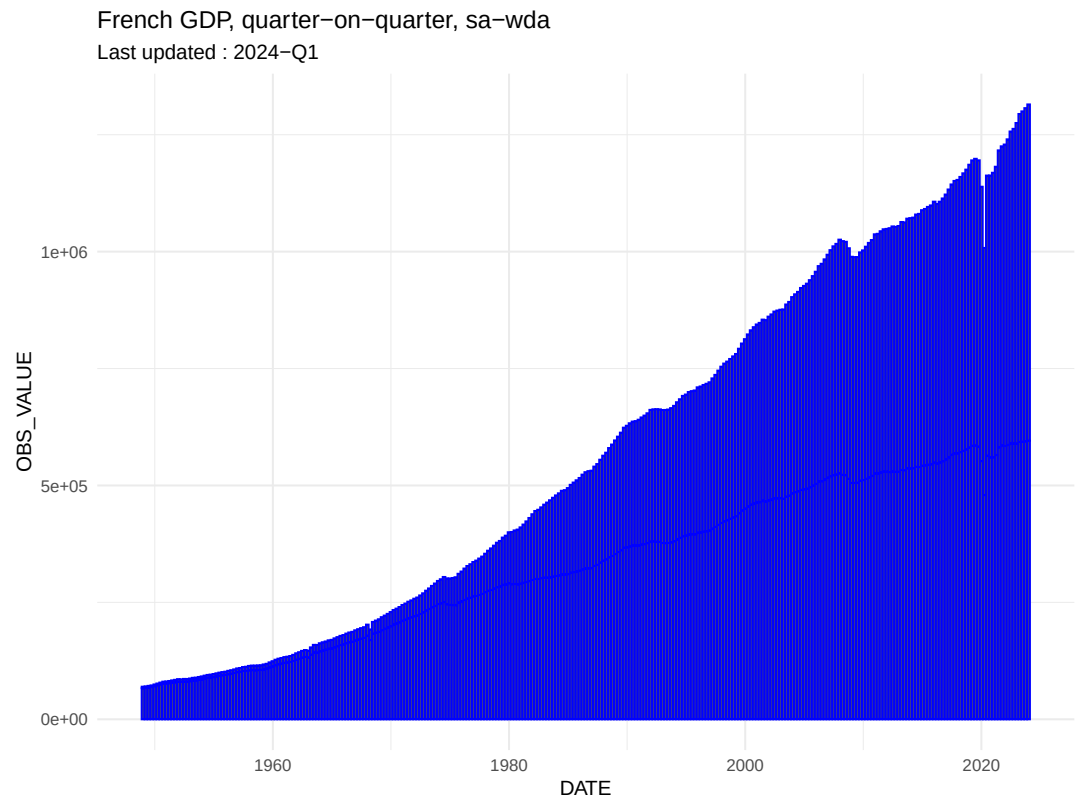
```
## # A tibble: 2 x 1
##   TITLE_EN
##   <chr>
## 1 Total gross domestic product - Volumes chained at previous year prices - SA-W~
## 2 Total gross domestic product - Values at current prices - SA-WDA series - Sto~
```

La librairie insee

PIB (4)

Visualisation avec ggplot()

```
ggplot(fr_pib, aes(x = DATE, y = OBS_VALUE)) +
  geom_col(color="blue") +
  ggtitle("French GDP, quarter-on-quarter, sa-wda") +
  labs(subtitle = sprintf("Last updated : %s", fr_pib$TIME_PERIOD[1]))
```



La librairie insee

Indice des prix à la consommation (1)

- Téléchargement de la série mensuelle de l'indice des prix à la consommation (attention, il y a énormément de séries, et il faut repérer l'identifiant de la série (idbank))

```
df_idbank_list_selected =
  get_idbank_list("IPC-2015") %>% # Gross domestic product balance
  filter(FREQ == "M" & idbank=="001759970") %>%
  add_insee_title()

idbank = df_idbank_list_selected %>% pull(idbank)

fr_cpi = get_insee_idbank(idbank)
```

La librairie `insee`

Indice des prix à la consommation (2)

- La série est sauvegardée dans le fichier `fr_cpi.RData`
- Pour cette série les données sont **mensuelles**

```
load("../data/fr_cpi.RData")
head(fr_cpi)
```

```
## # A tibble: 6 x 17
##   DATE       TIME_PERIOD OBS_VALUE OBS_STATUS OBS_QUAL OBS_TYPE IDBANK    FREQ
##   <date>     <chr>           <dbl> <chr>      <chr>    <chr>    <chr>     <chr>
## 1 2024-10-01 2024-10          120. P          P        A        001759970 M
## 2 2024-09-01 2024-09          120. P          P        A        001759970 M
## 3 2024-08-01 2024-08          121. A          DEF      A        001759970 M
## 4 2024-07-01 2024-07          120. A          DEF      A        001759970 M
## 5 2024-06-01 2024-06          120. A          DEF      A        001759970 M
## 6 2024-05-01 2024-05          120. A          DEF      A        001759970 M
## # i 9 more variables: TITLE_FR <chr>, TITLE_EN <chr>, LAST_UPDATE <chr>,
## #   UNIT_MEASURE <chr>, UNIT_MULT <chr>, REF_AREA <chr>, DECIMALS <chr>,
## #   DATE_JO <chr>, OBS_REV <chr>
```

La librairie insee

Indice des prix à la consommation (3)

- Les colonnes TITLE_xxx contiennent l'intitulé des séries

```
fr_cpi %>% distinct(TITLE_FR)
```

```
## # A tibble: 1 x 1
##   TITLE_FR
##   <chr>
## 1 Indice des prix à la consommation - Base 2015 - Ensemble des ménages - France~
```

- Début et fin de la série

```
fr_cpi %>% summarise(début=min(fr_cpi$DATE), fin=max(fr_cpi$DATE))
```

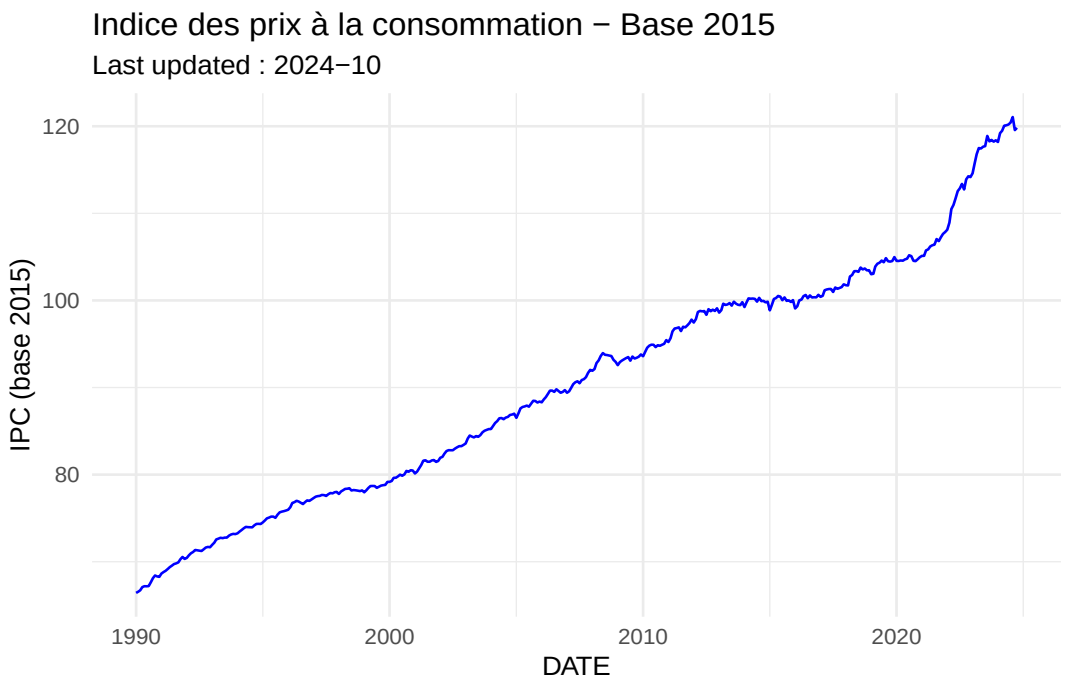
```
## # A tibble: 1 x 2
##   début      fin
##   <date>     <date>
## 1 1990-01-01 2024-10-01
```

La librairie insee

Indice des prix à la consommation (4)

Visualisation avec ggplot()

```
ggplot(fr_cpi, aes(x = DATE, y = OBS_VALUE)) +
  geom_line(color="blue") +
  ggtitle("Indice des prix à la consommation - Base 2015") +
  labs(subtitle = sprintf("Last updated : %s", fr_cpi$TIME_PERIOD[1])) +
  ylab("IPC (base 2015)")
```



Environnement de travail

Exercices

1. Le fichier `pib_fr.csv` contient la série temporelle du PIB et de ses composants depuis 1949 (valeurs aux prix courants - Source : INSEE) :
   - Importez les données à partir du fichier `csv`
   - Explorez les données avec des statistiques descriptives et des graphiques
2. La série de l'indice des prix à la consommation est contenue dans le fichier `fr_cpi.RData`
   - Importez les données
   - La fréquence de cette série est mensuelle. Transformez-là en série trimestrielle (moyenne sur les trois mois d'un trimestre). Les fonctions `quarter` et `year` vous seront utiles.
3. Fusionnez les séries `fr_pib`, `fr_unemp` et `fr_cpi` dans un jeu de données unique `fr_macro` (utilisez les fonctions '`_join`' de `dlpyr`)

Environnement de travail

Exercices : Solutions (1)

Importation du fichier csv

```
pib <- read.csv("../data/pib_fr.csv", sep=";", dec=',')
head(pib)
```

```
##   PERIODE PIB  P7 P3M P3P P31G P32G  P3 P51S P51B P51G P51M P51P P51 P54  P6
## 1  1949T1 3.2 0.4 1.9 0.1  0.2  0.2 2.4  0.4    0  0.1  0.1    0 0.6 0.1 0.4
## 2  1949T2 3.2 0.4 2.0 0.1  0.3  0.2 2.5  0.4    0  0.1  0.1    0 0.6 0.1 0.5
## 3  1949T3 3.3 0.4 2.1 0.1  0.3  0.2 2.6  0.4    0  0.1  0.1    0 0.6 0.1 0.5
## 4  1949T4 3.4 0.4 2.1 0.1  0.3  0.3 2.6  0.4    0  0.1  0.1    0 0.7 0.1 0.5
## 5  1950T1 3.6 0.5 2.1 0.1  0.3  0.3 2.7  0.5    0  0.1  0.1    0 0.7 0.2 0.5
## 6  1950T2 3.8 0.5 2.2 0.1  0.3  0.3 2.8  0.5    0  0.1  0.1    0 0.7 0.2 0.6
```

Environnement de travail

Exercices : Solutions (2)

Importation et transformation de la série `fr_cpi`

```
load("../data/fr_cpi.RData")
fr_cpi <- fr_cpi %>% mutate(Année=year(DATE), Trimestre=quarter(DATE)) %>%
  group_by(Année, Trimestre) %>%
  rename(cpi=OBS_VALUE) %>%
  summarise(cpi=mean(cpi))
```

```
## `summarise()` has regrouped the output.
## i Summaries were computed grouped by Année and Trimestre.
## i Output is grouped by Année.
## i Use `summarise(.groups = "drop_last")` to silence this message.
## i Use `summarise(.by = c(Année, Trimestre))` for per-operation grouping
##   (`?dplyr::dplyr_by`) instead.
```

```
head(fr_cpi)
```

```
## # A tibble: 6 x 3
## # Groups:   Année [2]
##   Année Trimestre   cpi
##   <dbl>     <int> <dbl>
## 1  1990         1  66.6
## 2  1990         2  67.2
## 3  1990         3  67.7
## 4  1990         4  68.3
## 5  1991         1  68.8
## 6  1991         2  69.4
```

Environnement de travail

Exercices : Solutions (3)

Fusion des jeux de données

```
load("../data/fr_pib.RData")
fr_pib <- fr_pib %>% mutate(Année=year(DATE), Trimestre=quarter(DATE)) %>%
  rename(gnp=OBS_VALUE) %>%
  filter(TITLE_FR==unique(fr_pib$TITLE_FR)[1]) %>%
  select(Année, Trimestre, gnp)

load("../data/fr_unemp.RData")
fr_unemp <- fr_unemp %>% mutate(Année=year(DATE), Trimestre=quarter(DATE)) %>%
  rename(unemp=OBS_VALUE) %>%
  filter(TITLE_FR2=='Ensemble') %>%
  select(Année, Trimestre, unemp)

fr_macro <- fr_pib %>%
  left_join(fr_unemp, by=c("Année", "Trimestre")) %>%
  left_join(fr_cpi, by=c("Année", "Trimestre")) %>%
  arrange(Année, Trimestre)

head(fr_macro)
```

```
## # A tibble: 6 x 5
##   Année Trimestre   gnp unemp   cpi
##   <dbl>     <int> <dbl> <dbl> <dbl>
## 1  1949         1 66592    NA    NA
## 2  1949         2 67170    NA    NA
## 3  1949         3 68183    NA    NA
## 4  1949         4 69046    NA    NA
## 5  1950         1 70999    NA    NA
## 6  1950         2 72780    NA    NA
```

3 Librairies spécialisées et structures de séries temporelles dans R

Liste des librairies

- En plus de la librairie standard `stats`, il existe plusieurs librairies R pour la manipulation et l'analyse des séries temporelles
  - tsibble
  - `forecasts`
  - `feasts`
  - `fable`
  - `ggfortify`
  - `tseries`
  - `zoo`
- Dans cette partie on se focalise sur les classes permettant de stocker les séries temporelles fournies par les librairies `stats` et `tsibble`

Les objets `ts` et `mts`

- La classe de base fournie par R pour représenter des séries temporelles s'appelle `ts` (ts = time series, série univariée) ou `mts` (mts = multiple time series, série multivariée)
- Cette classe est définie dans le package `stats`
- Elle concerne des séries temporelles qui sont échantillonnées à des périodes **équidistantes** dans le temps

Les objets `ts` et `mts`

Paramètres

- Les objets de classe `ts` ou `mts` possèdent trois paramètres caractéristiques :
  - `frequency` : nombre d'observations par unité de temps. Si l'unité de temps de la série est l'année, la valeur 4 correspond à des trimestres et la valeur 12 à des mois
  - `start` : date de début de la série temporelle (nombre unique ou vecteur de deux entiers représentant respectivement une unité temporelle (e.g. une année) et une subdivision de cette unité (mois ou trimestre selon la valeur du paramètre frequency)
  - `end` : date de fin de la série temporelle, exprimée comme pour le paramètre `start`

Les objets ts et mts

Données NelPlo (1)

- Les données Nelson-Plosser sont également présentes dans la librairie tseries
- Les données annuelles (frequency=1) sont contenues dans un objet de type mts

```
library(tseries)
```

```
## Registered S3 method overwritten by 'quantmod':
##   method            from
##   as.zoo.data.frame zoo
```

```
data(NelPlo)
class(NelPlo)
```

```
## [1] "mts" "ts"
```

- La fonction class() permet de connaître la classe d'un objet R

```
class(NelPlo)
```

```
## [1] "mts" "ts"
```

Les objets ts et mts

Données NelPlo (2)

- La fonction window() permet d'extraire une portion d'une série temporelle ts
- L'ensemble des séries est complet à partir de 1909 (il y a des valeurs manquantes sur certaines séries avant)

```
window(NelPlo, start=1905, end=1910)
```

```
## Time Series:
## Start = 1905
## End = 1910
## Frequency = 1
##           cpi       ip  gnp.nom       vel      emp int.rate nom.wages  gnp.def
## 1905 3.295837 2.219203       NA 0.7561220 10.37274     3.50  6.329721 3.277145
## 1906 3.332205 2.282382       NA 0.8241754 10.42671     3.55  6.357842 3.303217
## 1907 3.367296 2.302585       NA 0.8241754 10.44497     3.80  6.393591 3.342862
## 1908 3.332205 2.140066       NA 0.7227060 10.41169     3.95  6.306275 3.335770
## 1909 3.332205 2.302585 10.41631 0.7839015 10.46516     3.77  6.395262 3.370738
## 1910 3.367296 2.360854 10.47164 0.7747272 10.48464     3.80  6.478510 3.397858
##      money.stock gnp.real stock.prices gnp.capita real.wages     unemp
## 1905    2.326302       NA     2.196113         NA   3.033991 1.4586150
## 1906    2.405142       NA     2.265921         NA   3.025776 0.5306283
## 1907    2.451005       NA     2.059239         NA   3.026261 1.0296194
## 1908    2.437116       NA     2.051556         NA   2.973998 2.0794415
## 1909    2.540026 4.760463     2.273156   7.163172   3.062924 1.6292405
## 1910    2.590767 4.788325     2.235376   7.170120   3.111291 1.7749524
```

Les objets ts et mts

Données growthofmoney (Growth of Money Supply)

- Deux séries temporelles concernant le contrôle de la monnaie par la réserve fédérale aux USA
- Article original : R.L. Hetzel (1981), The Federal Reserve System and Control of the Money Supply in the 1970's, *Journal of Money, Credit and Banking*
- Série temporelle multivariée, données trimestrielles

```
library(lmtest)
data(growthofmoney)
window(growthofmoney, end=c(1971,4))
```

```
##         TG1.TG0 AG0.TG0
## 1970 Q2     0.0    -0.4
## 1970 Q3     1.0    -1.0
## 1970 Q4     1.0     1.1
## 1971 Q1     2.5     5.8
## 1971 Q2    -6.0    -4.4
## 1971 Q3     4.5    -1.6
## 1971 Q4    -0.5     1.6
```

Les objets ts et mts

Données USIncExp (US Income and Expenditure)

- Deux séries macro-économiques (USA) de janvier 1959 à février 2001, corrigées des variations saisonnières :
  - Revenus personnels mensuels aggrégés (millions de dollars)
  - Dépences de consommation aggrégées (millions de dollars)

```
library(strucchange)
data("USIncExp")
window(USIncExp, start=c(1959,6), end=c(1960,6))
```

```
##          income expenditure
## Jun 1959  396.3       319.2
## Jul 1959  396.5       318.8
## Aug 1959  395.0       321.2
## Sep 1959  396.2       325.2
## Oct 1959  397.8       323.8
## Nov 1959  401.2       323.9
## Dec 1959  405.7       323.9
## Jan 1960  407.0       324.6
## Feb 1960  407.7       326.4
## Mar 1960  408.6       331.2
## Apr 1960  411.3       337.6
## May 1960  412.8       331.1
## Jun 1960  413.1       331.2
```

Les objets ts et mts

Conversion des données retail

- Pour les données retail, frequency=12 (mois)

```
retail_ts <- ts(retail['RSXFSN'], frequency=12, start=c(1992,1))
window(retail_ts, start=2020)
```

```
##         Jan    Feb    Mar    Apr    May    Jun    Jul    Aug    Sep    Oct
## 2020 417123 413923 426258 376867 461785 479784 492689 487325 474038 493601
## 2021 461308 437031 560218 554269 565177 557446 552179 550169 529133 553588
## 2022 516923 507901 598541 596690 616626 609743 599929 613508 577966 597170
## 2023 547156 529374 604084 588220 631496 612243 605403 628892 592660
##         Nov    Dec
## 2020 490941 557696
## 2021 574978 627113
## 2022 605205 654825
## 2023
```

Les objets ts et mts

Extraction de l'index

- La fonction tsp permet d'extraire les propriétés d'un objet ts ou mts

```
tsp(retail_ts)
```

```
## [1] 1992.000 2023.667   12.000
```

- Extraction de l'index

```
retail2022 <- window(retail_ts, start=c(2022,1), end=c(2022,12))
time(retail2022)
```

```
##           Jan      Feb      Mar      Apr      May      Jun      Jul      Aug
## 2022 2022.000 2022.083 2022.167 2022.250 2022.333 2022.417 2022.500 2022.583
##           Sep      Oct      Nov      Dec
## 2022 2022.667 2022.750 2022.833 2022.917
```

- Conversion de l'index au format numérique

```
library(lubridate)
as.numeric(time(retail2022))
```

```
##  [1] 2022.000 2022.083 2022.167 2022.250 2022.333 2022.417 2022.500 2022.583
##  [9] 2022.667 2022.750 2022.833 2022.917
```

Les objets `tsibble`

- La librairie `tsibble` fournit une structure de type ‘tidy‘ pour les données temporelles ainsi que des outils pour le traitement de ces données
- Les objets de la classe `tsibble` possèdent deux attributs principaux :
  - `index` est la variable qui représente le temps (qui permet d'ordonner du passé au présent)
  - `key` identifie (éventuellement) la série (unité d'observation)
- Chaque observation est identifiée de manière unique par la combinaison `index` et `key`

Les objets `tsibble`

Conversion des données `growthofmoney`

- On utilise la fonction `as_tsibble()`

```
library(tsibble)
gmoney_tsbl <- growthofmoney %>% as_tsibble()
gmoney_tsbl
```

```
## # A tsibble: 38 x 3 [1Q]
## # Key:       key [2]
##      index key     value
##      <qtr> <chr>   <dbl>
##  1 1970 Q2 TG1.TG0   0
##  2 1970 Q3 TG1.TG0   1
##  3 1970 Q4 TG1.TG0   1
##  4 1971 Q1 TG1.TG0   2.5
##  5 1971 Q2 TG1.TG0  -6
##  6 1971 Q3 TG1.TG0   4.5
##  7 1971 Q4 TG1.TG0  -0.5
##  8 1972 Q1 TG1.TG0  -1
##  9 1972 Q2 TG1.TG0   0.5
## 10 1972 Q3 TG1.TG0  -1.5
## # i 28 more rows
```

- On obtient un objet de classe `tbl_ts`

```
class(gmoney_tsbl)
```

```
## [1] "tbl_ts"     "tbl_df"     "tbl"        "data.frame"
```

- La variable `key` référence les séries

```
gmoney_tsbl %>% distinct(key)
```

```
## # A tibble: 2 x 1
##   key
##   <chr>
## 1 TG1.TG0
## 2 AG0.TG0
```

Les objets tsibble

Conversion des données NelPlo (1)

- La transformation d'un objet de type mts en objet tsibble produit un jeu de données au format long

```
nelplo_tsbl <- NelPlo %>% as_tsibble()
nelplo_tsbl %>% filter(index>1980 & key=="gnp.capita")
```

```
## # A tsibble: 8 x 3 [1Y]
## # Key:       key [1]
##   index key         value
##   <dbl> <chr>       <dbl>
## 1  1981 gnp.capita  8.34
## 2  1982 gnp.capita  8.31
## 3  1983 gnp.capita  8.33
## 4  1984 gnp.capita  8.39
## 5  1985 gnp.capita  8.41
## 6  1986 gnp.capita  8.43
## 7  1987 gnp.capita  8.46
## 8  1988 gnp.capita  8.49
```

Les objets `tsibble`

Conversion des données `NelPlo` (2)

- Les 14 séries sont identifiées par la variable `key`

```
nelplo_tsbl %>% distinct(key)
```

```
## # A tibble: 14 x 1
##    key
##    <chr>
##  1 cpi
##  2 ip
##  3 gnp.nom
##  4 vel
##  5 emp
##  6 int.rate
##  7 nom.wages
##  8 gnp.def
##  9 money.stock
## 10 gnp.real
## 11 stock.prices
## 12 gnp.capita
## 13 real.wages
## 14 unemp
```

Les objets tsibble

Conversion des données retail

- Les données retail ne contiennent qu'une seule série, il n'y a pas de variable key

```
retail_tsbl <- retail_ts %>% as_tsibble()
head(retail_tsbl)
```

```
## # A tsibble: 6 x 2 [1M]
##        index  value
##        <mth>  <dbl>
## 1 1992 janv. 130683
## 2 1992 févr. 131244
## 3  1992 mars 142488
## 4 1992 avril 147175
## 5   1992 mai 152420
## 6  1992 juin 151849
```

Librairies spécialisées et strucutre de séries temporelles

Exercices

- Convertissez les données `pib` (série du PIB et de ses composantes) en objets `mts` et `tsibble`
- Convertissez les données `fr_macro` (série de l'index des prix à la consommation, du taux de chômage et du pib) en objets `mts` et `tsibble`

Librairies spécialisées et structure de séries temporelles

Exercices : Solutions (1)

- Conversion des données `pib` (série du PIB et de ses composantes) en objet `mts`

```
pib_ts <- pib %>% select(-PERIODE) %>% ts(start=1949, frequency=4)
window(pib_ts, start=1949, end=1950)
```

```
##         PIB  P7 P3M P3P P31G P32G  P3 P51S P51B P51G P51M P51P P51 P54  P6
## 1949 Q1 3.2 0.4 1.9 0.1  0.2  0.2 2.4  0.4    0  0.1  0.1    0 0.6 0.1 0.4
## 1949 Q2 3.2 0.4 2.0 0.1  0.3  0.2 2.5  0.4    0  0.1  0.1    0 0.6 0.1 0.5
## 1949 Q3 3.3 0.4 2.1 0.1  0.3  0.2 2.6  0.4    0  0.1  0.1    0 0.6 0.1 0.5
## 1949 Q4 3.4 0.4 2.1 0.1  0.3  0.3 2.6  0.4    0  0.1  0.1    0 0.7 0.1 0.5
## 1950 Q1 3.6 0.5 2.1 0.1  0.3  0.3 2.7  0.5    0  0.1  0.1    0 0.7 0.2 0.5
```

```
pib_tsbl <- as_tsibble(pib_ts)
pib_tsbl
```

```
## # A tsibble: 4,470 x 3 [1Q]
## # Key:       key [15]
##      index key   value
##      <qtr> <chr> <dbl>
##  1 1949 Q1 PIB     3.2
##  2 1949 Q2 PIB     3.2
##  3 1949 Q3 PIB     3.3
##  4 1949 Q4 PIB     3.4
##  5 1950 Q1 PIB     3.6
##  6 1950 Q2 PIB     3.8
##  7 1950 Q3 PIB     4
##  8 1950 Q4 PIB     4.2
##  9 1951 Q1 PIB     4.4
## 10 1951 Q2 PIB     4.8
## # i 4,460 more rows
```

Librairies spécialisées et strucutre de séries temporelles

Exercices : Solutions (2)

- Conversion des données `fr_macro` (série de l'index des prix à la consommation, du taux de chômage et du pib) en objets `mts`

```
fr_macro_ts <- fr_macro %>% select(-c(Année, Trimestre)) %>% ts(start=1949, frequency=4)
window(fr_macro_ts, start=1989, end=1992)
```

```
##            gnp unemp      cpi
## 1989 Q1 353355   8.1       NA
## 1989 Q2 357704   7.9       NA
## 1989 Q3 361651   7.8       NA
## 1989 Q4 366833   7.8       NA
## 1990 Q1 367883   7.7 66.56667
## 1990 Q2 369798   7.7 67.15333
## 1990 Q3 372019   7.6 67.65333
## 1990 Q4 371377   7.6 68.33000
## 1991 Q1 372215   7.6 68.80333
## 1991 Q2 373446   7.7 69.36333
## 1991 Q3 374761   8.0 69.82000
## 1991 Q4 376881   8.2 70.38333
## 1992 Q1 379939   8.4 70.71667
```

Librairies spécialisées et structure de séries temporelles

Exercices : Solutions (3)

- Conversion des données `fr_macro` (série de l'index des prix à la consommation, du taux de chômage et du pib) en objets `tsibble`

```
fr_macro_tsbl <- as_tsibble(fr_macro_ts)
fr_macro_tsbl
```

```
## # A tsibble: 903 x 3 [1Q]
## # Key:       key [3]
##      index key   value
##      <qtr> <chr> <dbl>
##  1 1949 Q1 gnp   66592
##  2 1949 Q2 gnp   67170
##  3 1949 Q3 gnp   68183
##  4 1949 Q4 gnp   69046
##  5 1950 Q1 gnp   70999
##  6 1950 Q2 gnp   72780
##  7 1950 Q3 gnp   74579
##  8 1950 Q4 gnp   76400
##  9 1951 Q1 gnp   76599
## 10 1951 Q2 gnp   77201
## # i 893 more rows
```

```
save(fr_macro_ts, fr_macro_tsbl, file="../data/fr_macro.RData")
```

4 Concepts théoriques

Série temporelle et processus stochastique

- Série temporelle univariée à temps discret = ensemble d'observations dans $\mathbb{R}$ enregistrées à un temps spécifique $t \in \mathbb{Z}$
- En statistique l'observation y est considérée comme la réalisation d'une **variable aléatoire** Y
- Une série temporelle $(y_t)_{t\in\mathbb{Z}}$ sera considérée comme la réalisation d'un **processus stochastique** $(Y_t)_{t\in\mathbb{Z}}$
- Processus stochastique $=>$ pour tout $t \in \mathbb{Z}$ fixé, Y_t est une variable aléatoire réelle
- L'objectif est d'étudier les caractéristiques principales de ce processus (tendance, variation saisonnière), de le modéliser et de faire des prévisions

Stationnarité

- Dans de très nombreux cas, on ne peut pas renouveler la suite de mesures dans des conditions identiques
- Pour que le modèle déduit à partir d'une suite d'observations ait un sens, il faut que toute portion de la trajectoire observée fournisse des informations sur la loi du processus et que des portions différentes, mais de même longueur, fournissent les mêmes indications. D'où la notion de stationnarité.
- Un processus $(Y_t)_{t\in\mathbb{Z}}$ est (faiblement) stationnaire si son espérance et ses autocovariances sont invariantes par translation dans le temps :
  - $\forall t \in \mathbb{Z} : \mathbb{E}(Y_t) = \mu$
  - $\forall t \in \mathbb{Z}, \forall h \in \mathbb{Z} : Cov(Y_t, Y_{t-h})$ dépend de l'intervalle h, mais pas de t

4 Concepts théoriques

- Autocovariance et autocorrélation
- Bruit blanc
- Marche aléatoire
- Marche aléatoire avec dérive

Fonction d'autocovariance et d'autocorrélation

- La fonction d'autocorrelation (ACF) est la corrélation entre y_t et y_{t-1}, y_{t-2}, y_{t-3}, etc ... :

$$\rho_j = \frac{Cov(y_t, y_{t-j})}{\sqrt{Var(y_t) \cdot Var(y_{t-j})}}$$

- Elle permet d'identifier une structure dans la série temporelle

Fonction d'autocorrélation avec `acf()`

- Pour les objets de la classe `ts` ou `mts` on peut utiliser la fonction `acf()` (cette fonction produit un graphique par défaut)

```
acf(NelPlo[,'cpi'], plot=FALSE)
```

```
##
## Autocorrelations of series 'NelPlo[, "cpi"]', by lag
##
##     0     1     2     3     4     5     6     7     8     9    10    11    12
## 1.000 0.966 0.928 0.889 0.853 0.818 0.784 0.748 0.712 0.677 0.645 0.615 0.585
##    13    14    15    16    17    18    19    20    21
## 0.558 0.533 0.509 0.489 0.469 0.450 0.431 0.412 0.396
```

- La corrélation entre le cpi à l'instant t et le cpi à l'instant $t-1$ (une année avant) est de 0.966

Fonction d'autocorrélation avec ACF()

- Pour les objets de la classe tsibble on utilise la fonction ACF() de la librairie feasts

```
library(feasts)

## Le chargement a nécessité le package : fabletools

nelplo_tsbl %>%
  filter(key=="cpi") %>%
  ACF(value)

## # A tsibble: 21 x 3 [1Y]
## # Key:       key [1]
##    key        lag   acf
##    <chr> <cf_lag> <dbl>
##  1 cpi         1Y 0.966
##  2 cpi         2Y 0.928
##  3 cpi         3Y 0.889
##  4 cpi         4Y 0.853
##  5 cpi         5Y 0.818
##  6 cpi         6Y 0.784
##  7 cpi         7Y 0.748
##  8 cpi         8Y 0.712
##  9 cpi         9Y 0.677
## 10 cpi        10Y 0.645
## # i 11 more rows
```

4 Concepts théoriques

- Autocovariance et autocorrélation
- Bruit blanc
- Marche aléatoire
- Marche aléatoire avec dérive

Bruit blanc

Définition

- Un bruit blanc (white noise) ϵ_t est une série de variables aléatoires **non corrélées** de moyenne nulle et de variance constante
- $(\epsilon_t)_{t\in\mathbb{Z}}$ est un bruit blanc faible si :
  - Son espérance est égale à 0 :
$$\forall t \in \mathbb{Z} : \mathbb{E}(\epsilon_t) = 0$$
  - Sa variance est constante :
$$\mathbb{E}(\sigma_t^2) = \sigma^2$$
  - La covariance entre (ϵ_t) et (ϵ_{t-h}) est nulle :
$$\forall t \in \mathbb{Z}, \forall h \in \mathbb{Z} : Cov(\epsilon_t, \epsilon_{t-h}) = 0$$
- Un bruit blanc gaussien ϵ_t est une série de variables aléatoires indépendantes et identiquement distribuées (i.i.d), suivant une loi normale $N(0, \sigma_z^2)$ de moyenne nulle et de variance σ_z^2
- Un bruit blanc gaussien est une série **strictement stationnaire**

Bruit blanc gaussien

Simulation (1)

- On simule un bruit blanc gaussien en générant 100 nombres aléatoires issus d'une loi normale $N(0, 1)$

```
bbg <- ts(rnorm(100))
window(bbg, start=1, end=10)
```

```
## Time Series:
## Start = 1
## End = 10
## Frequency = 1
##  [1]  0.37391442  0.05089372  0.50324144 -0.68692579 -1.67800009 -0.39751342
##  [7] -0.02776786 -1.48239942 -0.98601421  0.82722578
```

- On peut calculer la moyenne et l'écart type de la série

```
mean(bbg)
```

```
## [1] -0.1730682
```

```
sd(bbg)
```

```
## [1] 0.9271054
```

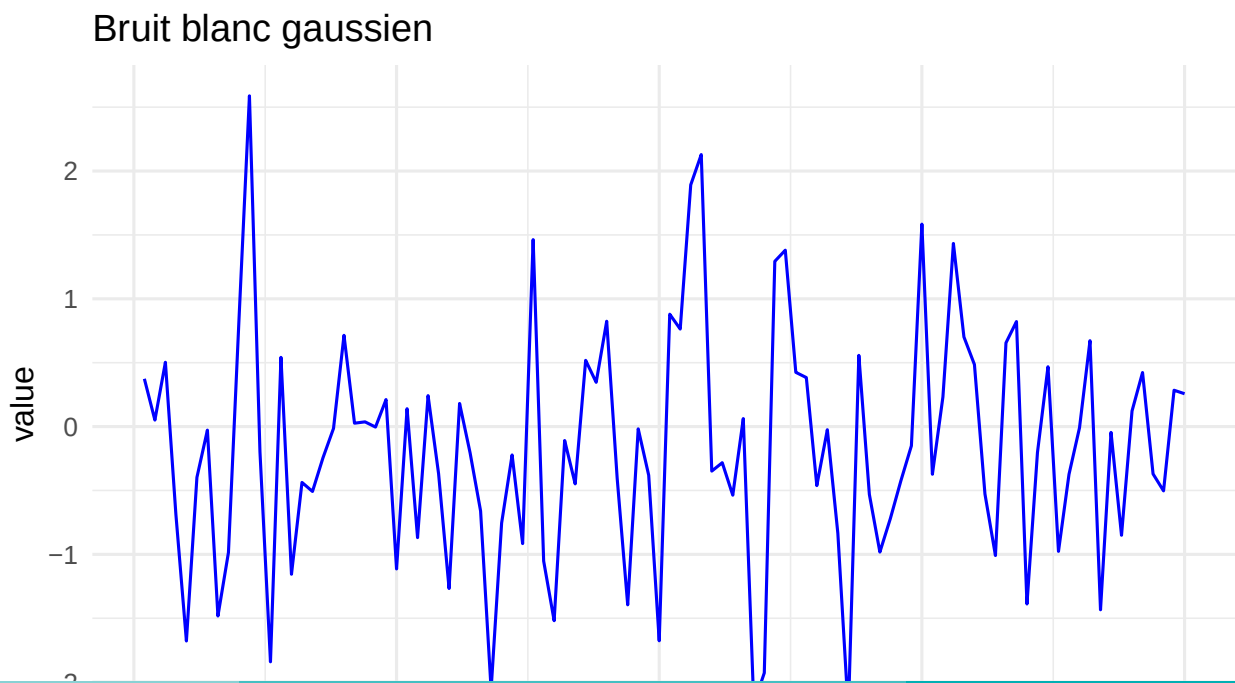
Bruit blanc gaussien

Simulation (2)

- Pour la représentation graphique, on convertit l'objet `ts` en objet tsibble puis on utilise la fonction `autoplot()` de la librairie `ggfortify`

```
library(ggfortify)
bbg %>% as_tsibble() %>% autoplot(color="blue") +
  ggtitle("Bruit blanc gaussien") +
  xlab("t")
```

```
## Warning: ‘autoplot.tbl_ts()‘ was deprecated in fabletools 0.6.0.
## i Please use ‘ggtime::autoplot.tbl_ts()‘ instead.
## i Graphics functions have been moved to the {ggtime} package. Please use
##   ‘library(ggtime)‘ instead.
## This warning is displayed once per session.
## Call ‘lifecycle::last_lifecycle_warnings()‘ to see where this warning was
## generated.
```



Bruit blanc gaussien

Simulation (3)

- On vérifie que la série ne présente pas d'autocorrélation avec la fonction `acf()`
- Cette fonction produit un graphique par défaut (voir section suivante), ici on utilise `plot=FALSE`

```
acf(bbg, plot=FALSE)
```

```
##
## Autocorrelations of series 'bbg', by lag
##
##      0      1      2      3      4      5      6      7      8      9     10
##  1.000  0.143 -0.146  0.003 -0.093 -0.124 -0.229 -0.116  0.129  0.181  0.103
##     11     12     13     14     15     16     17     18     19     20
## -0.056  0.014  0.041 -0.175 -0.122 -0.038  0.064 -0.007 -0.194 -0.017
```

Bruit blanc

Test de Portmanteau

- A l'issue d'une modélisation, il nous faudrait idéalement obtenir un signal résiduel qui ne contient plus d'information temporelle
- Dans le cadre des modèles ARMA, on souhaite que le résidu soit un bruit blanc (faible), c'est-à-dire sans dépendance temporelle linéaire
- On peut tester la **blancheur** d'une série $y_t, t = 1, \ldots, T$ en utilisant le test de Portmanteau
- La statistique de Portmanteau est calculée à partir des k premiers coefficients d'autocorrélation $Q_k = T \sum_{h=1}^{k} \hat{\rho}^2(h)$ où k est un décalage choisi par l'utilisateur et ρ_j l'estimateur du coefficient d'autocorrélation d'ordre h de la série y_t

Bruit blanc

Test de Portmanteau : Exemple

- On peut utiliser les fonctions `box_pierce()` et `ljung_box()` (pour les petits échantillons) de la librairie `feasts`
- Par défaut $k = 1$ (argument `lag=1`)
- Hypothèse H0 : pas d'autocorrélation
- On teste ici la série bbg de 100 nombres aléatoires issus d'une loi normale $N(0, 1)$

```
box_pierce(bbg)
```

```
##   bp_stat bp_pvalue
## 2.0306949 0.1541501
```

- La p-valeur est supérieure à 0.05, on accepte H0
- A noter : quand le test est appliqué non sur des v.a. indépendantes, mais sur les résidus d'un ajustement estimant m paramètres, on utilise le paramètre 'dof' (degrees of freedom, degrés de liberté)

4 Concepts théoriques

- Autocovariance et autocorrélation
- Bruit blanc
- Marche aléatoire
- Marche aléatoire avec dérive

Définition

- La série $y_t = y_{t-1} + \epsilon_t$ est une **marche aléatoire** (random walk)
- Une série suivant une marche aléatoire prend, à deux dates consécutives, des valeurs proches et la variation par rapport à la date précédente est **indépendante du passé** (c'est un bruit blanc)
- En exprimant y_{t-1} en fonction de $y_{t-2}, \ldots$ et d'une valeur initiale y_0 on obtient :

$$y_t = y_0 + \epsilon_1 + \epsilon_2 + \ldots + \epsilon_t$$

- Espérance :

$$E(y_t) = y_0$$

- Variance :

$$var(y_t) = t \cdot \sigma_\epsilon^2$$

- Covariance :

$$cov(y_t, y_{t+k}) = t \cdot \sigma_\epsilon^2, (k > 0)$$

- Il s'agit d'une série **non-stationnaire** car ni la variance ni l'autoccorélation ne sont constantes

Simulation (1)

- On simule une marche aléatoire avec un bruit blanc gaussien de moyenne 0 et d'écart type 1

```
set.seed(2025)
tmax <- 100
wnoise <- rnorm(99, mean=0, sd=1)
y <- rep(0,tmax)

for (t in 2:tmax) {
  y[t] = y[t-1] + wnoise[t-1]
}
rw <- tibble(time=1:tmax, y=y)
head(rw)
```

```
## # A tibble: 6 x 2
##    time     y
##   <int> <dbl>
## 1     1 0
## 2     2 0.621
## 3     3 0.656
## 4     4 1.43
## 5     5 2.70
## 6     6 3.07
```

Simulation (1)

- La moyenne observée peut s'écarter sensiblement de sa valeur théorique $y_0 = 0$

```
mean(rw$y)
```

```
## [1] 5.012416
```

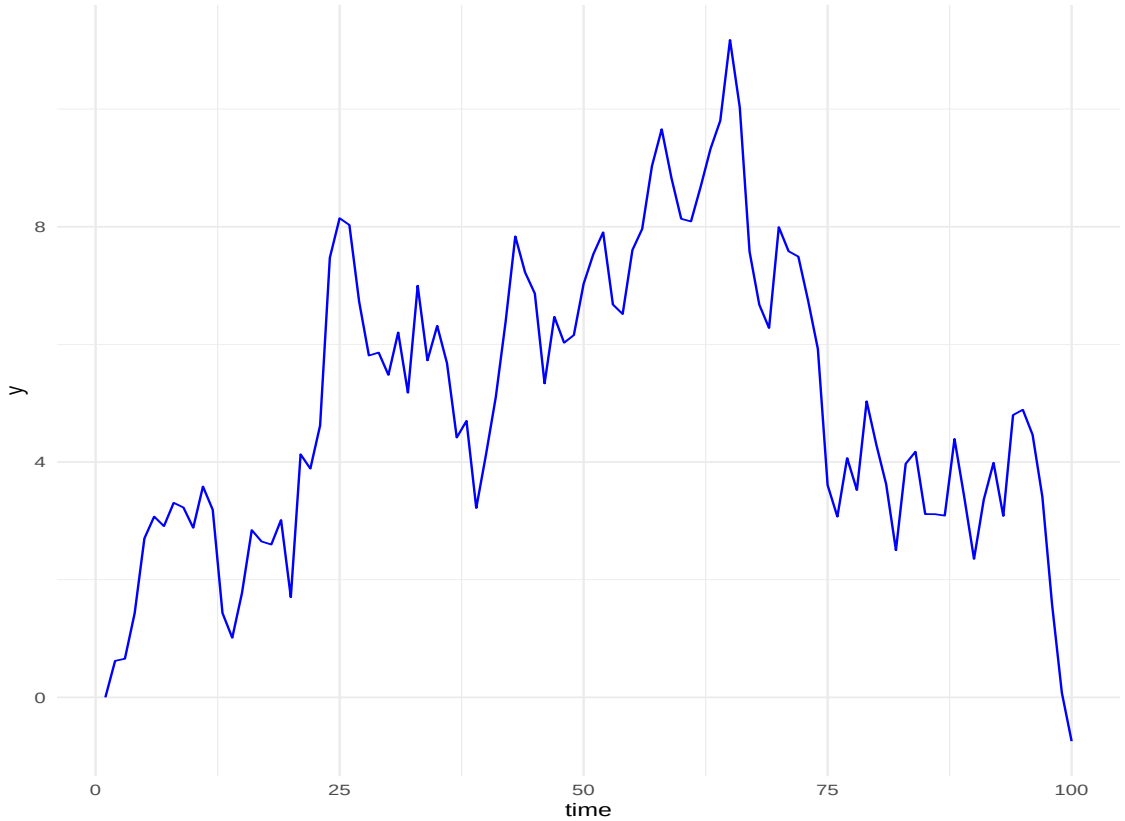
- L'autocorrélation est significative

```
acf(rw[,"y"], plot=FALSE)
```

```
##
## Autocorrelations of series 'rw[, "y"]', by lag
##
##     0     1     2     3     4     5     6     7     8     9    10
## 1.000 0.872 0.745 0.629 0.557 0.540 0.533 0.540 0.493 0.441 0.361
##    11    12    13    14    15    16    17    18    19    20
## 0.292 0.266 0.244 0.224 0.178 0.134 0.105 0.071 0.023 -0.033
```

Simulation (2)

```
ggplot(rw) +
  geom_line(aes(x=time, y=y), colour="blue")
```



4 Concepts théoriques

- Autocovariance et autocorrélation
- Bruit blanc
- Marche aléatoire
- Marche aléatoire avec dérive

Définition

- La série $y_t = \alpha + y_{t-1} + \epsilon_t$ est une **marche aléatoire** avec dérive (random walk with drift)
- En exprimant y_{t-1} en fonction de $y_{t-2}, \ldots$ et d'une valeur initiale y_0 on obtient :

$$y_t = \alpha \cdot t + y_0 + \epsilon_1 + \epsilon_2 + \ldots + \epsilon_t$$

- Espérance :

$$E(y_t) = \alpha \cdot t + y_0$$

- Variance :

$$var(y_t) = t \cdot \sigma_z^2$$

- Covariance :

$$cov(y_t, y_{t+k}) = t \cdot \sigma_z^2, (k > 0)$$

Simulation (1)

- On simule une marche aléatoire avec dérive ($alpha = 0.8$), on réutilise le bruit blanc gaussien `wnoise` généré pour l'exemple précédent

```
tmax <- 100
alpha <- 0.8

y <- rep(0,tmax)

for (t in 2:tmax) {
  y[t] = alpha + y[t-1] + wnoise[t-1]
}
rwd <- tibble(time=1:tmax, y=y)
head(rwd)
```

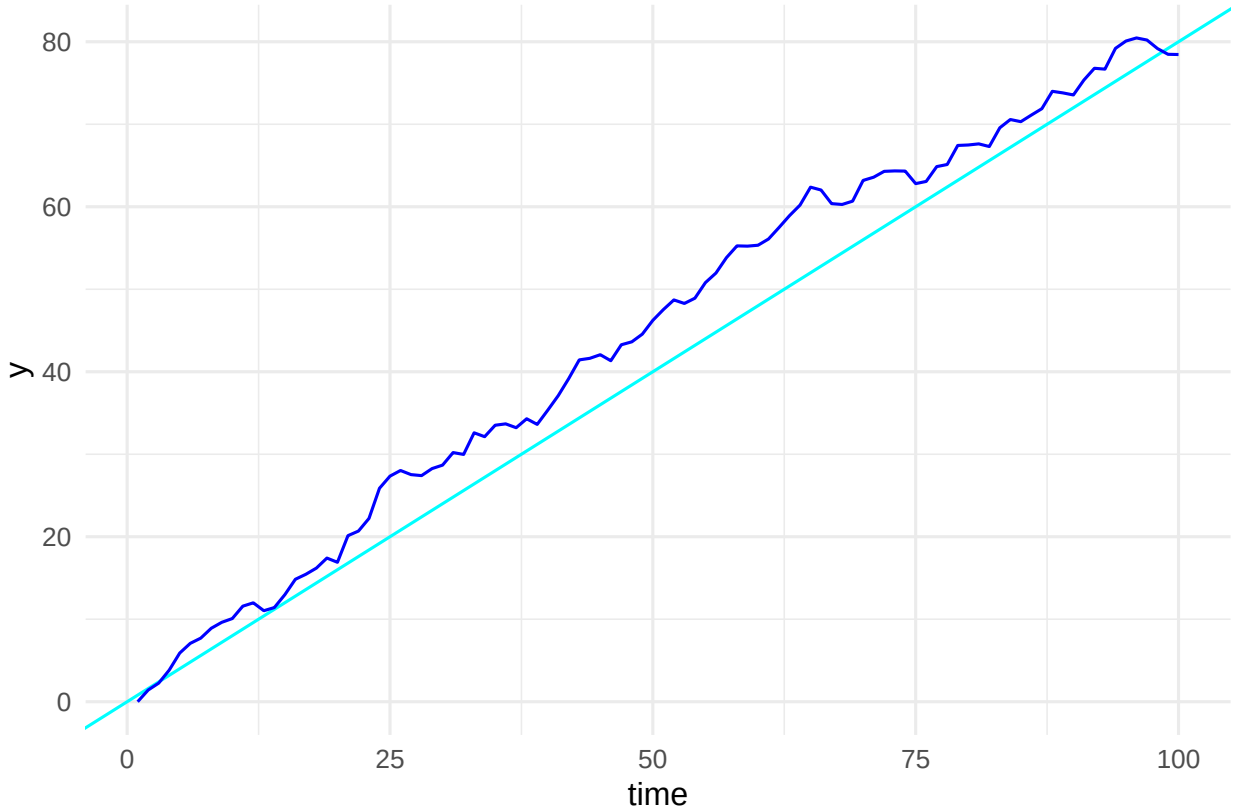
```
## # A tibble: 6 x 2
##    time     y
##   <int> <dbl>
## 1     1  0
## 2     2  1.42
## 3     3  2.26
## 4     4  3.83
## 5     5  5.90
## 6     6  7.07
```

Marche aléatoire avec dérive

Simulation (2)

- Le graphique de y_t en fonction du temps est donc celui d'une droite à laquelle est superposée une marche aléatoire

```
ggplot(rwd) +
  geom_abline(slope=0.8, color="cyan") +
  geom_line(aes(x=time, y=y), colour="blue")
```



Modèles théoriques

Exercices

1. Simulez une série temporelle $y_t, t = 1, \ldots, 1000$ représentant un bruit blanc gaussien suivant une loi normale $N(0, 3)$ de moyenne 0 et d'écart type 3
   - Calculez la moyenne et l'écart-type pour les observations $y_0, \ldots, y_{99}$, $y_{100}, \ldots, y_{199}$, etc ... et représentez les graphiquement
   - Calculez les coefficients d'autocorrélation de la série
2. Construisez une série $y_t, t = 1, \ldots, 1000$ représentant une marche aléatoire avec $y_0 = 1.39$ et $\sigma_\epsilon^2 = 0.41$
   - Calculez la moyenne et l'écart-type pour les observations $y_0, \ldots, y_{99}$, $y_{100}, \ldots, y_{199}$, etc ... et représentez les graphiquement
   - Calculez les coefficients d'autocorrélation de la série
   - Réalisez un test de Portmanteau sur la série, que concluez vous ?

5 Analyse descriptive et représentations graphiques

Introduction

- Les différentes représentations graphiques d'une série temporelle permettent de détecter une *saisonnalité*, une *tendance*, l'existence de *cycles* (voir par exemple)
  - Tendance : croissance ou décroissance sur le long terme (pas forcément linéaire)
  - Saisonnalité : la série est affectée par des facteurs saisonniers tels que le mois de l'année ou le jour de la semaine (toujours avec une fréquence fixe et connue)
  - Cycles : la série présente des épisodes de croissance ou décroissance, sans fréquence fixe

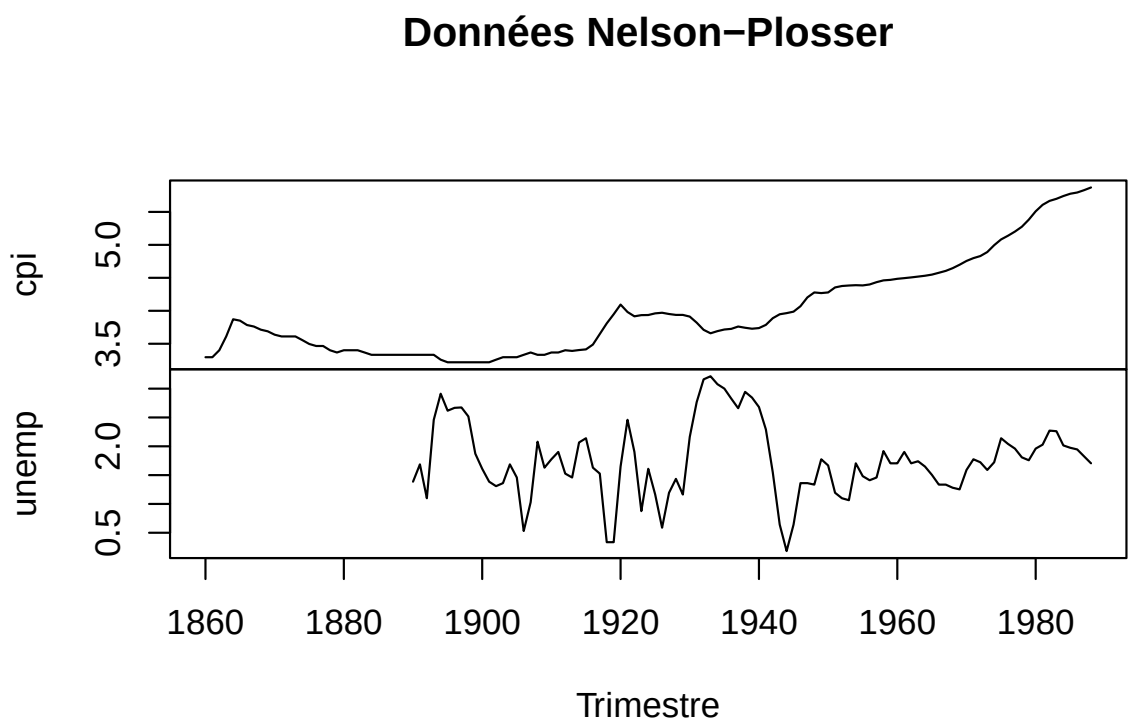
5 Analyse descriptive et représentations graphiques

- Chronogramme
- Seasonal plots
- Lag plot
- Fonction d'autocorrélation

Définition

- Chronogramme : diagramme des points (x=date, y=valeur de l'observation)
- Pour un objet de la classe `ts` on peut utiliser la méthode générique `plot()`

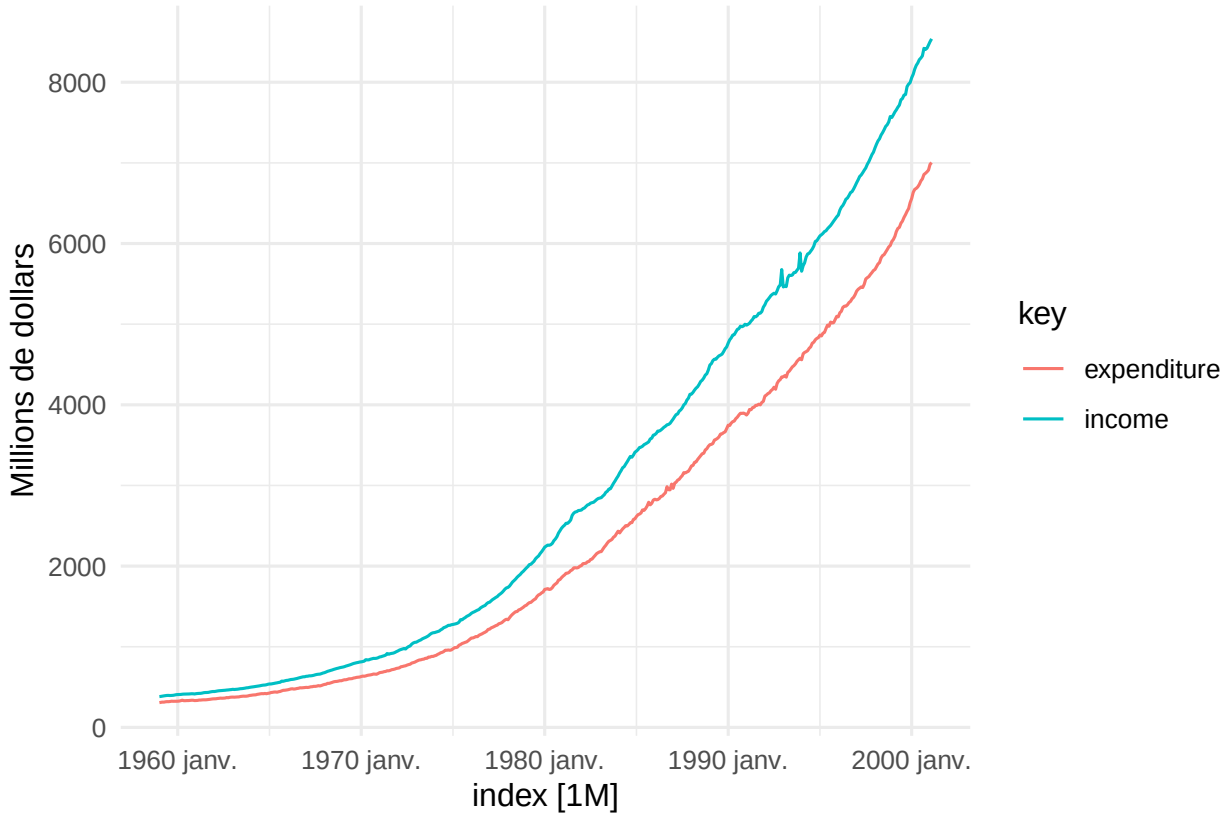
```
plot(NelPlo[,c("cpi", "unemp")], xlab="Trimestre", main="Données Nelson-Plosser")
```



Données USIncExp

- Pour un objet de la classe `tsibble`, on peut utiliser la fonction `autoplot()`

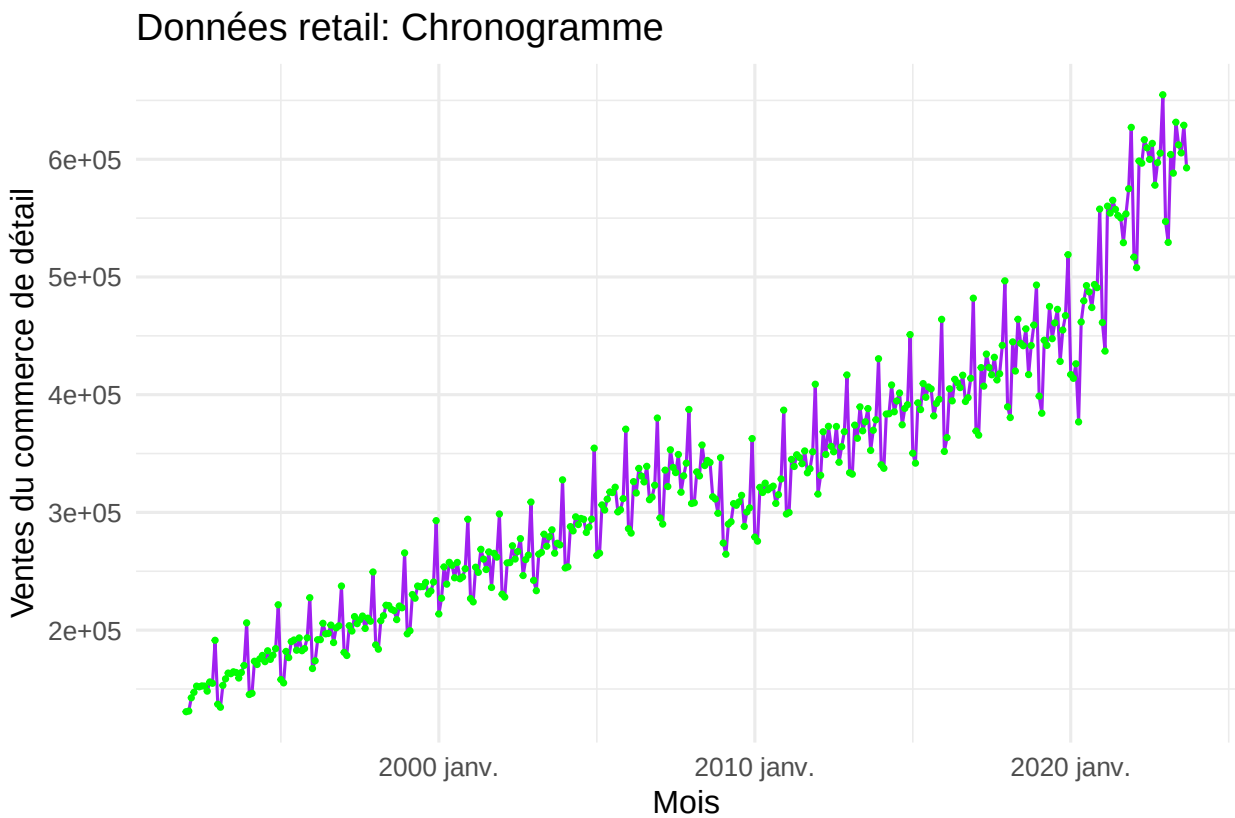
```
USIncExp %>% as_tsibble() %>% autoplot() + ylab("Millions de dollars")
```



Données retail

- Pour un objet de la classe `tsibble`, on peut également utiliser `ggplot` pour plus de contrôle

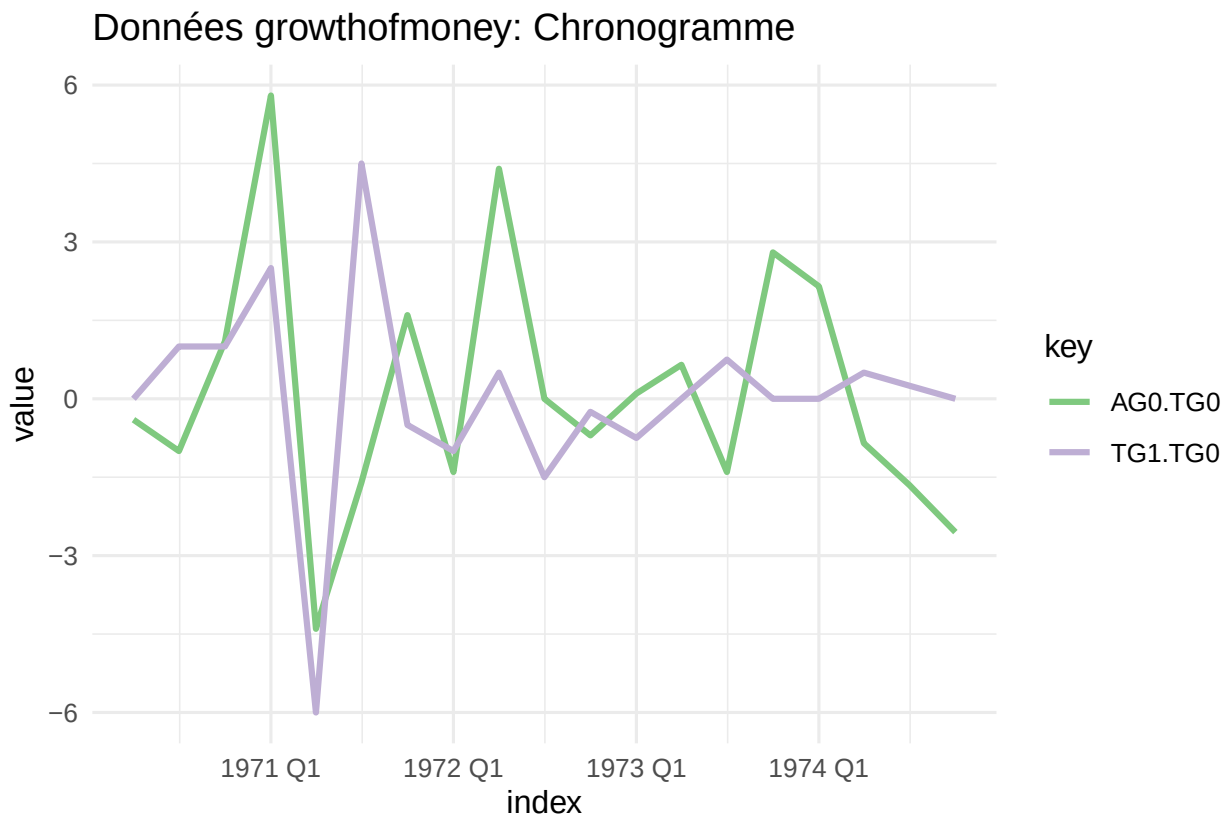
```
ggplot(retail_tsbl, aes(x = index, y = value)) +
    geom_line(color = "purple", size = 0.5) +
    geom_point(color = "green", size = 0.5) +
    xlab("Mois") + ylab("Ventes du commerce de détail") +
    ggtitle("Données retail: Chronogramme")
```



Données growthofmoney

Spécification de la palette avec la fonction scale_colour_brewer

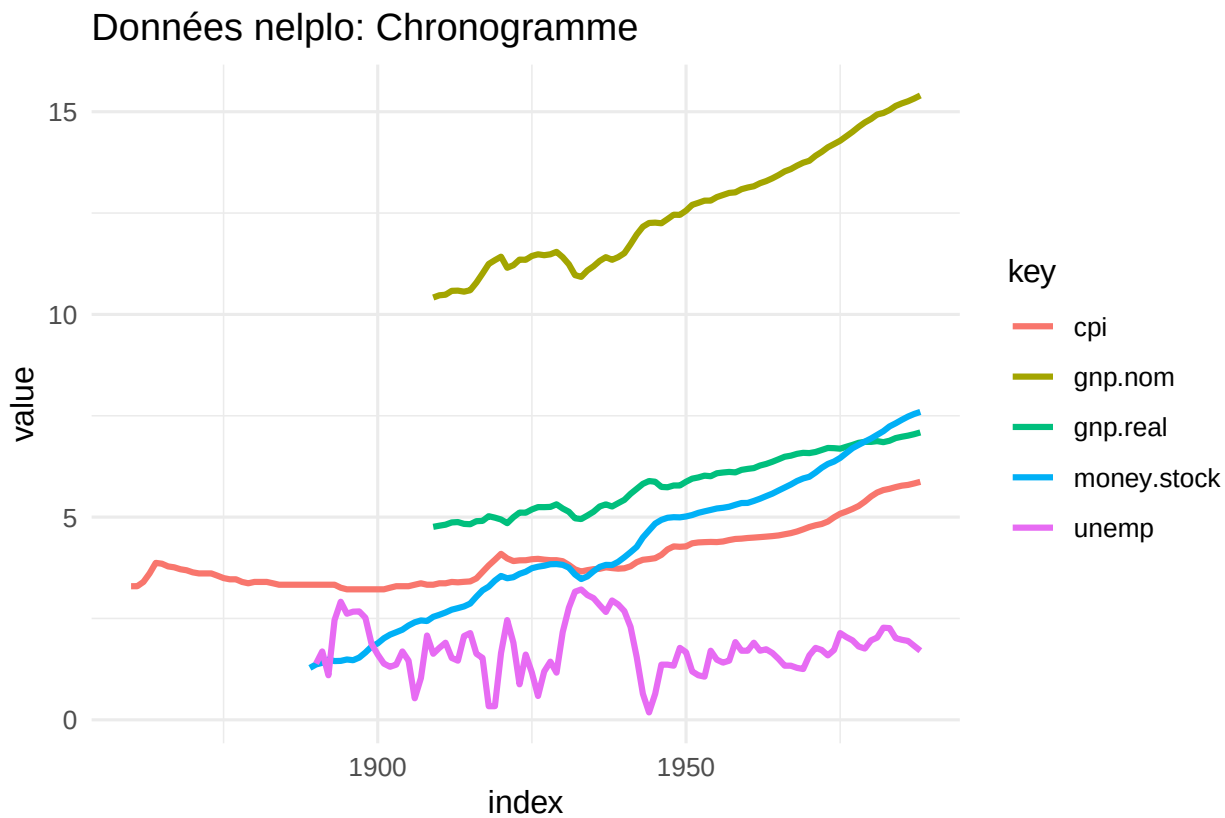
```
ggplot(gmoney_tsbl, aes(x = index, y = value)) +
  geom_line(aes(color = key), size = 1) +
  scale_colour_brewer(palette="Accent") +
  ggtitle("Données growthofmoney: Chronogramme")
```



Données nelplo (Nelson-Plosser)

Sélection des séries avec la fonction filter

```
nelplo_tsbl %>% filter(key %in% c("gnp.nom", "gnp.real", "unemp", "cpi", "money.stock")) %>%
  ggplot(aes(x = index, y = value)) +
  geom_line(aes(color = key), size = 1) +
  ggtitle("Données nelplo: Chronogramme")
```



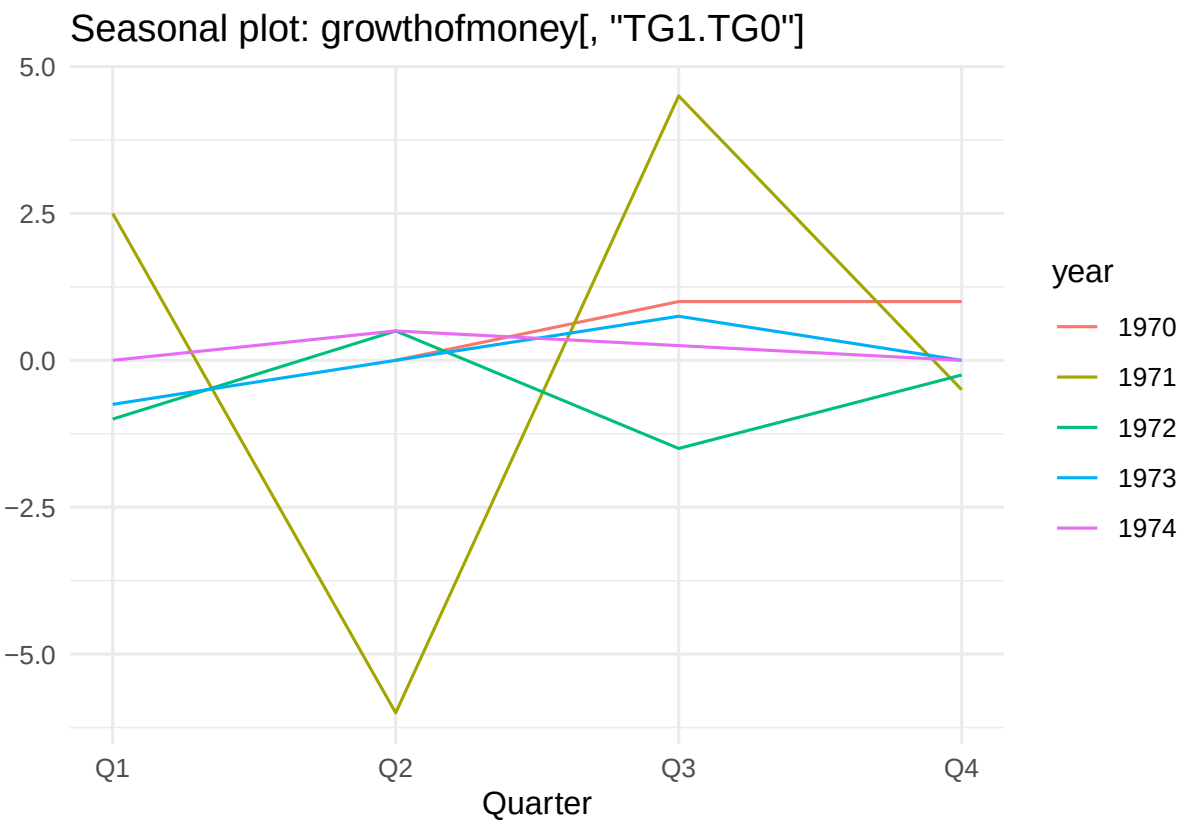
5 Analyse descriptive et représentations graphiques

- Chronogramme
- Seasonal plots
- Lag plot
- Fonction d'autocorrélation

Données growthofmoney

- Similaire à un chronogramme, mais l'axe temporel représente les 'saisons' (ici les trimestres)
- On obtient un *seasonal plot* avec la fonction `ggseasonplot` de la librairie `forecast`

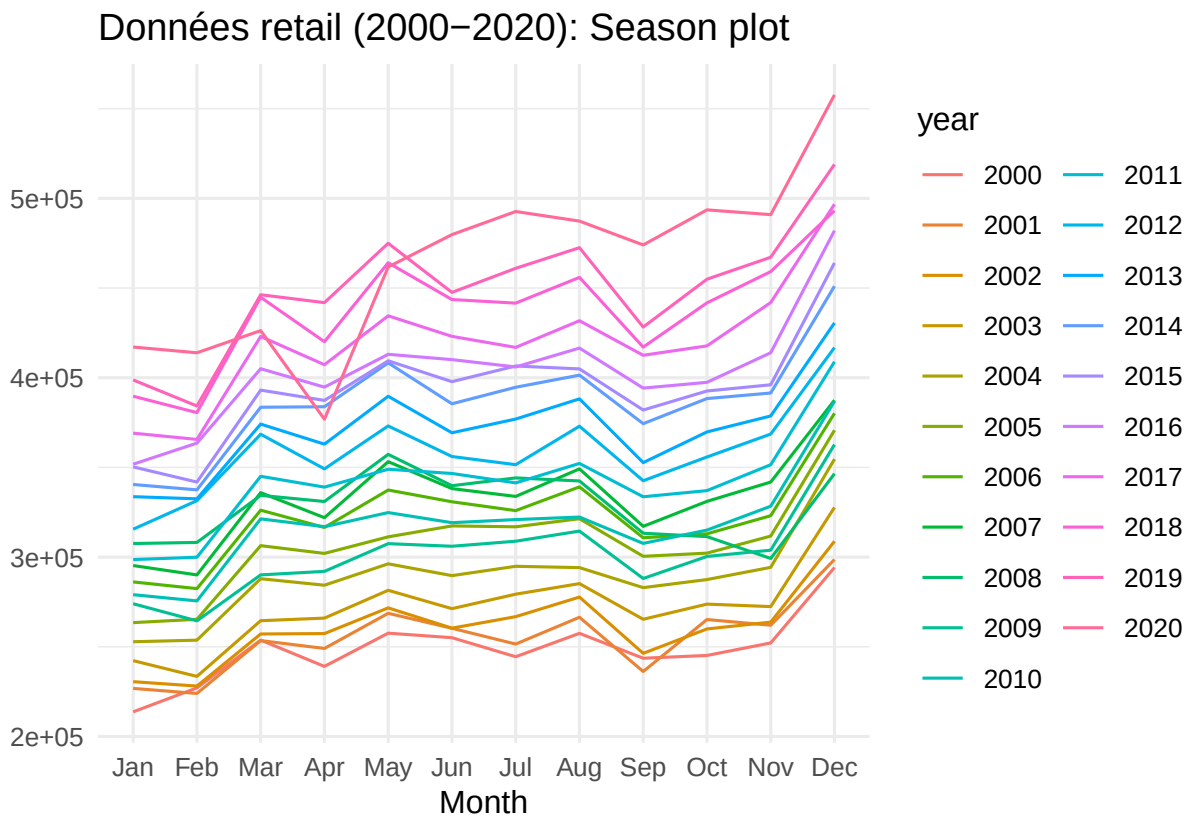
```
library(forecast)
ggseasonplot(growthofmoney[,"TG1.TG0"])
```



Données retail (1)

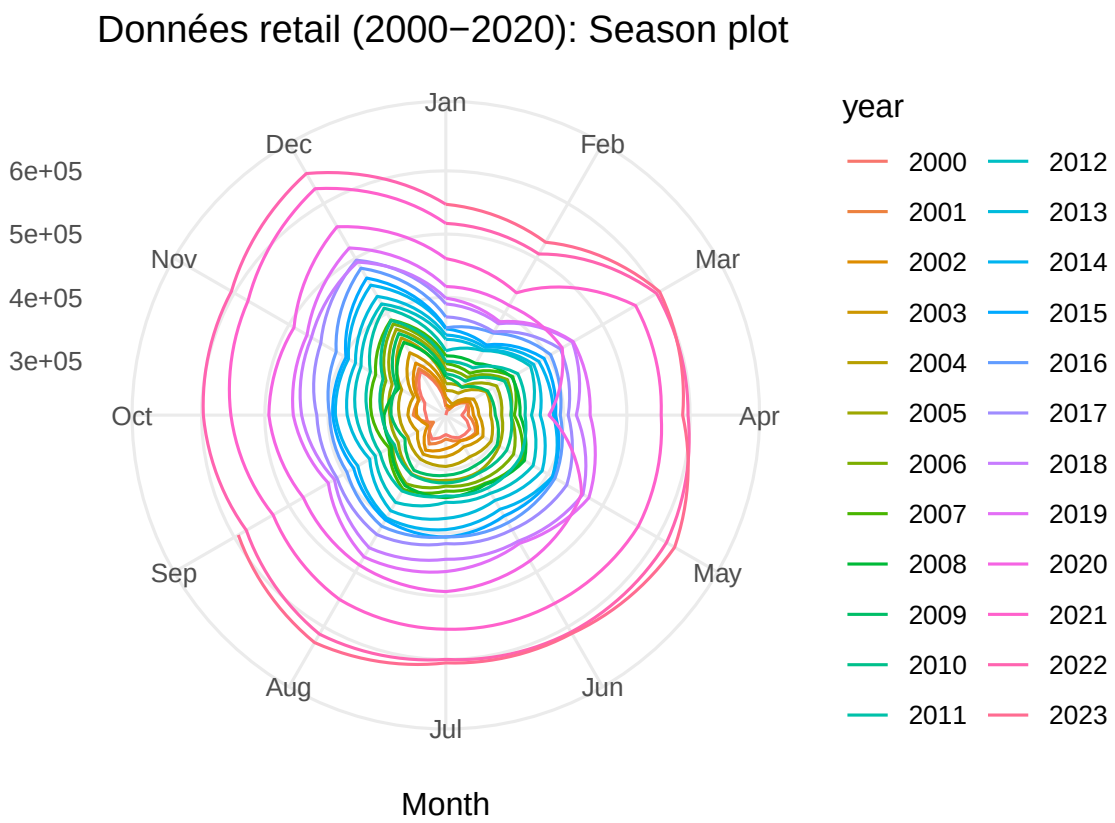
- Ici les 'saisons' sont les mois de l'année

```
ggseasonplot(window(retail_ts, start=c(2000,1), end=c(2020,12))) +
  ggtitle("Données retail (2000-2020): Season plot")
```



Utilisation de l'option `polar=TRUE`

```
ggseasonplot(window(retail_ts, start=c(2000,1)), polar=TRUE) +
  ggtitle("Données retail (2000-2020): Season plot")
```



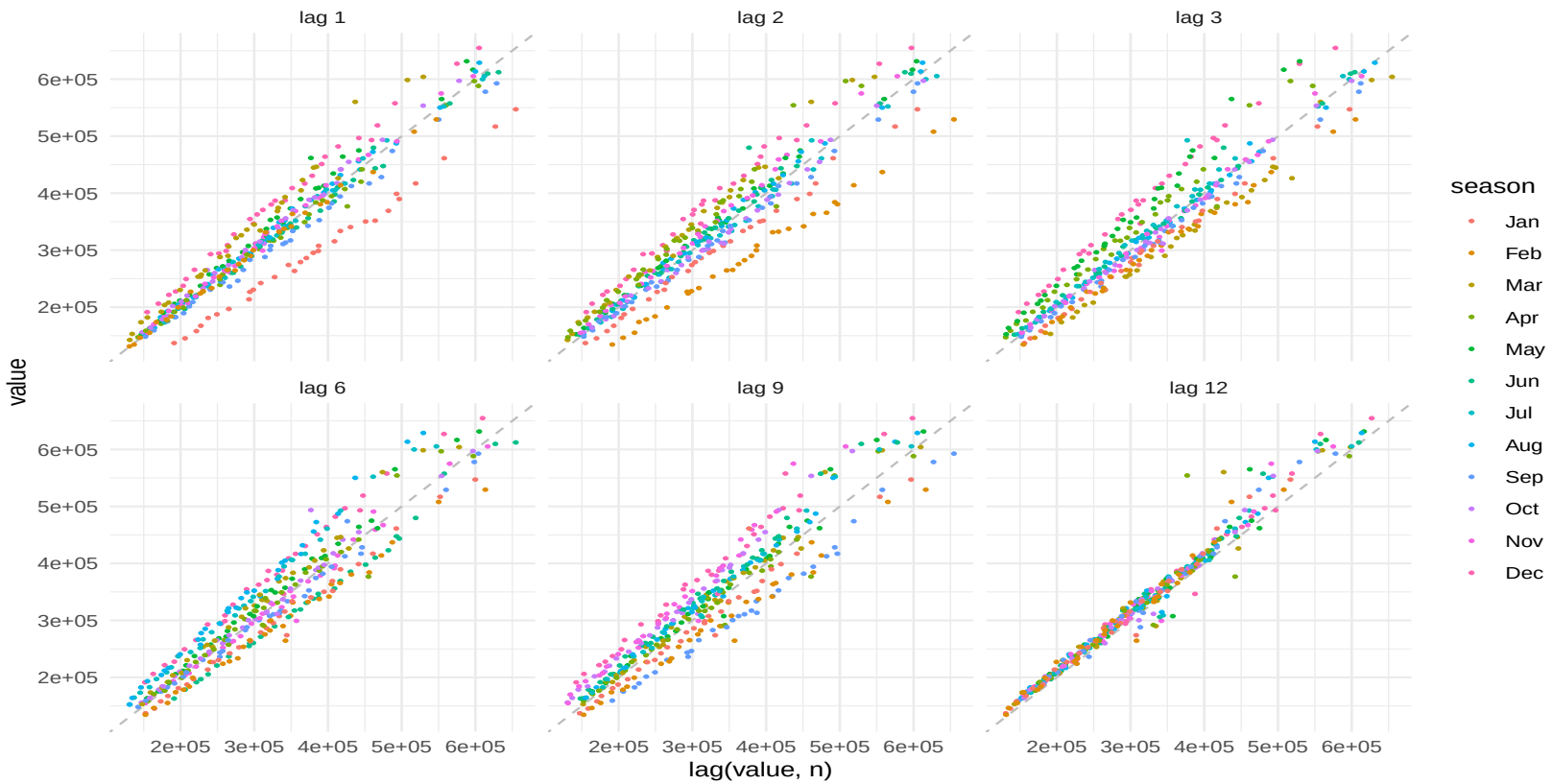
5 Analyse descriptive et représentations graphiques

- Chronogramme
- Seasonal plots
- Lag plot
- Fonction d'autocorrélation

Données `retail`

- Le **lag plot** représente la relation entre une série temporelle et ses valeurs précédentes
- Sur les données `retail`, on voit que la corrélation est forte entre y_t et y_{t-12}

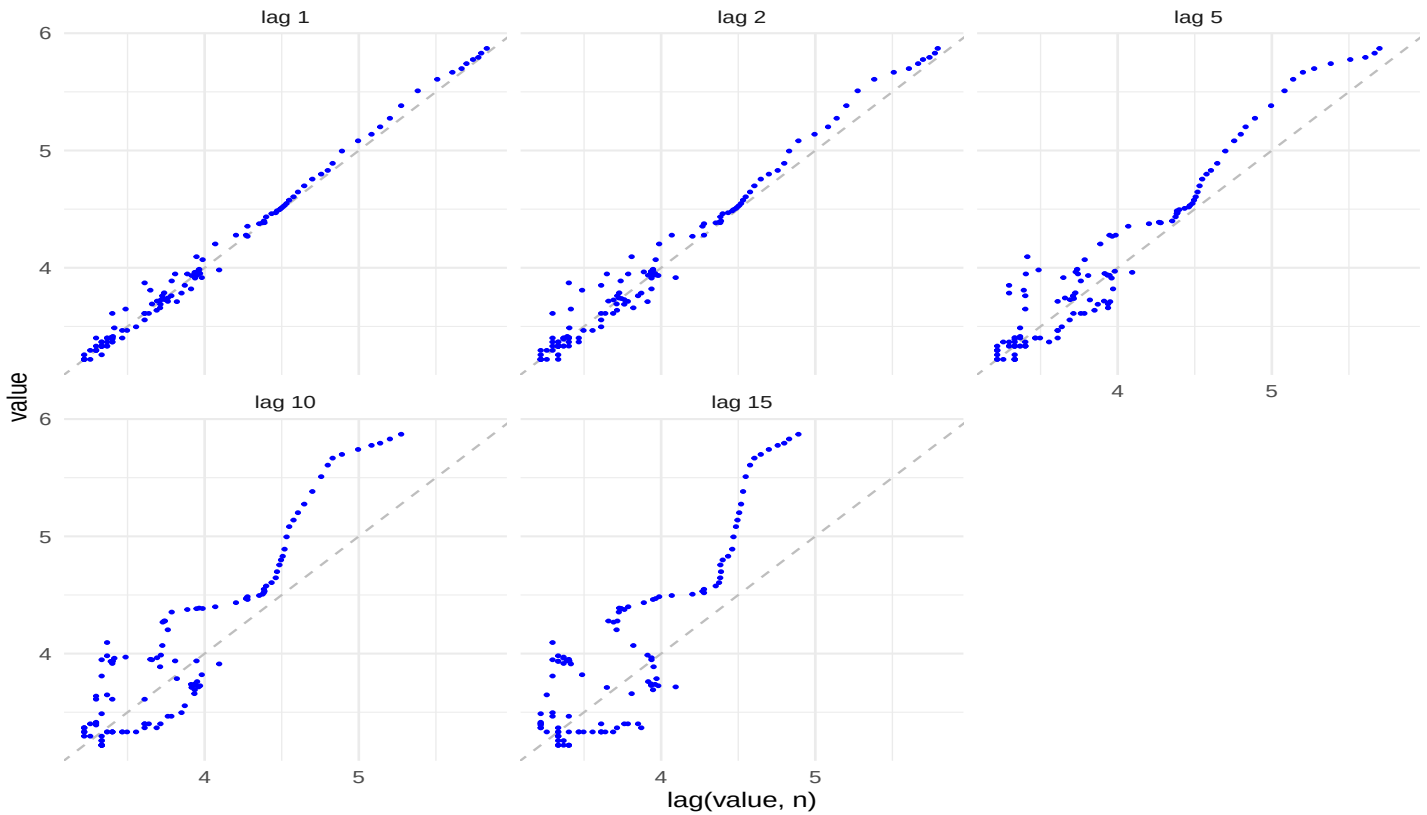
```
library(ggtime)
retail_tsbl %>% ggtime::gg_lag(y=value, geom="point", size=0.5, lags=c(1,2, 3,6,9,12))
```



Données Nelson-Plosser (série cpi)

- Sur la série cpi (données nelplo), on voit que la corrélation est forte entre y_t et y_{t-1},y_{t-2}

```
nelplo_tsbl %>% filter(key=='cpi') %>%
  ggtime::gg_lag(y=value, geom="point", size=0.5, colour="blue", lags=c(1,2,5,10,15))
```



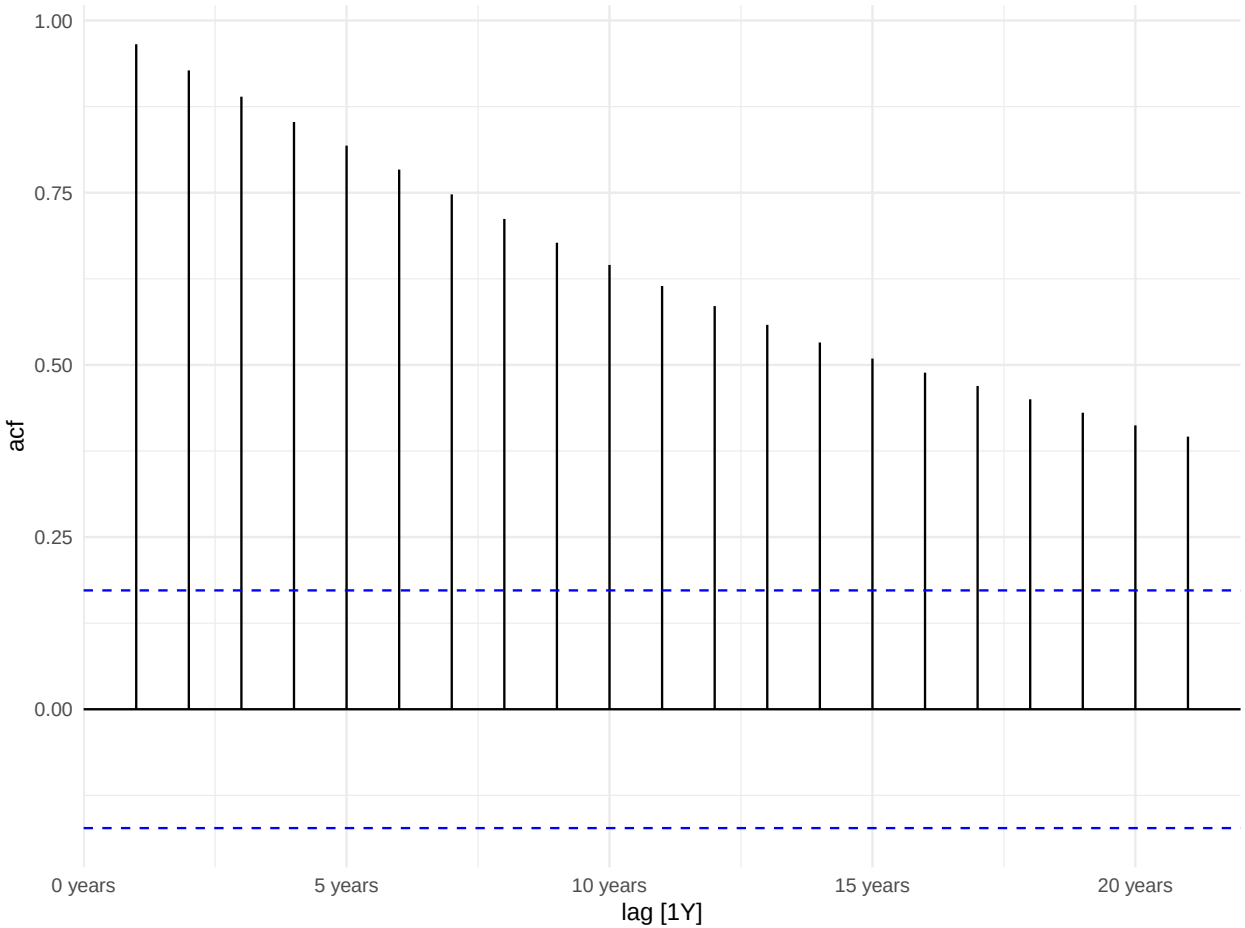
5 Analyse descriptive et représentations graphiques

- Chronogramme
- Seasonal plots
- Lag plot
- Fonction d'autocorrélation

Données Nelson-Plosser

- La fonction `autoplot()` permet de représenter la fonction d'autocorrélation obtenue avec `ACF()`, ici pour la série `cpi` (données Nelson-Plosser)

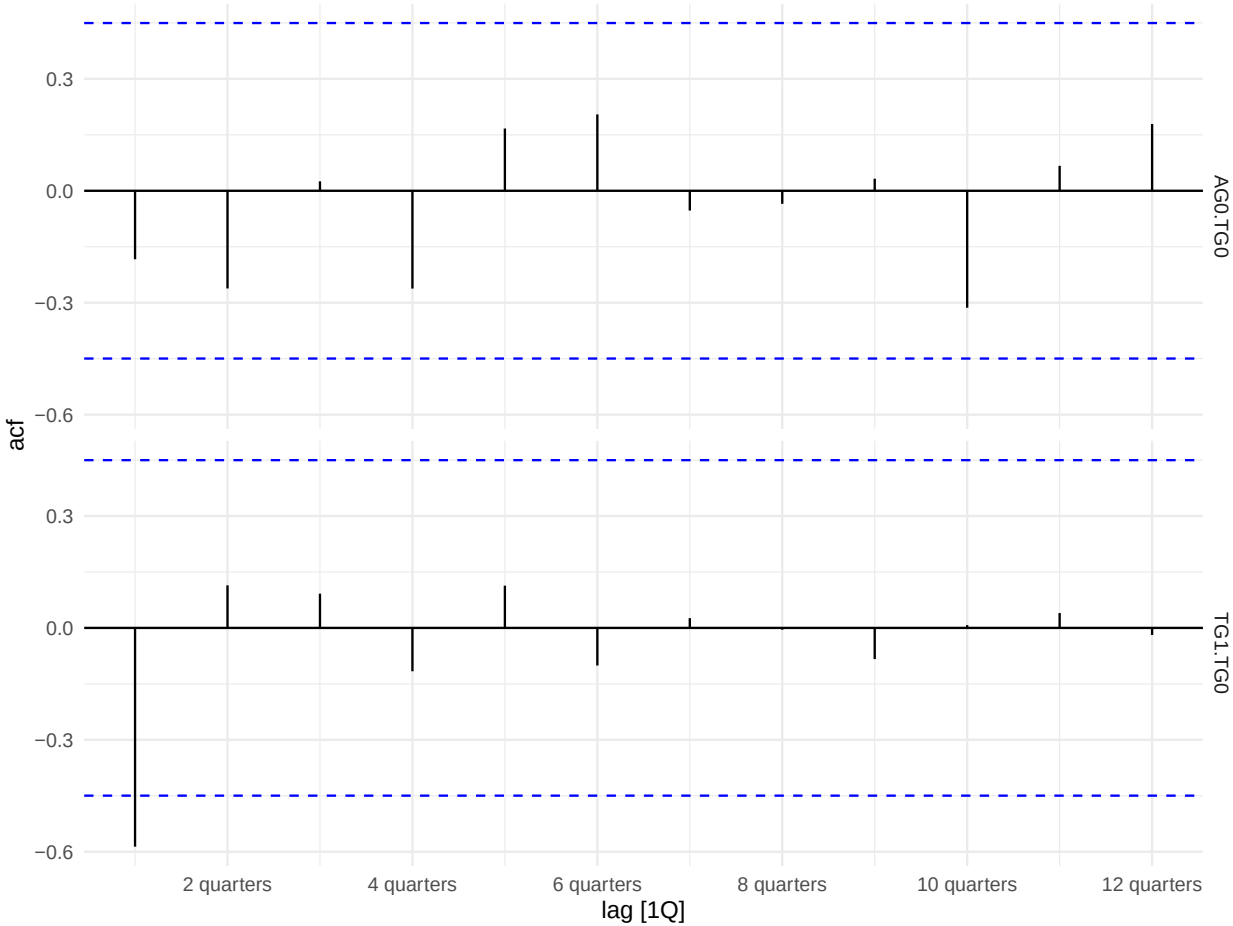
```
nelplo_tsbl %>% filter(key=="cpi") %>%
  ACF(value) %>% autoplot()
```



Données growthofmoney

Autocorrélation des deux séries des données growthofmoney

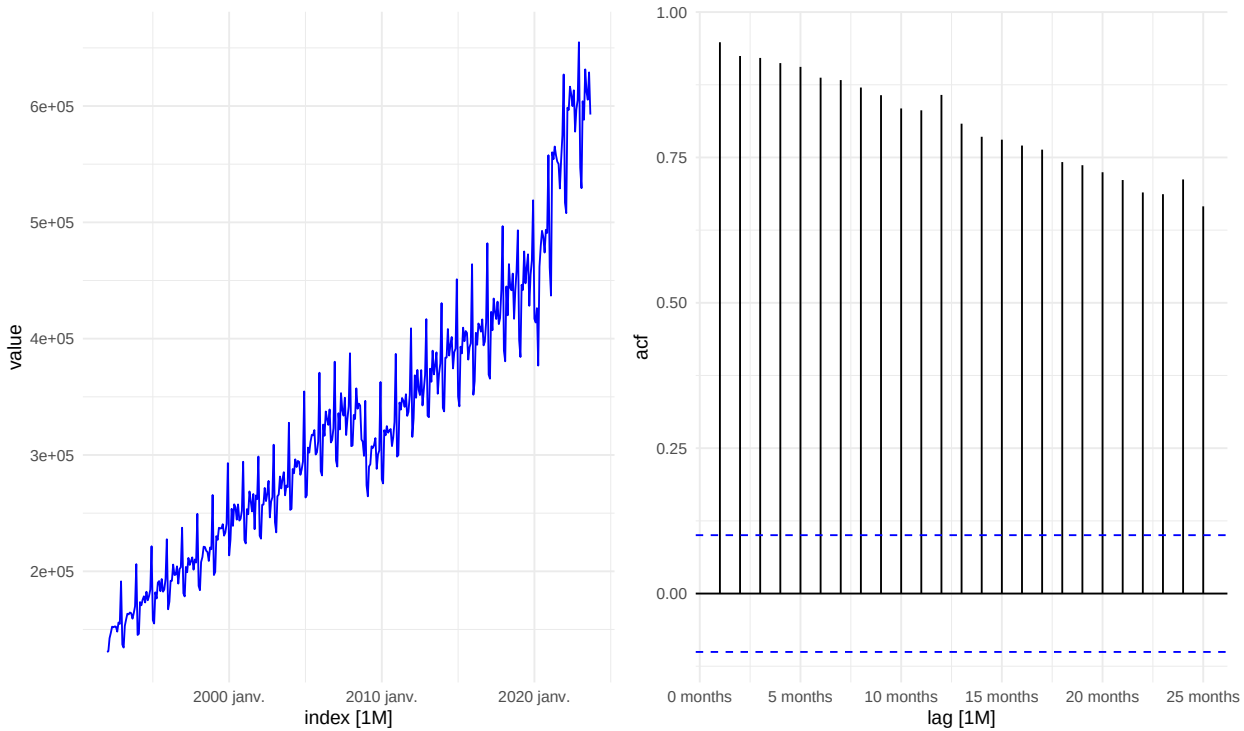
```
gmoney_tsbl %>% ACF() %>% autoplot()
```



Données `retail`

- Ici on utilise la librairie `patchwork` pour combiner deux graphiques

```
library(patchwork)
g1 <- retail_tsbl %>% autoplot(color='blue')
g2 <- ACF(retail_tsbl) %>% autoplot()
g1+g2
```



Exercice

- Représenter et décrire les séries des données PIB (chronogramme, fonction d'autocorrélation, lag plot)
- Représenter et décrire la série du taux de chômage (chronogramme, fonction d'autocorrélation, lag plot)

6 Régression Linéaire

6 Régression Linéaire
- Régression Linéaire simple
- Modèle de régression multivarié
- Test de changement structurel

Le modèle de régression linéaire simple

- Modèle de régression linéaire simple (une seule variable indépendante) :

$$y_i = \beta_1 + \beta_2 \cdot x_i + \epsilon_i$$

  - Les observations sont indicées par i, $(i = 1, \ldots, N)$
  - y_i est la variable **expliquée** (dépendante)
  - x_i est la variable **explicative** (indépendante)
  - β_1 et β_2 sont les **paramètres** (à estimer)
  - ϵ_i est le **résidu** (écart aléatoire, erreur)
- L'équation de la droite de régression est déterminée par la pente (slope) (β_0) et l'intercept (β_1) :

$$E(y|x) = \beta_1 + \beta_2 \cdot x$$

Hypothèses de base du modèle linéaire

- On parle de Moindres Carrés Ordinaires (MCO) ou Ordinary Least Square model (OLS) car l'objectif lors de l'estimation est de minimiser la somme des erreurs au carré $\sum_i \epsilon_i^2$
- Les hypothèses de base concernent la distribution de probabilité des résidus ϵ_i :
  - Hypothèse 1 : ϵ_i suit une distribution normale $N(\mu, \sigma^2)$
  - Hypothèse 2 : l'espérance de ϵ_i est nulle : $\forall i, E(\epsilon_i) = 0$
  - Hypothèse 3 : la variance de ϵ_i est constante (homoscédasticité) : $\forall i, V(\epsilon_i) = \sigma^2$
  - Hypothèse 4 : la covariance entre deux observations est nulle : $\forall i \neq j, Cov(\epsilon_i, \epsilon_j) = 0$, il n'y a pas d'autocorrélation des résidus, ils sont sériellement indépendants

Données Nelson-Plosser

- On considère les données Nelson-Plosser à partir de l'année 1909 (toutes les séries complètes)
- On utilise la fonction `spread` pour passer du format "long" au format "large"

```
nelplo1909 <- nelplo_tsbl %>%
  filter(index>=1909) %>%
  spread(key = key, value=value)
head(nelplo1909)
```

```
## # A tsibble: 6 x 15 [1Y]
##   index   cpi   emp gnp.capita gnp.def gnp.nom gnp.real int.rate    ip
##   <dbl> <dbl> <dbl>      <dbl>   <dbl>   <dbl>    <dbl>    <dbl> <dbl>
## 1  1909  3.33  10.5       7.16    3.37    10.4     4.76     3.77  2.30
## 2  1910  3.37  10.5       7.17    3.40    10.5     4.79     3.8   2.36
## 3  1911  3.37  10.5       7.18    3.39    10.5     4.81     3.9   2.32
## 4  1912  3.40  10.5       7.22    3.43    10.6     4.87     3.9   2.46
## 5  1913  3.39  10.6       7.21    3.44    10.6     4.88     4     2.53
## 6  1914  3.40  10.5       7.14    3.45    10.6     4.83     4.1   2.46
## # i 6 more variables: money.stock <dbl>, nom.wages <dbl>, real.wages <dbl>,
## #   stock.prices <dbl>, unemp <dbl>, vel <dbl>
```

Données Nelson-Plosser

Modèle

- Régression linéaire simple sur les données Nelson-Plosser :
  - Variable expliquée y = gnp.nom (Log(PNB), en millions de dollars)
  - Variable explicative x = index (année)

```
nelplo1909 %>% select(index, gnp.nom)
```

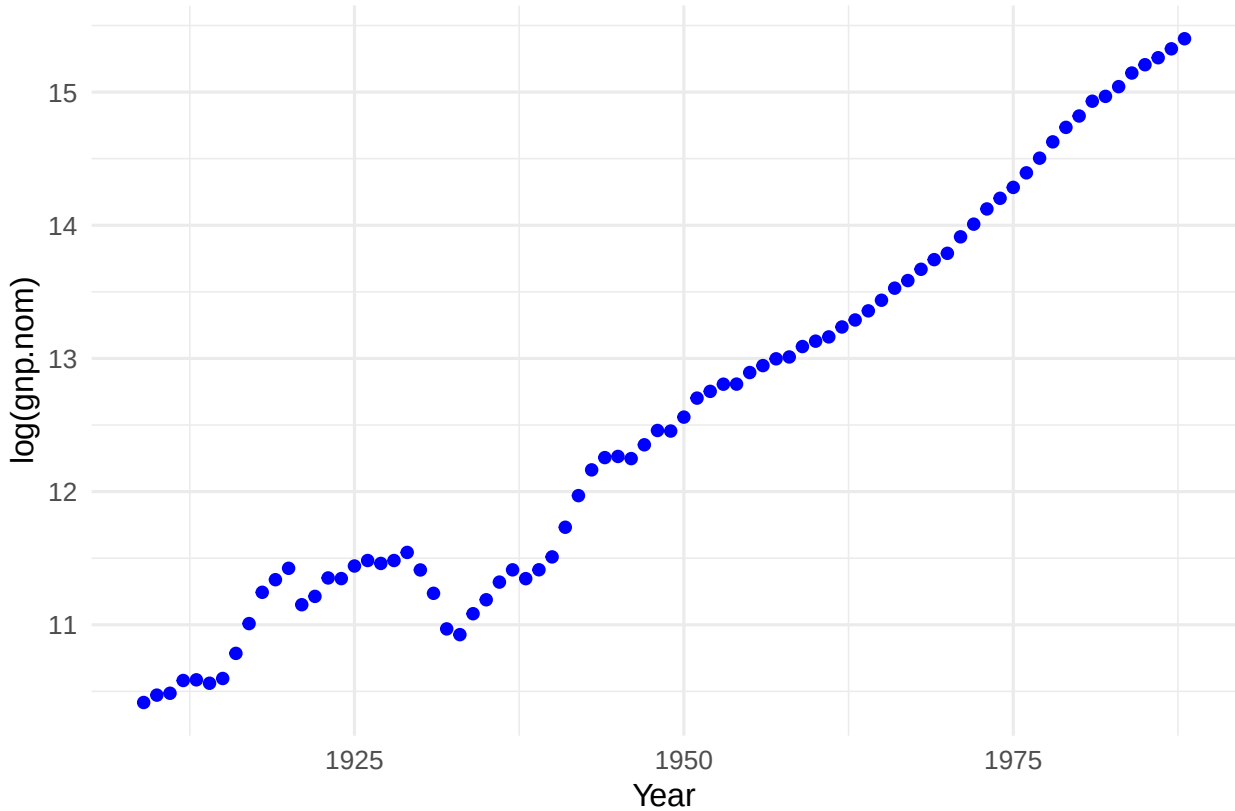
```
## # A tsibble: 80 x 2 [1Y]
##    index gnp.nom
##    <dbl>   <dbl>
##  1  1909    10.4
##  2  1910    10.5
##  3  1911    10.5
##  4  1912    10.6
##  5  1913    10.6
##  6  1914    10.6
##  7  1915    10.6
##  8  1916    10.8
##  9  1917    11.0
## 10  1918    11.2
## # i 70 more rows
```

Données Nelson-Plosser

Visualisation

- Nous n'avons qu'une seule variable indépendante, on peut visualiser simplement la relation entre y et x

```
nelplo1909 %>%
  ggplot(aes(index, gnp.nom)) +
    geom_point(colour="blue") +
  xlab("Year") + ylab('log(gnp.nom)')
```



Données Nelson-Plosser

Estimation

- L'estimation des paramètres se fait avec la fonction `lm()` (Linear Models)
- Le premier argument `gnp.nom ~ index` représente l'équation de la régression

```
mod1 <- lm(gnp.nom ~ index, data=nelplo1909)
```

- On obtient un résumé du modèle et de ses résultats avec la fonction `summary()`

```
summary(mod1)
```

```
##
## Call:
## lm(formula = gnp.nom ~ index, data = nelplo1909)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.70453 -0.16422 -0.04624  0.26225  0.59689
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept) -1.078e+02  2.840e+00  -37.96   <2e-16 ***
## index        6.179e-02  1.458e-03   42.39   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.301 on 78 degrees of freedom
## Multiple R-squared:  0.9584, Adjusted R-squared:  0.9579
## F-statistic:  1797 on 1 and 78 DF,  p-value: < 2.2e-16
```

Données Nelson-Plosser

Résultats

- On peut accéder aux différents éléments du résumé, dont les noms sont renvoyés par la fonction `names()`

```
names(summary(mod1))
```

```
##  [1] "call"          "terms"         "residuals"     "coefficients"
##  [5] "aliased"       "sigma"         "df"            "r.squared"
##  [9] "adj.r.squared" "fstatistic"    "cov.unscaled"
```

- Un test avec H0 : $\beta = 0$ est réalisé pour chacun des coefficients

```
summary(mod1)$coefficients
```

```
##                  Estimate  Std. Error   t value     Pr(>|t|)
## (Intercept) -107.81083108 2.840221275 -37.95860 4.766536e-52
## index          0.06179065 0.001457543  42.39371 1.276677e-55
```

- Le coefficient associé à la variable explicative `index` (année) est statistiquement significatif (p-valeur < 0.001)
- La valeur de R^2 (variance expliquée) est élevée

```
summary(mod1)$adj.r.squared
```

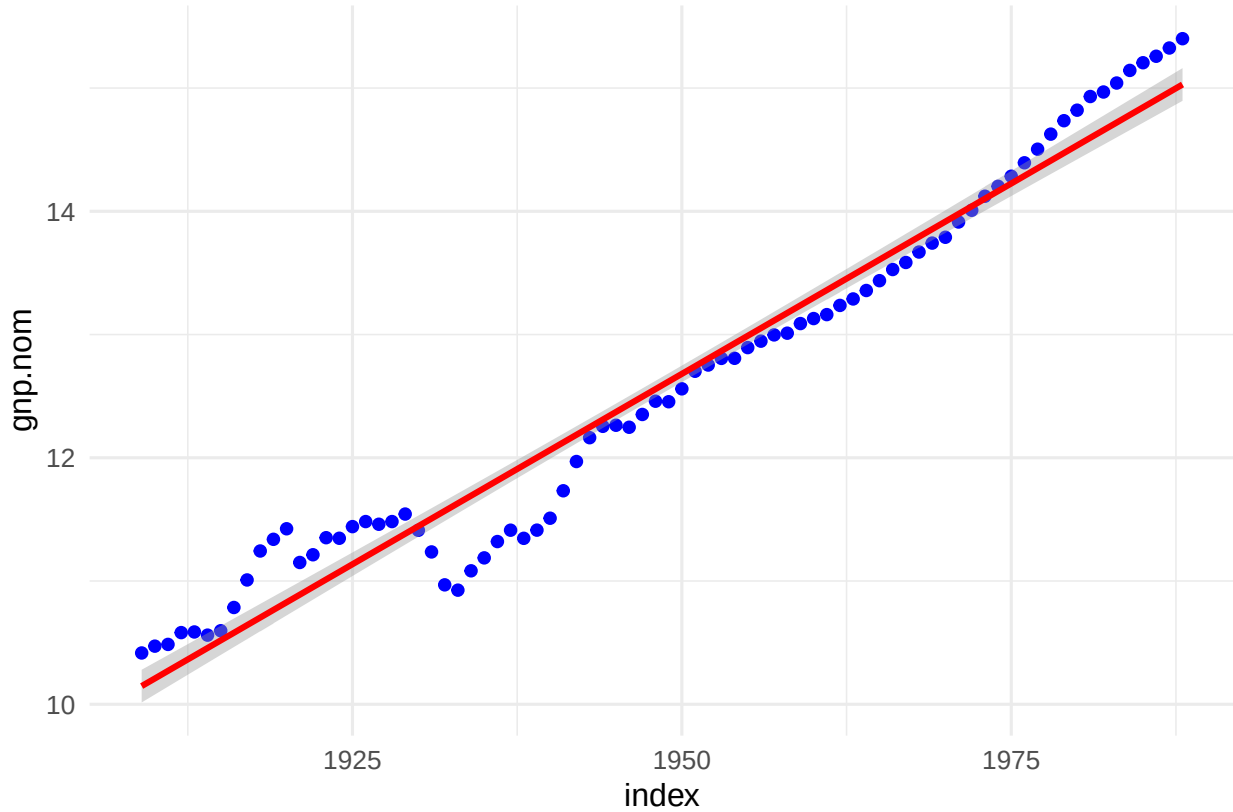
```
## [1] 0.9578718
```

Données Nelson-Plosser

Droite de régression

- La fonction `geom_smooth()` avec l'argument `method='lm'` permet d'ajouter la droite des MCO au graphique

```
ggplot(nelplo1909, aes(x=index, y=gnp.nom)) +
  geom_point(colour="blue") +
  geom_smooth(method='lm', color="red")
```



Données Nelson-Plosser

Valeurs observées, valeurs prédites, résidus

- Les valeurs prédites par le modèle se trouvent dans l'attribut `fitted.values`
- Les résidus représentent la différence valeur observée-valeur prédite, ils se trouvent dans l'attribut `residuals`

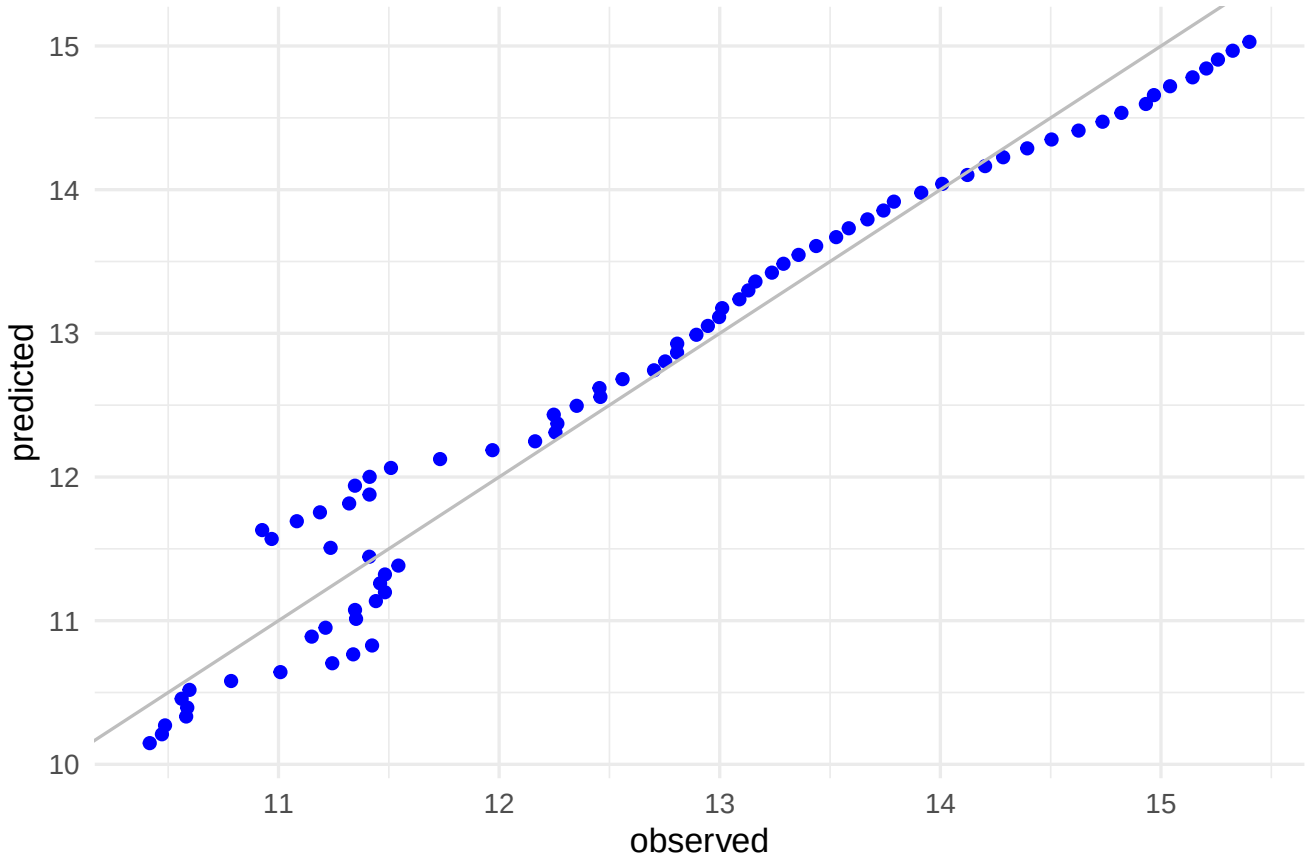
```
mod1_diag <- tibble(observed=nelplo1909$gnp.nom, predicted=mod1$fitted.values,
                    residual=mod1$residuals)
mod1_diag
```

```
## # A tibble: 80 x 3
##    observed predicted residual
##       <dbl>     <dbl>    <dbl>
##  1     10.4      10.1   0.269
##  2     10.5      10.2   0.262
##  3     10.5      10.3   0.215
##  4     10.6      10.3   0.249
##  5     10.6      10.4   0.192
##  6     10.6      10.5   0.105
##  7     10.6      10.5   0.0784
##  8     10.8      10.6   0.205
##  9     11.0      10.6   0.367
## 10     11.2      10.7   0.540
## # i 70 more rows
```

Données Nelson-Plosser

Valeurs prédites x observées

```
ggplot(mod1_diag) +
  geom_point(aes(x=observed, y=predicted), colour="blue") +
  geom_abline(colour="grey")
```

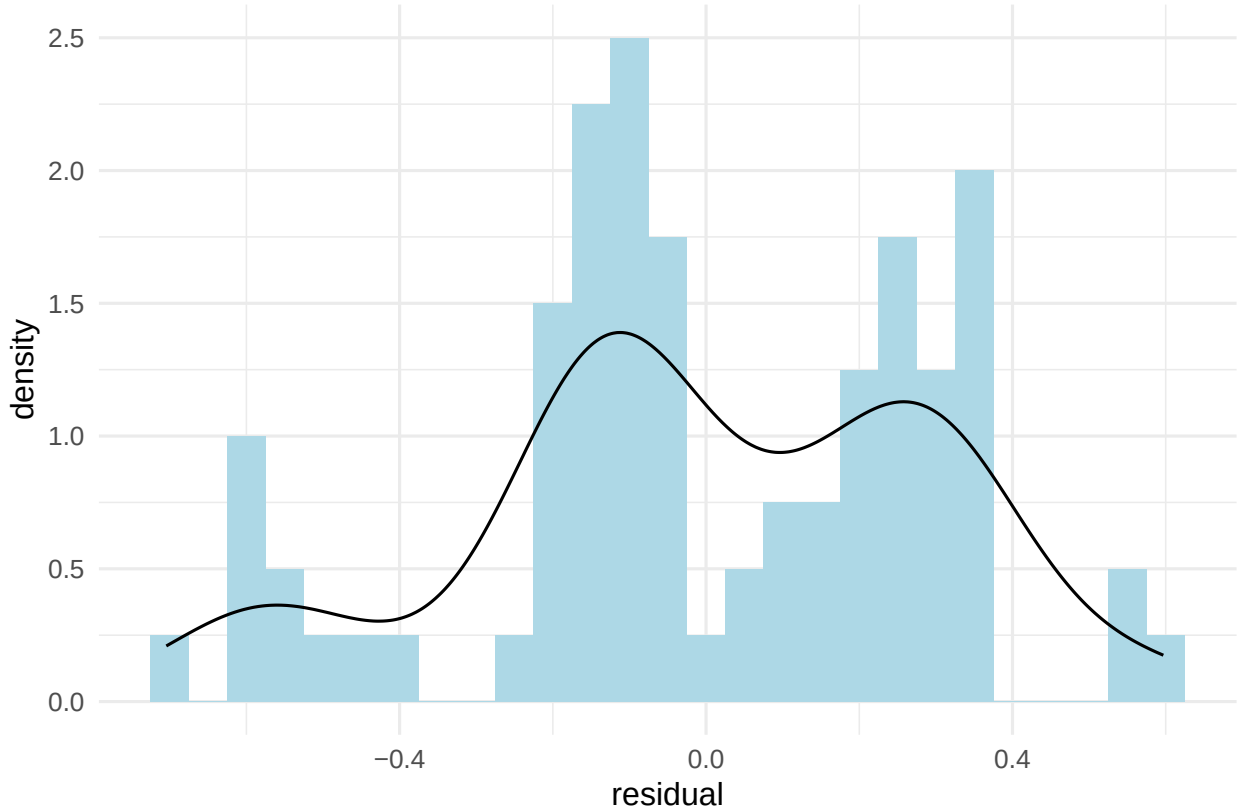


Données Nelson-Plosser

Distribution des résidus

- On représente la distribution des résidus sous la forme d'un histogramme et d'une courbe de densité

```
ggplot(mod1_diag, aes(x=residual)) +
  geom_histogram(aes(y = after_stat(density)), fill="lightblue", binwidth = 0.05) +
  geom_density(aes(x=residual))
```



Données Nelson-Plosser

Normalité des résidus

- Pour tester la normalité des résidus on utilise le test de **Shapiro-Wilk** (on peut également utiliser le test de **Jarque-Bera**)
- Hypothèse H0 : les résidus suivent une distribution normale
- La p-valeur est inférieure à 0.05, on rejette H0 : les résidus ne sont pas distribués normalement

```
shapiro.test(mod1_diag$residual)
```

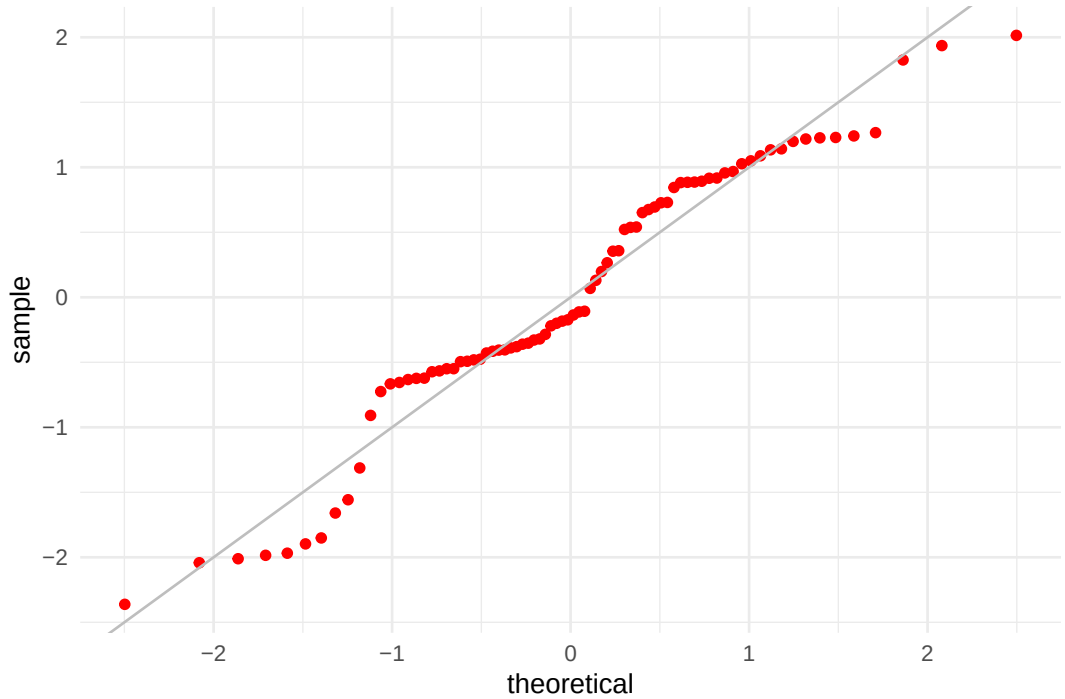
```
##
##  Shapiro-Wilk normality test
##
## data:  mod1_diag$residual
## W = 0.95726, p-value = 0.009177
```

Données Nelson-Plosser

Normalité des résidus : Quantile-Quantile plot

- Le quantile-quantile plot permet de comparer les quantiles des résidus à ceux d'une loi normale
- On utilise la fonction `geom_qq()` avec les résidus standardisés du modèle de régression `mod1`

```
ggplot() +
  geom_qq(aes(sample=rstandard(mod1)), color="red") +
  geom_abline(color = "grey")
```



Données Nelson-Plosser

Homoscédasticité des résidus

- L'homoscédasticité des résidus est une des hypothèses fondamentales du modèle OLS/MCO
- **Homoscédasticité** = la variance des résidus est constante (sur la plage des valeurs prédites par le modèle) :

$$Var(\epsilon_i) = \sigma^2$$

- **Hétéroscédasticité** = la variance des résidus n'est pas constante :

$$Var(\epsilon_i) = \sigma_i^2$$

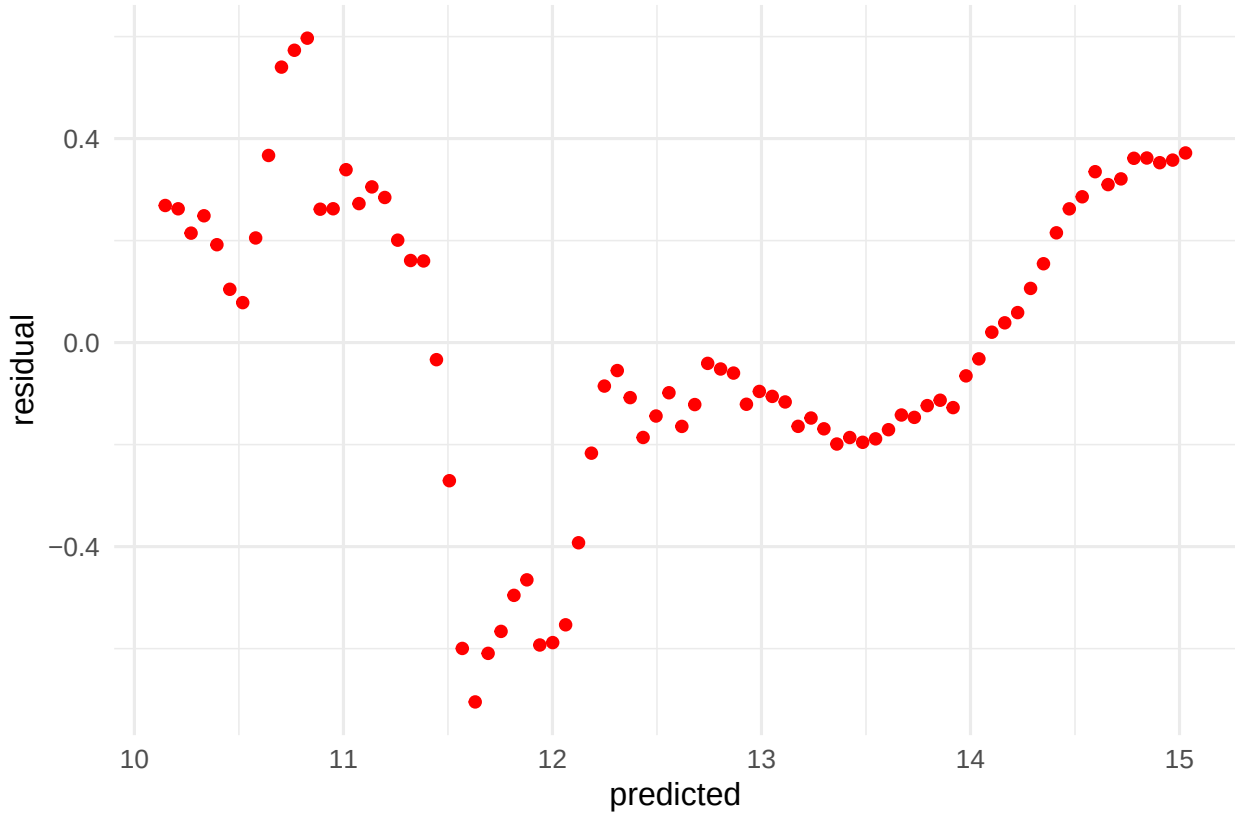
- Si l'hypothèse d'homoscédasticité des résidus est violée, les tests d'hypothèse sur les coéfficients β du modèle OLS ne sont plus valides

Données Nelson-Plosser

Résidus x valeurs prédites

- Pour visualiser l'heteroscédasticité éventuelle des résidus, on produit un graphique 'valeurs prédites par le modèle x résidus'

```
ggplot(mod1_diag) +
  geom_point(aes(x=predicted, y=residual), colour="red")
```



Données Nelson-Plosser

Homoscédasticité des résidus : Le test de **Breusch-Pagan-Godfrey**

- Le test de **Breusch-Pagan-Godfrey** permet de tester l'homoscédasticité des résidus
- Hypothèse nulle (H0) : homoscédasticité (les résidus ont une variance constante)
- On utilise la function `bptest()` de la librairie R `lmtest`
- La p-valeur du test est inférieure à 0.05, on rejette H0

```
library(lmtest)
bptest(mod1)
```

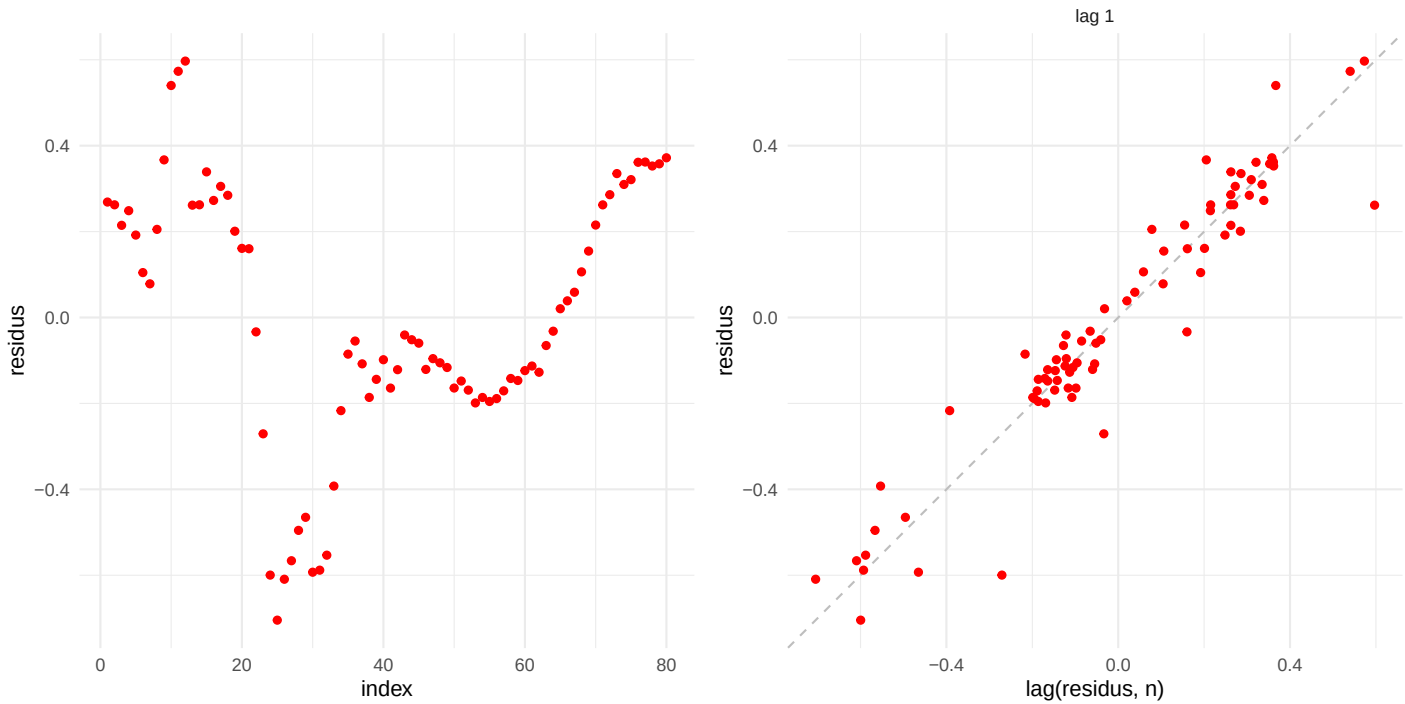
```
##
##  studentized Breusch-Pagan test
##
## data:  mod1
## BP = 5.3717, df = 1, p-value = 0.02047
```

Données Nelson-Plosser

Autocorrélation des résidus (1)

Série des résidus et lag plot

```
gdata <- tibble(index=1:nrow(mod1_diag), residus=mod1_diag$residual)
g1 <- gdata %>% ggplot() + geom_point(aes(x=index, y=residus), color="red")
g2 <- gg_lag(tsibble(gdata, index=index), y=residus, color="red", lags=1, geom="point")
g1 + g2
```

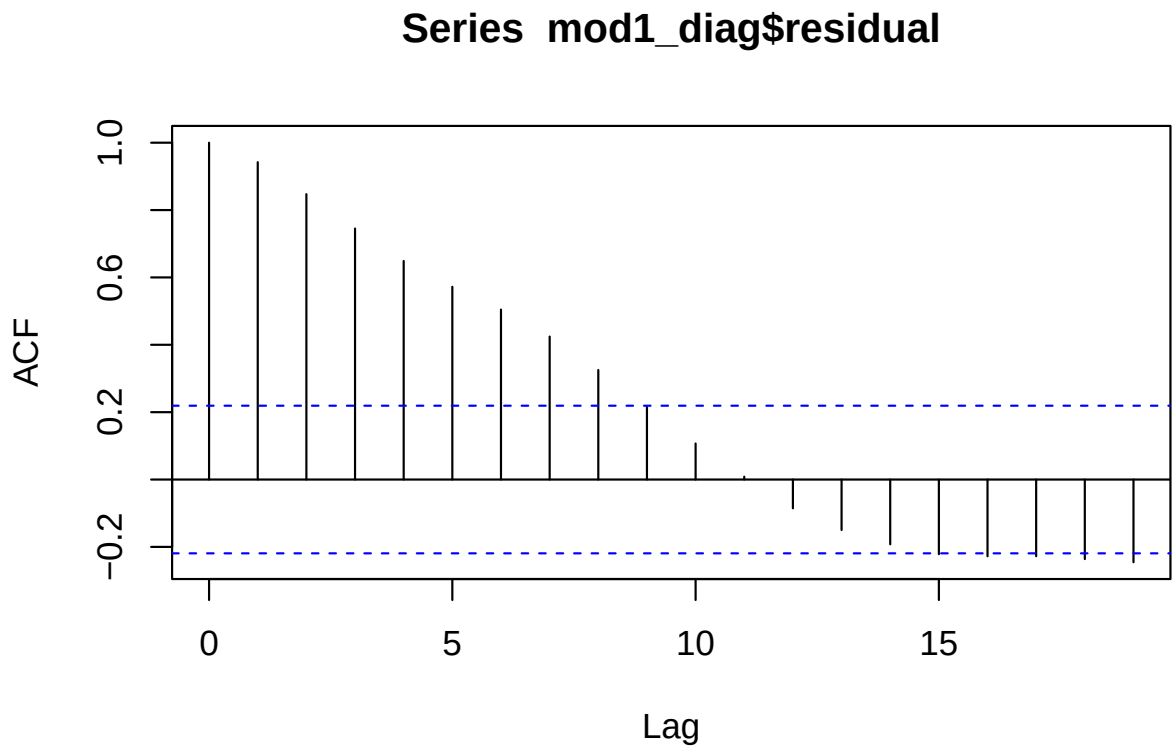


Données Nelson-Plosser

Autocorrélation des résidus (2)

- Un graphique `acf` permet de visualiser la fonction d'autocorrélation des résidus
- La ligne bleue représente le seuil de significativité

```
acf(mod1_diag$residual)
```



Données Nelson-Plosser

Autocorrélation des résidus : Le test de Durbin-Watson

- Le test de `Durbin-Watson` est un test d'absence d'autocorrélation d'ordre 1 sur les résidus d'une régression linéaire
- On utilise la fonction `dwtest` de la librairie `lmtest`
- Hypothèse nulle (H0) : pas d'autocorrélation des résidus

```
dwtest(mod1)
```

```
##
##  Durbin-Watson test
##
## data:  mod1
## DW = 0.085604, p-value < 2.2e-16
## alternative hypothesis: true autocorrelation is greater than 0
```

- Ici on rejette l'hypothèse H0, il y a autocorrélation des résidus
- Le modèle n'a pas capturé les dépendances dynamiques existantes

Données growthofmoney (1)

- Nous allons reproduire le modèle de régression simple utilisé par [13] sur les données `growthofmoney`
- La question de départ est la suivante :
  This paper adresses the question of whether the Federal Reserve System in the 1970s used the money supply as its predominant intermediate target of policy. Did the Federal Reserve "control" the money supply in the 1970s, where "control" is defined as requiring that the behavior of the money supply accurately reflect the stance of monetary policy desired by the Federal Open Market Committee (FOMC) of the Federal Reserve System ?

Données growthofmoney (2)

- Le modèle de régression est décrit par [18] :
  If the Federal Open Market Committee had indeed viewed the money supply as its primary intermediate target of policy, it would have provided for operating procedures that would have ensured control of the money supply.
  This implies a negative value for the coefficient β_2 in the regression

$$(TG_1 - TG_0) = \beta_1 + \beta_2(AG_0 - TG_0)$$

  where TG_1 is the current target for the growth rate of the money supply, TG_0 is the target of the preceding period, and AG_0 is the actual growth rate of the preceding period.

Données growthofmoney (3)

- Les deux variables utilsées dans le modèle sont :
  - Variable dépendante : `TG1.TG0`, difference of current and preceding target for the growth rate of the money supply
  - Variable indépendante : `AG0.TG0`, difference of actual growth rate and target growth rate for the preceding period

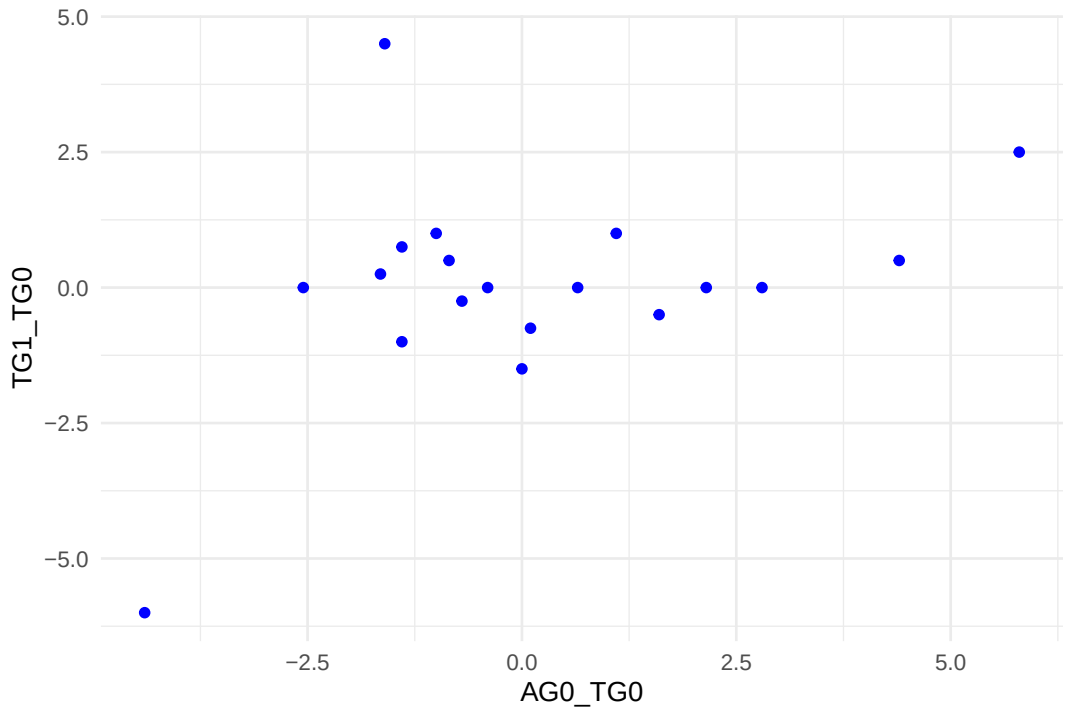
```
data("growthofmoney")
head(growthofmoney)
```

```
##         TG1.TG0 AG0.TG0
## 1970 Q2     0.0    -0.4
## 1970 Q3     1.0    -1.0
## 1970 Q4     1.0     1.1
## 1971 Q1     2.5     5.8
## 1971 Q2    -6.0    -4.4
## 1971 Q3     4.5    -1.6
```

Données growthofmoney (4)

- On visualise la relation entre x_i et y_i avec un nuage de points

```
gdata <- tibble(AG0_TG0=growthofmoney[,"AG0.TG0"], TG1_TG0=growthofmoney[,"TG1.TG0"])
ggplot(gdata) + geom_point(aes(x=AG0_TG0, y=TG1_TG0), color="blue")
```



Données growthofmoney

Estimation

- On créé un objet contenant l'équation du modèle (on l'utilisera par la suite)

```
modelHetzel <- TG1.TG0 ~ AG0.TG0
class(modelHetzel)
```

```
## [1] "formula"
```

- Estimation du dèle utilisé par Hetzel dans son article

```
gom.mod1 <- lm(modelHetzel, data=growthofmoney)
```

- Le coefficient associé à la variable `AG0.TG0` n'est pas significatif au seuil de 5%

```
summary(gom.mod1)
```

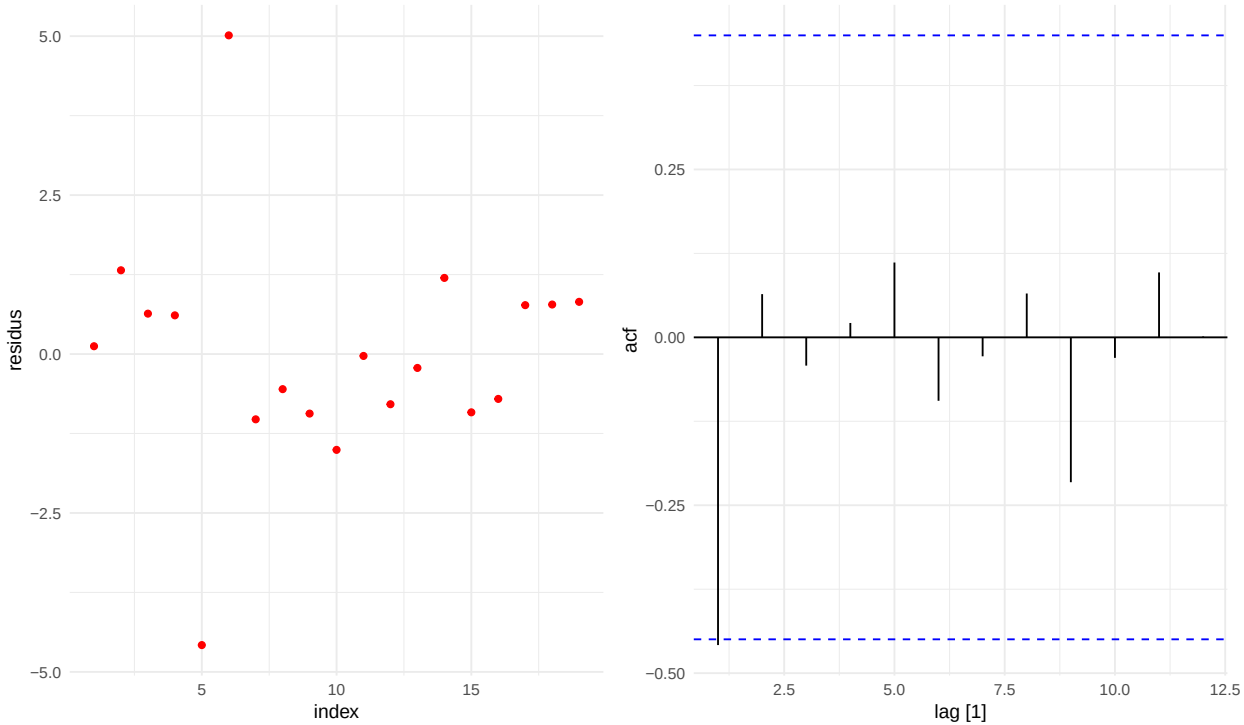
```
##
## Call:
## lm(formula = modelHetzel, data = growthofmoney)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -4.5779 -0.8534 -0.0299  0.7737  5.0125
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept) 0.007322   0.426070   0.017    0.986
## AG0.TG0     0.324858   0.179456   1.810    0.088 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.854 on 17 degrees of freedom
## Multiple R-squared:  0.1616, Adjusted R-squared:  0.1123
## F-statistic: 3.277 on 1 and 17 DF,  p-value: 0.08797
```

Données growthofmoney

Autocorrélation des résidus (1)

- Graphique combiné de la série des résidus et des autocorrélations (utilisation de la librairie patchwork)

```
gdata <- tibble(index=1:nrow(growthofmoney), residus=gom.mod1$residual)
g1 <- gdata %>% ggplot() + geom_point(aes(x=index, y=residus), color="red")
g2 <- gdata %>% as_tsibble(index=index) %>% ACF() %>% autoplot()
g1 + g2
```



Données growthofmoney

Autocorrélation des résidus (2)

- La p-valeur du test de **Durbin-Watson** est largement supérieure à 0.05, on accepte H0, il n'y a pas d'autocorrélation des résidus

```
dwtest(modelHetzel, data=growthofmoney)
```

```
##
##  Durbin-Watson test
##
## data:  modelHetzel
## DW = 2.9046, p-value = 0.9839
## alternative hypothesis: true autocorrelation is greater than 0
```

Données growthofmoney

Diagnostic du modèle

- La distance de Cook (Cook's D) permet d'estimer l'influence d'une observation dans un modèle de régression linéaire
- La distance de Cook peut être utilisée :
  - pour indiquer les observations pour lesquelles une vérification de la validité est nécessaire
  - pour indiquer les zones ou il serait utile d'avoir plus de données d'observations

Données growthofmoney

Distance de Cook (1)

- Calcul des distances de Cook avec la fonction `cooks.distance()`

```
dcook <- gdata <- data.frame(obs=1:nrow(growthofmoney), dcook=cooks.distance(gom.mod1))
head(dcook)
```

```
##   obs        dcook
## 1   1 0.0001356853
## 2   2 0.0187077141
## 3   3 0.0040829364
## 4   4 0.0453754681
## 5   5 1.3164884600
## 6   6 0.3504193583
```

- On peut utiliser un seuil pour identifier les observations influentes :
  - $D > 1$
  - $D > 4/N$ ou $D > 4/(N - K - 1)$, ou N est le nombre d'observations et K le nombre de variables indépendantes

```
tcook <- 4/nrow(growthofmoney)
tcook
```

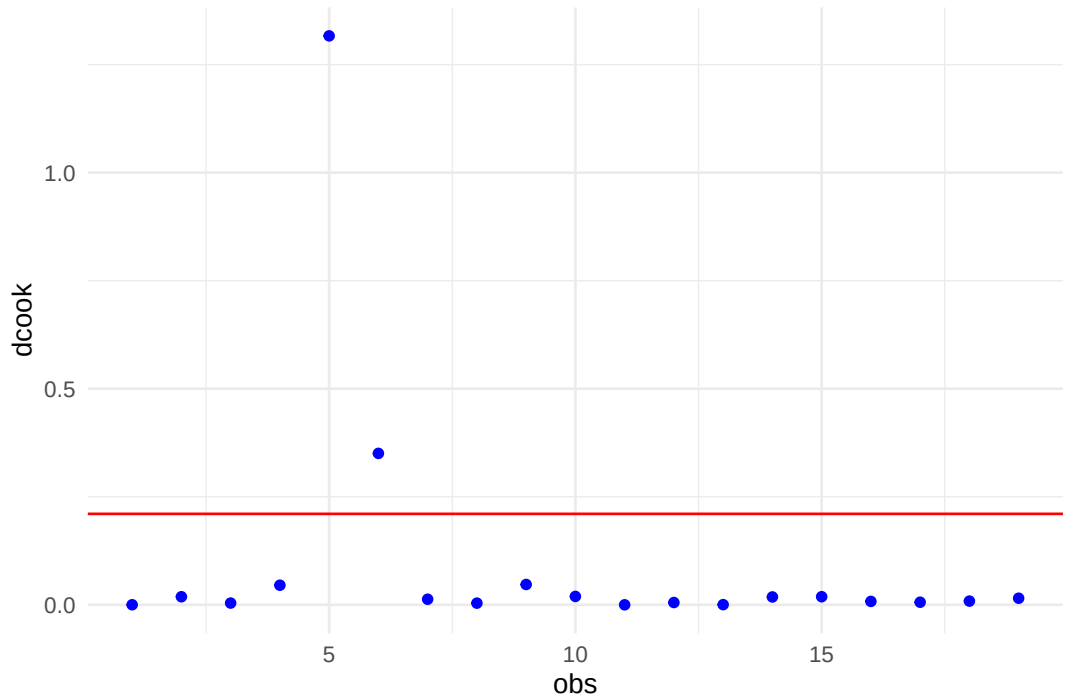
```
## [1] 0.2105263
```

Régression linéaire simple

Données growthofmoney : Distance de Cook (2)

- Les observations particulièrement influentes sont celles dont la distance de Cook est au dessus du seuil matérialisé par la ligne rouge

```
ggplot(dcook, aes(x=obs, y=dcook)) +
  geom_point(color="blue") +
  geom_hline(aes(yintercept=tcook), color="red")
```



Données growthofmoney

Résidus studentisés

- Les résidus *studentisés* sont calculés en divisant les résidus par leur écart type

```
stres <- data.frame(obs=1:nrow(growthofmoney), stres=rstudent(gom.mod1))
head(stres)
```

```
##   obs      stres
## 1   1  0.0660265
## 2   2  0.7245157
## 3   3  0.3444020
## 4   4  0.3977586
## 5   5 -3.8084498
## 6   6  3.7506138
```

- Un résidu studentisé supérieur à 3 (2) est considéré comme une valeur aberrante

```
stres[abs(stres$stres)>3,]
```

```
##   obs     stres
## 5   5 -3.808450
## 6   6  3.750614
```

- On retrouve les observations 5 et 6 identifiées également par la distance de Cook, elles correspondent au 2ème et 3ème trimestre 1971

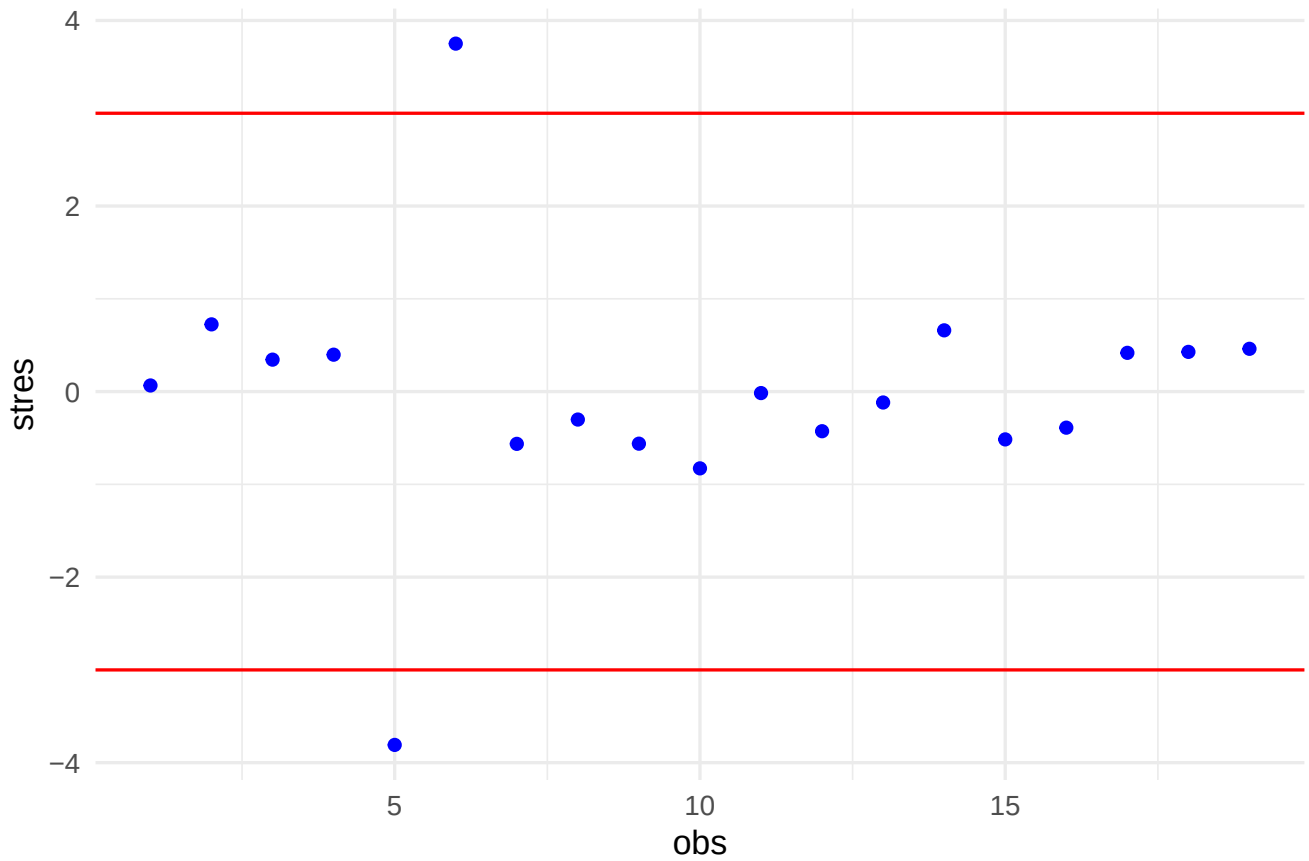
```
as_tsibble(growthofmoney)[5:6,]
```

```
## # A tsibble: 2 x 3 [1Q]
## # Key:       key [1]
##     index key     value
##     <qtr> <chr>   <dbl>
## 1 1971 Q2 TG1.TG0  -6
## 2 1971 Q3 TG1.TG0   4.5
```

Régression linéaire simple

Données growthofmoney : Résidus studentisés

```
ggplot(stres, aes(x=obs, y=stres)) +
  geom_point(color="blue") +
  geom_hline(aes(yintercept=3), color="red") +
  geom_hline(aes(yintercept=-3), color="red")
```



6 Régression Linéaire
- Régression Linéaire simple
- Modèle de régression multivarié
- Test de changement structurel

Modèle de régression multivarié

- Modèle de régression multivarié = plusieurs variables indépendantes :

$$y = \beta_0 + \beta_1 \cdot x_1 + \beta_2 \cdot x_2 + \cdot + \beta_k \cdot x_k + \epsilon$$

- qu'on peut également représenter par

$$y = \beta_0 + \beta_1^T \cdot x + \epsilon$$

où β_1 est un vecteur de coefficients et $x = x_1, x_2 + \ldots + x_k$ est un vecteur de variables indépendantes

Données Nelson-Plosser

Modèle de régression multivarié

- On souhaite prédire le PNB (`gnp.nom`) par la population active (`emp`, log), l'indice de production industrielle (`ip`, log), le niveau de salaires (`nom.wages`, log) et l'année (`index`)
- On utilise la fonction `spread` pour passer du format "long" au format "large"

```
nelplo1909 <- nelplo_tsbl %>%
  filter(index>=1909) %>%
  spread(key = key, value=value)
regdata <- nelplo1909 %>%
  select(index, gnp.nom, emp, ip, nom.wages)
```

Données Nelson-Plosser

Estimation

- L'équation (formule) du modèle prend la forme `gnp.nom index+emp+ip+nom.wages`

```
mod2 <- lm(gnp.nom ~ index+emp+ip+nom.wages, data=regdata)
summary(mod2)
```

```
##
## Call:
## lm(formula = gnp.nom ~ index + emp + ip + nom.wages, data = regdata)
##
## Residuals:
##       Min        1Q    Median        3Q       Max
## -0.145762 -0.046282 -0.007339  0.043974  0.147261
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  5.347255   4.257390   1.256  0.21302
## index       -0.006329   0.001903  -3.325  0.00137 **
## emp          0.942652   0.219361   4.297 5.13e-05 ***
## ip           0.101593   0.067941   1.495  0.13903
## nom.wages    1.086870   0.049022  22.171  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.06323 on 75 degrees of freedom
## Multiple R-squared:  0.9982, Adjusted R-squared:  0.9981
## F-statistic: 1.061e+04 on 4 and 75 DF,  p-value: < 2.2e-16
```

Modèle de régression multivarié

Données Nelson-Plosser : Sélection du modèle (1)

- On élimine (successivement) les variables dont le coefficient n'est pas significativement différent de 0 (p-valeur > 0.05), ici la variable `ip`

```
mod3 <- lm(gnp.nom ~ index+emp+nom.wages, data=regdata)
summary(mod3)
```

```
##
## Call:
## lm(formula = gnp.nom ~ index + emp + nom.wages, data = regdata)
##
## Residuals:
##       Min        1Q    Median        3Q       Max
## -0.152528 -0.042715  0.002419  0.041428  0.164197
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  0.540320   2.813876   0.192  0.84824
## index       -0.004975   0.001688  -2.947  0.00426 **
## emp          1.190645   0.144738   8.226 4.06e-12 ***
## nom.wages    1.064772   0.047120  22.597  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.06374 on 76 degrees of freedom
## Multiple R-squared:  0.9982, Adjusted R-squared:  0.9981
## F-statistic: 1.392e+04 on 3 and 76 DF,  p-value: < 2.2e-16
```

Modèle de régression multivarié

Données Nelson-Plosser : Sélection du modèle (2)

- Pour comparer des modèles **imbriqués**, on peut utiliser un critère d'information (AIC ou BIC)
- Ici on compare le modèle `mod1` ne contenant que l'année comme variable indépendante ; et les deux modèles multivariés `mod2` et `mod3`

```
AIC(mod1, mod2, mod3)
```

```
##      df        AIC
## mod1  3   38.92556
## mod2  6 -207.89003
## mod3  5 -207.53988
```

- On retient le modèle ayant le plus faible AIC, ici le modèle `mod3`

Données Nelson-Plosser

Valeurs observées, valeurs prédites, résidus

- Les valeurs prédites par le modèle se trouvent dans l'attribut `fitted.values`
- Les résidus représentent la différence valeur observée-valeur prédite, ils se trouvent dans l'attribut `residuals`

```
mod3.diag <- tibble(observed=nelplo1909$gnp.nom, predicted=mod3$fitted.values,
                    residual=mod3$residuals)
mod3.diag
```

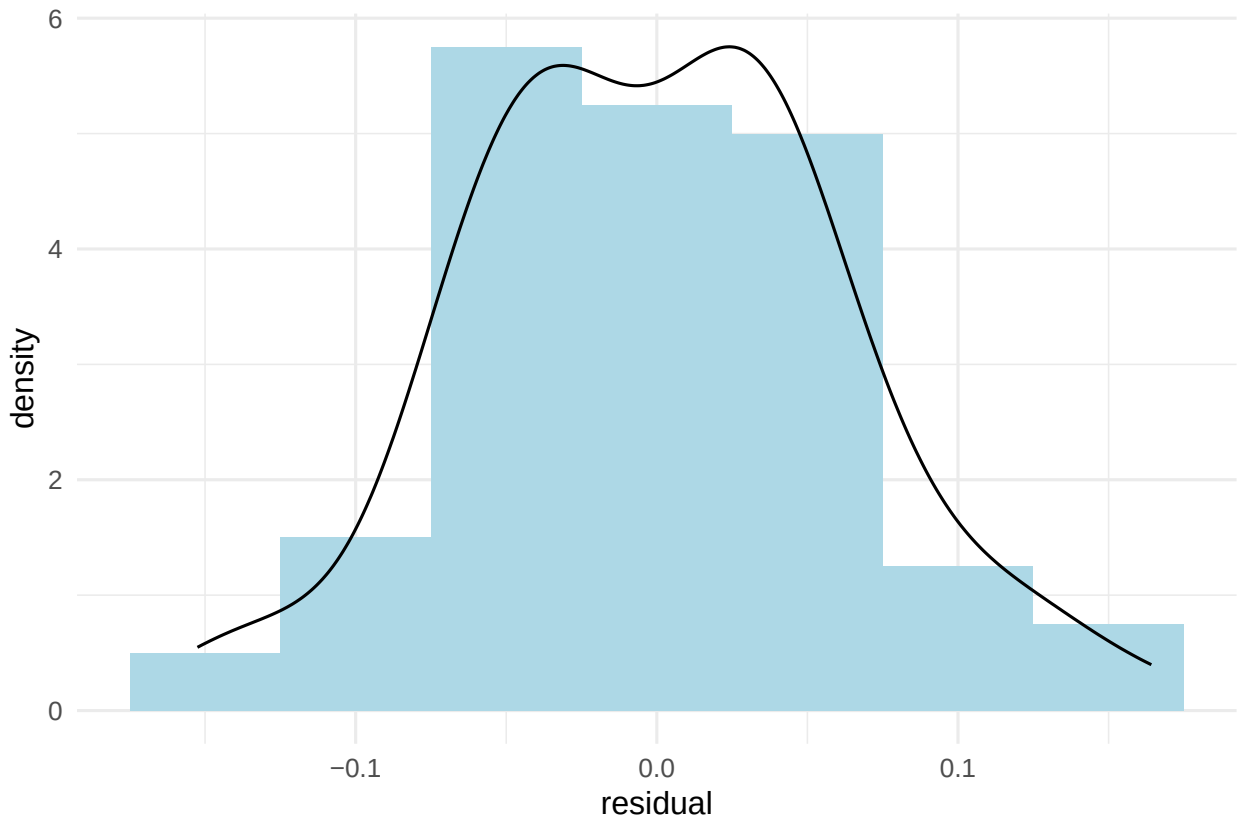
```
## # A tibble: 80 x 3
##    observed predicted residual
##       <dbl>     <dbl>    <dbl>
##  1     10.4      10.3   0.104
##  2     10.5      10.4   0.0527
##  3     10.5      10.4   0.0894
##  4     10.6      10.5   0.118
##  5     10.6      10.5   0.0406
##  6     10.6      10.5   0.0324
##  7     10.6      10.5   0.129
##  8     10.8      10.7   0.130
##  9     11.0      10.8   0.164
## 10     11.2      11.2   0.0878
## # i 70 more rows
```

Données Nelson-Plosser

Distribution des résidus

- On représente la distribution des résidus sous la forme d'un histogramme et d'une courbe de densité

```
ggplot(mod3.diag, aes(x=residual)) +
  geom_histogram(aes(y = after_stat(density)), fill="lightblue", binwidth = 0.05) +
  geom_density(aes(x=residual))
```



Données Nelson-Plosser

Test de normalité des résidus

- La p-valeur du test est supérieure à 0.05, on accepte H0, les résidus suivent une loi normale

```
shapiro.test(mod3.diag$residual)
```

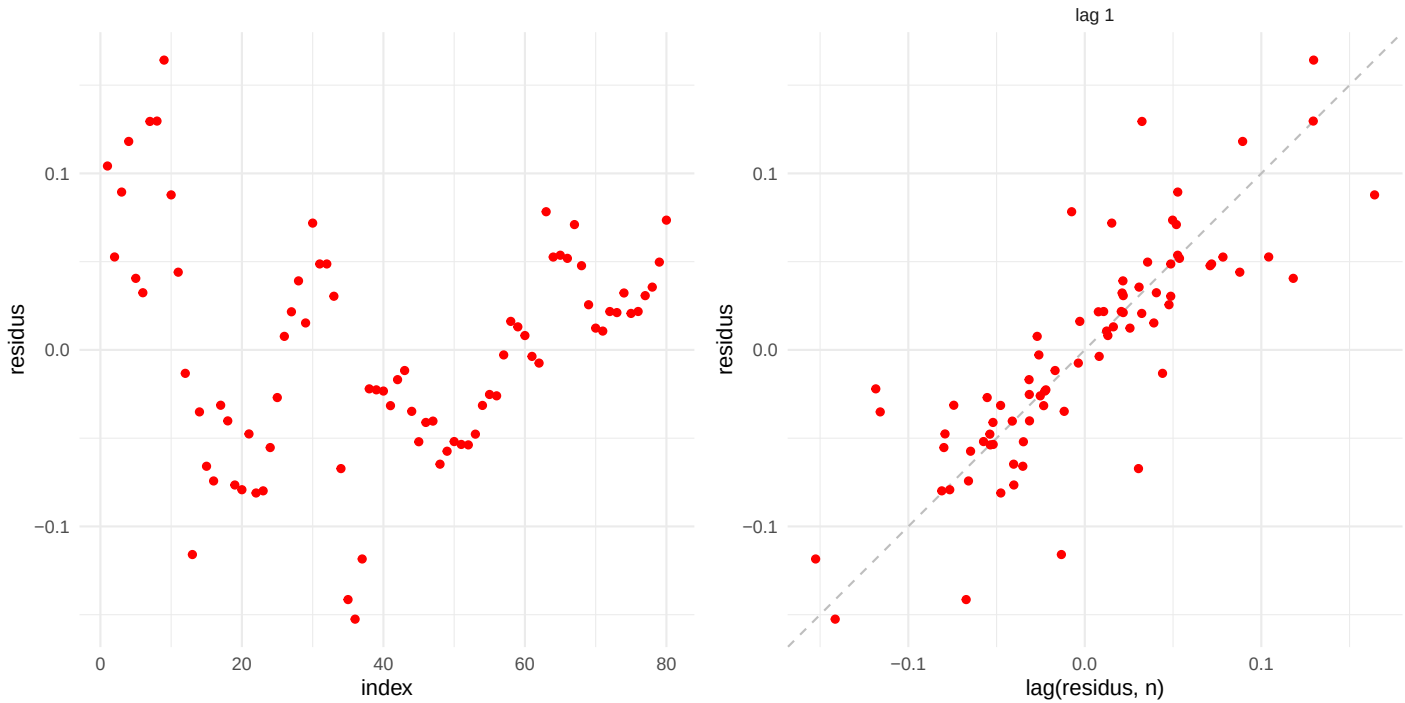
```
##
##  Shapiro-Wilk normality test
##
## data:  mod3.diag$residual
## W = 0.99156, p-value = 0.884
```

Données Nelson-Plosser

Autocorrélation des résidus (1)

Série des résidus et lag plot

```
gdata <- tibble(index=1:nrow(mod3.diag), residus=mod3.diag$residual)
g1 <- gdata %>% ggplot() + geom_point(aes(x=index, y=residus), color="red")
g2 <- gg_lag(tsibble(gdata, index=index), y=residus, color="red", lags=1, geom="point")
g1 + g2
```

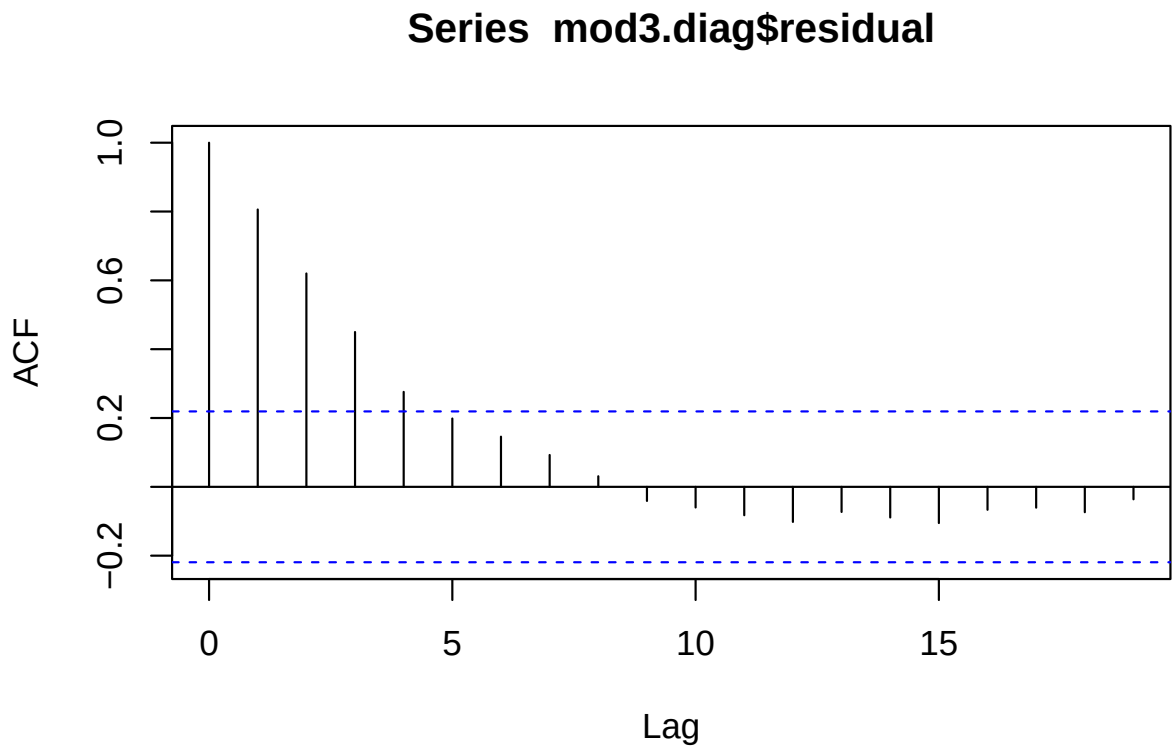


Données Nelson-Plosser

Autocorrélation des résidus (2)

- Un graphique `acf` permet de visualiser la fonction d'autocorrélation des résidus
- La ligne bleue représente le seuil de significativité

```
acf(mod3.diag$residual)
```



Données Nelson-Plosser

Autocorrélation des résidus (3)

- Test de `Durbin-Watson` d'absence d'autocorrélation d'ordre 1 sur les résidus
- Hypothèse nulle (H0) : pas d'autocorrélation des résidus

```
dwtest(mod3)
```

```
##
##  Durbin-Watson test
##
## data:  mod3
## DW = 0.3355, p-value < 2.2e-16
## alternative hypothesis: true autocorrelation is greater than 0
```

- Ici on rejette l'hypothèse H0, il y a autocorrélation des résidus
- Le modèle n'a pas capturé les dépendances dynamiques existantes

6 Régression Linéaire
- Régression Linéaire simple
- Modèle de régression multivarié
- Test de changement structurel

Définition

- Soit le modèle de régression linéaire standard :

$$y_i = x_i^T \beta_i + u_i \quad (i = 1, \ldots, N)$$

où y_i est l'observation de la variable dépendante et le vecteur $x_i = (x_{i,1}, x_{i,2}, \ldots, x_{i,k})$ l'obsevation des variables indépendantes au temps i

- Les tests de changement structurel proposent de tester l'hypothèse

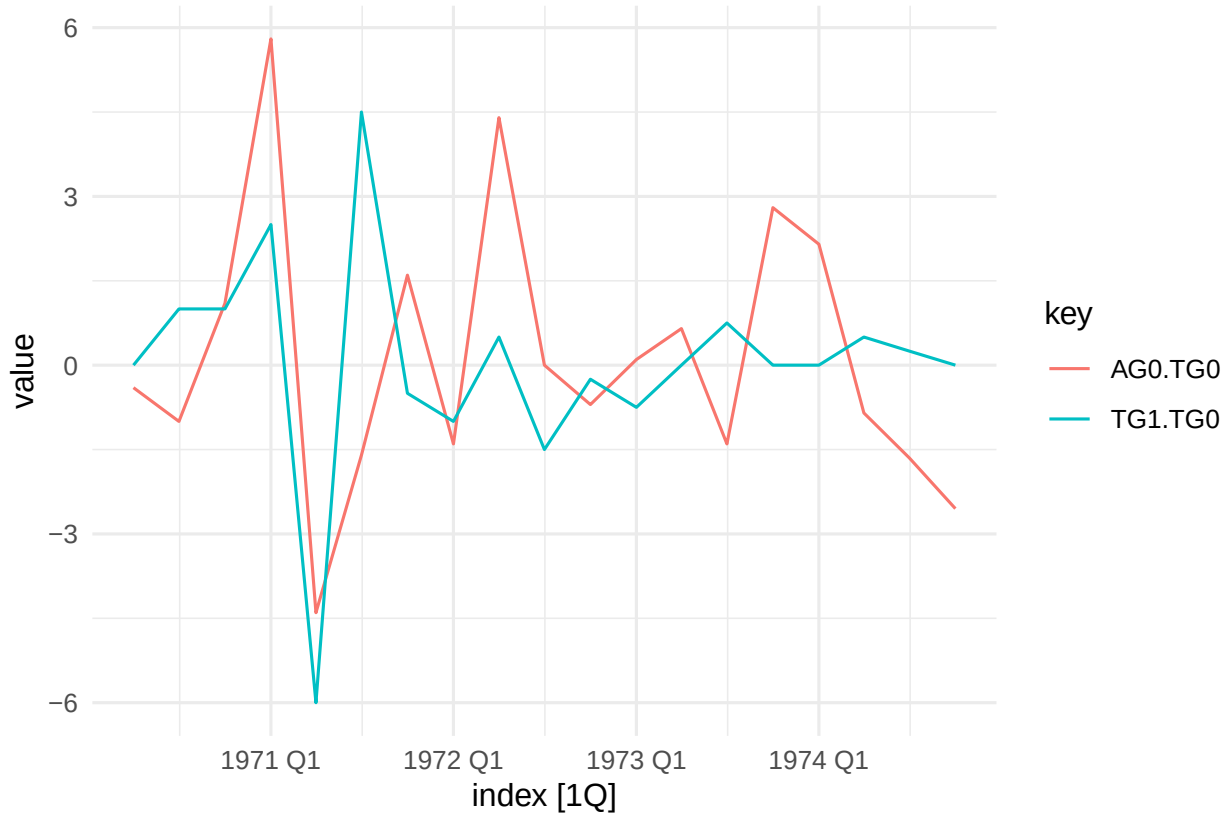
$$H0 : \beta_i = \beta_0 \quad (i = 1, \ldots, N)$$

- Le test de **Chow** permet d'identifier un éventuel changement structurel dans les données au point fourni **a priori** par l'utilisateur
- Le rejet de l'hypotèse H0 signifie qu'un meilleur ajustement peut être obtenu avec deux droites de régression (i.e. les paramètres du modèle ne sont pas stables)

Données growthofmoney (1)

- On utilise la librairie strucchange (voir article)
- On reproduit ici l'exemple de [18] : test de changement structurel au premier trimestre 1974
- Voici pour rappel le chronogramme des deux séries

```
growthofmoney %>% as_tsibble() %>% autoplot(value)
```



Données growthofmoney (2)

- La formule de la régression est contenue dans l'objet `modelHetzel`

```
modelHetzel
```

```
## TG1.TG0 ~ AG0.TG0
```

- Le test est réalisé avec la fonction `sctest()`

```
sctest(modelHetzel, point=c(1973,4), data=growthofmoney, type="Chow")
```

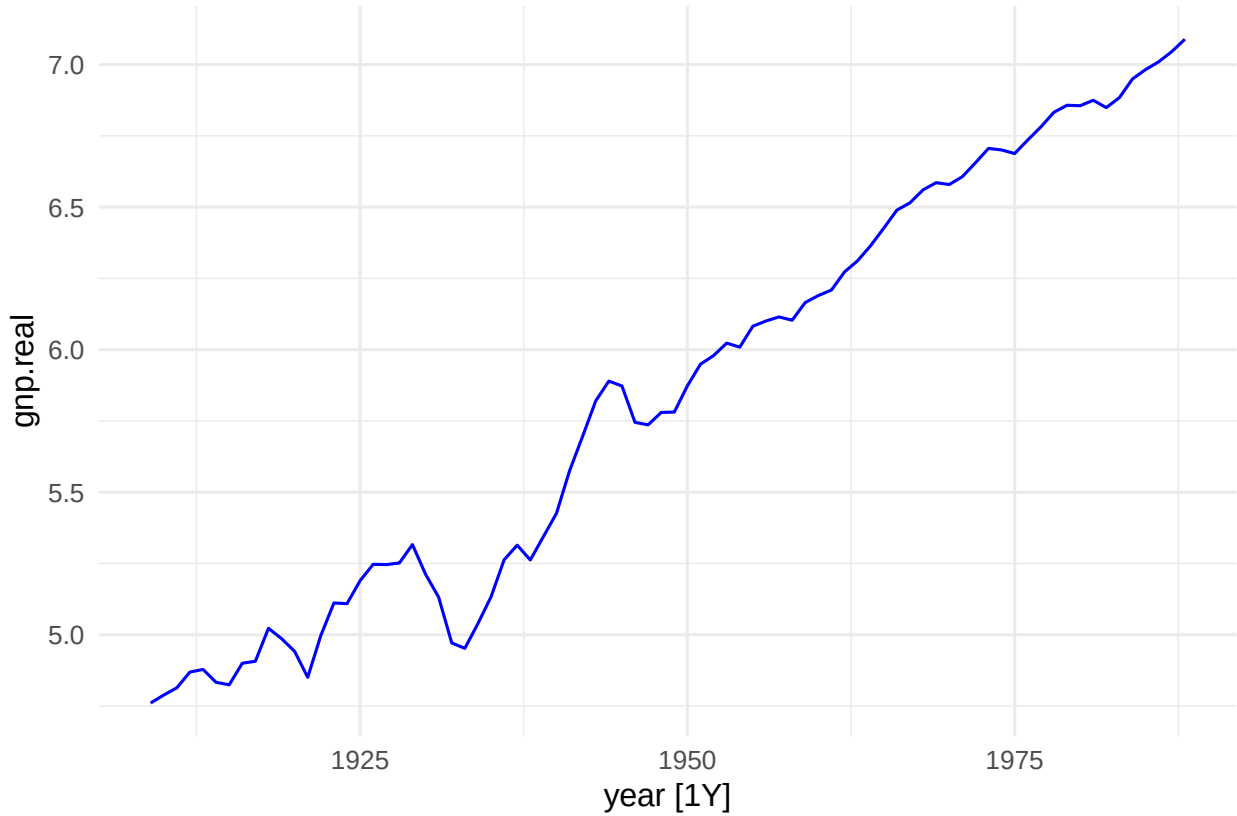
```
##
##  Chow test
##
## data:  modelHetzel
## F = 0.37876, p-value = 0.6911
```

- La p-valeur est supérieure à 0.05, on accepte H0, il n'y a pas de changement structurel au premier trimestre 1974

Données Nelson-Plossser (1)

- Pour la série du PNB des USA (log) on teste un changement structurel en 1933

```
rdata <- Nelson_Plosser[,c("year", "gnp.real")]
rdata <- rdata[rdata$year>=1909,]
tsibble(rdata, index=year) %>% autoplot(gnp.real, color="blue")
```



Modèle de régression linéaire

Test de changement structurel : Données Nelson-Plossser (2)

- Ici on spécifie l'index correspondant à l'année 1933 en utilisant la fonction which()
- La p-valeur du test est inférieure à 0.01, on rejette H0 : il y a un changement structurel en 1933

```
library(strucchange)
model <- gnp.real ~ year
sctest(model, point=which(rdata$year==1933), data=rdata, type="Chow")
```

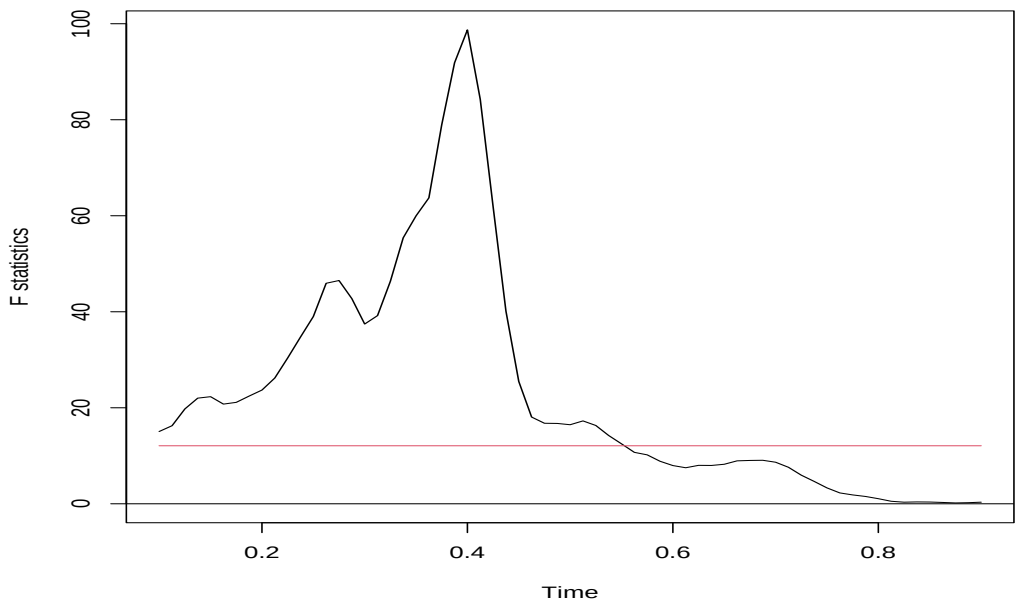
```
##
##  Chow test
##
## data:  model
## F = 19.598, p-value = 1.369e-07
```

Modèle de régression linéaire

Test de changement structurel - Extension

- La librairie `strucchange` propose une extension du test de Chow : le changement structurel est recherché sur un intervalle
- La statistique F est calculée pour chacun des points de l'intervalle
- On rejette H0 pour des valeurs élevées de F (supérieure au seuil représenté par la ligne rouge)

```
fs <- Fstats(model, from=0.1, data = rdata)
plot(fs)
```



Modèle de régression linéaire

Exercice

- Utilisez les données `fr_macro` :
  1. Ajustez un modèle $gnp = \beta_0 + \beta_1 * index$,
  2. Ajustez un modèle $gnp = \beta_0 + \beta_1 * index + \beta_2 * unemp + \beta_3 * cpi$
- Vérifiez les hypothèses de base pour ces modèles :
  - Les résidus sont-ils normalement distribués ?
  - Sont-ils autocorrélés ?
  - Leur variance est-elle constante ?
- Réalisez un test de changement structurel sur le modèle 1

7 Décomposition, tendances, transformation des données

7 Décomposition, tendances, transformation des données

- Décomposition d'une série temporelle
- Tendances
- Saisonnalité

Composants d'une série temporelle

- Une série temporelle peut être décomposée en trois éléments :
  - Tendence
  - Saisonnalité
  - Résidus
- La méthode classique de décomposition (voir la fonction `decompose()`) utilise un filtre basé sur les moyennes mobiles

Décomposition avec la librairie `feasts` (1)

- La librarie `feast` propose deux méthodes de décomposition (classique et STL)
- La décomposition classique utilise les moyennes mobiles, la saisonalité peut être additive ou multiplicative

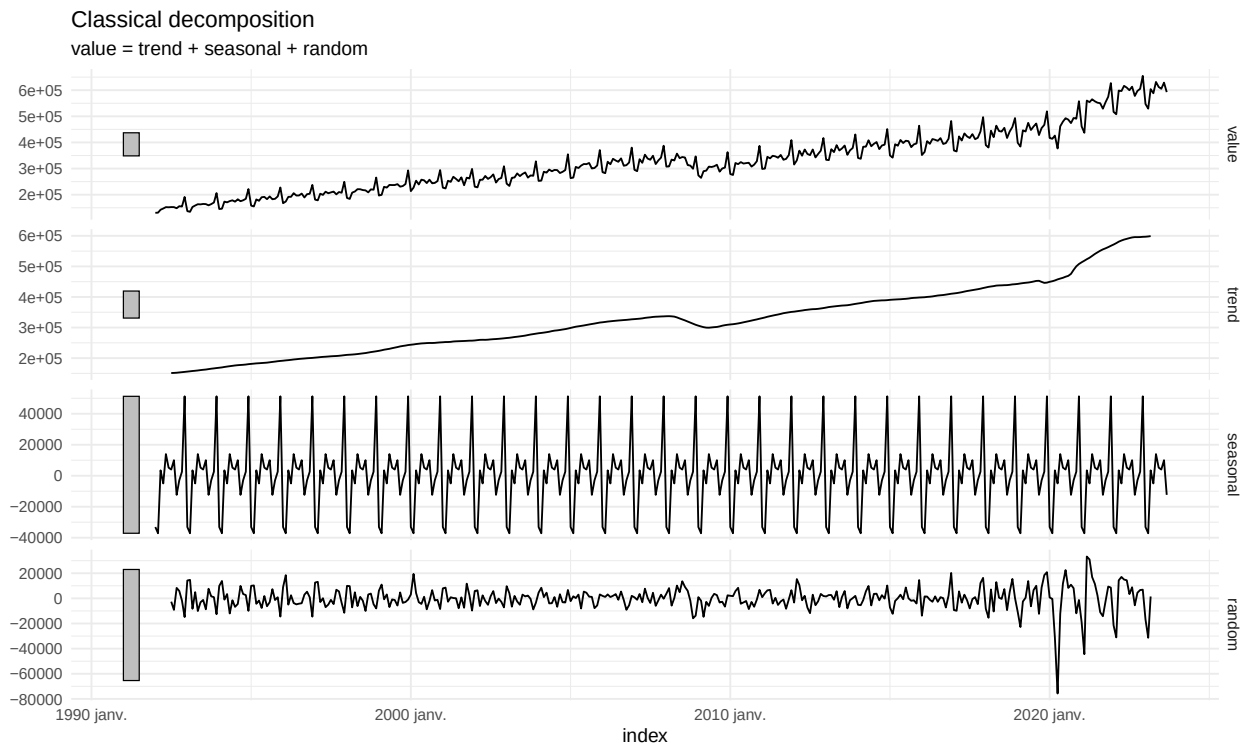
```
dcmp <- retail_tsbl %>%
  model(classical_decomposition(value))
components(dcmp)
```

```
## # A dable: 381 x 7 [1M]
## # Key:     .model [1]
## # :        value = trend + seasonal + random
##    .model                     index  value   trend seasonal random season_adjust
##    <chr>                      <mth>  <dbl>   <dbl>    <dbl>  <dbl>         <dbl>
##  1 classical_decomposit~ 1992 janv. 130683     NA   -33112.    NA        163795.
##  2 classical_decomposit~ 1992 févr. 131244     NA   -37134.    NA        168378.
##  3 classical_decomposit~  1992 mars 142488     NA     3459.    NA        139029.
##  4 classical_decomposit~ 1992 avril 147175     NA    -4965.    NA        152140.
##  5 classical_decomposit~   1992 mai 152420     NA    13966.    NA        138454.
##  6 classical_decomposit~  1992 juin 151849     NA     5222.    NA        146627.
##  7 classical_decomposit~ 1992 juil. 152586 151200.    4004. -2619.       148582.
##  8 classical_decomposit~  1992 août 152476 151599.   10017. -9140.       142459.
##  9 classical_decomposit~ 1992 sept. 148158 152172.  -12342.  8329.       160500.
## 10 classical_decomposit~  1992 oct. 155987 153087.   -2997.  5897.       158984.
## # i 371 more rows
```

Décomposition avec la librairie `feasts` (2)

- On obtient une représentation graphique de la décomposition avec la méthode générique `autoplot()`

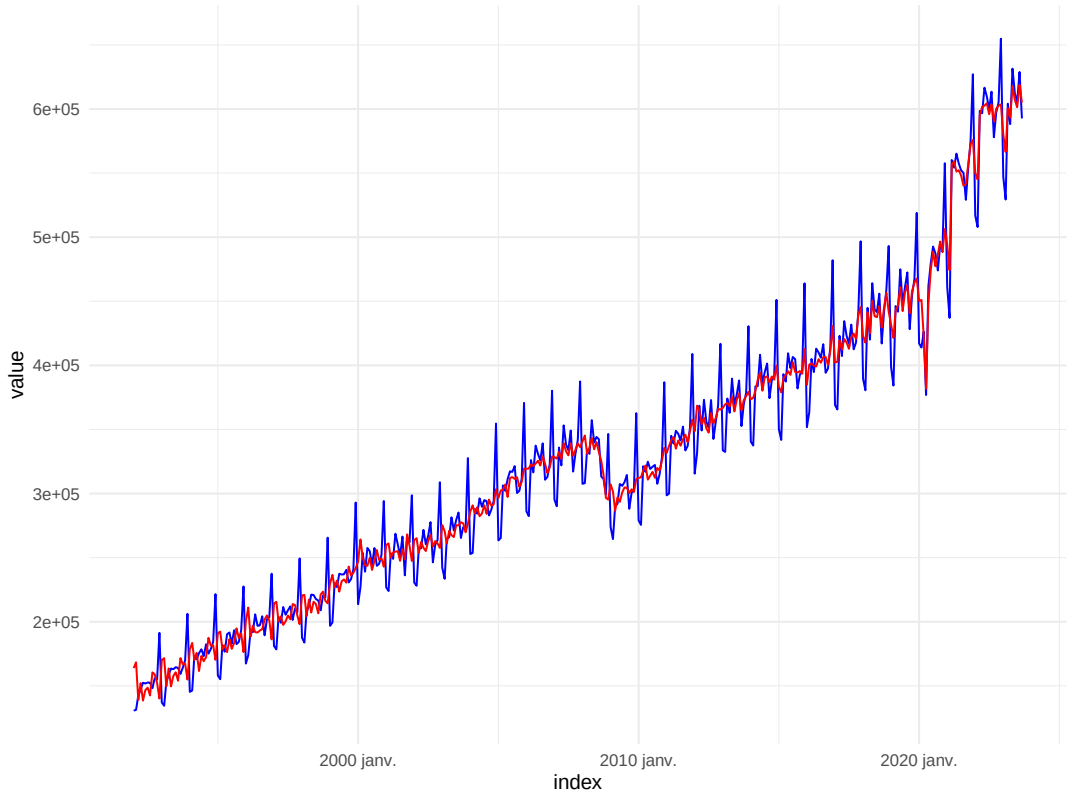
```
components(dcmp) %>% autoplot()
```



Décomposition avec la librairie `feasts` (3)

- La colonne `season_adjust` de l'objet renvoyé par `components` contient les données CVS

```
components(dcmp) %>% ggplot() +
  geom_line(aes(x=index,y=value), colour="blue") +
  geom_line(aes(x=index,y=season_adjust), colour="red")
```



7 Décomposition, tendances, transformation des données

- Décomposition d'une série temporelle
- Tendances
- Saisonnalité

Tendance linéaire

- Une série temporelle dont l'évolution est une fonction **déterministe** du temps est non-stationnaire
- Une série dont l'évolution autour d'une fonction déterministe du temps est stationnaire est dite stationnaire à une tendance près (trend stationary)
- On peut décrire une tendance linéaire par le modèle

$$y_t = \beta_0 + \beta_1 \cdot t + \epsilon_t$$

- y_t est non-stationnaire si $\beta 1 \neq 0$, la moyenne de y_t est

$$\mu_t = \beta_0 + \beta_1 \cdot t$$

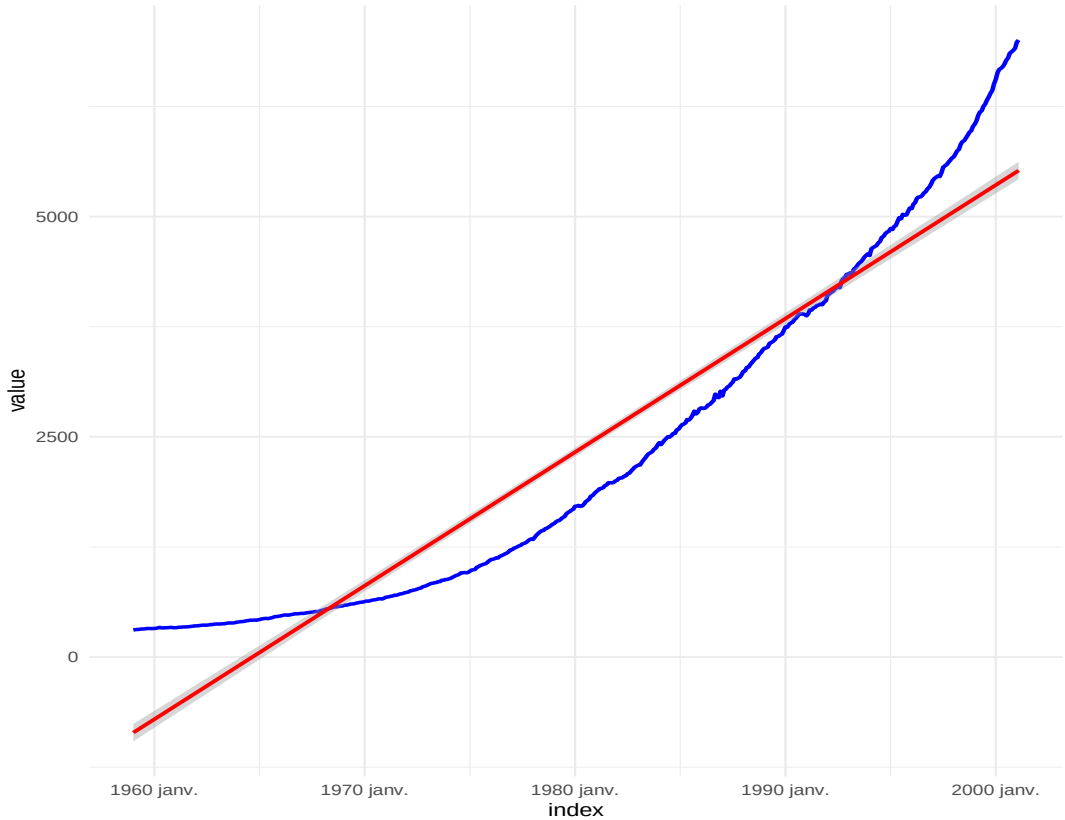
- On peut tester l'hypothèse $\beta_1 \neq 0$ (existence d'une tendance) en ajustant un modèle MCO sur les données
- On peut ajuster un modèle pour estimer β_0 et β_1 puis analyser les résidus comme un processus stationnaire $\epsilon_t = y_t - \beta_0 - \beta_1 \cdot t$

Tendance linéaire

Données USIncExp

- Données USIncExp, dépenses aggrégées de consommation en millions de dollars

```
USIncExp %>% as_tsibble() %>% filter(key=='expenditure') %>%
  ggplot(aes(x = index, y = value)) +
  geom_line(color = "blue", size = 1) +
  geom_smooth(method='lm', color="red")
```



Tendance linéaire

Données USIncExp : Estimation du modèle

- Estimation du modèle par MCO

```
regdata <- USIncExp %>% as_tsibble() %>% filter(key=='expenditure')
summary(lm(value ~ index, data=regdata))
```

```
##
## Call:
## lm(formula = value ~ index, data = regdata)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -688.01 -507.37  -97.57  411.41 1480.36
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) 8.101e+02  3.277e+01   24.72   <2e-16 ***
## index       4.151e-01  5.686e-03   73.00   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 568.7 on 504 degrees of freedom
## Multiple R-squared:  0.9136, Adjusted R-squared:  0.9134
## F-statistic:  5329 on 1 and 504 DF,  p-value: < 2.2e-16
```

- Le coefficient associé au temps (index) est significativement différent de 0 (p-valeur < 0.05)

Transformation logarithmique (1)

- On utilise souvent le logarithme (naturel) d'une série temporelle pour l'analyse
- Par exemple dans l'article original de Nelson et Plosser, toutes les séries à l'exception de la série `stock.price` (prix des actions) sont transformées en log :
  The tendency of economic time series to exhibit variation that increases in mean and dispersion in proportion to absolute level motivates the transformation to natural logs and the assumption that trends are linear in the transformed data.
- L'utilisation du log d'une série :
  - diminue l'hétéroscédasticité (stabilisation de la variance) (cf. par ex. [27], p. 15)
  - transforme une tendance exponentielle en tendance linéaire

Transformation logarithmique (2)

- *Many economic time series (such as consumption, national income and expenditure, or the price level) do grow over time, but the amount by which they grow in each period also tends to rise.* [3]
- $\Delta x_t = x_t - x_{t-1}$ est stationnaire uniquement si la valeur absolue de la croissance est stationnaire, dans ce cas pour $\mu > 0$, σ / x_t tendra vers zero.
- Au contraire, la croissance exprimée en %, ne présente généralement pas de tendance, et est donc plus susceptible d'être stationnaire
- Puisque les niveaux de nombreuses variables économiques sont initialement positifs, et que

$$\Delta log(x_t) = log(x_t) - log(x_{t-1}) = log(x_t / x_{t-1})$$

  la stationnarité du taux de croissance implique la stationnarité de $\Delta log(x_t)$
- Les changements dans le logarithme des séries économiques sont donc plus susceptible d'être stationnaires que les changements en valeur absolue

Tendance exponentielle

- Si la tendance est exponentielle, on peut ramener à une tendance linéaire en utilisant le log :

$$\log(y_t) = \beta_0 + \beta_1 \cdot t + \epsilon_t$$

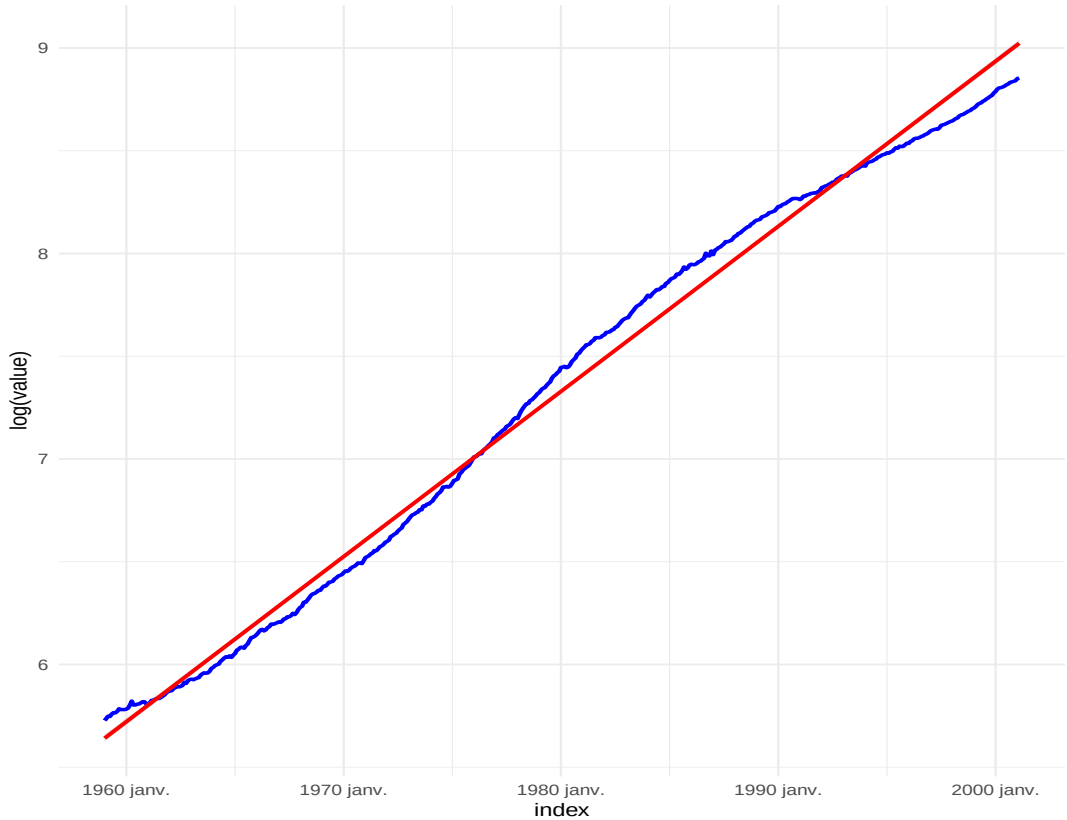
- Note : β_1 dans le modèle à tendance exponentielle est le taux de croissance annuel moyen (si t est exprimé en années)

Tendance exponentielle

Transformation logarithmique

- L'utilisation du logarithme de la série `USIncExp` transforme la tendance

```
USIncExp %>% as_tsibble() %>% filter(key=='expenditure') %>%
  ggplot(aes(x = index, y = log(value))) +
  geom_line(color = "blue", size = 1) +
  geom_smooth(method='lm', color="red")
```



Tendance exponentielle - Modèle linéaire

- La valeur de R^2 est nettement plus élevée que pour le modèle précédent sur la série brute

```
regdata <- USIncExp %>% as_tsibble() %>%
  filter(key=='expenditure') %>%
  mutate(value=log(value))
summary(lm(value ~ index, data=regdata))
```

```
##
## Call:
## lm(formula = value ~ index, data = regdata)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.16858 -0.07288 -0.01324  0.09148  0.15056
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) 6.525e+00  5.134e-03    1271   <2e-16 ***
## index       2.200e-04  8.908e-07     247   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.08909 on 504 degrees of freedom
## Multiple R-squared:  0.9918, Adjusted R-squared:  0.9918
## F-statistic: 6.099e+04 on 1 and 504 DF,  p-value: < 2.2e-16
```

7 Décomposition, tendances, transformation des données

- Décomposition d'une série temporelle
- Tendances
- Saisonnalité

Désaisonnalisation

- De nombreuses séries économiques présentent des comportements périodiques, rendant difficile la comparaison de deux instants successifs
- Cela peut être le cas particulièrement lorsque la série est trimestrielle ou mensuelle (par exemple données `retail`)
- Le recours à une `désaisonnalisation` permet d'obtenir des séries dites corrigées des variations saisonnières (CVS)

Désaisonnalisation

Régression linéaire

- On inclus la saisonnalité dans un modèle linéaire tendenciel avec des variables 'dummy' (0 ou 1) représentant les mois, trimestres, etc ...
- Par exemple pour des trimestres on ajoute $4 - 1 = 3$ variables 'dummy' (modèle avec constante) :

$$y_t = \beta_0 + \delta_1 \cdot Q1_t + \delta_2 \cdot Q2_t + \delta_3 \cdot Q3_t + \beta_1 \cdot t + \epsilon_t$$

Désaisonnalisation

Données retail (1)

- Données `retail` sur le commerce de détail
- On créé une variable catégorielle (factor) pour le mois
- Le temps est un index t de 1 à 381 (nombre de mois dans la série)

```
retail$mois <- factor(month(retail$DATE), labels=month(1:12, label=TRUE))
retail$t <- c(1:nrow(retail))
head(retail)
```

```
##         DATE RSXFSN  mois t
## 1 1992-01-01 130683  janv 1
## 2 1992-02-01 131244  févr 2
## 3 1992-03-01 142488  mars 3
## 4 1992-04-01 147175 avril 4
## 5 1992-05-01 152420   mai 5
## 6 1992-06-01 151849  juin 6
```

- Dans la formule, on ajoute -1 pour un modèle sans constante (intercept)

```
modst <- lm(RSXFSN ~ t+ mois, data=retail)
```

Désaisonnalisation

Données retail (2)

```
summary(modst)

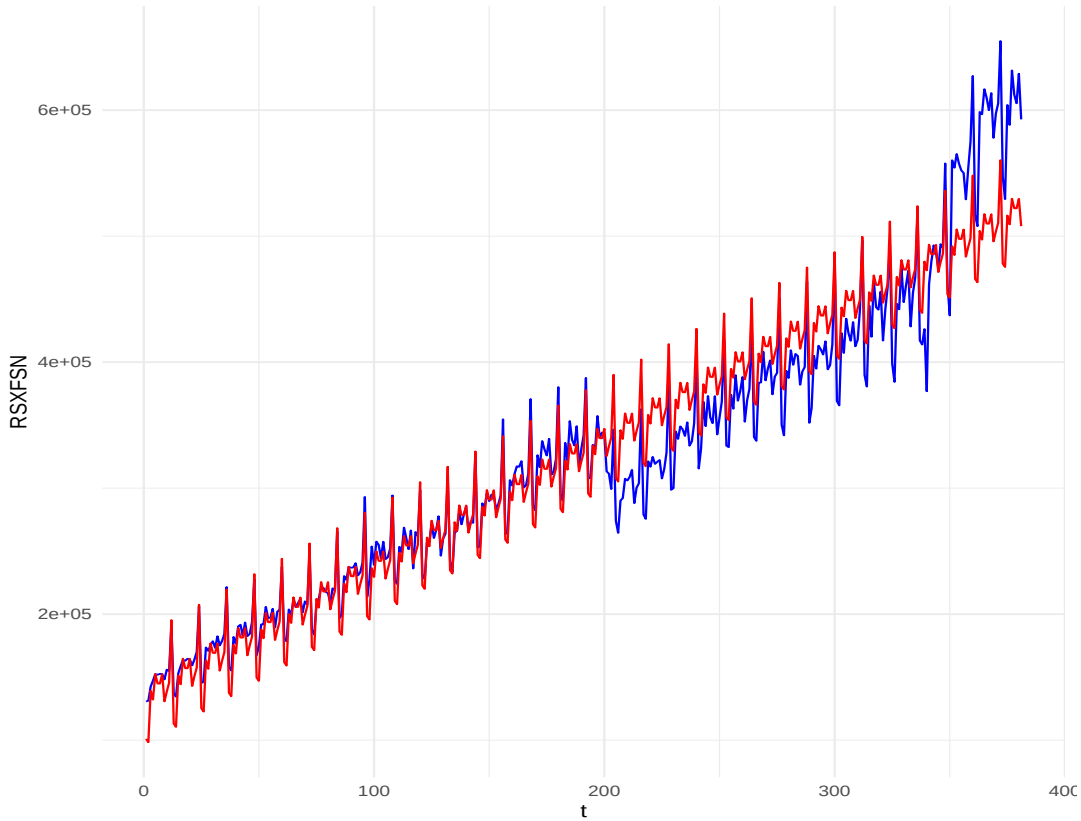
##
## Call:
## lm(formula = RSXFSN ~ t + mois, data = retail)
##
## Residuals:
##    Min     1Q Median     3Q    Max
## -95922 -22769    753  10775 101781
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept) 99954.45    6105.34  16.372  < 2e-16 ***
## t            1013.92      14.43  70.249  < 2e-16 ***
## moisfévr    -3682.36    7744.75  -0.475 0.634738
## moismars    36178.00    7744.79   4.671 4.20e-06 ***
## moisavril   28100.95    7744.86   3.628 0.000326 ***
## moismai     47512.13    7744.96   6.135 2.21e-09 ***
## moisjuin    38870.58    7745.08   5.019 8.12e-07 ***
## moisjuil    37742.22    7745.23   4.873 1.64e-06 ***
## moisaoût    44347.86    7745.40   5.726 2.14e-08 ***
## moissept    21660.47    7745.60   2.796 0.005437 **
## moisoct     28471.16    7807.07   3.647 0.000304 ***
## moisnov     34316.47    7807.16   4.396 1.45e-05 ***
## moisdéc     83196.71    7807.28  10.656  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 30980 on 368 degrees of freedom
## Multiple R-squared:  0.9332, Adjusted R-squared:  0.931
## F-statistic: 428.3 on 12 and 368 DF,  p-value: < 2.2e-16
```

Désaisonnalisation

Données retail (3)

- Données originales (bleu) et valeurs prédites par le modèle (rouge)

```
retail$prediction <- predict.lm(modst)
ggplot(retail)+
  geom_line(mapping=aes(x=t,y=RSXFSN),color="blue")+
  geom_line(mapping=aes(x=t,y=prediction), color="red")
```



Désaisonnalisation

Données retail : Série CVS (1)

- Creation des variables indicatrices pour le mois (‘dummies‘)

```
retail_1992_2022 <- retail %>% filter(year(DATE)<2023)
annees = nrow(retail_1992_2022)/12
t=1:annees

for (i in 1:12)
{
  su=rep(0,times=12)
  su[i]=1
  s=rep(su,times=annees)
  assign(paste("s",i,sep=""),s)
}
cbind(retail_1992_2022[,"RSXFSN"],s1,s2,s3,s4,s5,s6,s7,s8,s9,s10,s11,s12)[1:12,]
```

```
##              s1 s2 s3 s4 s5 s6 s7 s8 s9 s10 s11 s12
##  [1,] 130683  1  0  0  0  0  0  0  0  0   0   0   0
##  [2,] 131244  0  1  0  0  0  0  0  0  0   0   0   0
##  [3,] 142488  0  0  1  0  0  0  0  0  0   0   0   0
##  [4,] 147175  0  0  0  1  0  0  0  0  0   0   0   0
##  [5,] 152420  0  0  0  0  1  0  0  0  0   0   0   0
##  [6,] 151849  0  0  0  0  0  1  0  0  0   0   0   0
##  [7,] 152586  0  0  0  0  0  0  1  0  0   0   0   0
##  [8,] 152476  0  0  0  0  0  0  0  1  0   0   0   0
##  [9,] 148158  0  0  0  0  0  0  0  0  1   0   0   0
## [10,] 155987  0  0  0  0  0  0  0  0  0   1   0   0
## [11,] 154824  0  0  0  0  0  0  0  0  0   0   1   0
## [12,] 191347  0  0  0  0  0  0  0  0  0   0   0   1
```

Désaisonnalisation

Données retail : Série CVS (1)

- Pour obtenir les données CVS on extrait les coefficients

```
coefst <- modst$coefficients
coefst
```

```
## (Intercept)           t    moisfévr    moismars   moisavril     moismai
##   99954.447    1013.922   -3682.359   36178.000   28100.954   47512.126
##    moisjuin    moisjuil    moisaoût    moissept     moisoct     moisnov
##   38870.579   37742.220   44347.860   21660.470   28471.164   34316.468
##     moisdéc
##   83196.708
```

- On soustrait ensuite les coefficients saisonniers à la série originale

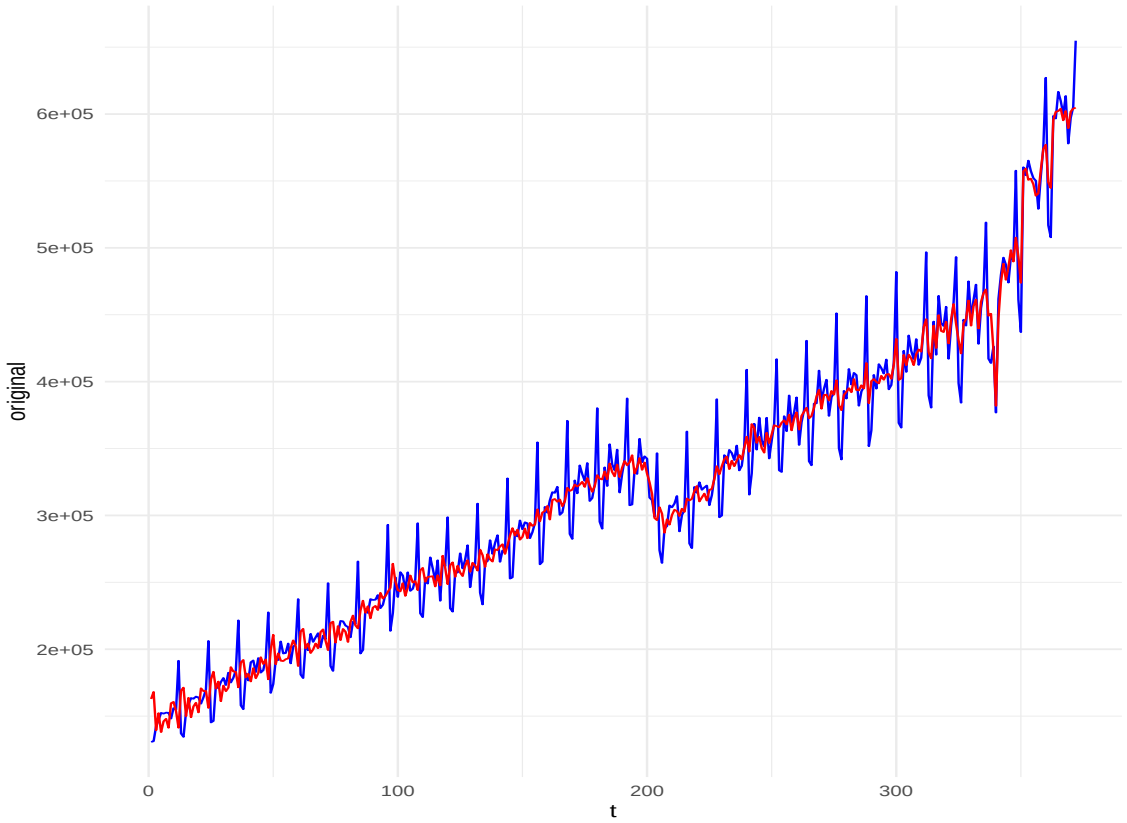
```
a <- mean(coefst[2:13])
b <- coefst[1]
c <- coefst[2:13]-mean(coefst[2:13])
y_cvs <- retail_1992_2022$RSXFSN-(c[1]*s1+c[2]*s2+c[3]*s3+c[4]*s4+c[5]*s5+c[6]*s6+
                                  c[7]*s7+c[8]*s8+c[9]*s9+c[10]*s10+c[11]*s11+c[12]*s12)
```

Désaisonnalisation

Données retail : Série CVS (2)

```
gdata <- tibble(t=retail_1992_2022$t,
                original=retail_1992_2022$RSXFSN, cvs = y_cvs)

ggplot(gdata)+
  geom_line(mapping=aes(x=t,y=original),color="blue")+
  geom_line(mapping=aes(x=t,y=cvs), color="red")
```



Désaisonnalisation

La fonction `tslm()` (1)

- On peut également utiliser la fonction `tslm()` de la librairie `forecast`
- Cette fonction ajuste un modèle linéaire incluant la saisonalité et la tendance (et éventuellement la tendance au carré), à partir d'un objet de classe `ts`
- Modèle linéaire avec saisonnalité et tendance - données `retail`

```
library(forecast)
bhat <- tslm(retail_ts ~ trend + season)
```

Désaisonnalisation

La fonction `tslm` (2)

Résultats du modèle

```
summary(bhat)
```

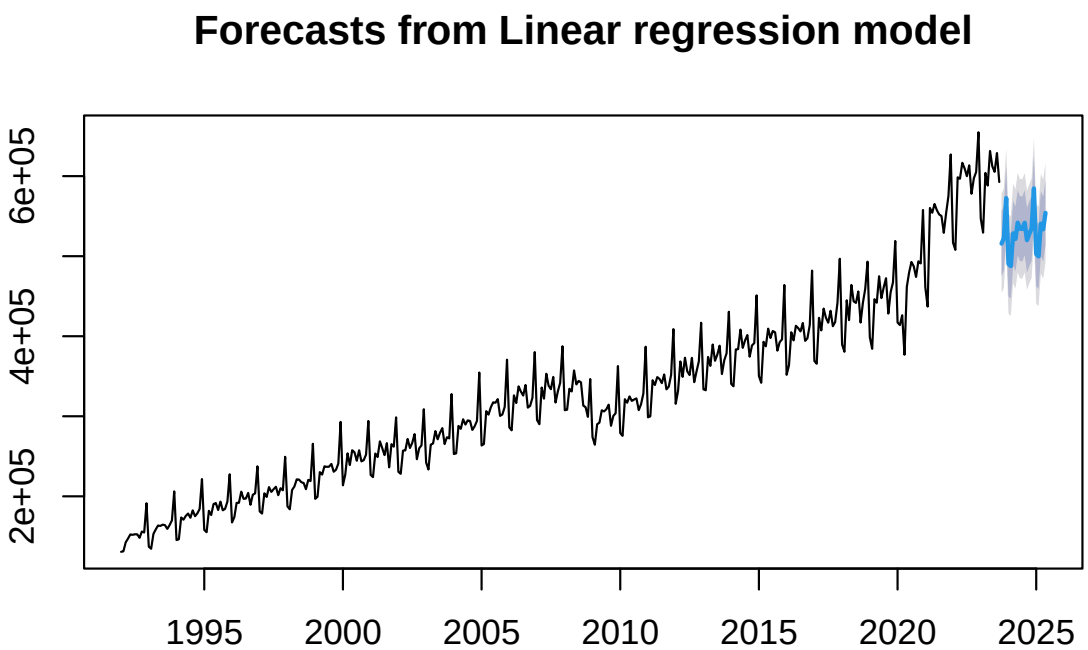
```
##
## Call:
## tslm(formula = retail_ts ~ trend + season)
##
## Residuals:
##    Min     1Q Median     3Q    Max
## -95922 -22769    753  10775 101781
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept) 99954.45    6105.34  16.372  < 2e-16 ***
## trend        1013.92      14.43  70.249  < 2e-16 ***
## season2     -3682.36    7744.75  -0.475 0.634738
## season3     36178.00    7744.79   4.671 4.20e-06 ***
## season4     28100.95    7744.86   3.628 0.000326 ***
## season5     47512.13    7744.96   6.135 2.21e-09 ***
## season6     38870.58    7745.08   5.019 8.12e-07 ***
## season7     37742.22    7745.23   4.873 1.64e-06 ***
## season8     44347.86    7745.40   5.726 2.14e-08 ***
## season9     21660.47    7745.60   2.796 0.005437 **
## season10    28471.16    7807.07   3.647 0.000304 ***
## season11    34316.47    7807.16   4.396 1.45e-05 ***
## season12    83196.71    7807.28  10.656  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 30980 on 368 degrees of freedom
## Multiple R-squared:  0.9332, Adjusted R-squared:  0.931
## F-statistic: 428.3 on 12 and 368 DF,  p-value: < 2.2e-16
```

Désaisonnalisation

La fonction `tslm()` (3)

- La fonction `forecast()` permet d'obtenir une prévision

```
plot(forecast(bhat, h=20))
```



8 Séries temporelles non-stationnaires

Stationnarité des séries macroéconomiques

- De nombreuses séries temporelles macroéconomiques ne sont pas stationnaires
- Il existe différents types de non-stationnarité
- C'est l'objet notamment de l'article de [26], qui arguent que les séries temporelles macroéconomiques sont plus souvent caractétisées par une non-stationnarité de type stochastique (unit-root nonstationarity) que par une tendance déterministe [17, p. 164]

Tendance déterministe et tendance stochastique (1)

- On consière le processus y_t comme la réalisation d'une tendance déterministe et d'un composant stochastique

$$y_t = TD_t + z_t$$

où TD_t désigne la tendance déterministe $TD_t = \beta_1 + \beta_2 t$ et z_t désigne le composant stochastique sous la forme d'un processus ARMA

- On distingue deux cas :
  1. Le polynome autorégressif ne comporte pas de racine unitaire : y_t est stationnaire autour d'une tendance déterministe (trend-stationary), on peut éliminer la tendance de la série originale et ajuster un modèle $ARMA(p, q)$ sur les résidus
  2. Le polynome autorégressif comporte une racine unitaire : $\Delta z_t = (1 - L)z_t$ est stationnaire autour d'une moyenne constante, la série y_t est stationnaire par différence (difference-stationary)

Tendance déterministe et tendance stochastique (2)

- Différence entre un processus stationnaire à une tendance près (trend stationnary) et un processus stationnaire par différence :

$$y_t = y_{t-1} + \mu = y_0 + \mu t \quad (1)$$

$$y_t = y_{t-1} + \epsilon_t = y_0 + \sum_{s=1}^{t} \epsilon_s \quad (2)$$

où μ est une constante et ϵ_t un bruit blanc

- Dans l'equation (1), y_t est representé par une tendance **déterministe**
- Dans l'équation (2) la série est expliquée par ses chocs passés, i.e. une tendance **stochastique**

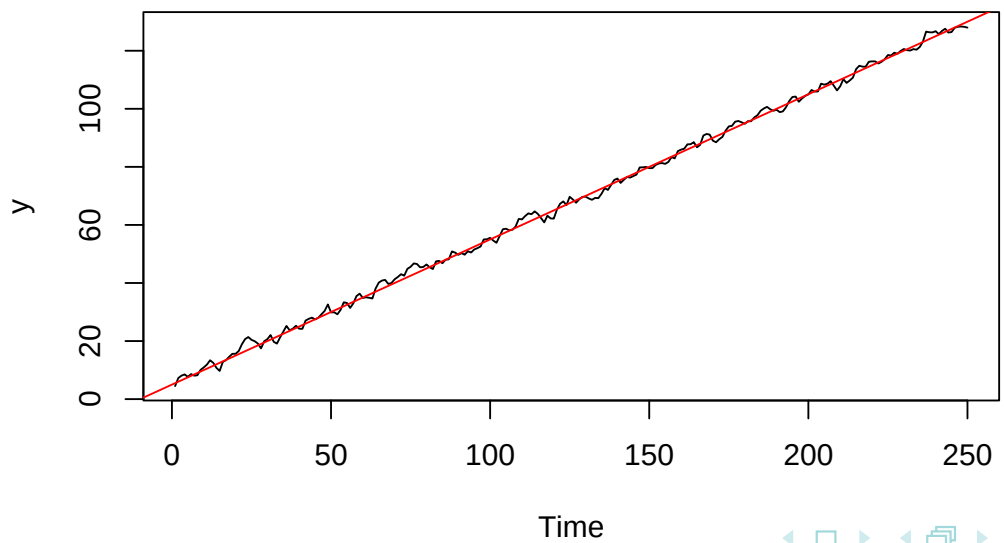
Tendance déterministe et tendance stochastique

Tendance déterministe : Simulation (1)

- Simulation d'une série présentant une **tendance déterministe** (trend stationary time series, exemple tiré de [27])
- La série est la combinaison d'une fonction déterministe du temps et d'un processus stationnaire (auto-régressif)

```
set.seed(12345)
y.tsar2 <- 5 + 0.5 * seq(250) +
  arima.sim(list(ar = c(0.8, -0.2)), n = 250)

plot(y.tsar2, ylab="y", xlab = "Time")
abline(a=5, b=0.5, col = "red")
```



Tendance déterministe et tendance stochastique

Tendance stochastique : Simulation (1)

- Simulation d'une série présentant une **tendance stochastique** (difference stationary time series, exemple tiré de [27])
- La série `y1` est l'accumulation d'erreurs passées
- La série `y1.d` inclu une **dérive**

```
set.seed(12345)
u.ar2 <- arima.sim(list(ar = c(0.8, -0.2)), n = 250)
y1 <- cumsum(u.ar2)
TD <- 5.0 + 0.7 * seq(250)
y1.d <- y1 + TD
tstoch <- ts(data.frame(nodrift=y1, drift=y1.d))
head(tstoch)
```

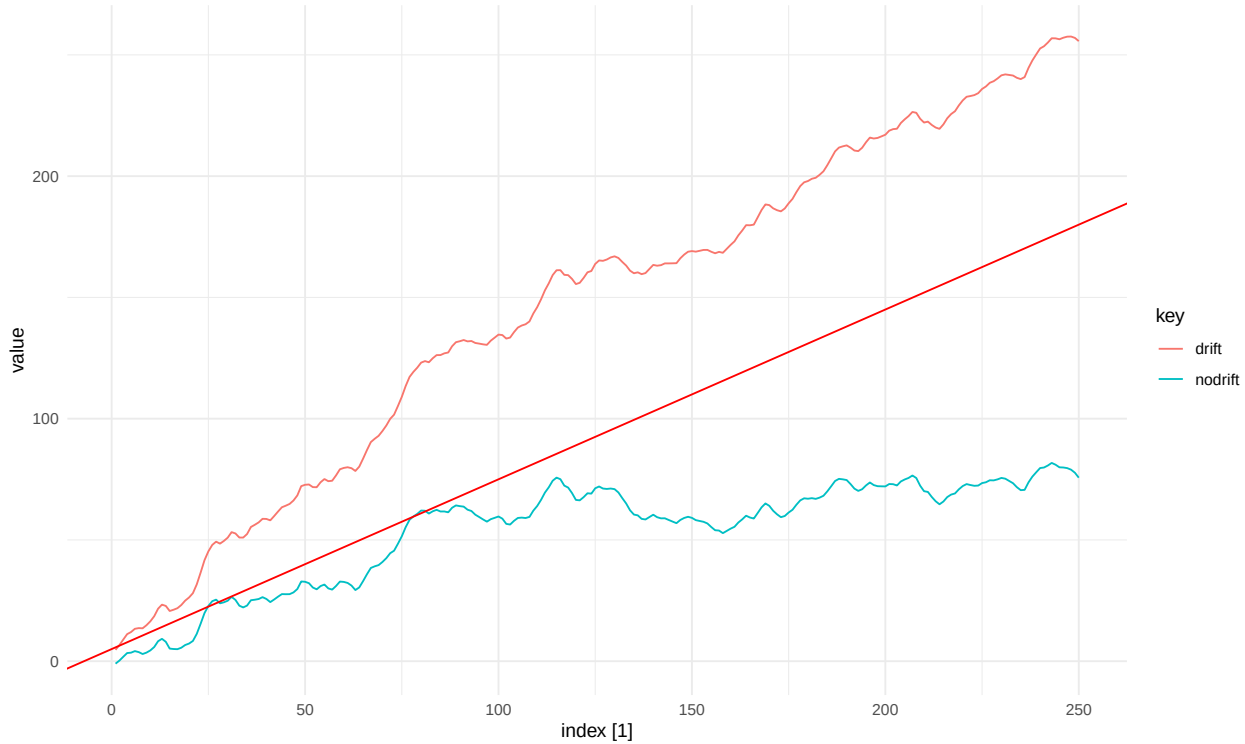
```
## Time Series:
## Start = 1
## End = 6
## Frequency = 1
##     nodrift     drift
## 1 -1.014384  4.685616
## 2  0.243365  6.643365
## 3  1.823069  8.923069
## 4  3.355498 11.155498
## 5  3.514969 12.014969
## 6  4.152960 13.352960
```

Tendance déterministe et tendance stochastique

Tendance stochastique : Simulation (2)

- Séries présentant une tendance stochastique, avec (`drift`) ou sans (`nodrift`) dérive

```
as_tsibble(tstoch) %>% autoplot(value) +
  geom_abline(intercept=5, slope=0.7, col = "red")
```



Stationarisation

- Une série présentant une tendance et/ou une saisonnalité ne pourra pas être modélisée par un processus stationnaire
- Soit le processus $(X_t)_{t\in\mathbf{Z}}$ vérifiant :

$$X_t = at + b + \epsilon_t$$

avec $a \neq 0$

- On a :

$$\mathbf{E}(X_t) = \mathbf{E}(at + b + \epsilon_t) = at + b$$

et $Cov(X_t, X_{t-h}) = \sigma^2$ si $h = 0$ et $Cov(X_t, X_{t-h}) = 0$ si $h \neq 0$

- Le processus $(X_t)_{t\in Z}$ n'est pas stationnaire car $E(X_t)$ dépend de t

Stationarisation par différenciation (1)

- Si on considère maintenant le processus $Y_t = X_t - X_{t-1}$, on a :

$$\begin{aligned} Y_t &= X_t - X_{t-1} \\ Y_t &= at + b + \epsilon_t - a(t-1) + b + \epsilon_{t-1} \\ &= a + \epsilon_t - \epsilon_{t-1} \end{aligned}$$

  et $\mathbf{E}(Y_t) = a$

- La covariance $Cov(Y_t, Y_{t-h})$ est également constante dans le temps
- On obtient un processus stationnaire Y_t par **différenciation** de X_t
- Une série est dite **intégrée** d'ordre d, notée $I(d)$, s'il faut la différencier d fois pour obtenir une série stationnaire

Stationarisation par différentiation

Données Nelson-Plosser, indice des prix à la consommation (1)

- La fonction `diff()` permet d'obtenir la série intégrée, ici la série `cpi` des données Nelson-Plosser
- La série intégrée débute au temps $t + 1$

```
cpi_diff <- diff(NelPlo[,"cpi"],1)
window(cpi_diff, start=1861, end=1900)
```

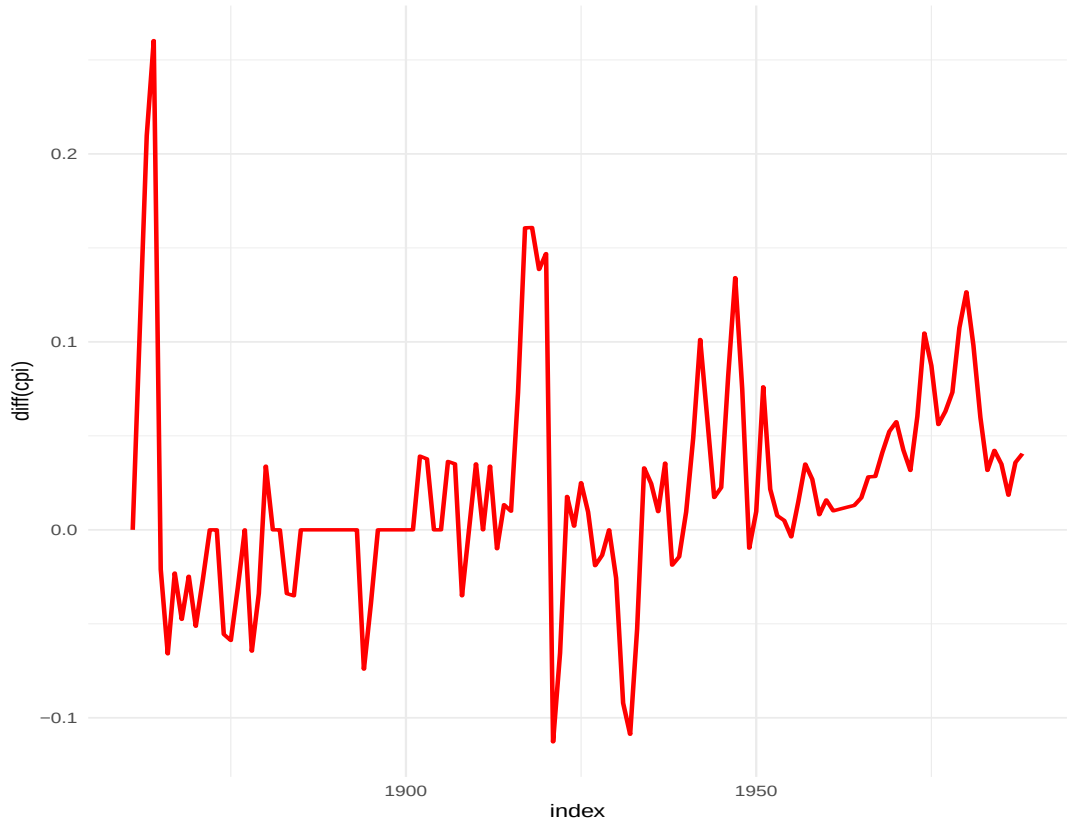
```
## Time Series:
## Start = 1861
## End = 1900
## Frequency = 1
##  [1]  0.0000000  0.1053605  0.2097205  0.2602831 -0.0210534 -0.0659580
##  [7] -0.0229895 -0.0476280 -0.0246926 -0.0512933 -0.0266683  0.0000000
## [13]  0.0000000 -0.0555698 -0.0588405 -0.0307717  0.0000000 -0.0645385
## [19] -0.0339016  0.0339016  0.0000000  0.0000000 -0.0339016 -0.0350913
## [25]  0.0000000  0.0000000  0.0000000  0.0000000  0.0000000  0.0000000
## [31]  0.0000000  0.0000000  0.0000000 -0.0741080 -0.0392207  0.0000000
## [37]  0.0000000  0.0000000  0.0000000  0.0000000
```

Stationarisation par différentiation

Données Nelson-Plosser, indice des prix à la consommation (2)

- Pour représenter la série on la transforme en objet `tsibble`

```
cpi_diff %>% as_tsibble() %>%
  ggplot(aes(x = index, y = value)) +
    geom_line(colour="red", size = 1) + ylab("diff(cpi)")
```



8 Séries temporelles non-stationnaires

- Test de non-stationnarité

Deux cas de non-stationnarité

- Avant de réaliser un test de non-stationnarité, il faut examiner le **chronogramme** de la série pour voir si la série présente une **tendance**
  - soit stochastique (marche aléatoire avec dérive),
  - soit déterministe (série stationnaire à une tendance déterministe près)
- La série y_t est stationnaire autour d'une tendance déterministe (pas de racine unitaire) : on peut éliminer la tendance de la série originale y_t et ajuster un modèle $ARMA(p, q)$ sur les résidus
- La série y_t est intégrée (d'ordre d) : la série est stationnaire par différence

Le test ADF (Augmented Dickey-Fuller)

- Les tests de racine unitaire (unit-root tests) permettent de tester la non-stationnarité d'une série
- Le test ADF (Augmented Dickey-Fuller) est le plus couramment utilisé
- Il est basé sur la régression (cf. [27])

$$\Delta y_t = \beta_1 + \beta_2 t + \pi y_{t-1} + \sum_{j=1}^{k} \gamma_j \Delta y_{t-j} + u_t \quad (1)$$

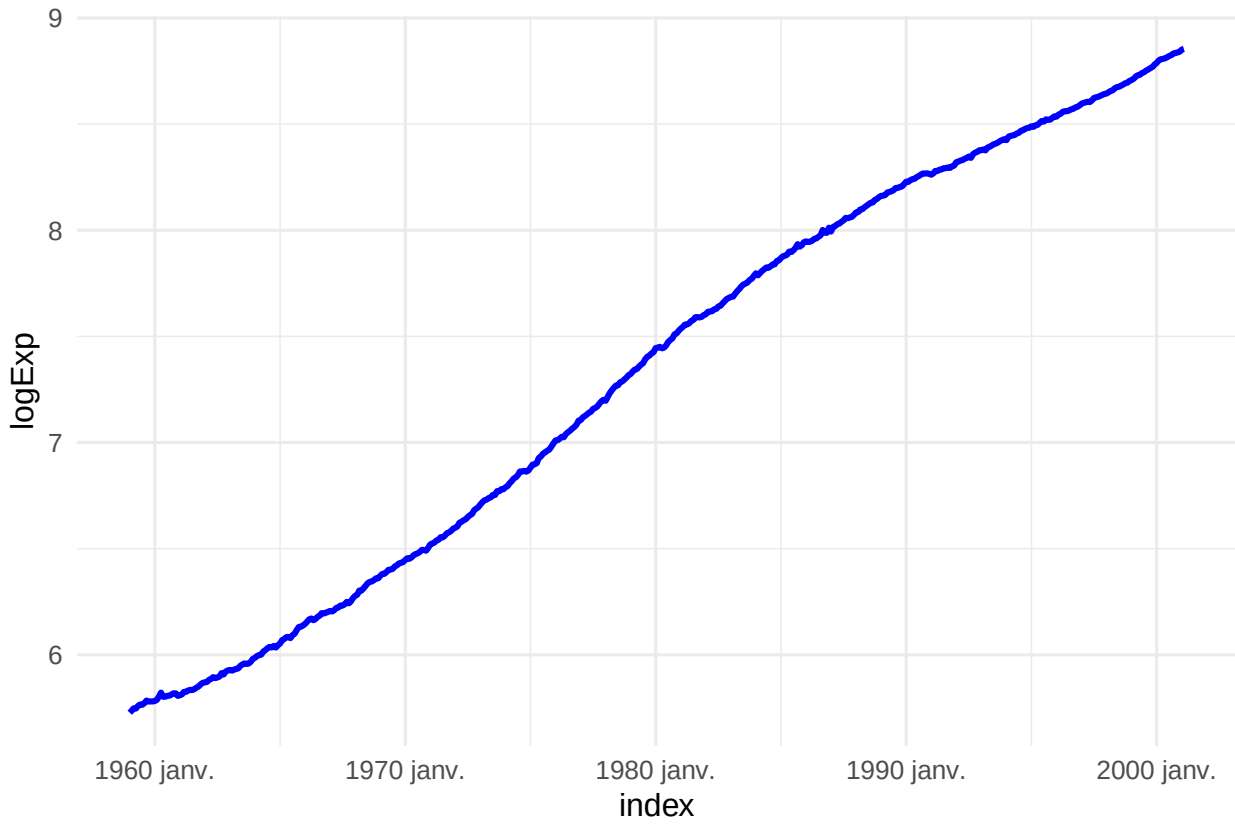
- Le test comporte plusieurs étapes (cf Pfaff-2008)
- On commence par tester l'hypothèse $H_0 : \pi = 0$ avec la statistique τ (tau) (les valeurs critiques sont fournies par Fuller (1976))
  - Si H_0 est rejettée, la procédure de test s'arrête

Dépenses de consommation aux USA

Données

- La série du log des dépenses de consommation aux USA présente une tendance

```
USExp <- USIncExp %>% as_tsibble() %>% filter(key=='expenditure') %>%
  mutate(logExp=log(value))
ggplot(USExp, aes(x = index, y = logExp)) +
  geom_line(color = "blue", size = 1)
```



Dépenses de consommation aux USA (USExp)

Test ADF (1)

- On utilise la fonction `ur.df` (librairie `urca`)
- Nous allons tester les hypothèses :
  - Hypothèse nulle H0 : la série est **non stationnaire avec dérive**
  - Hypothèse alternative H1 : la série est **stationnaire avec trend déterministe**
- Le test considère un modèle autorégressif d'ordre p (p repésenté par le lag est inconnu)
- On choisi pour commencer une valeur de p élevée et on retient la première valeur du lag fortement significative
- On choisit dans un premier temps p=12

```
library(urca)
exp.df <- ur.df(y=as.data.frame(USExp)[,"logExp"], lags=12, type='trend')
exp.df
```

```
##
## ###############################################################
## # Augmented Dickey-Fuller Test Unit Root / Cointegration Test #
## ###############################################################
##
## The value of the test statistic is: -0.4615 5.0639 0.7187
```

Dépenses de consommation aux USA (USExp)

Test ADF (1)

- La fonction `attr()` permet d'accéder aux éléments de l'objet `exp.df` contenant le résultat du test (`attributes(exp.df)` renvoie la liste des éléments)
- L'élément `testreg` contient le modèle de régression utilisé pour le test, et l'élément `call` de `testreg` contient la formule

```
names(attr(exp.df, "testreg"))
```

```
##  [1] "call"          "terms"         "residuals"     "coefficients"
##  [5] "aliased"       "sigma"         "df"            "r.squared"
##  [9] "adj.r.squared" "fstatistic"    "cov.unscaled"
```

```
attr(exp.df, "testreg")$call
```

```
## lm(formula = z.diff ~ z.lag.1 + 1 + tt + z.diff.lag)
```

Dépenses de consommation aux USA (USExp)

Test ADF (2)

- On extrait l'attribut `testreg`

```
attr(exp.df, "testreg")
```

```
##
## Call:
## lm(formula = z.diff ~ z.lag.1 + 1 + tt + z.diff.lag)
##
## Residuals:
##       Min         1Q     Median         3Q        Max
## -0.0208491 -0.0030259 -0.0000296  0.0034857  0.0233391
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept)   1.130e-02  1.613e-02   0.701 0.483735
## z.lag.1      -1.339e-03  2.901e-03  -0.461 0.644675
## tt            7.074e-06  1.965e-05   0.360 0.719044
## z.diff.lag1  -2.085e-01  4.567e-02  -4.565 6.37e-06 ***
## z.diff.lag2  -4.538e-02  4.627e-02  -0.981 0.327235
## z.diff.lag3  -1.030e-02  4.614e-02  -0.223 0.823428
## z.diff.lag4  -2.322e-03  4.589e-02  -0.051 0.959664
## z.diff.lag5   6.135e-02  4.562e-02   1.345 0.179309
## z.diff.lag6   1.505e-01  4.517e-02   3.332 0.000931 ***
## z.diff.lag7   1.621e-01  4.515e-02   3.590 0.000365 ***
## z.diff.lag8   1.202e-01  4.569e-02   2.631 0.008776 **
## z.diff.lag9   4.947e-02  4.593e-02   1.077 0.281906
## z.diff.lag10  3.670e-02  4.598e-02   0.798 0.425159
## z.diff.lag11  1.181e-01  4.592e-02   2.573 0.010389 *
## z.diff.lag12  4.532e-02  4.521e-02   1.002 0.316631
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.005535 on 478 degrees of freedom
## Multiple R-squared:  0.09946, Adjusted R-squared:  0.07308
## F-statistic: 3.771 on 14 and 478 DF,  p-value: 4.668e-06
```

Dépenses de consommation aux USA (USExp)

Test ADF (3)

- La première valeur p fortement significative est 8, on refait le test avec p=8

```
library(urca)
exp.df <- ur.df(y=as.data.frame(USExp)[,"logExp"], lags=8, type='trend')
```

- Valeur de la statistique du test

```
attr(exp.df, "teststat")
```

```
##                 tau3     phi2      phi3
## statistic -0.4287925 7.896003 0.3583672
```

- Valeurs critiques

```
attr(exp.df, "cval")
```

```
##       1pct  5pct 10pct
## tau3 -3.96 -3.41 -3.12
## phi2  6.09  4.68  4.03
## phi3  8.27  6.25  5.34
```

- La valeur de tau3 est supérieure au seuil de 10%, on retient l'hypothèse H0, la série ne présente pas de tendance déterministe et peut être rendue stationaire par différentiation

Dépenses de consommation aux USA (USExp)

Test ADF (4)

- Le modèle de régression utilisé pour le test est contenu dans l'attribut `testreg`

```
attr(exp.df, "testreg")
```

```
##
## Call:
## lm(formula = z.diff ~ z.lag.1 + 1 + tt + z.diff.lag)
##
## Residuals:
##        Min         1Q     Median         3Q        Max
## -0.0205542 -0.0030498 -0.0001604  0.0036908  0.0222845
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept)  1.117e-02  1.609e-02   0.694 0.487809
## z.lag.1     -1.236e-03  2.883e-03  -0.429 0.668264
## tt           7.040e-06  1.949e-05   0.361 0.718134
## z.diff.lag1 -1.886e-01  4.512e-02  -4.180 3.46e-05 ***
## z.diff.lag2 -2.393e-02  4.538e-02  -0.527 0.598191
## z.diff.lag3  2.050e-02  4.467e-02   0.459 0.646565
## z.diff.lag4  2.866e-02  4.455e-02   0.643 0.520301
## z.diff.lag5  8.980e-02  4.455e-02   2.016 0.044395 *
## z.diff.lag6  1.715e-01  4.466e-02   3.841 0.000139 ***
## z.diff.lag7  1.632e-01  4.530e-02   3.604 0.000346 ***
## z.diff.lag8  1.080e-01  4.500e-02   2.399 0.016807 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.005581 on 486 degrees of freedom
## Multiple R-squared:  0.08049, Adjusted R-squared:  0.06157
## F-statistic: 4.254 on 10 and 486 DF,  p-value: 1.076e-05
```

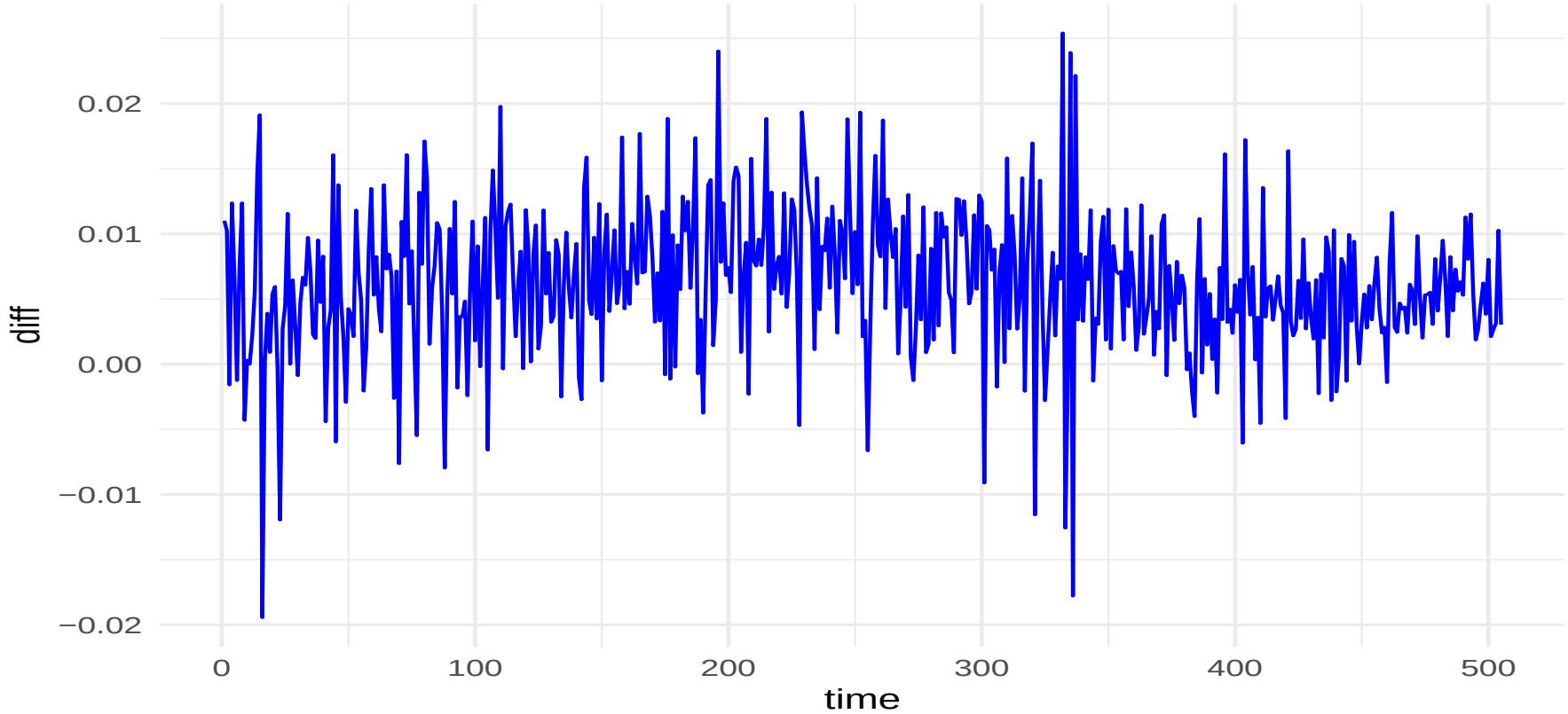
- On peut obtenir plus de détails avec la méthode générique `summary`

Dépenses de consommation aux USA (USExp)

Série différenciée

- On utilise la fonction `diff` pour obtenir la série intégrée

```
USExp.diff <- tibble(time=1:(nrow(USExp)-1), diff=diff(as.data.frame(USExp)[, "logExp"]))
ggplot(USExp.diff, aes(x = time, y = diff)) +
  geom_line(color = "blue", size = 0.5)
```



Dépenses de consommation aux USA (USExp)

Série différenciée : Test ADF

- On réalise un test ADF sur la série différenciée
  - Hypothèse H0 : la série présente une tendance stochastique (racine unitaire)
  - Hypothèse H1 : la série est stationnaire

```
USExp.diff.df <- ur.df(y=as.data.frame(USExp.diff)[, "diff"], lags=1, type='drift')
```

- Valeur de la statistique du test

```
attr(USExp.diff.df, "teststat")
```

```
##               tau2     phi1
## statistic -16.8418 141.8248
```

- Valeurs critiques

```
attr(USExp.diff.df, "cval")
```

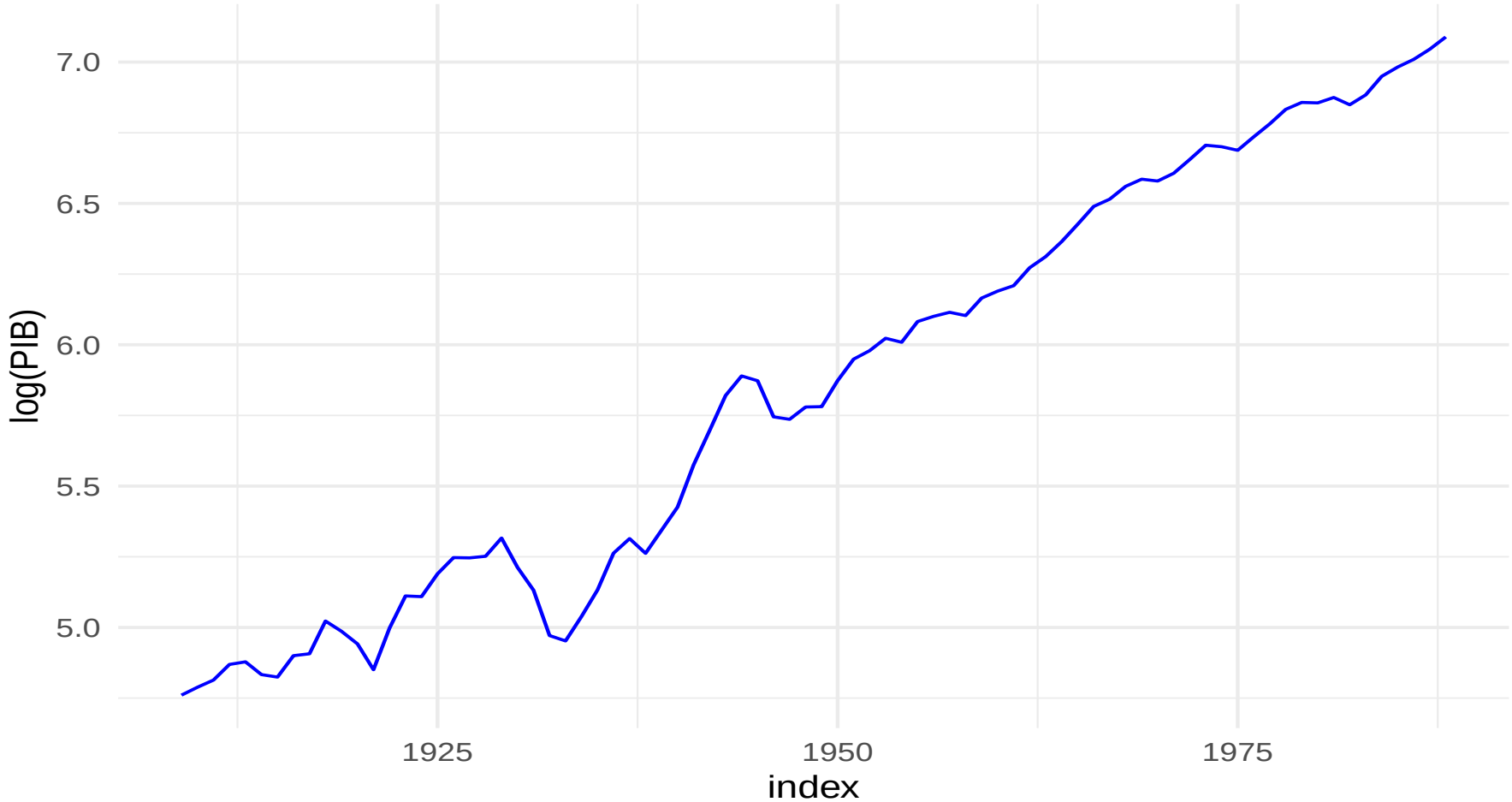
```
##       1pct  5pct 10pct
## tau2 -3.43 -2.86 -2.57
## phi1  6.43  4.59  3.78
```

- La valeur de tau2 est inférieure à la valeur critique à 1% donc on rejete H0 et on conclut à la stationnarité de la série

Données nelplo, série gnp.real

- La série du log du PIB des USA en milliards de dollars 1958 présente une tendance

```
nelplo_tsbl %>% filter(key=="gnp.real" & index>=1909) %>%
  ggplot(aes(x = index, y = value)) +
    geom_line(color='blue') + ylab("log(PIB)")
```



Données nelplo, série gnp.real

Test ADF

- On réalise un test ADF (on choisit dans un premier temps p=12) :
  - Hypothèse nulle H0 : la série est **non stationnaire avec dérive**
  - Hypothèse alternative H1 : la série est **stationnaire avec trend déterministe**
- La première valeur *p* fortement significative est 1, on refait le test avec p=1

```
gnpreal.df <- ur.df(y=as.data.frame(nelplo1909)[,"gnp.real"], lags=1, type='trend')
```

- Valeur de la statistique du test

```
attr(gnpreal.df, "teststat")
```

```
##                tau3     phi2   phi3
## statistic -3.454521 7.017575 6.0513
```

- Valeurs critiques

```
attr(gnpreal.df , "cval")
```

```
##       1pct  5pct 10pct
## tau3 -4.04 -3.45 -3.15
## phi2  6.50  4.88  4.16
## phi3  8.73  6.49  5.47
```

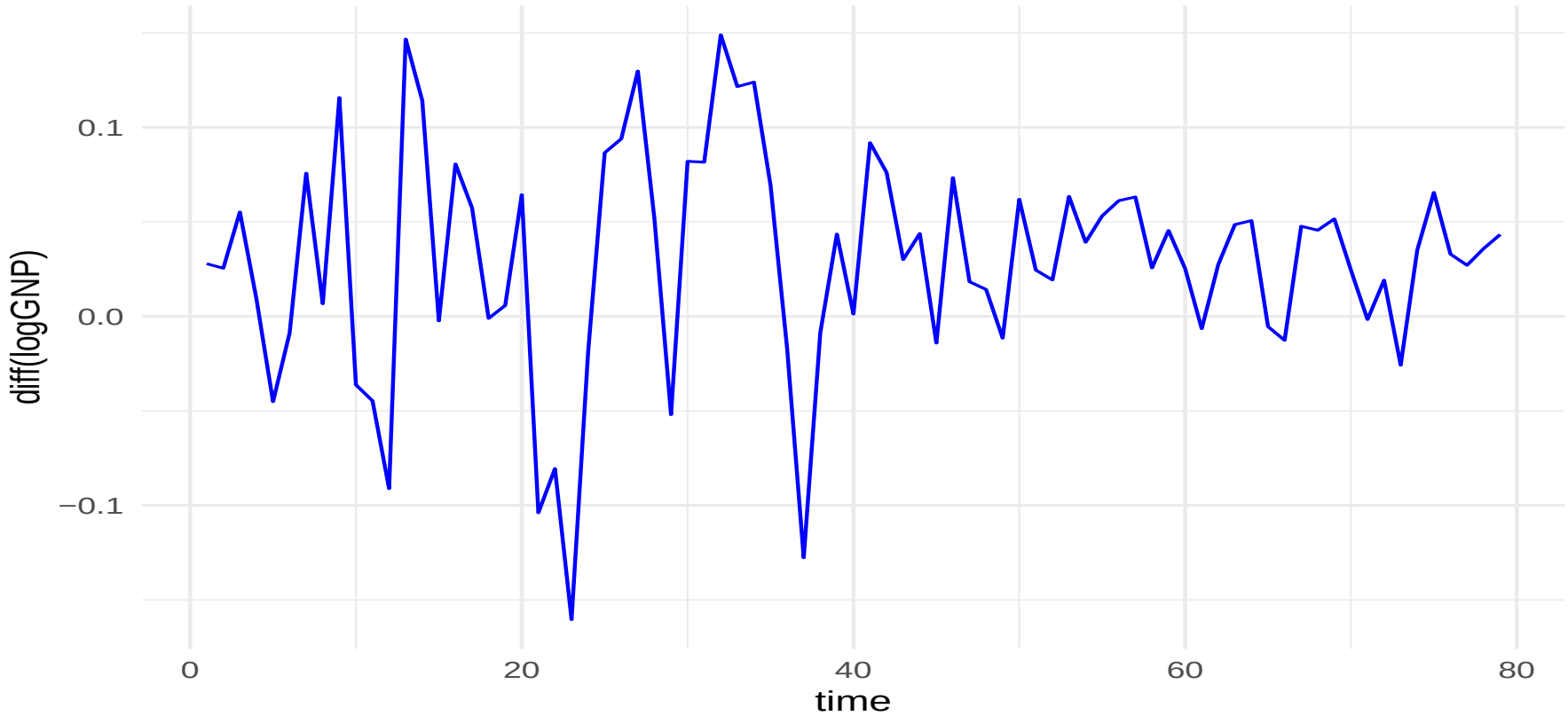
- La valeur de tau3 est supérieure au seuil de 5%, on retient l'hypothèse H0, la série ne présente pas de tendance déterministe et peut être rendue stationaire par différentiation

Données nelplo, série gnp.real

Série différenciée

- On utilise la fonction `diff()` pour obtenir la série intégrée

```
gnpreal.diff <- tibble(time=1:(nrow(nelplo1909)-1), diff= diff(as.data.frame(nelplo1909)$gnp.real))
ggplot(gnpreal.diff, aes(x = time, y = diff)) +
  geom_line(color = "blue", size = 0.5) + ylab("diff(logGNP)")
```



Test de non-stationnarité

Données `nelplo`, série différenciée `gnp.real`

- On réalise un test ADF sur la série différenciée
  - Hypothèse H0 : la série présente une tendance stochastique (racine unitaire)
  - Hypothèse H1 : la série est stationnaire

```
gnpreal.diff.df <- ur.df(y=as.data.frame(gnpreal.diff)[, "diff"], lags=1, type='drift')
```

- Valeur de la statistique du test

```
attr(gnpreal.diff.df, "teststat")
```

```
##                tau2    phi1
## statistic -5.471812 14.9712
```

- Valeurs critiques

```
attr(gnpreal.diff.df , "cval")
```

```
##       1pct  5pct 10pct
## tau2 -3.51 -2.89 -2.58
## phi1  6.70  4.71  3.86
```

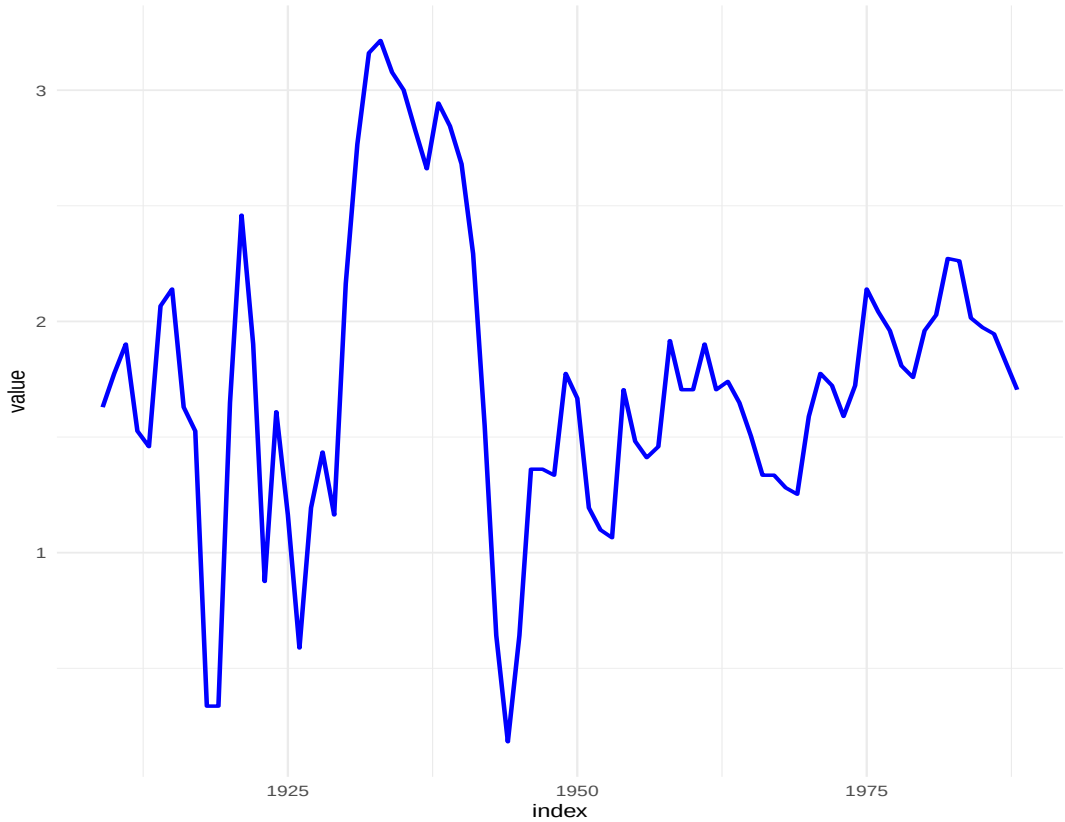
- La valeur de `tau2` est inférieure à la valeur critique à 1% donc on rejete H0 et on conclut à la stationnarité de la série

Test de non-stationnarité

Taux de chômage aux USA

- La série du taux de chômage aux USA ne présente pas de tendance

```
Unemp <- nelplo_tsbl %>% filter(key=='unemp' & index >= 1909)
ggplot(Unemp, aes(x = index, y = value)) +
  geom_line(color = "blue", size = 1)
```



Test de non-stationnarité

Taux de chômage aux USA : Test ADF (1)

- Après un premier test avec `lags=6`, le premier lag significatif est 1

```
unemp.df <- ur.df(y=as.data.frame(Unemp)[,"value"], lags=1, type='drift')
```

- Valeur de la statistique du test

```
attr(unemp.df, "teststat")
```

```
##                tau2     phi1
## statistic -3.909361 7.642016
```

- Valeurs critiques

```
attr(unemp.df, "cval")
```

```
##       1pct  5pct 10pct
## tau2 -3.51 -2.89 -2.58
## phi1  6.70  4.71  3.86
```

- La statistique tau2 est inférieure à la valeur critique à 1% donc on rejete H0 et on conclut à la stationnarité de la série

Test de non-stationnarité

Taux de chômage aux USA : Test ADF (2)

- On peut également utiliser la fonction `adf.test()` de la librairie `tseries`
- H0 : la série a une racine unitaire (unit root)
- La p-valeur est inférieure au seuil critique à 5%, on rejette H0 : la série est stationnaire

```
adf.test(as.data.frame(Unemp)[,"value"], k=1)
```

```
##
##  Augmented Dickey-Fuller Test
##
## data:  as.data.frame(Unemp)[, "value"]
## Dickey-Fuller = -3.8903, Lag order = 1, p-value = 0.01899
## alternative hypothesis: stationary
```

Test de stationnarité

Le test KPSS

- Le test KPSS proposé par Kwiatkowski est disponible dans les librairies `urca` et `tseries`
- Hypothèse nulle H0 : la série est stationnaire (soit à une tendance près, soit à une moyenne non nulle près)
- Le test suppose que la série est la somme d'une marche aléatoire R_t, d'un trend déterministe et d'une erreur stationnaire U_t :

$$y_t = R_t + \beta_1 + \beta_2 t + U_t$$

où $R_t = R_{t-1} + z_t$

- Pour tester que la série y_t est stationnaire à une tendance près, l'hypothèse nulle est $\sigma_z^2 = 0$

Test de stationnarité

Le test KPSS : Série USExp (1)

- On teste ici l'hypothèse H0 : la série des dépenses de consommation aux USA est stationaire à une tendance près (option `type="tau"`)

```
ktest <- ur.kpss(USExp$logExp, type="tau", lags="long")
summary(ktest)
```

```
##
## ########################
## # KPSS Unit Root Test #
## ########################
##
## Test is of type: tau with 17 lags.
##
## Value of test-statistic is: 0.488
##
## Critical value for a significance level of:
##                 10pct  5pct 2.5pct  1pct
## critical values 0.119 0.146  0.176 0.216
```

- La valeur de la statistique est nettement supérieure à la valeur critique au seuil de 1%, on rejette H0, la série ne présente pas de tendance linéaire

Test de stationnarité

Test KPSS : Série USExp (2)

- On teste maintenant l'hypothèse H0 : la série des dépenses de consommation aux USA est stationaire de moyenne constante (option `type="mu"`)

```
ktest <- ur.kpss(USExp$logExp, type="mu", lags="long")
summary(ktest)
```

```
##
## ########################
## # KPSS Unit Root Test #
## ########################
##
## Test is of type: mu with 17 lags.
##
## Value of test-statistic is: 2.9218
##
## Critical value for a significance level of:
##                 10pct  5pct 2.5pct  1pct
## critical values 0.347 0.463  0.574 0.739
```

- La valeur de la statistique est nettement supérieure à la valeur critique au seuil de 1%, on rejette H0, la série n'est pas stationnaire

Test de stationnarité

Test KPSS : Série USExp intégrée

- On teste maintenant la stationnarité de la série intégrée

```
ktest <- ur.kpss(USExp.diff$diff, type="mu", lags="long")
summary(ktest)
```

```
##
## ########################
## # KPSS Unit Root Test #
## ########################
##
## Test is of type: mu with 17 lags.
##
## Value of test-statistic is: 0.5706
##
## Critical value for a significance level of:
##                 10pct  5pct 2.5pct  1pct
## critical values 0.347 0.463  0.574 0.739
```

- La valeur de la statistique se situe entre les valeurs critiques à 2.5% et 5%, on ne peut pas conclure à la stationnarité de la série différenciée

Test de stationnarité

Test KPSS : Série USExp intégrée

- On teste la stationnarité de la série intégrée d'ordre 2

```
ktest <- ur.kpss(diff(USExp.diff$diff), type="mu", lags="long")
summary(ktest)
```

```
##
## ########################
## # KPSS Unit Root Test #
## ########################
##
## Test is of type: mu with 17 lags.
##
## Value of test-statistic is: 0.0213
##
## Critical value for a significance level of:
##                 10pct  5pct 2.5pct  1pct
## critical values 0.347 0.463  0.574 0.739
```

- La valeur de la statistique est inférieure à la valeur critique à 10%, on peut conclure à la stationnarité de la série différenciée d'ordre 2

Séries temporelles non-stationnaires

Exercices

1. Les séries du PIB français présentent-t-elle une tendance ?
   - Cette tendance est-elle déterministe ou stochastique ?
   - Appliquez le test de stationnarité approprié
   - Le cas échéant, differenciez la série pour la rendre stationnaire
2. Même question pour la série de l'index des prix à la consommation
3. Même question pour la série du taux de chômage

9 Modélisation de séries temporelles stationnaires

Introduction

- Les modèles AR (AutoRegressive), MA (Moving Average) et ARMA (AutoRegressive Moving Average) sont des modèles fondamentaux pour étudier et décrire le comportement des séries temporelles
- Ces modèles ont été popularisés par George Box et Gwilym Jenkins
- Leur principale limitation est qu'ils ne peuvent modéliser que des séries stationnaires, cependant, on peut transformer des séries non-stationnaires par différenciation pour les étudier avec ce cadre (modèles ARIMA)

9 Modélisation de séries temporelles stationnaires
- Processus autorégressif (AR)
- Processus moyenne mobile
- Processus ARMA
- Prévision

Processus autorégressif $AR(1)$ stationnaire

- Dans un modèle auto régressif, les variables explicatives sont des valeurs passées (lags) de la variable expliquée
- Dans un modèle AR(1) (autorégressif d'ordre 1), y est retardé d'une période

$$y_t = \alpha + \phi_1 \cdot y_{t-1} + \epsilon_t$$

avec $\epsilon_t \sim N(0, \sigma^2)$

- La valeur de $|\phi_1|$ détermine l'évolution du processus
- Si $|\phi_1| < 1$, le processus AR(1) est **stationnaire**
- Si $|\phi_1| \geq 1$, le processus est **non-stationnaire** (les chocs s'accumulent dans le temps)
  - Si $|\phi_1| > 1$ le processus croît sans limite
  - Si $|\phi_1| = 1$ le processus a une **racine unitaire** (unit root), c'est une marche aléatoire

Processus autorégressif $AR(1)$ stationnaire

Propriétés

- Propriétés d'un processus $AR(1)$ stationnaire :
  - Moyenne de y_t

$$\mu = \frac{\alpha}{1 - \phi_1}$$

  - Variance :

$$Var(y_t) = \frac{\sigma_w^2}{1 - \phi_1^2}$$

  - Corrélation :

$$\rho_h = \phi_1^h$$

- Si $(Y_t)_{t \in \mathbb{Z}}$ est un processus $AR(p)$ ses **autocorrélations partielles** s'annulent à partir du rang $p + 1$
- Les **autocorrélations simples** décroissent rapidement vers 0 (de manière exponentielle ou sinusoïdale amortie)

Processus autorégressif $AR(1)$ stationnaire

Simulation (1)

- On utilise la fonction `arima.sim()` pour simuler un processus $AR(1)$ avec $\phi_1 = 0.9$ (cf. [27])
- Le bruit blanc gaussien `wn` représente les innovations
- La fonction `set.seed()` permet d'initialiser le générateur de nombres aléatoires (la série de 100 nombres aléatoires sera toujours la même)

```
set.seed(13)
wn <- rnorm(100)
AR1sim <- arima.sim(n = 100, list(ar = 0.9), innov=wn)
```

- Les corrélations partielles s'annulent bien à partir du rang 2

```
pacf(AR1sim, plot=FALSE)
```

```
##
## Partial autocorrelations of series 'AR1sim', by lag
##
##      1      2      3      4      5      6      7      8      9     10     11
##  0.784 -0.271 -0.132 -0.116  0.005 -0.094  0.152 -0.007  0.028 -0.011 -0.194
##     12     13     14     15     16     17     18     19     20
##  0.222 -0.106 -0.081 -0.024  0.038  0.016 -0.052 -0.091 -0.098
```

Processus autorégressif $AR(1)$ stationnaire

Simulation (2)

- Le code suivant permet de visualiser la série ainsi que ses autocorrélations (partielles)
- On utilise la foncion `layout()` pour définir l'emplacement des différents graphiques

```
op <- par(no.readonly=TRUE)

layout(matrix(c(1, 1, 2, 3), 2, 2, byrow=TRUE))

plot.ts(AR1sim, ylab="", main="Processus AR(1) avec B1=0.9")

acf(AR1sim, main="Autocorrelations", ylab="",
    ylim=c(-1, 1), ci.col = "black")

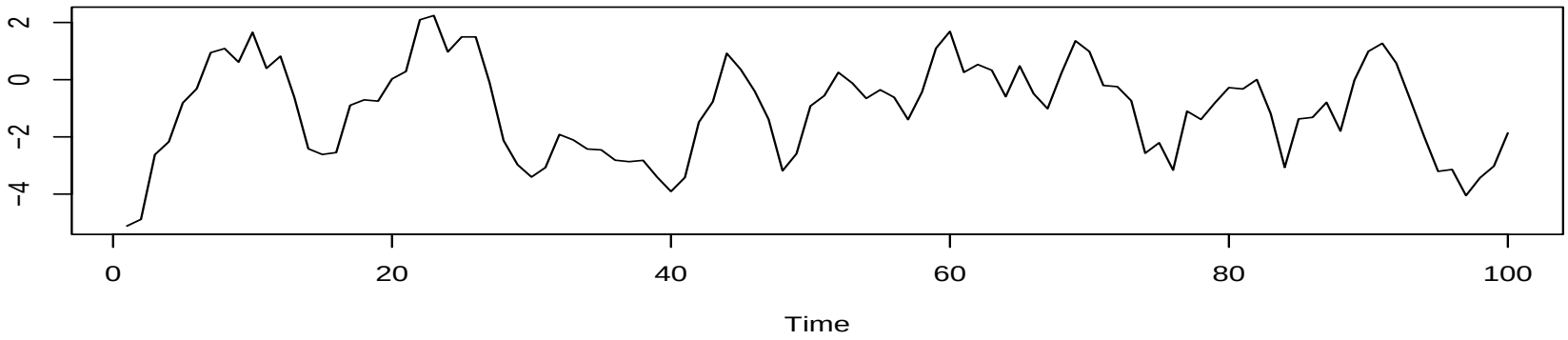
pacf(AR1sim, main="Partial Autocorrelations", ylab="",
     ylim=c(-1, 1), ci.col = "black")

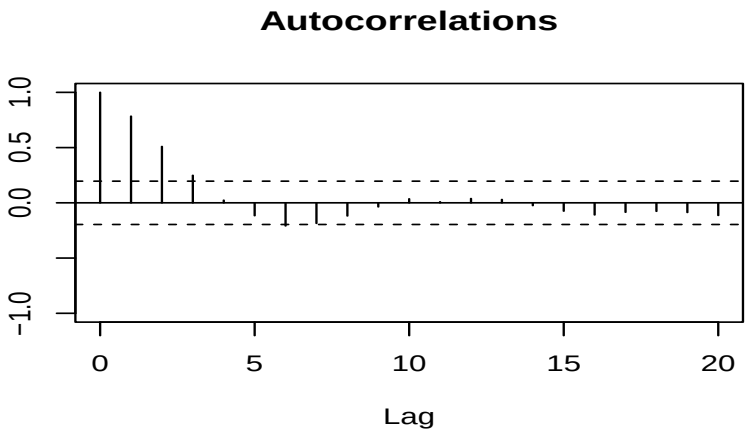
par(op)
```

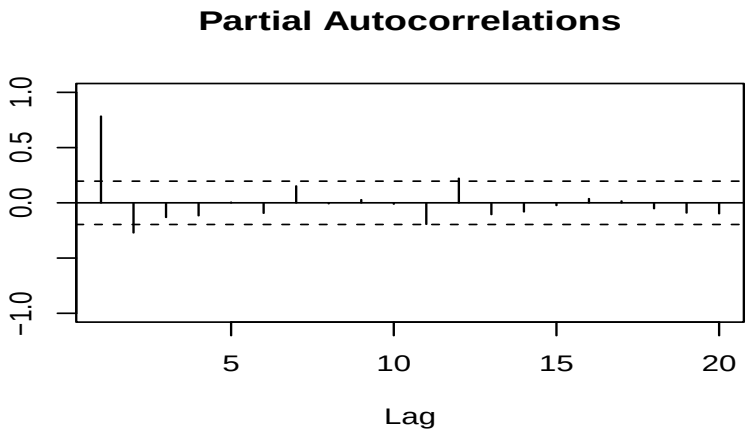
Processus autorégressif $AR(1)$ stationnaire

Simulation (3)

- On constate le déclin régulier des autocorrélations, et les autocorrélations partielles ne sont plus significatives pour $\ell > 1$





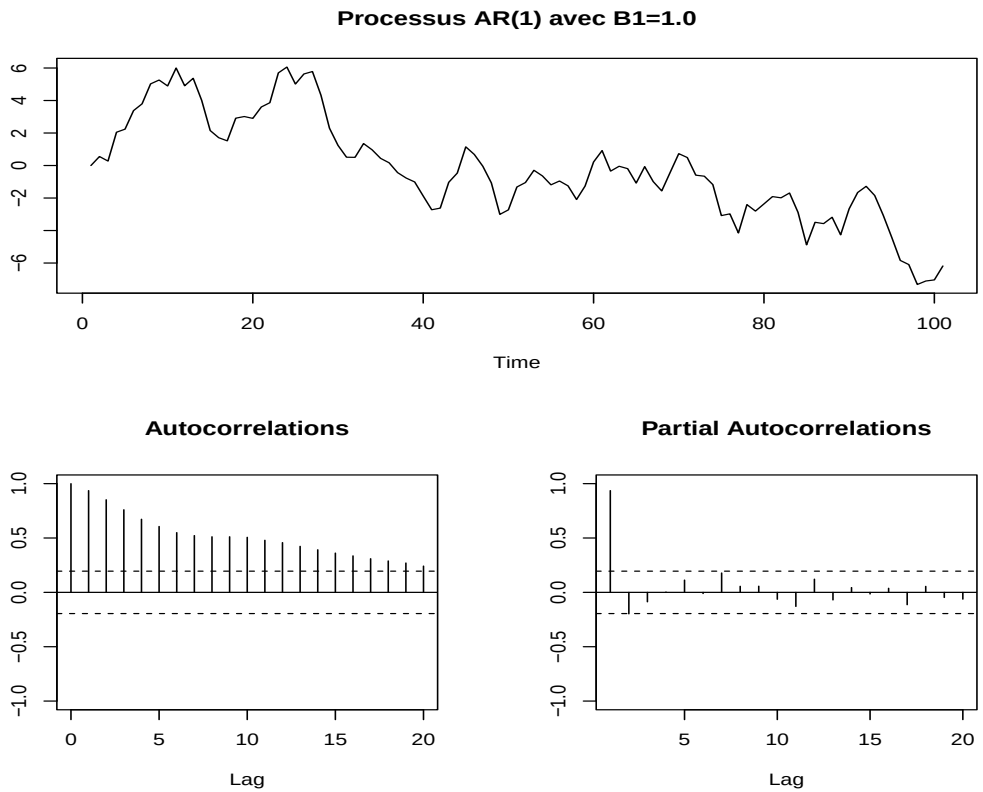


Processus autorégressif $AR(1)$ avec racine unitaire

Simulation

- Lorsque $\phi_1 = 1.0$ il s'agit d'un processus avec **racine unitaire**, i.e. une marche aléatoire (cf [3])
- La fonction `arima.sim()` produit une erreur si le paramètre `ar` est ≥ 1

```
rusim <- 0
for (i in 1:length(wn)) {rusim[i+1] <- rusim[i] + wn[i]}
```

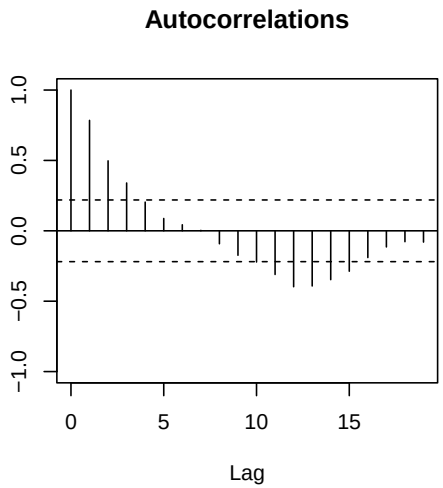


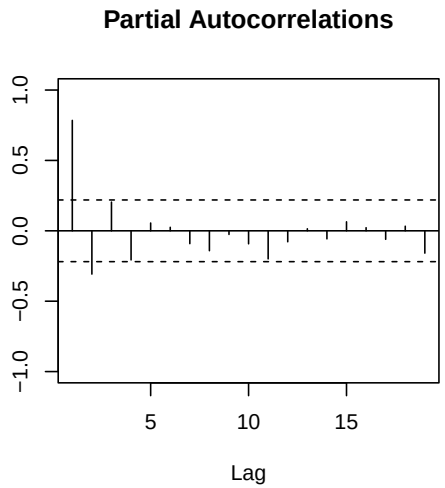
Modèle autorégressif

Données Nelson-Plosser : ACF et PACF

- On modélise le taux de chômage aux USA (données Nelson-Plosser)





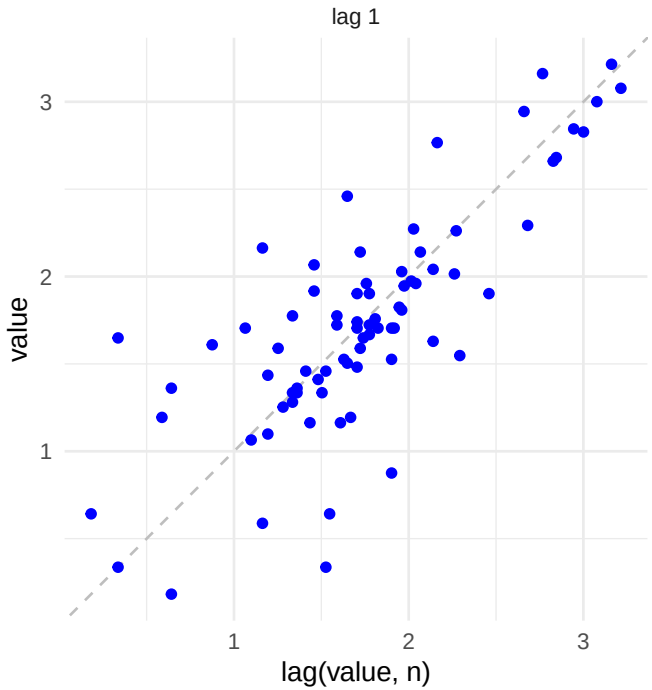


Modèle autorégressif

Données Nelson-Plosser

- On visualise y x y_{t-1} à l'aide de la fonction `gg_lag`

```
unemp1909p <- nelplo_tsbl %>% filter(index>1909 & key=='unemp')
unemp1909p %>% gg_lag(y=value, lags=1, geom="point", colour="blue")
```



Données Nelson-Plosser

Estimation du modèle $AR(1)$ avec `lm()` (1)

- On peut estimer le modèle $AR(1)$ avec les fonctions `lm` (linear model) et `lag` :

```
nelplo.lm <- lm(value ~ lag(value), data=unemp1909p)
summary(nelplo.lm)
```

```
##
## Call:
## lm(formula = value ~ lag(value), data = unemp1909p)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.23467 -0.16288 -0.01882  0.18815  1.01130
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)  0.37324    0.13168   2.834  0.00588 **
## lag(value)   0.78497    0.07107  11.046  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.4001 on 76 degrees of freedom
##   (1 observation effacée parce que manquante)
## Multiple R-squared:  0.6162, Adjusted R-squared:  0.6111
## F-statistic:   122 on 1 and 76 DF,  p-value: < 2.2e-16
```

- Le coefficient `lag(value)` associé à y_{t-1} est significatif

Données Nelson-Plosser

Estimation du modèle $AR(1)$ avec `lm()` (2)

- L'objet `nelplo.lm` contenant la régression linéaire est de classe `lm`

```
class(nelplo.lm)
```

```
## [1] "lm"
```

- La fonction `names()` permet d'obtenir la liste des éléments contenus dans l'objet `nelplo.lm`

```
names(nelplo.lm)
```

```
##  [1] "coefficients"  "residuals"     "effects"       "rank"
##  [5] "fitted.values" "assign"        "qr"            "df.residual"
##  [9] "na.action"     "xlevels"       "call"          "terms"
## [13] "model"
```

- Les estimations des coefficients sont contenues dans l'élément `coefficients`

```
nelplo.lm$coefficients
```

```
## (Intercept)  lag(value)
##   0.3732404   0.7849675
```

Données Nelson-Plosser

Estimation du modèle $AR(1)$ avec `arima()` (1)

- On peut également estimer le modèle $AR(1)$ avec la fonction `arima()` :

```
nelplo.ar1 <- arima(unemp1909p[,"value"], c(1,0,0))
summary(nelplo.ar1)
```

```
##
## Call:
## arima(x = unemp1909p[, "value"], order = c(1, 0, 0))
##
## Coefficients:
##          ar1  intercept
##       0.7756     1.7394
## s.e.  0.0680     0.1887
##
## sigma^2 estimated as 0.154:  log likelihood = -38.66,  aic = 83.32
##
## Training set error measures:
##                        ME      RMSE       MAE       MPE     MAPE      MASE
## Training set -0.0004783172 0.3924347 0.2859196 -14.03967 28.36622 0.9722669
##                    ACF1
## Training set 0.2482824
```

- Le coefficient `ar1` associé à y_{t-1} est significatif

Données Nelson-Plosser

Estimation du modèle $AR(1)$ avec `arima()` (2)

- Les estimations des coefficients sont contenues dans l'élément `coef`

```
names(nelplo.ar1)
```

```
##  [1] "coef"      "sigma2"    "var.coef"  "mask"      "loglik"    "aic"
##  [7] "arma"      "residuals" "call"      "series"    "code"      "n.cond"
## [13] "nobs"      "model"
```

```
nelplo.ar1$coef
```

```
##       ar1 intercept
## 0.7755955 1.7393568
```

Représentation $MA(\infty)$ d'un processus $AR(1)$

- Un processus $AR(1)$ stationnaire (covariance-stationary) peut-être représenté sous la forme d'une somme infinie des erreurs passées avec des poids diminuant dans le temps (voir par exemple [27, p.6])
- Soit le processus $AR(1)$

$$y_t = \alpha + \phi_1 \cdot y_{t-1} + \epsilon_t$$

- En utilisant l'opérateur lag L

$$(1 - \phi_1 L) y_t = \alpha + \epsilon_t$$

- La solution stable de ce processus est

$$y_t = (\alpha + \epsilon_t) + \phi_1(\alpha + \epsilon_{t-1}) + \phi_1^2(\alpha + \epsilon_{t-2}) + \phi_1^3(\alpha + \epsilon_{t-3}) + \dots$$

$$y_t = \frac{\alpha}{1 - \phi_1} + \epsilon_t + \phi_1 \epsilon_{t-1} + \phi_1^2 \epsilon_{t-2} + \phi_1^3 \epsilon_{t-3} + \dots$$

- On retouve la moyenne d'un processus $AR(1)$ $\mu = \frac{\alpha}{1-\phi_1}$

9 Modélisation de séries temporelles stationnaires

- Processus autorégressif (AR)
- Processus moyenne mobile
- Processus ARMA
- Prévision

Définition

- y_t est un processus moyenne mobile (Moving Average, MA) d'ordre q noté $MA(q)$ si

$$y_t = \mu + \epsilon_t + \theta_1 \cdot \epsilon_{t-1} + \ldots + \theta_q \cdot \epsilon_{t-q}$$

- On considère que le processus est la résultante d'une combinaison linéaire de perturbations decorrélées (un bruit blanc et son passé)
- Un $MA(q)$ est **toujours stationnaire** quelles que soient les valeurs des θ
- Les propriétés d'un modèle MA(1)

$$y_t = \mu + \epsilon_t + \theta_1 \cdot \epsilon_{t-1}$$

  - $E[y_t] = \mu$
  - $Var(y_t) = \sigma_\epsilon^2(1 + \theta_1^2)$
  - ACF : $\rho_1 = \theta_1/(1 + \theta_1^2)$ et $\rho_h = 0$ pour $h \geq 2$

Fonction d'autocorrélation

- Si $(X_t)_{t\in\mathbb{Z}}$ est un processus $MA(q)$, sa fonction d'autocorrélation est égale à 0 pour les rangs supérieurs à q
- Les autocorrélations observées sont un bon indicateur de l'ordre du processus
- Les autocorrélations partielles tendent vers 0 exponentiellement lorsque le rang augmente (comme les corrélations simples d'un processus AR) (cf par ex. [6])
- Plus généralement, on peut montrer que les autocorrélations partielles d'un modèle $MA(q)$ se comportent comme les autocorrélations d'un modèle $AR(q)$

Simulation (1)

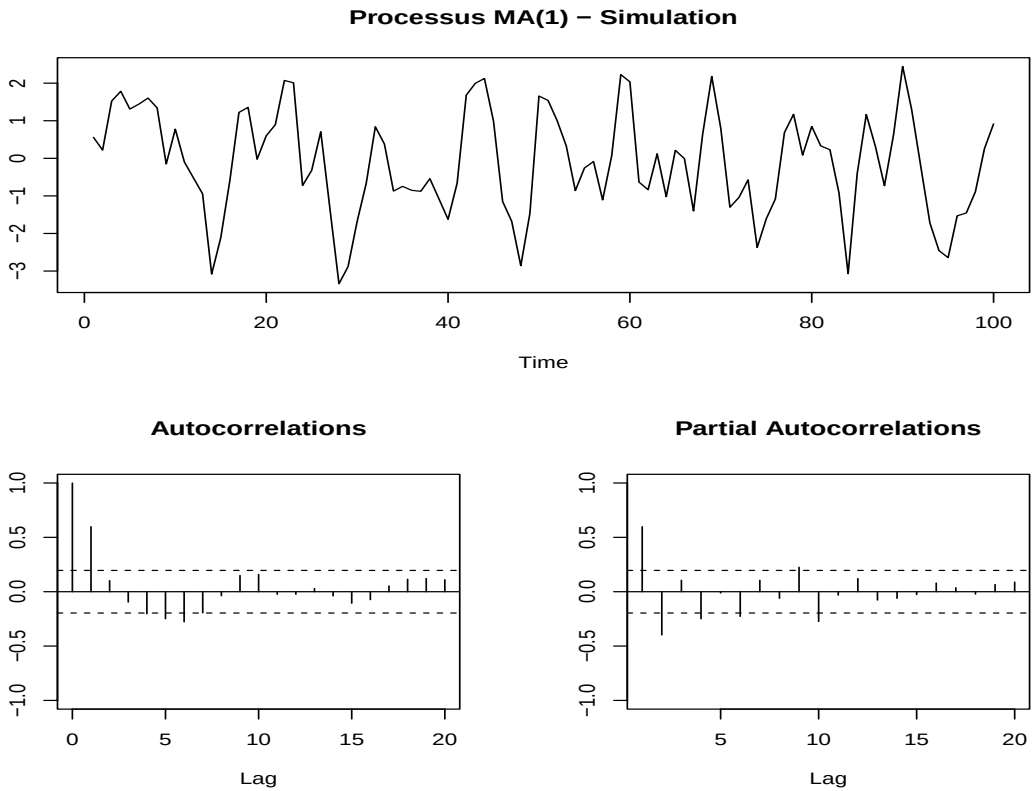
- En utilisant la même série `wn` de nombre aléatoires, on simule un processus $MA(1)$

```
MA1sim <- arima.sim(n = 100, list(ma = 0.9), innov=wn)
```

- Le code suivant permet de visualiser la série ainsi que ses autocorrélations (partielles)

```
op <- par(no.readonly=TRUE)
layout(matrix(c(1, 1, 2, 3), 2, 2, byrow=TRUE))
plot.ts(MA1sim, ylab=")
acf(MA1sim, main="Autocorrelations", ylab="", main="Processus MA(1) - Simulation", ylim=c(-1, 1), ci.col = "black") pacf(MA1sim
main="Partial Autocorrelations", ylab="", ylim=c(-1, 1), ci.col = "black")
par(op)
```

Simulation (2)



9 Modélisation de séries temporelles stationnaires
- Processus autorégressif (AR)
- Processus moyenne mobile
- Processus ARMA
- Prévision

Définition

- Un processus ARMA (Auto Regressive Moving Average) est une synthèse des processus AR et MA
- y_t obéit à un modèle ARMA(p, q) s'il est stationnaire et vérifie :

$$y_t = \alpha + \phi_1 \cdot y_{t-1} + \ldots + \phi_p \cdot y_{t-p} + \epsilon_t + \theta_1 \cdot \epsilon_{t-1} + \ldots + \theta_q \cdot \epsilon_{t-q}$$

Identification d'un modèle ARMA

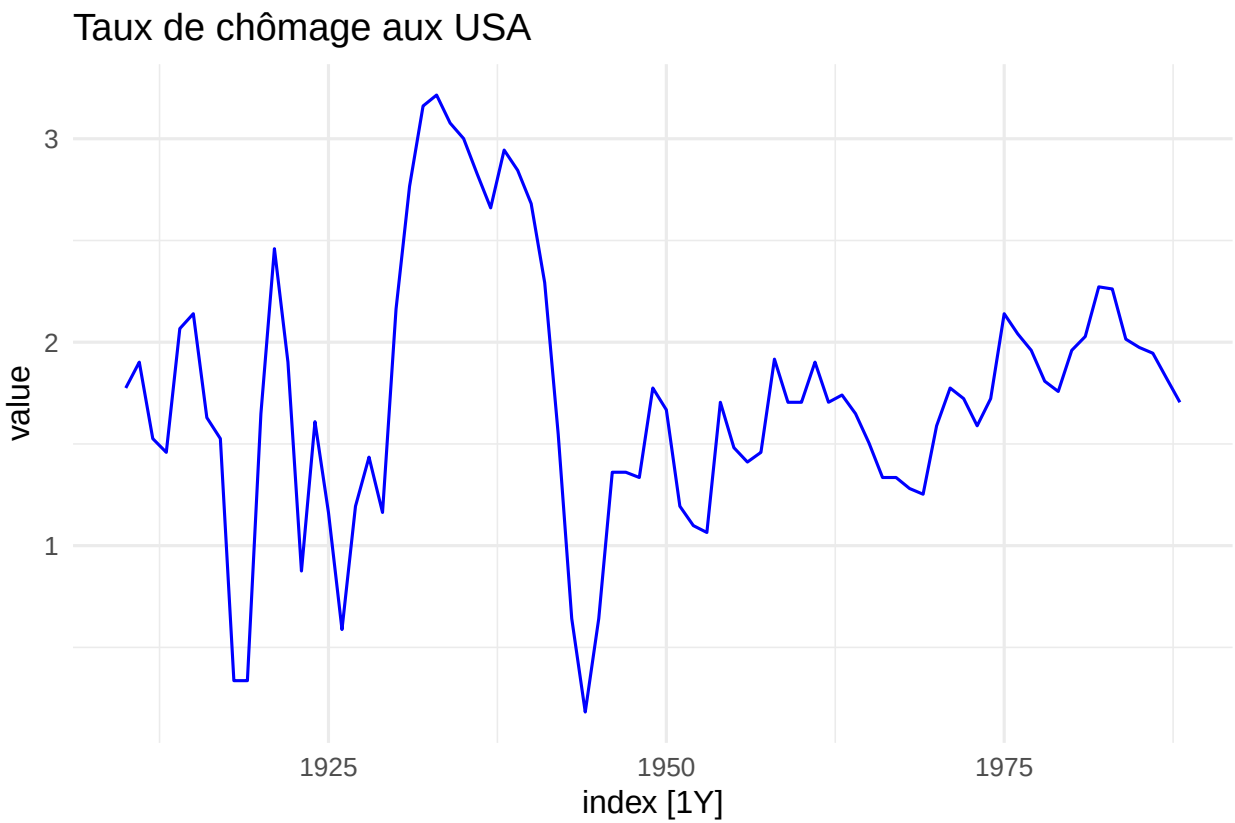
- Soit la trajectoire observée $y_1, \ldots, y_t$ d'une série y_t, éventuellement transformée par passage en log
- Si cette trajectoire peut-être considérée comme stationnaire, on peut lui ajuster un modèle ARMA(p, q) (on ne traite pas ici des séries présentant une saisonnalité)
- La première étape consiste à choisir les ordres p et q
- Le choix de p et q n'est pas unique, il faut comparer plusieurs modèles
- Le premier critère pour juger de la qualité d'un modèle est la blancheur du résidu obtenu (voir la section définitions)

Taux de chômage aux USA

Données

- Nous avons vu que la série des taux de chômage aux USA peut-être considérée comme stationnaire

```
unemp1909p %>% autoplot(value, col="blue") +
  ggtitle("Taux de chômage aux USA")
```

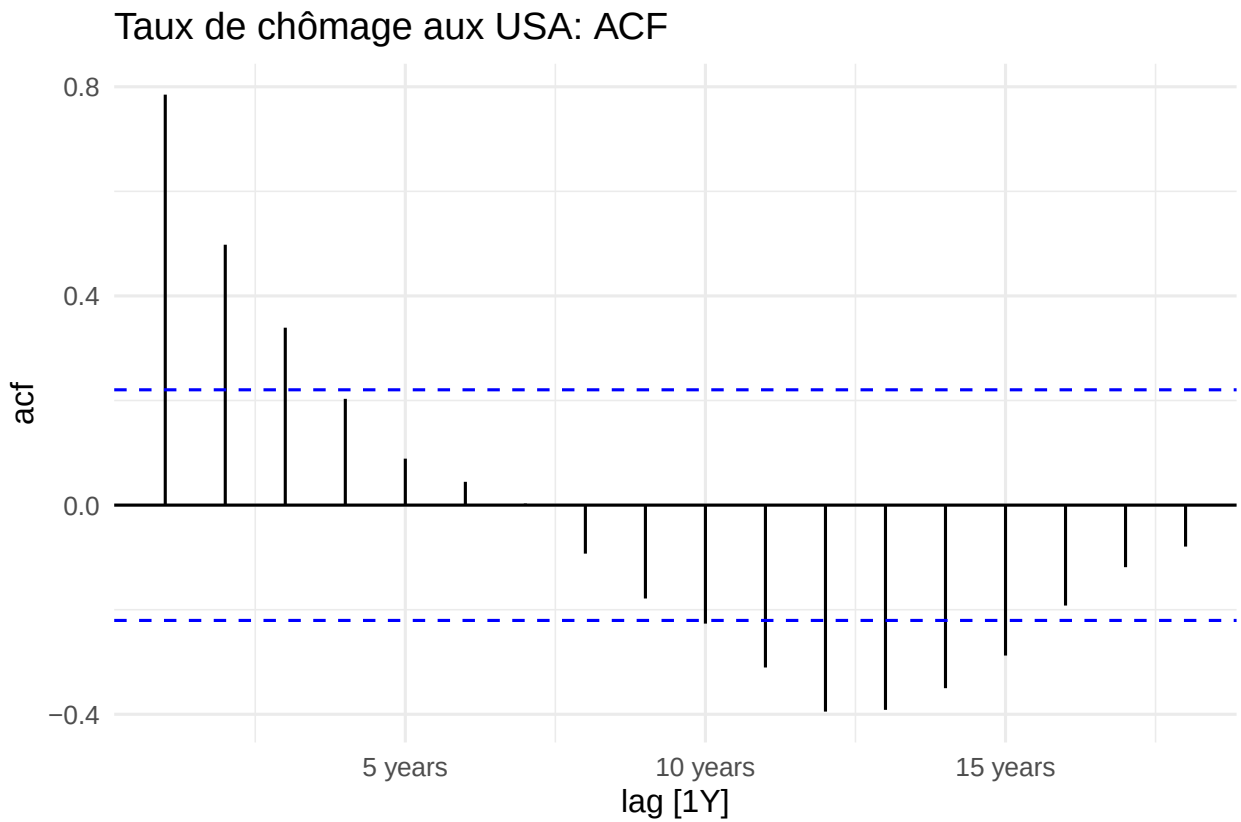


Taux de chômage aux USA

Autocorrélation

- On commence par examiner sa fonction d'autocorrélation

```
unemp1909p %>% ACF(value) %>% autoplot() +
  ggtitle("Taux de chômage aux USA: ACF")
```

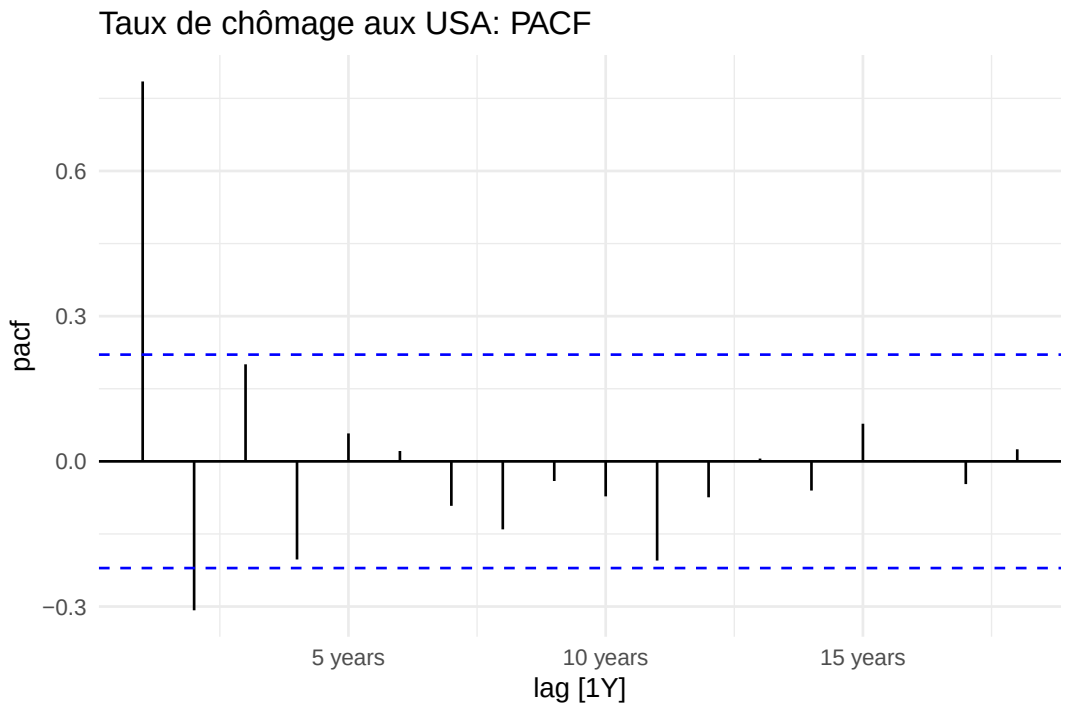


Taux de chômage aux USA

Autocorrélation partielle

- Le coefficient d'autocorrélation partielle $\phi k, k$ représente l'apport d'explication de y_{t-k} à y_t, **toutes choses égales par ailleurs**
- La fonction d'autocorrélation partielle permet d'identifier l'ordre p d'un processus $AR(p)$

```
unemp1909p %>% PACF(value) %>% autoplot() +
  ggtitle("Taux de chômage aux USA: PACF")
```



Modèle ARMA

Taux de chômage aux USA : Estimation du modèle (1)

- On commence par un modèle ARMA(2,2)
- L'erreur standard est élevée par rapport à la valeur du coefficient

```
arma.mod1 <- arima(unemp1909p[,"value"], c(2,0,2))
summary(arma.mod1)
```

```
##
## Call:
## arima(x = unemp1909p[, "value"], order = c(2, 0, 2))
##
## Coefficients:
##          ar1      ar2     ma1      ma2  intercept
##       0.7228  -0.0376  0.4009  -0.1253     1.7357
## s.e.  0.9293   0.5663  0.9222   0.5102     0.1598
##
## sigma^2 estimated as 0.1296:  log likelihood = -32.05,  aic = 76.11
##
## Training set error measures:
##                       ME      RMSE       MAE       MPE     MAPE     MASE
## Training set 0.0004553661 0.3599465 0.2635755 -10.65611 24.60626 0.896286
##                    ACF1
## Training set 0.01310071
```

Modèle ARMA

Taux de chômage aux USA : Modèle $ARMA(2,2)$ - significativité des paramètres

- Les p-valeurs ne sont pas fournies directement par les fonctions `arima()` et `auto.arima()`
- On peut les calculer en utilisant les erreurs standard des coefficients

```
# Extract standard errors
se <- sqrt(diag(vcov(arma.mod1)))

# Calculate the t-values
t_values <- coef(arma.mod1) / se

# Calculate the p-values
p_values <- 2 * (1 - pnorm(abs(t_values)))

# Create a data frame with coefficients, standard errors, t-values, and p-values
results <- data.frame(
  Coefficient = coef(arma.mod1),
  Std_Error = se,
  T_Value = t_values,
  P_Value = p_values
)
print(results)
```

```
##           Coefficient Std_Error     T_Value   P_Value
## ar1        0.72276639 0.9292858  0.77776541 0.4367073
## ar2       -0.03759621 0.5663260 -0.06638616 0.9470704
## ma1        0.40092547 0.9222413  0.43472947 0.6637588
## ma2       -0.12526964 0.5102430 -0.24550974 0.8060618
## intercept  1.73571612 0.1597567 10.86474781 0.0000000
```

Modèle ARMA

Taux de chômage aux USA : Estimation du modèle (2)

- On ajuste un modèle ARMA(1,1)
- L'erreur standard des coefficients a diminué fortement

```
arma.mod2 <- arima(unemp1909p[,"value"], c(1,0,1))
summary(arma.mod2)
```

```
##
## Call:
## arima(x = unemp1909p[, "value"], order = c(1, 0, 1))
##
## Coefficients:
##          ar1     ma1  intercept
##       0.5895  0.5555     1.7356
## s.e.  0.1032  0.1103     0.1503
##
## sigma^2 estimated as 0.13:  log likelihood = -32.19,  aic = 72.38
##
## Training set error measures:
##                       ME      RMSE       MAE       MPE     MAPE      MASE
## Training set 0.0005107137 0.3605539 0.2665423 -10.46991 24.69032 0.9063747
##                      ACF1
## Training set -0.006430895
```

Modèle ARMA

Taux de chômage aux USA : Sélection du modèle

- On utilise l'AIC (Akaike Information Criterion) pour sélectionner le modèle (la plus petite valeur de l'AIC)
- On retient le second modèle avec moins de paramètres ($p = 1$ et $q = 1$)

```
AIC(arma.mod1, arma.mod2)
```

```
##           df      AIC
## arma.mod1  6 76.10996
## arma.mod2  4 72.37670
```

- NOTE : le calcul de l'AIC requiert la log-vraissemblance (log-likelihood) du modèle, celle ci n'est pas disponible lorque le modèle est estimé par MCO

Modèle ARMA

Taux de chômage aux USA : Utilisation de la fonction `auto.arima()`

- La fonction `auto.arima()` recherche le meilleur modèle en utilisant un critère d'information
- Le résultat confirme le choix du modèle ARMA(1,1)

```
nelplo.arma <- auto.arima(unemp1909p$value)
nelplo.arma
```

```
## Series: unemp1909p$value
## ARIMA(1,0,1) with non-zero mean
##
## Coefficients:
##          ar1     ma1    mean
##       0.5895  0.5555  1.7356
## s.e.  0.1032  0.1103  0.1503
##
## sigma^2 = 0.1351:  log likelihood = -32.19
## AIC=72.38   AICc=72.92   BIC=81.85
```

Modèle ARMA

Taux de chômage aux USA : Significativité des paramètres

- Calcul des p-valeurs en utilisant les erreurs standard des coefficients

```r
# Extract standard errors
se <- sqrt(diag(vcov(nelplo.arma)))

# Calculate the t-values
t_values <- coef(nelplo.arma) / se

# Calculate the p-values
p_values <- 2 * (1 - pnorm(abs(t_values)))

# Create a data frame with coefficients, standard errors, t-values, and p-values
results <- data.frame(
  Coefficient = coef(nelplo.arma),
  Std_Error = se,
  T_Value = t_values,
  P_Value = p_values
)
print(results)
```

```
##           Coefficient Std_Error   T_Value      P_Value
## ar1         0.5894997 0.1032109  5.711603 1.119166e-08
## ma1         0.5554587 0.1103101  5.035431 4.767737e-07
## intercept   1.7355500 0.1503400 11.544166 0.000000e+00
```

Modèle ARMA

Taux de chômage aux USA : Diagnostic du modèle - Préparation

- Afin de vérifier la validité des modèles estimés, on doit vérifier au minimum la significativité des paramètres et la blancheur du résidu
- On prépare un data frame avec les valeurs observées, les valeurs prédites par le modèle, et les résidus

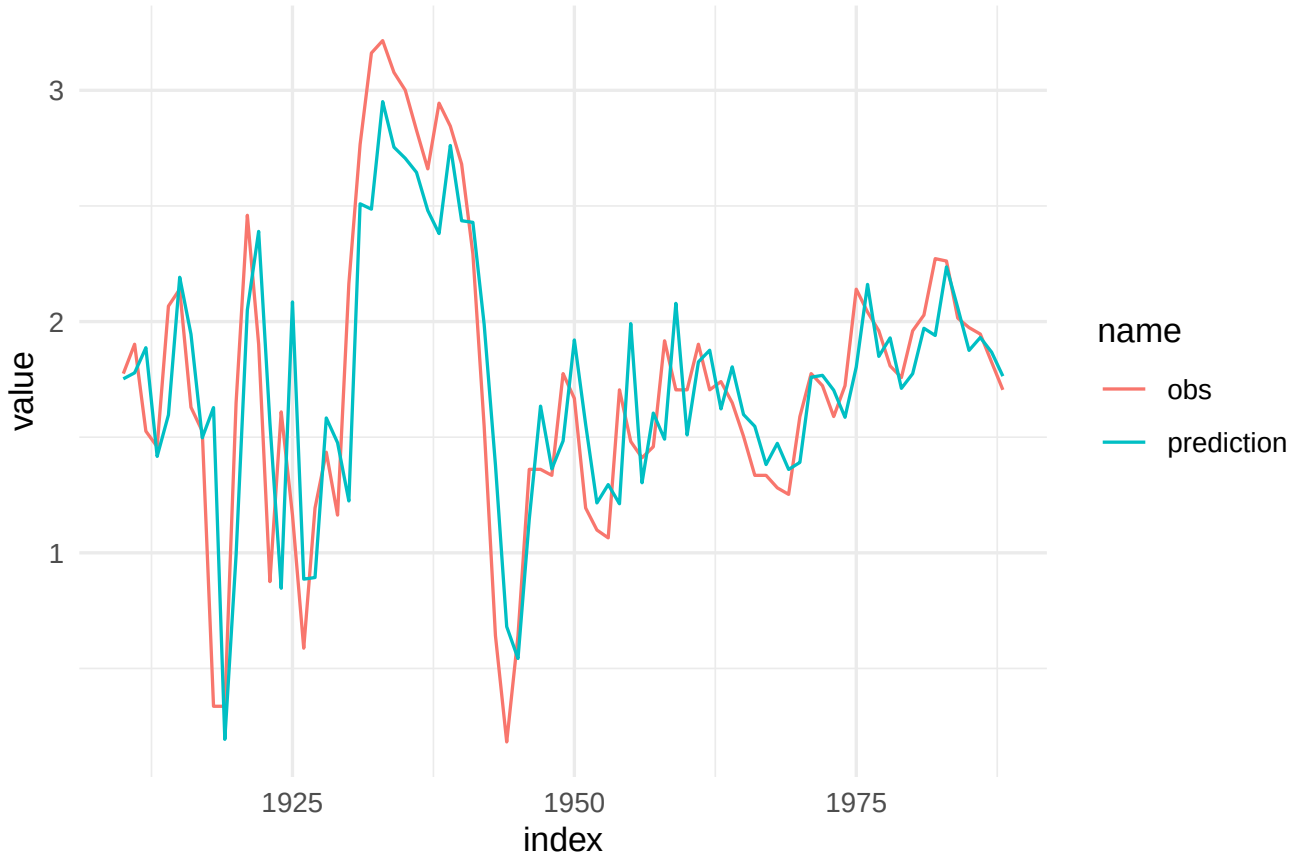
```
nelplo.arma.diag <- data.frame(index=unemp1909p$index, obs=unemp1909p$value,
                               prediction=as.data.frame(nelplo.arma$fitted)$x,
                               residus=as.data.frame(nelplo.arma$residuals)$x)
head(nelplo.arma.diag)
```

```
##   index      obs prediction     residus
## 1  1910 1.774952   1.752238  0.02271448
## 2  1911 1.902108   1.778215  0.12389255
## 3  1912 1.526056   1.887011 -0.36095467
## 4  1913 1.458615   1.416962  0.04165325
## 5  1914 2.066863   1.596360  0.47050309
## 6  1915 2.140066   2.191556 -0.05148945
```

Modèle ARMA

Taux de chômage aux USA : Valeurs observées x prédites

```
nelplo.arma.diag %>%
  pivot_longer(cols=2:4) %>%
  filter(name %in% c("prediction", "obs")) %>%
  ggplot() + geom_line(aes(x=index, y=value, color=name))
```



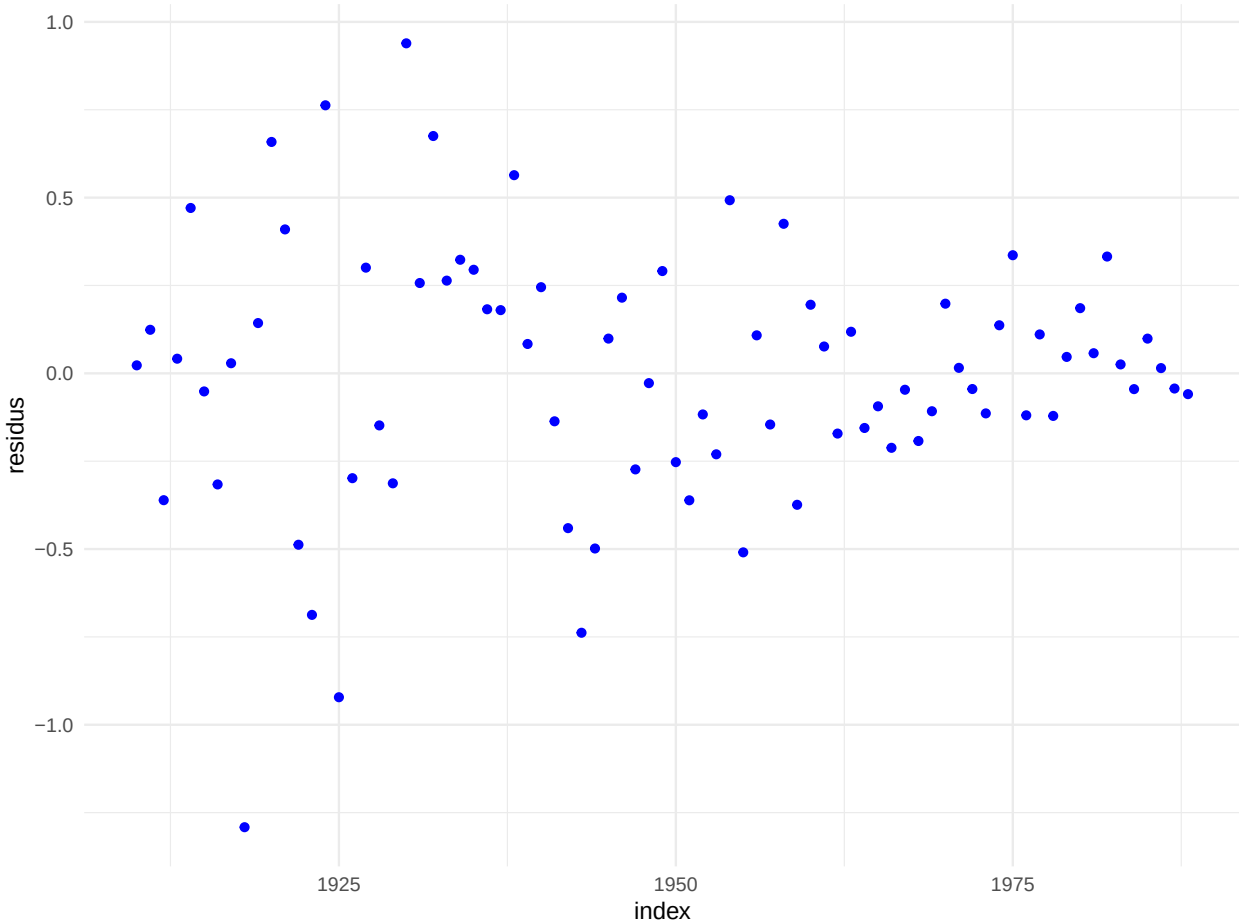
Modèle ARMA

Taux de chômage aux USA : Résidus (1)

- Si notre modèle est adéquat, les résidus devraient fluctuer autour de 0, sans tendance

```
nelplo.arma.diag %>% ggplot() +
  geom_point(aes(x=index, y=residus), color="blue")
```

```
## Don't know how to automatically pick scale for object of type <ts>. Defaulting
## to continuous.
```

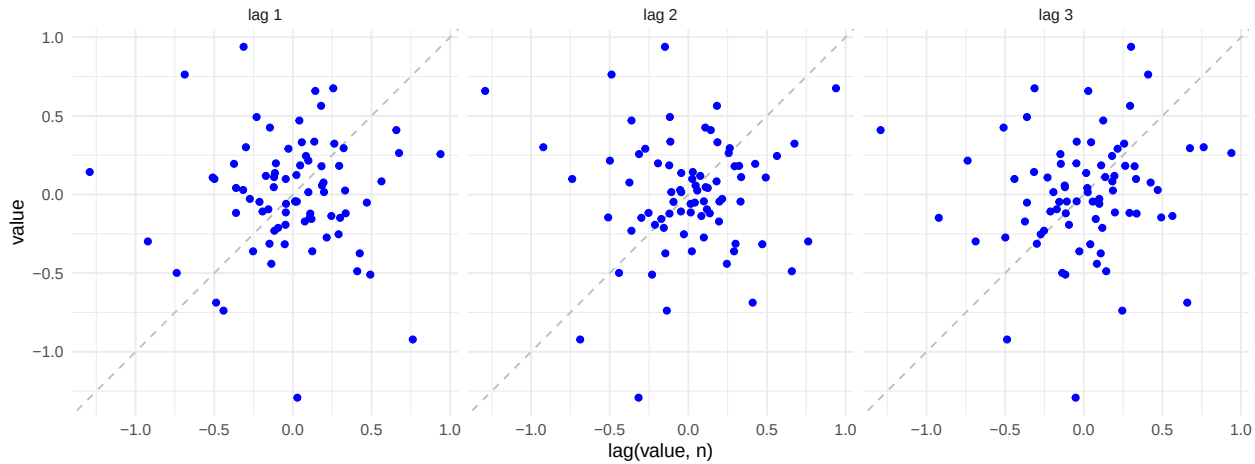


Modèle ARMA

Taux de chômage aux USA : Résidus (2)

- On visualise l'autocorrélation des résidus à l'aide de la fonction `gg_lag()`

```
nelplo.arma$residuals %>% as_tsibble() %>%
  gg_lag(y=value, lags=1:3, geom="point", colour="blue")
```



Modèle ARMA

Taux de chômage aux USA : Résidus (3)

- On teste la blancheur des résidus aux retards 1 à 3
- Toutes les p-valeurs sont largement supérieure à 0.05, on peut conclure à la blancheur des résidus

```
lags <- 1:3
pval <- NULL
for (l in lags) {
  pval <-c(pval, box_pierce(nelplo.arma.diag$residus, lag=l)["bp_pvalue"])
}
res <- data.frame(lags, pval)
res
```

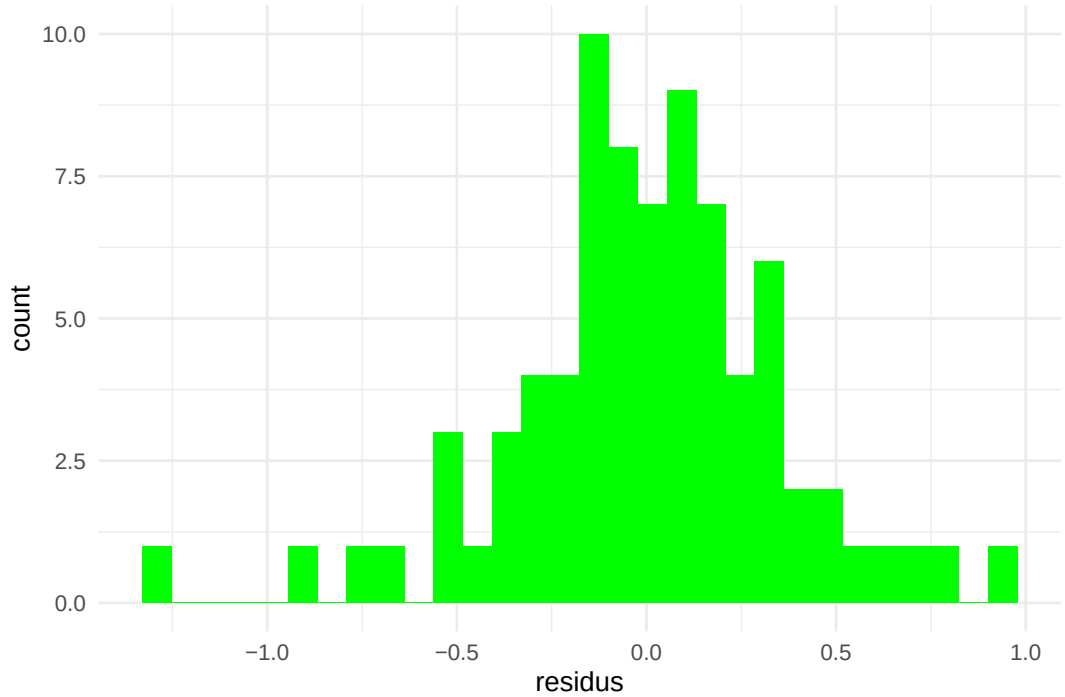
```
##   lags      pval
## 1    1 0.9544185
## 2    2 0.9906342
## 3    3 0.8891629
```

Modèle ARMA

Taux de chômage aux USA : Résidus (4)

- L'hypothèse de normalité des résidus (test de Jarque-Bera) est rejettée

```
ggplot(nelplo.arma.diag) + geom_histogram(aes(x=residus), fill='green')
```




```
jarque.bera.test(nelplo.arma.diag$residus)
```

```
##
##  Jarque Bera Test
##
## data:  nelplo.arma.diag$residus
## X-squared = 11.985, df = 2, p-value = 0.002498
```

9 Modélisation de séries temporelles stationnaires

- Processus autorégressif (AR)
- Processus moyenne mobile
- Processus ARMA
- Prévision

Modèle ARMA

Taux de chômage aux USA : Prévisions (1)

- On peut utiliser la fonction générique `forecast()` pour la prévision du taux de chômage
- On obtient la prédiction et les intervalles de confiance à 80% et 95%

```
forecast(nelplo.arma, h=10)
```

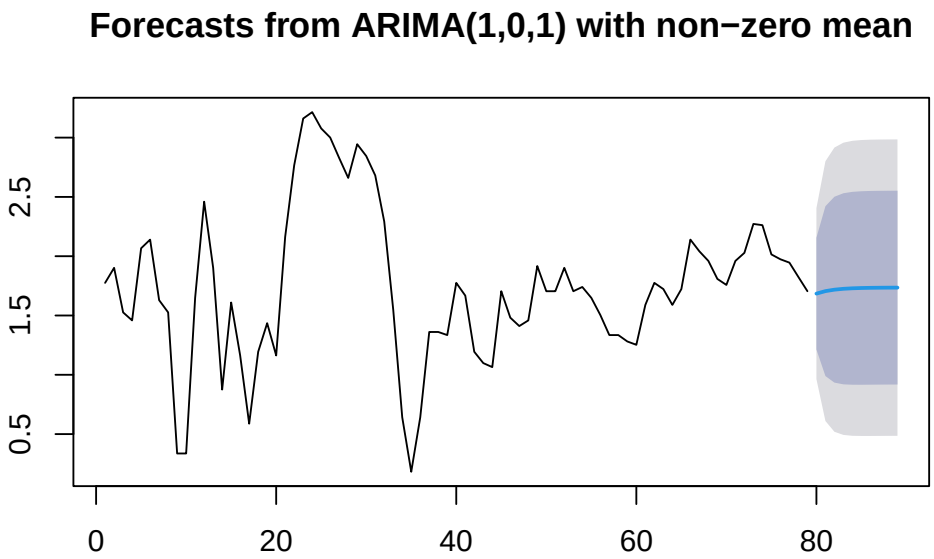
```
##    Point Forecast     Lo 80    Hi 80     Lo 95    Hi 95
## 80       1.684490 1.2133901 2.155590 0.9640049 2.404975
## 81       1.705450 0.9892962 2.421604 0.6101872 2.800713
## 82       1.717806 0.9342365 2.501376 0.5194397 2.916173
## 83       1.725090 0.9194124 2.530768 0.4929124 2.957268
## 84       1.729384 0.9161642 2.542603 0.4856717 2.973096
## 85       1.731915 0.9160908 2.547739 0.4842196 2.979611
## 86       1.733407 0.9166798 2.550135 0.4843304 2.982484
## 87       1.734287 0.9172458 2.551328 0.4847304 2.983843
## 88       1.734805 0.9176554 2.551955 0.4850823 2.984528
## 89       1.735111 0.9179232 2.552299 0.4853301 2.984892
```

Modèle ARMA

Taux de chômage aux USA : Projection (2)

- La méthode générique `plot()` permet de visualiser la projection

```
plot(forecast(nelplo.arma, h=10))
```



10 Modèlisation de séries non-stationnaires

Modèle ARIMA

Identification d'un modèle

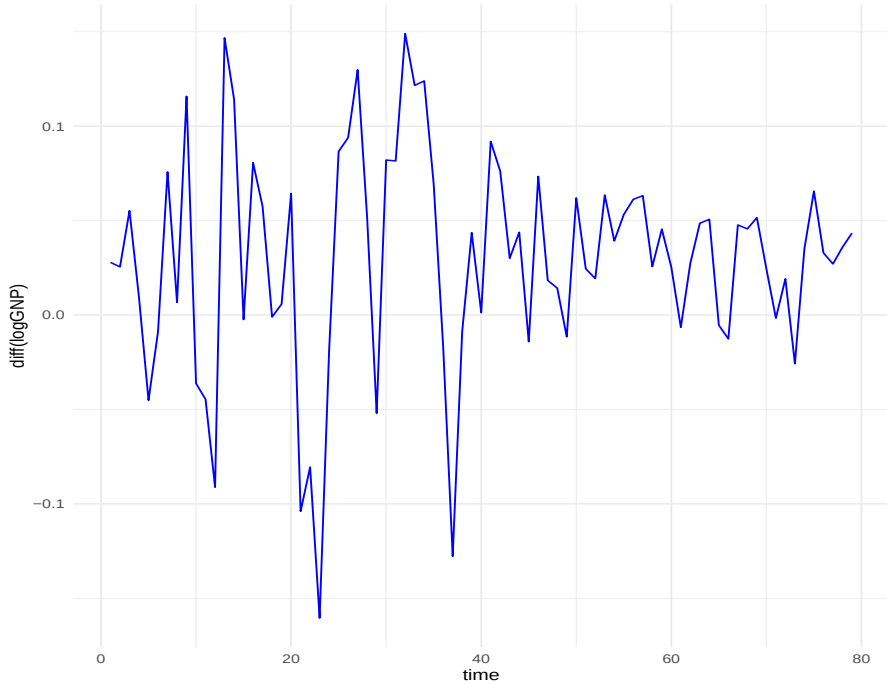
- Un modèle ARIMA(p,d,q) est un modèle ARMA sur la série différenciée, d est l'ordre de différentiation
- Soit la trajectoire observée $y_1, \ldots, y_t$ d'une série y_t , éventuellement obtenue après transformation d'une série initiale par passage en log
- La série n'est pas stationnaire et on veut lui ajuster un modèle ARIMA(p, d, q) (on ne traite pas ici des séries présentant une saisonnalité)
- Une fois d choisis, on est ramené à l'identification d'un ARMA sur la série différenciée
- Pour choisir d, on peut tester l'hypothèse d $= 1$ contre d $= 0$ avec un test ADF
- On peut également comparer les modèles avec et sans différenciation à l'aide d'un critère d'information ou de la valeur prédictive

Modèle ARIMA

PIB des USA

- Nous avons vu que cette série (après **transformation logarithmique**) est non stationnaire et qu'elle possède un trend stochastique
- On peut donc la différencier puis ajuster un modèle ARMA

```
ggplot(gnpreal.diff) + geom_line(aes(x=time, y=diff), color="blue") + ylab("diff(logGNP)")
```

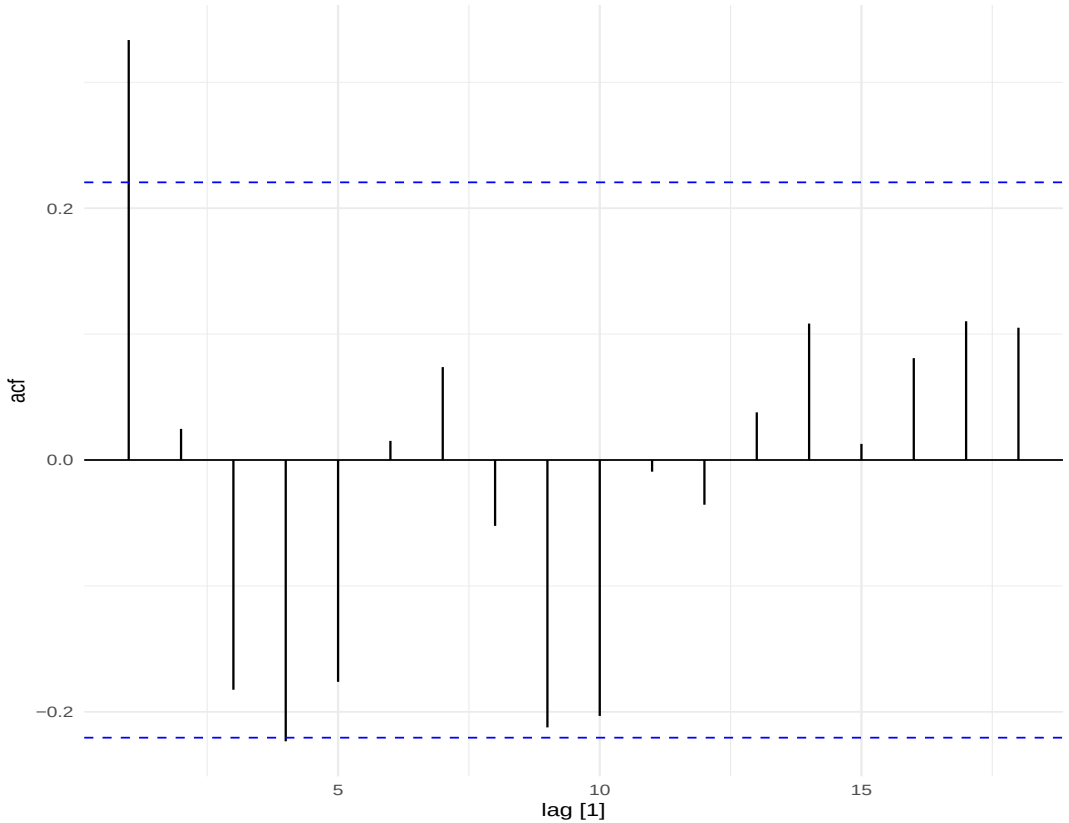


Modèle ARIMA

PIB des USA : Autocorrélation (série différenciée)

- L'autocorrélation est significative uniquement pour le lag 1

```
gnpreal.diff %>% as_tsibble(index=time) %>% ACF(diff) %>% autoplot()
```

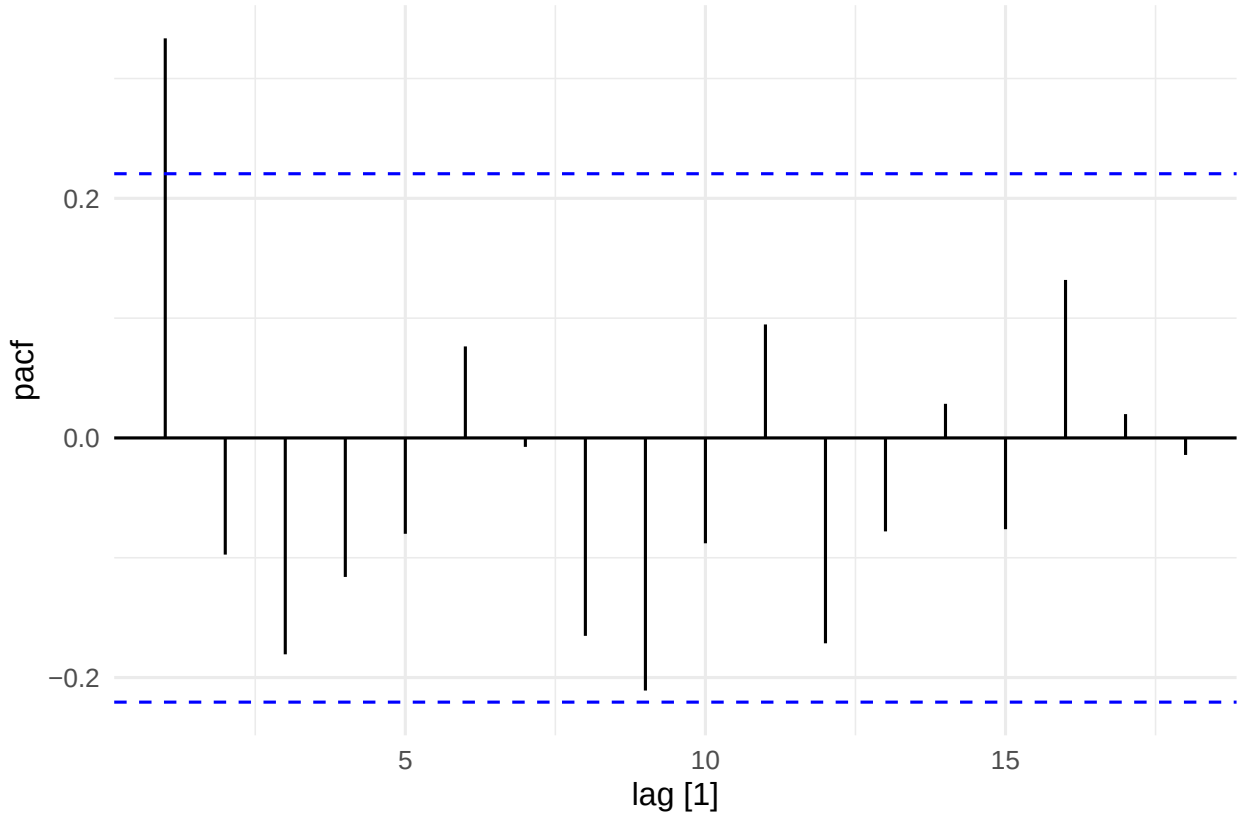


Modèle ARIMA

PIB des USA : Autocorrélation partielle (série différenciée)

- La fonction d'autocorrélation partielle suggère un modèle $AR(1)$

```
gnpreal.diff %>% as_tsibble(index=time) %>% PACF(diff) %>% autoplot()
```



Modèle ARIMA

PIB des USA : Modélisation

- On utilise la fonction `auto.arima()`
- Le modèle retenu est $ARIMA(1,1,0)$: un modèle $ARMA(1,0)$ sur la série intégrée $I(1)$

```
regdata <- nelplo1909$gnp.real
gnpreal.arima <- auto.arima(regdata)
gnpreal.arima
```

```
## Series: regdata
## ARIMA(1,1,0) with drift
##
## Coefficients:
##          ar1   drift
##       0.3296  0.0295
## s.e.  0.1052  0.0090
##
## sigma^2 = 0.003015:  log likelihood = 118.12
## AIC=-230.24   AICc=-229.92   BIC=-223.14
```

Modèle ARIMA

PIB des USA : Diagnostics (1)

- Calcul des p-valeurs en utilisant les erreurs standard des coefficients

```r
# Extract standard errors
se <- sqrt(diag(vcov(gnpreal.arima)))

# Calculate the t-values
t_values <- coef(gnpreal.arima) / se

# Calculate the p-values
p_values <- 2 * (1 - pnorm(abs(t_values)))

# Create a data frame with coefficients, standard errors, t-values, and p-values
results <- data.frame(
  Coefficient = coef(gnpreal.arima),
  Std_Error = se,
  T_Value = t_values,
  P_Value = p_values
)
print(results)
```

```
##       Coefficient   Std_Error  T_Value     P_Value
## ar1    0.32961164 0.105246210 3.131815 0.001737294
## drift  0.02954975 0.009042602 3.267837 0.001083726
```

Modèle ARIMA

PIB des USA : Diagnostics (2)

- On prépare un data frame avec les valeurs prédites par le modèle, et les résidus

```
gnpreal.arima.diag <- data.frame(index=nelplo1909$index,
                                 prediction=as.data.frame(gnpreal.arima$fitted)$x,
                                 residus=as.data.frame(gnpreal.arima$residuals)$x)
head(gnpreal.arima.diag)
```

```
##   index prediction      residus
## 1  1909   4.755732  0.004730911
## 2  1910   4.789920 -0.001595465
## 3  1911   4.817318 -0.003508918
## 4  1912   4.842019  0.027052834
## 5  1913   4.907097 -0.028850607
## 6  1914   4.901080 -0.067977600
```

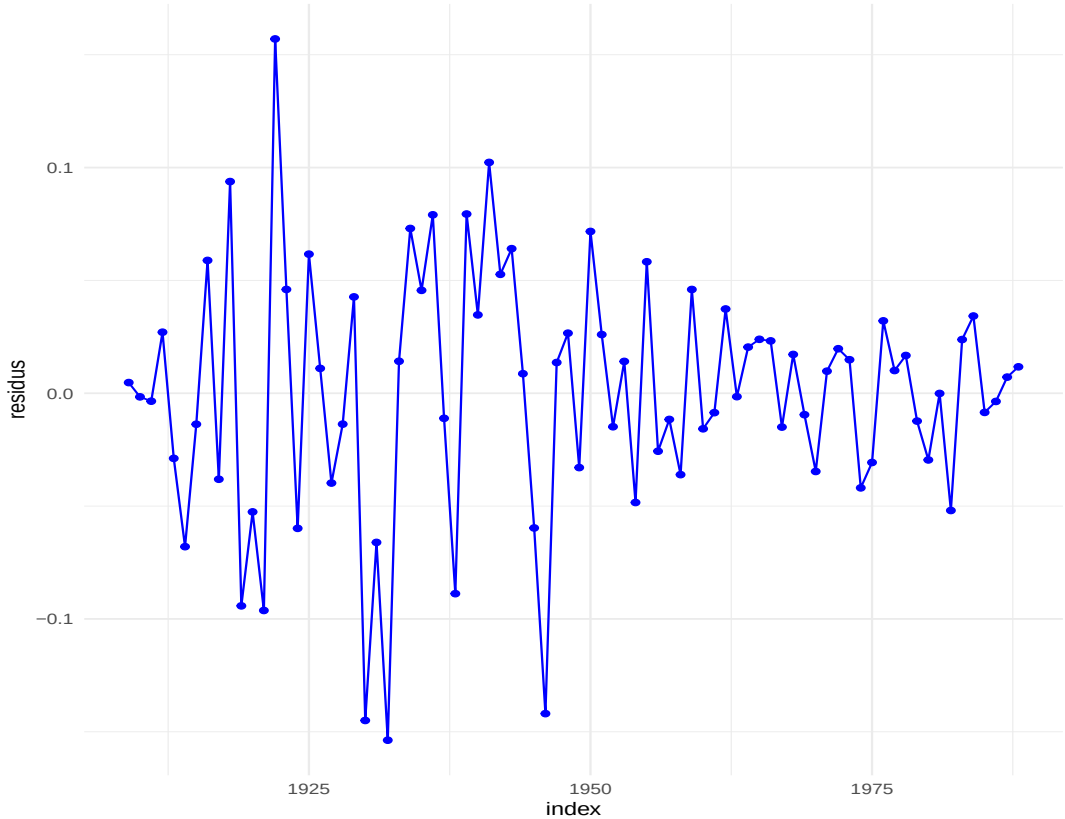
Modèle ARIMA

PIB des USA : Résidus (1)

- Si notre modèle est adéquat, les résidus devraient fluctuer autour de 0, sans tendance

```
gnpreal.arima.diag %>% ggplot(aes(x=index, y=residus)) +
  geom_point(color="blue") +
  geom_line(color="blue")
```

```
## Don't know how to automatically pick scale for object of type <ts>. Defaulting
## to continuous.
```

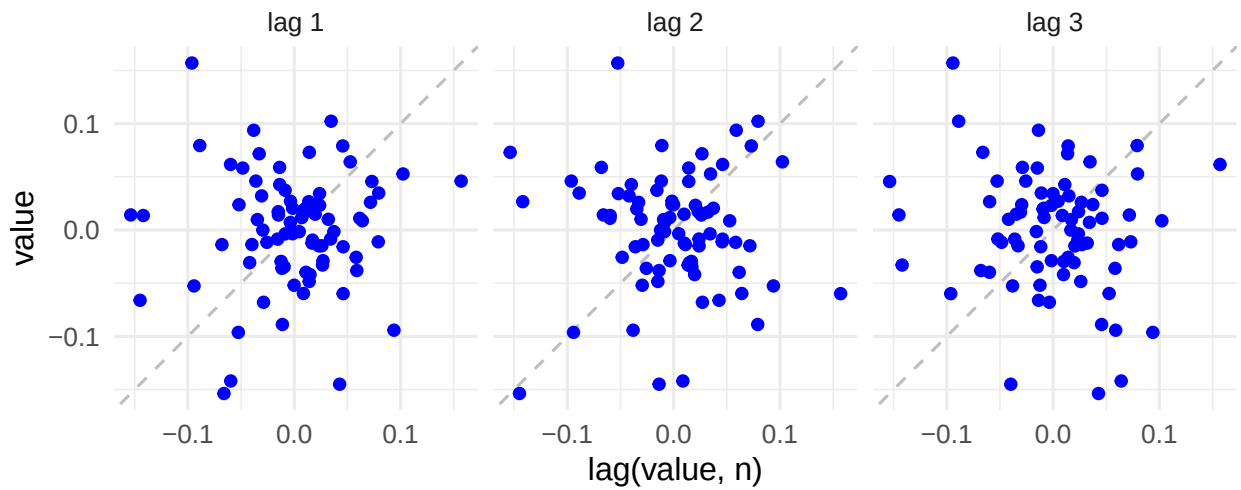


Modèle ARIMA

PIB des USA : Résidus (2)

- On visualise l'autocorrélation des résidus à l'aide de la fonction `gg_lag()`

```
gnpreal.arima$residuals %>% as_tsibble() %>%
  gg_lag(y=value, lags=1:3, geom="point", colour="blue")
```



Modèle ARIMA

PIB des USA : Résidus (3)

- On teste la blancheur des résidus aux retards 1 à 3
- Toutes les p-valeur sont largement supérieure à 0.05, on peut conclure à la blancheur des résidus

```
lags <- 1:3
pval <- NULL
for (l in lags) {
  pval <-c(pval, box_pierce(gnpreal.arima.diag$residus, lag=l)["bp_pvalue"])
}
res <- data.frame(lags, pval)
res
```

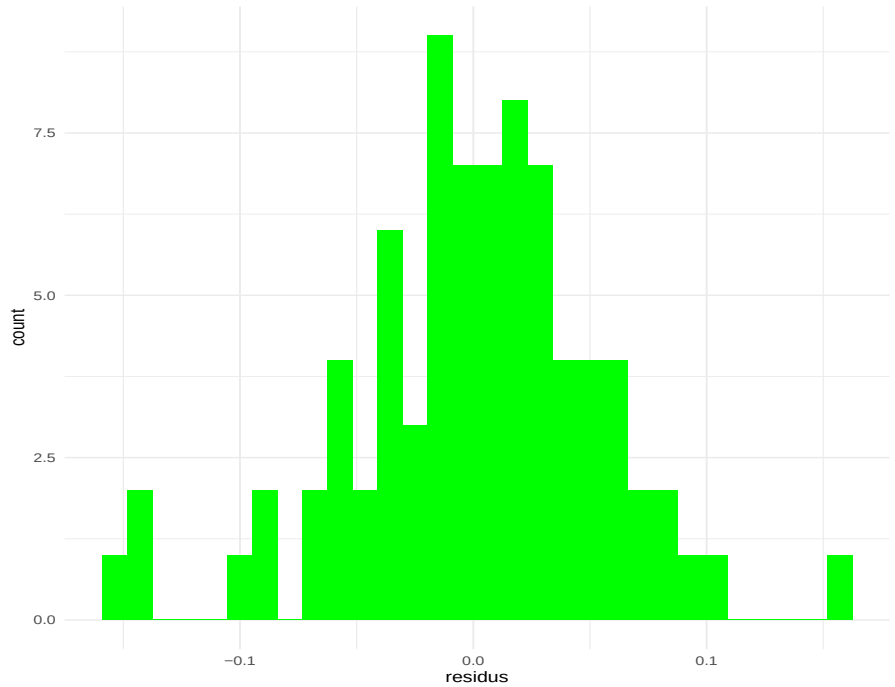
```
##   lags      pval
## 1    1 0.7460147
## 2    2 0.9252396
## 3    3 0.5645515
```

Modèle ARIMA

PIB des USA : Résidus (4)

- L'hypothèse de normalité des résidus (test de Jarque-Bera) est acceptée

```
ggplot(gnpreal.arima.diag) + geom_histogram(aes(x=residus), fill='green')
```




```
jarque.bera.test(gnpreal.arima.diag$residus)
```

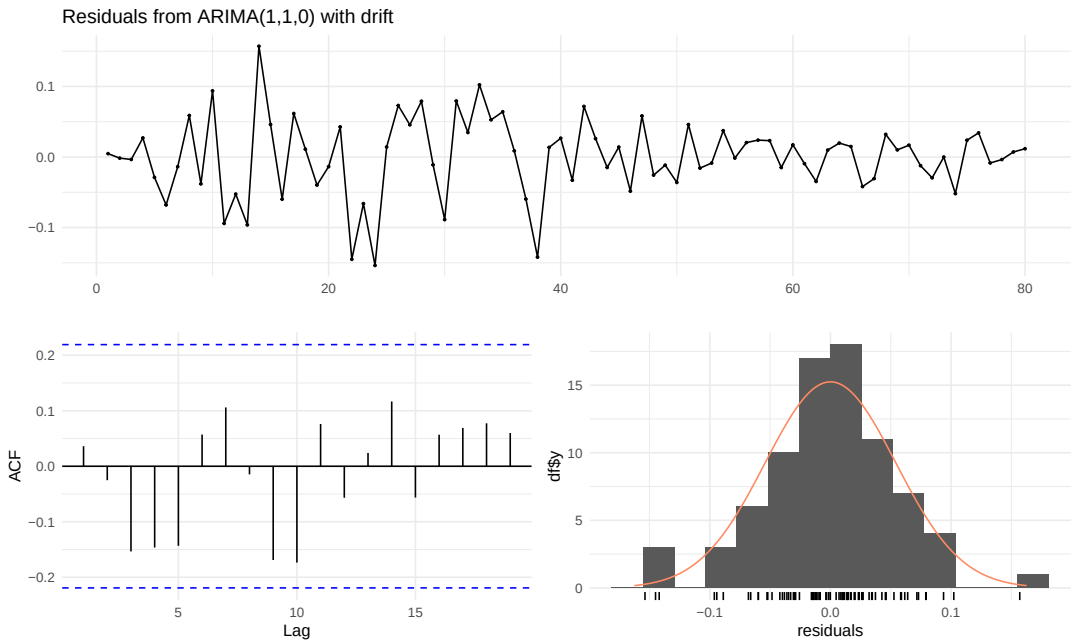
```
##
##  Jarque Bera Test
##
## data:  gnpreal.arima.diag$residus
## X-squared = 5.7526, df = 2, p-value = 0.05634
```

Modèle ARIMA

PIB des USA : Résidus (5)

- La fonction `checkresiduals()` (librairie `forecast`) produit également les graphiques de diagnostic

```
checkresiduals(gnpreal.arima, test=FALSE)
```



Modèle ARIMA

PIB des USA : Résidus (5)

- La fonction `checkresiduals` renvoie également les résultats du test de Ljung-Box (absence d'autocorrélation des résidus)

```
checkresiduals(gnpreal.arima, plot=FALSE)
```

```
##
##  Ljung-Box test
##
## data:  Residuals from ARIMA(1,1,0) with drift
## Q* = 12.585, df = 9, p-value = 0.1823
##
## Model df: 1.   Total lags used: 10
```

- Ici la p-valeur est supérieure à 5%, il n'y a pas d'autocorrélation

Modèle ARIMA

PIB des USA : Prédiction (1)

- La fonction `forecast` produit les prédiction à partir du modèle sélectionné, avec un intervalle de confiance
- Les données sont annuelles, on prédit à un horizon de 10 ans

```
forecast(gnpreal.arima, h=10)
```

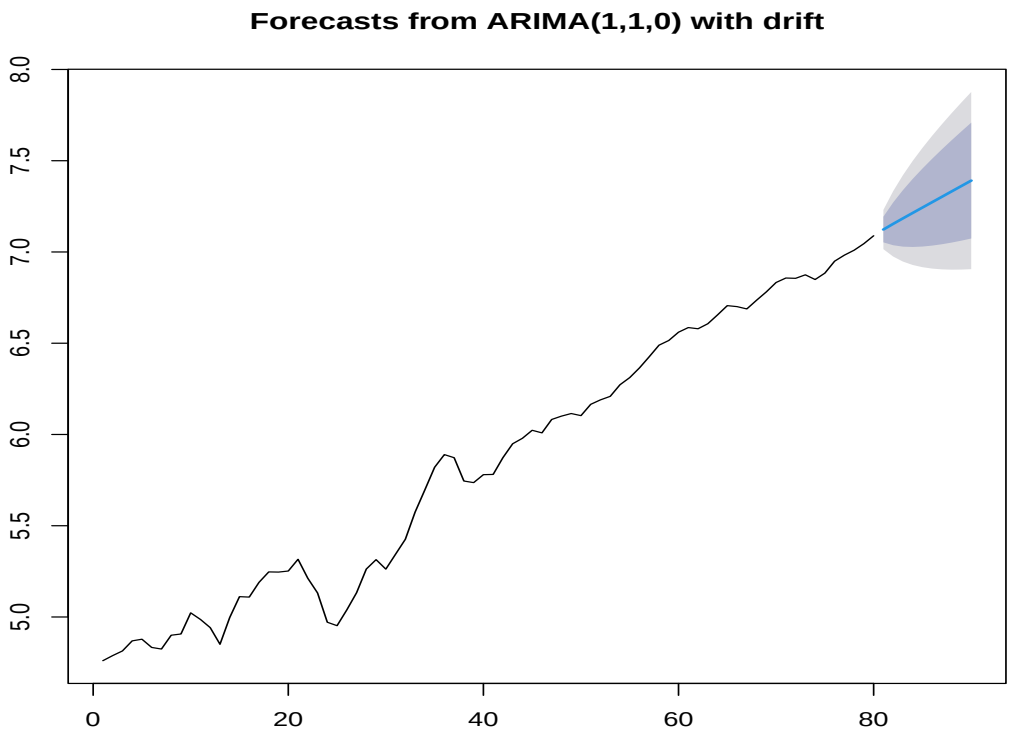
```
##    Point Forecast    Lo 80    Hi 80    Lo 95    Hi 95
## 81       7.122980 7.052606 7.193353 7.015353 7.230606
## 82       7.154028 7.036948 7.271107 6.974970 7.333085
## 83       7.184071 7.029307 7.338836 6.947380 7.420763
## 84       7.213784 7.027470 7.400098 6.928841 7.498727
## 85       7.243387 7.029736 7.457039 6.916636 7.570139
## 86       7.272955 7.034967 7.510943 6.908984 7.636926
## 87       7.302510 7.042418 7.562603 6.904733 7.700288
## 88       7.332062 7.051590 7.612535 6.903117 7.761008
## 89       7.361612 7.062141 7.661084 6.903610 7.819615
## 90       7.391162 7.073826 7.708499 6.905838 7.876487
```

Modèle ARIMA

Log du PIB des USA : Prédiction (2)

La méthode générique `plot` produit une représentation graphique de la prédiction

```
plot(forecast(gnpreal.arima, h=10))
```



Modèle ARIMA

Dépenses aggégées de consommation aux USA (1)

- Nous avons vu que cette série (après transformation logarithmique) est non stationnaire et qu'elle possède un trend stochastique
- La série peut être rendue stationnaire en la différenciant deux fois (on utilise l'argument diff=2)

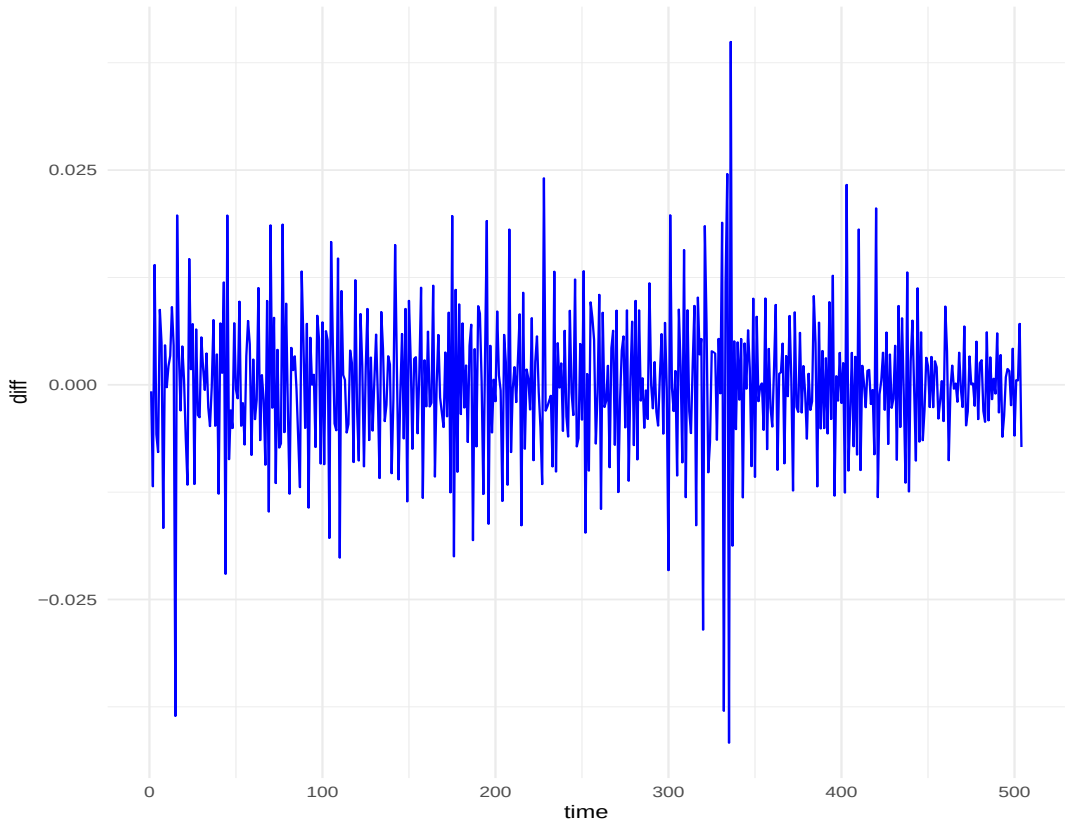
```
USExp.diff2 <- tibble(time=1:(nrow(USExp)-2), diff= diff(as.data.frame(USExp)[, "logExp"], diff=2))
```

Modèle ARIMA

Dépenses aggégées de consommation aux USA (2)

Chronogramme de la série intégrée d'ordre 2

```
ggplot(USExp.diff2, aes(x = time, y = diff)) +
  geom_line(color = "blue", size = 0.5)
```

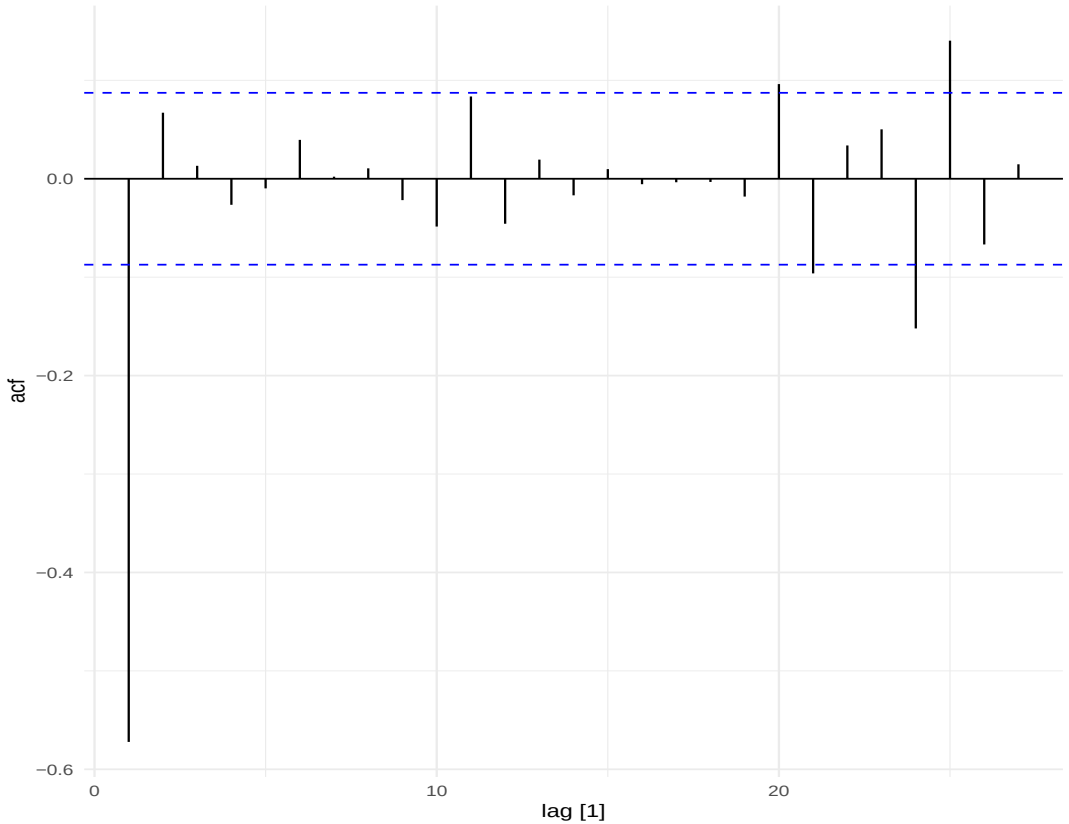


Modèle ARIMA

Dépenses aggégées de consommation aux USA : Fonction d'autocorrélation (série différenciée)

- La fonction d'autocorrélation montre une corrélation significative uniquement pour le lag 1 (pas de déclin régulier)

```
USExp.diff2 %>% as_tsibble(index=time) %>% ACF(diff) %>% autoplot()
```

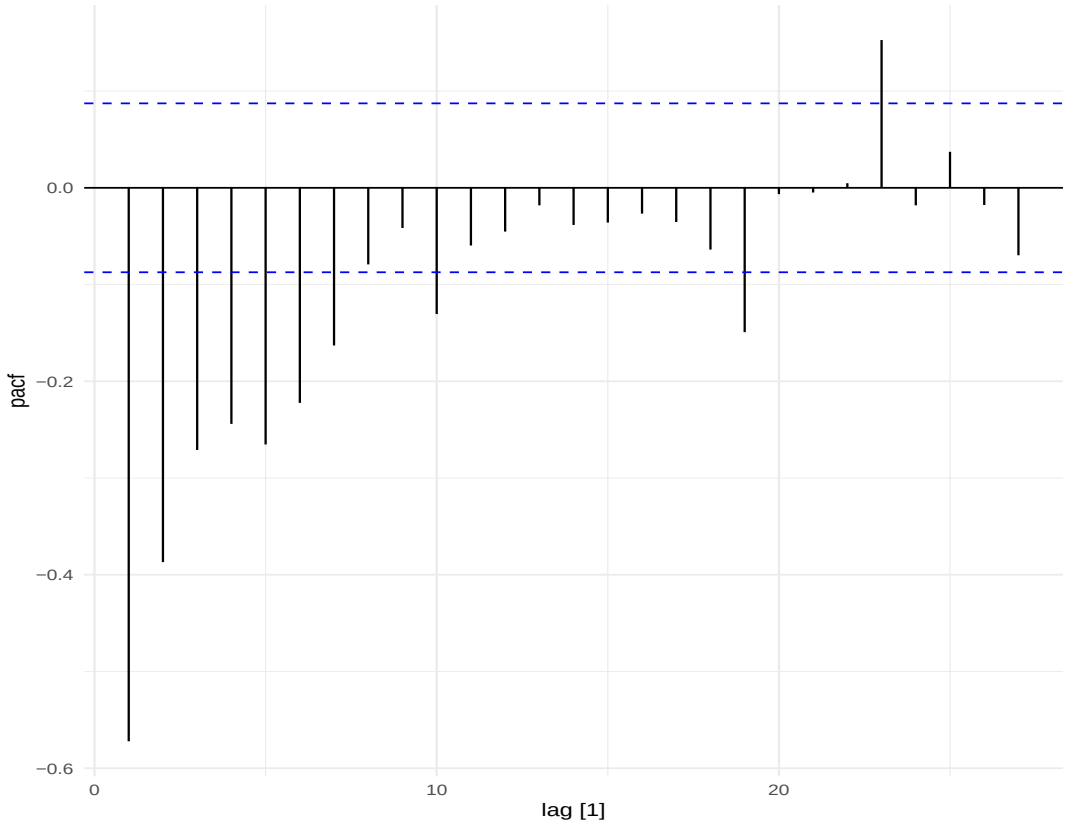


Modèle ARIMA

Dépenses aggégées de consommation aux USA : Fonction d'autocorrélation partielle

- La fonction d'autocorrélation partielle montre un déclin régulier

```
USExp.diff2 %>% as_tsibble(index=time) %>% PACF(diff) %>% autoplot()
```



Modèle ARIMA

Dépenses aggégées de consommation aux USA : Modélisation

- On utilise la fonction `auto.arima` sur la série non différenciée
- Le résultat suggère un modèle ARIMA(2,2,2), c'est à dire un modèle ARMA(2,2) sur la série intégrée d'ordre 2

```
arimod <- auto.arima(USExp[,"logExp"])
arimod
```

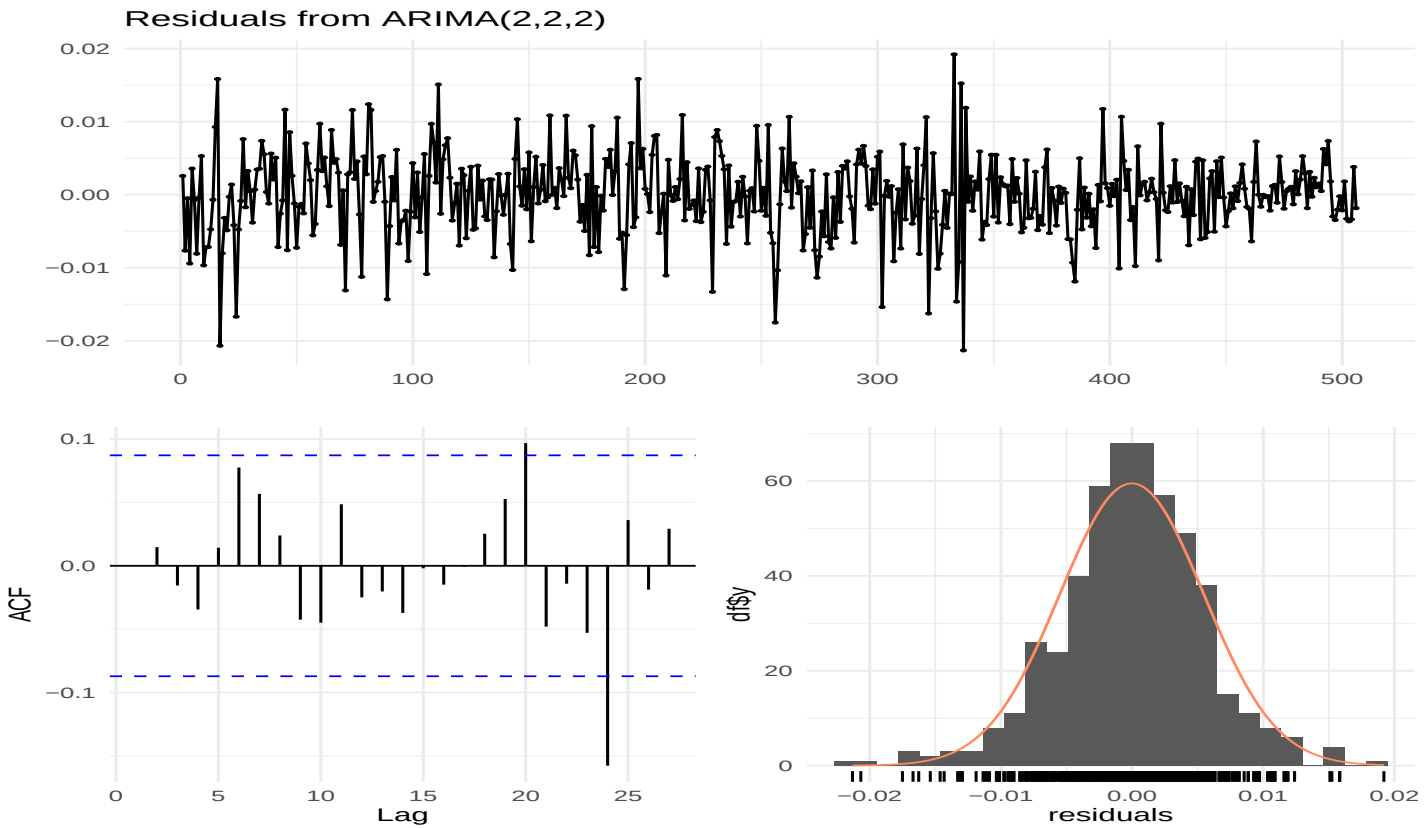
```
## Series: USExp[, "logExp"]
## ARIMA(2,2,2)
##
## Coefficients:
##          ar1     ar2      ma1     ma2
##       0.3384  0.0158  -1.5509  0.5725
## s.e.  0.1704  0.0679   0.1644  0.1578
##
## sigma^2 = 3.053e-05:  log likelihood = 1906.35
## AIC=-3802.69   AICc=-3802.57   BIC=-3781.58
```

Modèle ARIMA

Dépenses aggégées de consommation aux USA : Diagnostic (1)

- On obtient un diagnostic des résidus avec la fonction `checkresiduals` de la librairie `forecast`

```
checkresiduals(arimod, test=FALSE)
```



Modèle ARIMA

Dépenses aggégées de consommation aux USA : Diagnostic (2)

- La fonction `checkresiduals` renvoie également le résultat du test de Ljung-Box (absence d'autocorrélation des résidus)

```
checkresiduals(arimod, plot=FALSE)
```

```
##
##  Ljung-Box test
##
## data:  Residuals from ARIMA(2,2,2)
## Q* = 7.9665, df = 6, p-value = 0.2406
##
## Model df: 4.   Total lags used: 10
```

Modèle ARIMA

Dépenses aggégées de consommation aux USA : Prédiction (1)

- La fonction `forecast()` produit les prédiction à partir du modèle sélectionné, avec un intervalle de confiance

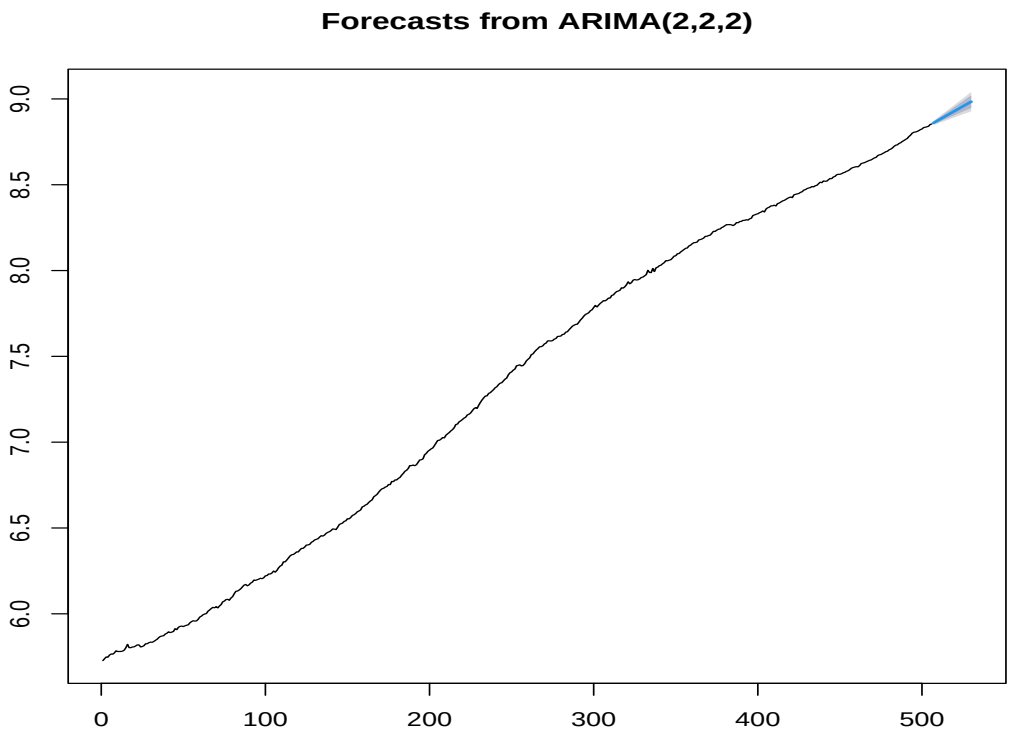
```
forecast(arimod, h=24)
```

```
##     Point Forecast    Lo 80    Hi 80    Lo 95    Hi 95
## 507       8.859892 8.852811 8.866972 8.849063 8.870721
## 508       8.865359 8.856346 8.874371 8.851575 8.879142
## 509       8.870780 8.860308 8.881252 8.854765 8.886796
## 510       8.876182 8.864410 8.887955 8.858178 8.894187
## 511       8.881577 8.868565 8.894590 8.861676 8.901478
## 512       8.886969 8.872742 8.901197 8.865210 8.908729
## 513       8.892361 8.876928 8.907794 8.868758 8.915963
## 514       8.897751 8.881115 8.914388 8.872308 8.923195
## 515       8.903142 8.885300 8.920984 8.875854 8.930429
## 516       8.908532 8.889479 8.927586 8.879393 8.937672
## 517       8.913923 8.893652 8.934194 8.882921 8.944925
## 518       8.919313 8.897816 8.940811 8.886435 8.952191
## 519       8.924704 8.901970 8.947438 8.889935 8.959473
## 520       8.930094 8.906114 8.954075 8.893419 8.966770
## 521       8.935485 8.910246 8.960723 8.896886 8.974084
## 522       8.940875 8.914367 8.967383 8.900335 8.981416
## 523       8.946266 8.918476 8.974055 8.903765 8.988766
## 524       8.951656 8.922573 8.980740 8.907177 8.996136
## 525       8.957047 8.926656 8.987437 8.910569 9.003525
## 526       8.962437 8.930727 8.994147 8.913941 9.010934
## 527       8.967828 8.934785 9.000870 8.917293 9.018362
## 528       8.973218 8.938829 9.007607 8.920625 9.025811
## 529       8.978609 8.942861 9.014356 8.923937 9.033280
## 530       8.983999 8.946879 9.021119 8.927229 9.040769
```

Modèle ARIMA

Dépenses aggégées de consommation aux USA : Prédiction (2)

```
plot(forecast(arimod, h=24))
```



Exercice

- Ajustez un modèle ARMA ou ARIMA sur les séries du PIB, de l'IPC et du taux de chômage français (données `fr_macro`)
- Réalisez le diagnostic des modèles
- Prédisez les indicateurs pour les trimestres à venir

11 Modèles multivariés pour l'analyse de séries temporelles stationnaires

11 Modèles multivariés pour l'analyse de séries temporelles stationnaires

- Introduction
- Série temporelle multivariée
- Processus VAR
- Test de causalité
- Estimation et sélection du modèle
- Diagnostics
- Fonction de réponse impulsionnelle
- Forecast Error Variance Decomposition
- Modèles VAR Bayesien (BVAR)
- Les modèles SVAR (Structural Vector Autoregressive)

Série temporelle multivariée

Introduction

- L'analyse de séries temporelles multivariées permet notamment
  1. D'étudier les relations **dynamiques** entre variables
  2. D'améliorer la précision des prédictions
- Nous allons nous focaliser ici sur l'analyse de séries temporelles multivariées **stationnaires**
- Les modèles VAR (Vector Autoregressive Models) représentent l'instrument principal pour l'études de séries temporelles multivariées stationnaires

11 Modèles multivariés pour l'analyse de séries temporelles stationnaires
- Introduction
- Série temporelle multivariée
- Processus VAR
- Test de causalité
- Estimation et sélection du modèle
- Diagnostics
- Fonction de réponse impulsionnelle
- Forecast Error Variance Decomposition
- Modèles VAR Bayesien (BVAR)
- Les modèles SVAR (Structural Vector Autoregressive)

PIB et taux de chômage aux USA (1)

- La série multivariée $y_t = (y_{1t}, \ldots, y_{kt}, \ldots, y_{Kt}), \quad k = 1, \ldots, K$ contient K variables
- Par exemple, soit y_{1t} le PIB et y_{2t} le taux de chômage aux USA (exemple tiré de [29] (voir ici)
- Ici $K = 2$
- Les données sont disponibles dans le fichier `q-gdpunemp.txt`

```
gdpunemp <- read.csv('../data/q-gdpunemp.txt', sep=' ')
head(gdpunemp)
```

```
##   year mon      gdp     rate
## 1 1948   1 1821.809 3.733333
## 2 1948   4 1855.345 3.666667
## 3 1948   7 1865.320 3.766667
## 4 1948  10 1868.184 3.833333
## 5 1949   1 1842.240 4.666667
## 6 1949   4 1835.512 5.866667
```

- En étudiant conjointement les deux séries, on peut estimer les relations temporelles et simultanées entre le PIB et le taux de chômage

PIB et taux de chômage aux USA (2)

- Les séries trimestrielles (log du PIB en millions de dollars 2005) CVS sont obtenues à partir de données mensuelles (moyenne), de 1948 à 2011
- Création d'un index et conversion en objet `mts`

```
gdpunemp$loggdp <- log(gdpunemp$gdp)
gdpunemp <- ts(gdpunemp[, c("loggdp", "rate")], start=1948, frequency = 4)
head(gdpunemp)
```

```
##           loggdp     rate
## 1948 Q1 7.507585 3.733333
## 1948 Q2 7.525826 3.666667
## 1948 Q3 7.531188 3.766667
## 1948 Q4 7.532722 3.833333
## 1949 Q1 7.518738 4.666667
## 1949 Q2 7.515079 5.866667
```

- Création d'un objet `tsibble`

```
gdpunemp_tsbl <- as_tsibble(gdpunemp)
head(gdpunemp_tsbl)
```

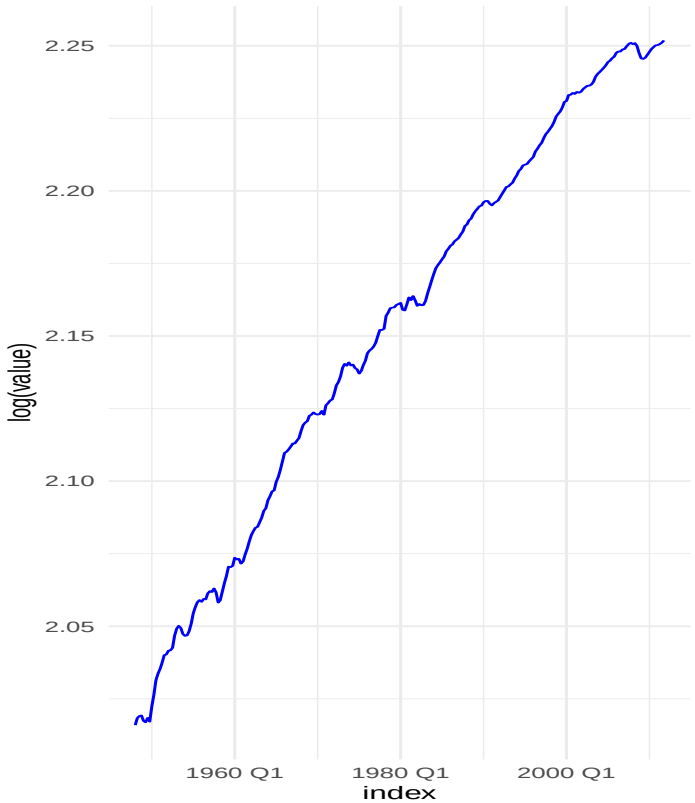
```
## # A tsibble: 6 x 3 [1Q]
## # Key:       key [1]
##     index key    value
##     <qtr> <chr>  <dbl>
## 1 1948 Q1 loggdp  7.51
## 2 1948 Q2 loggdp  7.53
## 3 1948 Q3 loggdp  7.53
## 4 1948 Q4 loggdp  7.53
## 5 1949 Q1 loggdp  7.52
## 6 1949 Q2 loggdp  7.52
```

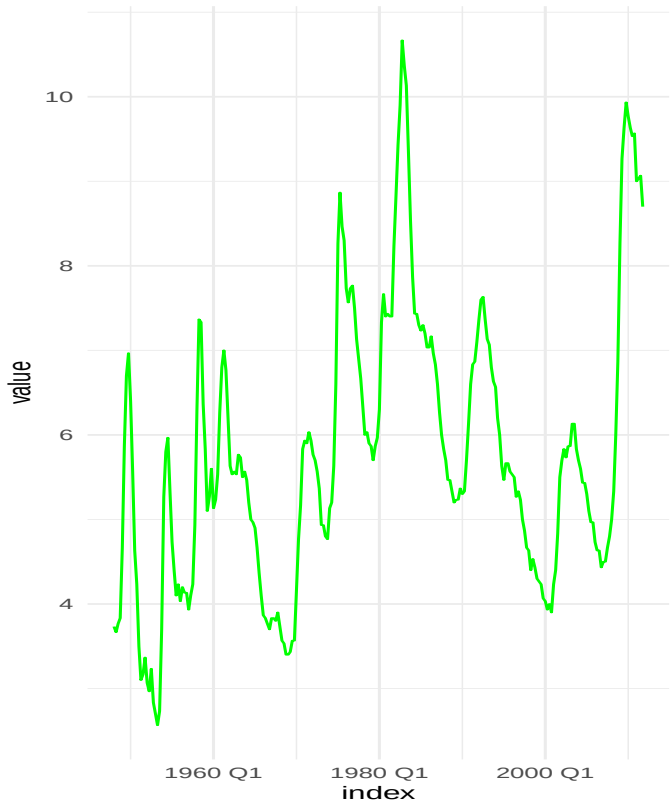
PIB et taux de chômage aux USA

Chronogramme

- Les chronogrammes des deux séries sont combinés avec le symbole '+'

```
g1 <- gdpunemp_tsbl %>% filter(key=="loggdp") %>%
  ggplot() + geom_line(aes(x=index, y=log(value)), color="blue")
g2 <- gdpunemp_tsbl %>% filter(key=="rate") %>%
  ggplot() + geom_line(aes(x=index, y=value), color="green")
g1 + g2
```



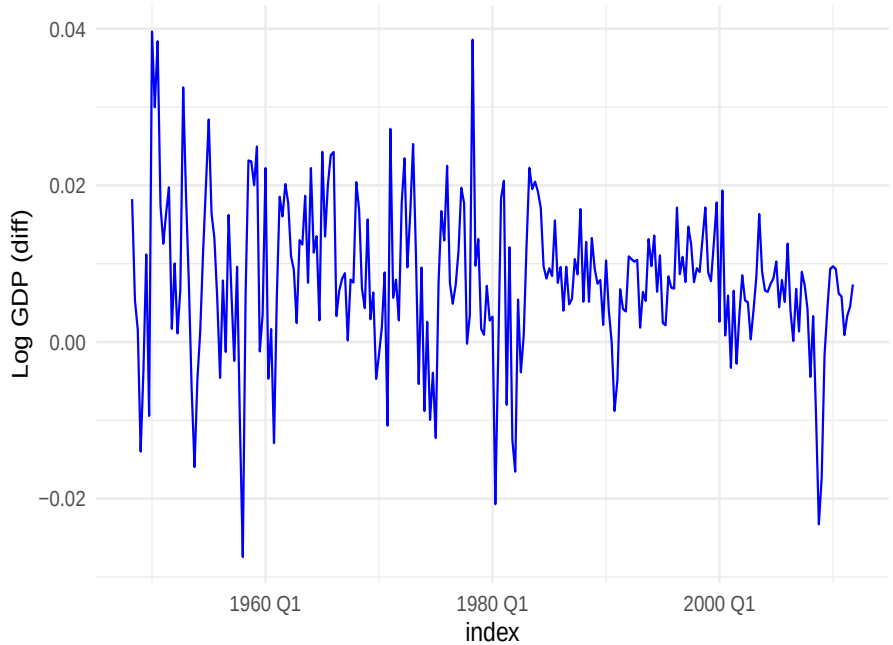


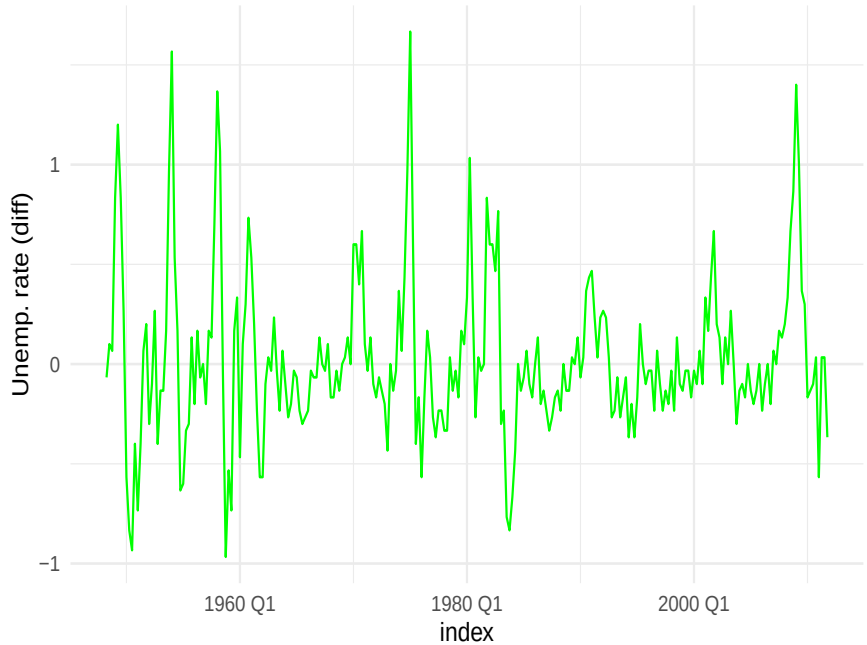
PIB et taux de chômage aux USA

Intégration

- On calcule le taux de croissance du PIB et la variation du taux de chômage en différenciant les séries

```
gdpunemp_i <- diff(gdpunemp) %>% as_tsibble()
g1 <- gdpunemp_i %>% filter(key=="loggdp") %>%
  ggplot() + geom_line(aes(x=index, y=value), color="blue") + ylab("Log GDP (diff)")
g2 <- gdpunemp_i %>% filter(key=="rate") %>%
  ggplot() + geom_line(aes(x=index, y=value), color="green") + ylab("Unemp. rate (diff)")
g1 + g2
```





Matrices de covariance et de corrélation (1)

- La matrice de covariance croisée (*cross-covariance matrix*) de lag ℓ mesure la dépendence linéaire dynamique d'une série temporelle mutlivariée stationnaire y_t

$$\tau_\ell = \text{Cov}(y_t, y_{t-\ell})$$

- Cette matrice de dimension $K \times K$ dépend de ℓ mais pas de t si la série est stationnaire
- Pour $\ell = 0$ nous avons la matrice de covariance τ_0 de y_t
- La matrice de corrélation croissée de lag ℓ est notée ρ_ℓ

$$\rho_\ell = \text{D}^{-1}\tau_{t-\ell}\text{D}^{-1}$$

où D est la matrice diagonale diag$(\sigma_1, , \sigma_k)$ contenant les écarts types des composants de y_t

Matrices de covariance et de corrélation

Exemple : Série d'une loi normale multivariée (1)

- Dans l'exemple ci-dessous (cf. [29]) on créé deux séries de 300 nombres aléatoires tirés d'une loi normale multivariée ($t = 1, ..., 300$ et $K = 2$)
- L'argument `sig` spécifie la matrice de covariance, ici une matrice diagonale 2 x 2 avec les écarts types de chacune des deux séries normales

```
library(MTS)
library(mvtnorm)
sig=diag(2)
x=rmvnorm(300,rep(0,2),sig)
head(x)
```

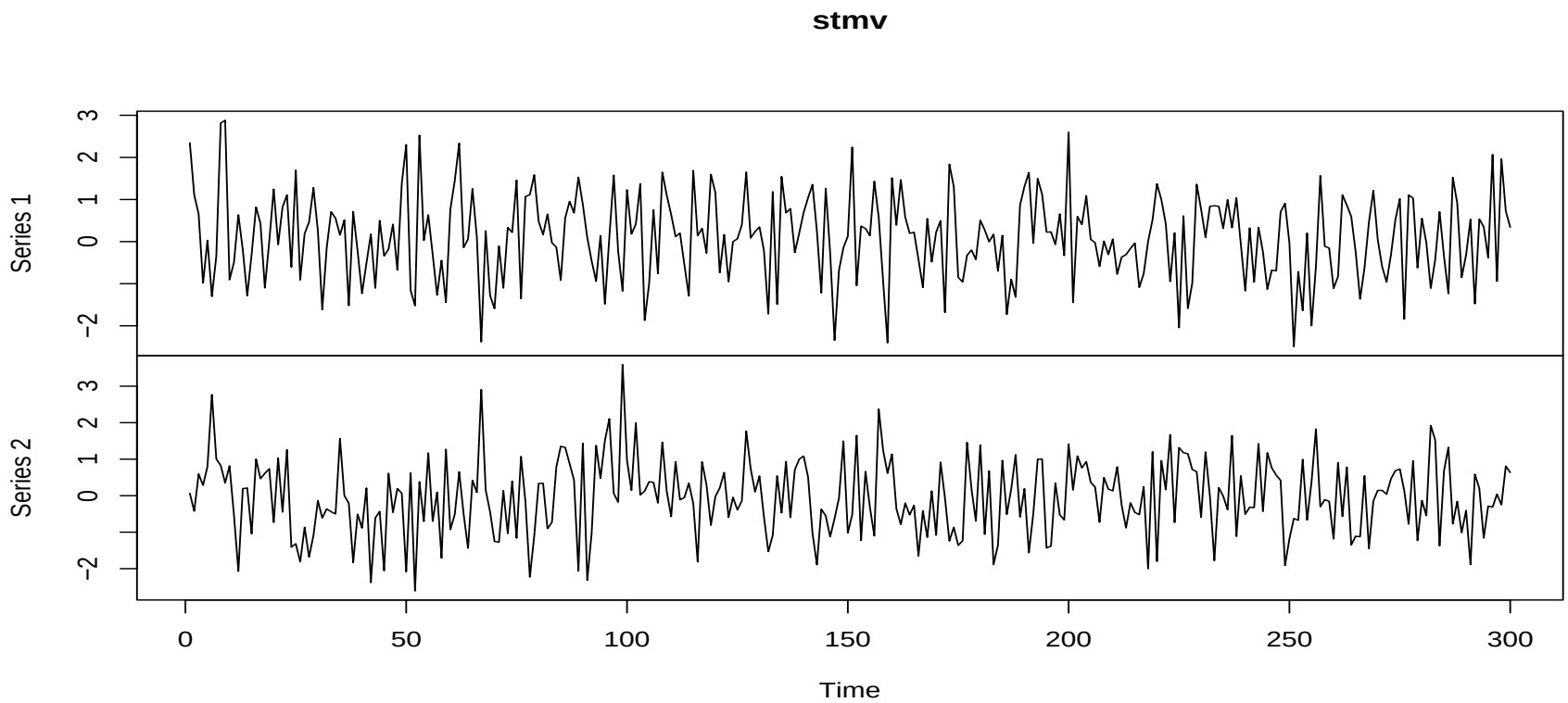
```
##              [,1]        [,2]
## [1,]  2.34596258  0.06393681
## [2,]  1.10922878 -0.42407435
## [3,]  0.65442149  0.59367640
## [4,] -0.98500970  0.28375686
## [5,]  0.02947319  0.79969336
## [6,] -1.30327260  2.77037716
```

Matrices de covariance et de corrélation

Exemple : Série d'une loi normale multivariée (2)

- Chronogramme de la série multivariée (on convertit préalablement en objet `mts`)

```
stmv <- ts(x)
plot(stmv)
```



Matrices de covariance et de corrélation

Exemple : Série d'une loi normale multivariée (3)

- On utilise la fontion `ccm` de la librairie `MTS` pour calculer les matrices de corrélation croisée
- Les coefficients de corrélation sont inférieurs au seuil de significativité (voir également le graphique produit)

```
ccm(x, lags=2, level=TRUE)
```

```
## [1] "Covariance matrix:"
##        [,1]   [,2]
## [1,]  1.020 -0.125
## [2,] -0.125  1.045
## CCM at lag:  0
##        [,1]   [,2]
## [1,]  1.000 -0.121
## [2,] -0.121  1.000
## Simplified matrix:
## CCM at lag:  1
## . .
## . .
## Correlations:
##         [,1]   [,2]
## [1,]  0.0222 0.0980
## [2,] -0.0188 0.0177
## CCM at lag:  2
## . .
## . +
## Correlations:
##          [,1]   [,2]
## [1,] -0.04584 0.0947
## [2,] -0.00961 0.1460
```

Matrices de covariance et de corrélation

Exemple : PIB et taux de chômage aux USA (1)

- Pour les séries du PIB et du taux de chômage différenciées, on obtient des coefficients significatifs

```
ccm(diff(gdpunemp), lags=3, level=TRUE)
```

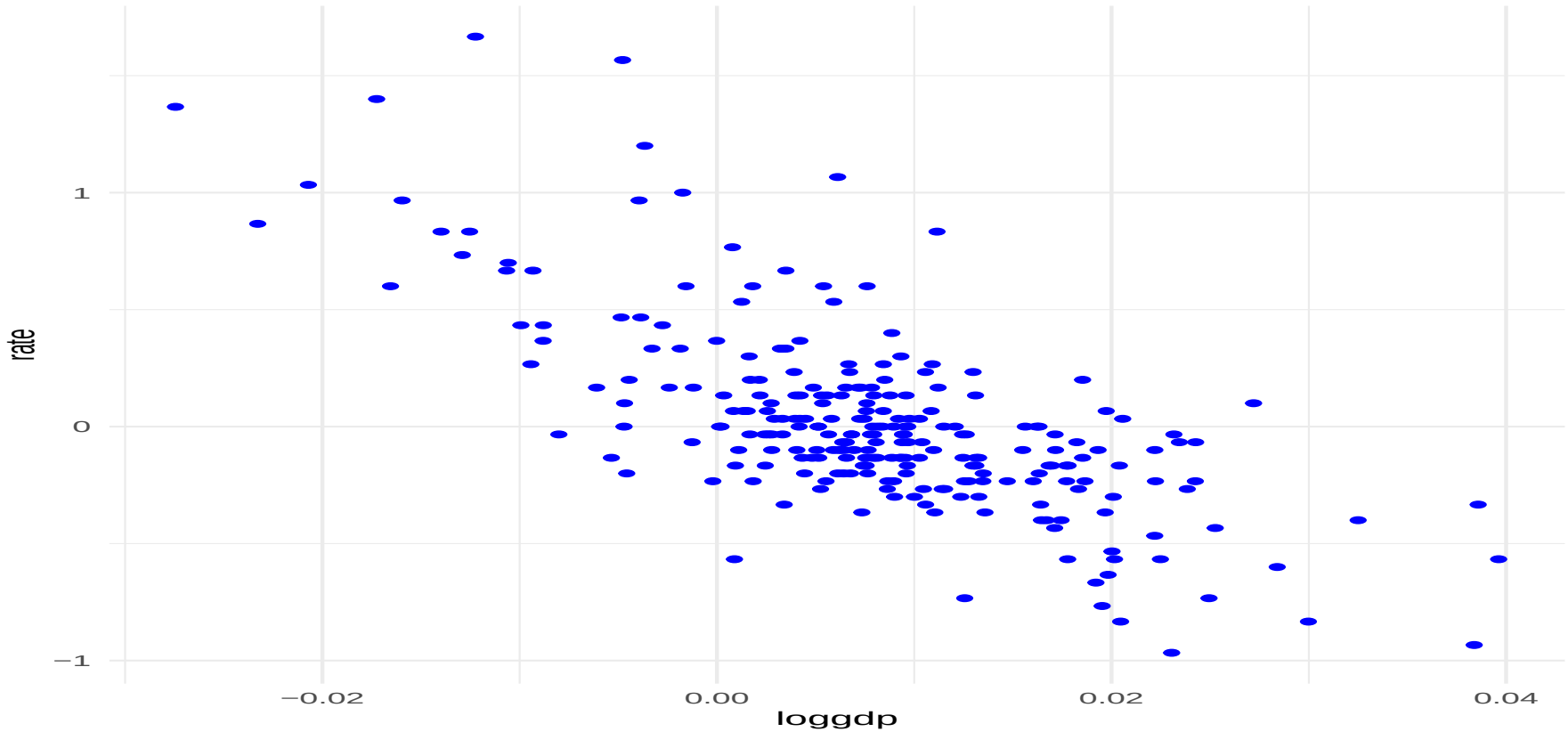
```
## [1] "Covariance matrix:"
##           loggdp     rate
## loggdp  9.95e-05 -0.00281
## rate   -2.81e-03  0.15858
## CCM at lag:  0
##        [,1]   [,2]
## [1,]  1.000 -0.709
## [2,] -0.709  1.000
## Simplified matrix:
## CCM at lag:  1
## + -
## - +
## Correlations:
##        [,1]   [,2]
## [1,]  0.373 -0.355
## [2,] -0.616  0.641
## CCM at lag:  2
## + .
## - +
## Correlations:
##       [,1]    [,2]
## [1,]  0.22 -0.0808
## [2,] -0.44  0.3057
## CCM at lag:  3
## . .
## - .
## Correlations:
##         [,1]   [,2]
## [1,]  0.0164 0.0992
## [2,] -0.1964 0.0373
```

Matrices de covariance et de corrélation

Exemple : PIB et taux de chômage aux USA (2)

- Les deux séries présentent une corrélation instantanée négative

```
diff(gdpunemp) %>% as.data.frame() %>%
  ggplot() + geom_point(aes(x=loggdp, y=rate), color="blue")
```



Test de dépendance linéaire

- Pour tester l'existence de dépendances linéaires dans la série multivariée, on utilise la généralisation du test de Portmanteau (test de Ljung-Box)
- La statistique du test est $Q_k(m)$, où m désigne le lag
- Dans le cas de la croissance du PIB et de la variation du taux de chômage aux USA, $k = 2$
- Hypothèse H_0 : pas de corrélation
- On utilise la fonction `mq()` de la librairie `MTS`

```
y <- diff(gdpunemp)
mq(y, lag=10)
```

```
## Ljung-Box Statistics:
##          m       Q(m)     df    p-value
##  [1,]     1        125      4          0
##  [2,]     2        184      8          0
##  [3,]     3        209     12          0
##  [4,]     4        227     16          0
##  [5,]     5        236     20          0
##  [6,]     6        239     24          0
##  [7,]     7        243     28          0
##  [8,]     8        252     32          0
##  [9,]     9        260     36          0
## [10,]    10        265     40          0
```



Série temporelle multivariée

Exercice

- Le fichier `q-fdebt.txt` contient les séries trimestrielle de la dette des USA détenue par (a) les investisseurs étrangers et internationaux, (b) la réserve fédérale et (c) les particuliers de 1970 à 2012, en milliards de dollars (données de la la réserve fédérale de Saint-Louis) [29]
- Les données ne sont pas CVS
  1. Charger les données, prendre le logarithme et différencier les séries
  2. Afficher le chronogramme des séries (brutes, logarithmes et différenciées)
  3. Afficher les matrices de corrélation croisée de lag 1 à 5
  4. Réaliser un test de Ljung-Box pour les lags 1 à 10

Série temporelle multivariée

Exercice : Solution (1)

- Charger les données dans un objet `tsibble`, prendre le logarithme et différencier la série

```
usdebt <- read.table("../data/q-fdebt.txt", header=T)
head(usdebt)
```

```
##   year mon hbfin hbfrbn   hbpun
## 1 1970   1  12.4   55.8 280.814
## 2 1970   4  14.0   57.7 274.924
## 3 1970   7  16.6   60.0 282.367
## 4 1970  10  19.7   62.1 291.242
## 5 1971   1  24.6   64.2 292.043
## 6 1971   4  31.8   65.5 294.417
```

- Conversion en objet `mts` et transformations

```
usdebt <- ts(usdebt[, 3:5], frequency=4, start=c(1970,1))
window(usdebt, start=c(1970,1), end=c(1971,4))
```

```
##         hbfin hbfrbn   hbpun
## 1970 Q1  12.4   55.8 280.814
## 1970 Q2  14.0   57.7 274.924
## 1970 Q3  16.6   60.0 282.367
## 1970 Q4  19.7   62.1 291.242
## 1971 Q1  24.6   64.2 292.043
## 1971 Q2  31.8   65.5 294.417
## 1971 Q3  41.5   67.6 304.952
## 1971 Q4  46.0   70.2 317.118
```

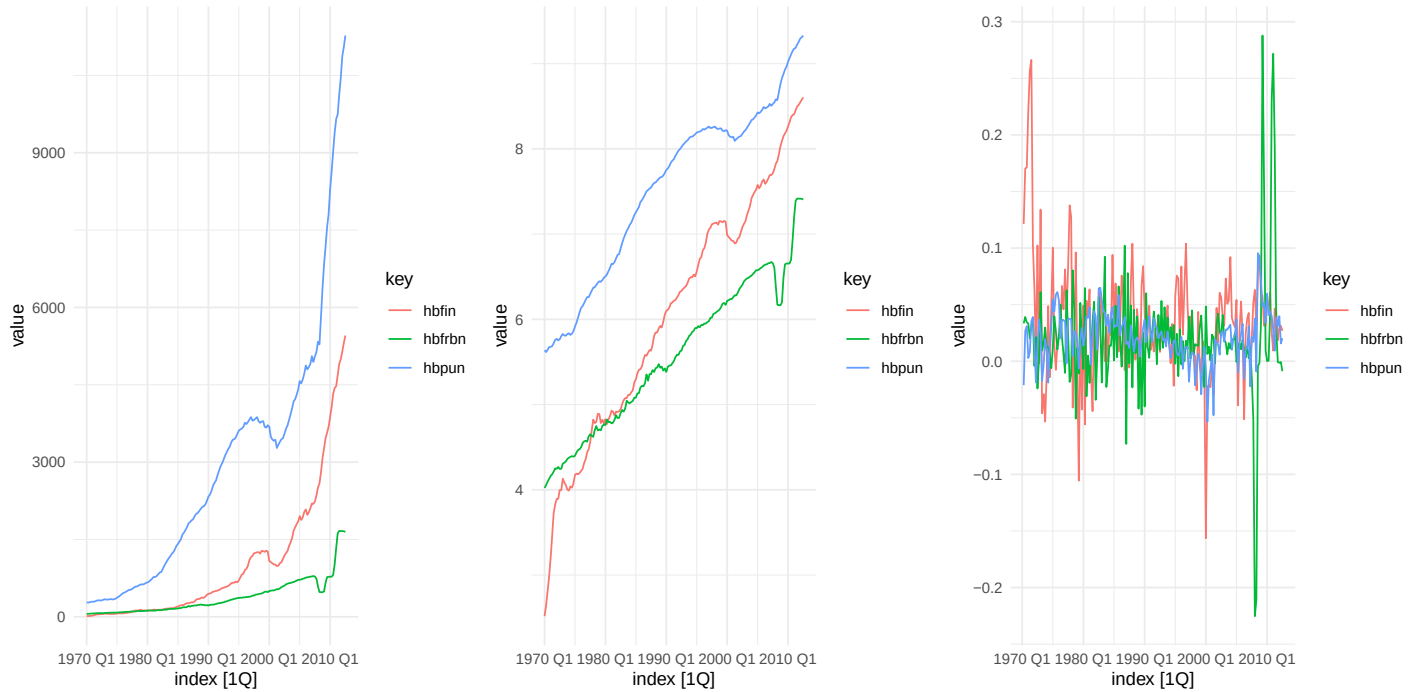
```
log_usdebt <- log(usdebt)
diff_log_usdebt <- diff(log_usdebt)
```

Série temporelle multivariée

Exercice : Solution (2)

- Transformation en objet `tsibble` et chronogramme (on utilise ici la librairie `patchwork` pour combiner les trois graphiques)

```
library(patchwork)
g1 <- as_tsibble(usdebt) %>% autoplot()
g2 <- as_tsibble(log_usdebt) %>% autoplot()
g3 <- as_tsibble(diff_log_usdebt) %>% autoplot()
g1+g2+g3
```



Série temporelle multivariée

Exercice : Solution (3)

Matrices de corrélation croisée

```
ccm(as.data.frame(diff_log_usdebt), lag=5)
```

```
## [1] "Covariance matrix:"
##            hbfin    hbfrbn    hbpun
## hbfin   0.002745 -0.000175 0.000350
## hbfrbn -0.000175  0.002709 0.000104
## hbpun   0.000350  0.000104 0.000488
## CCM at lag:  0
##         [,1]    [,2]   [,3]
## [1,]  1.0000 -0.0642 0.3027
## [2,] -0.0642  1.0000 0.0906
## [3,]  0.3027  0.0906 1.0000
## Simplified matrix:
## CCM at lag:  1
## + . .
## . + .
## + . +
## CCM at lag:  2
## + . .
## . . +
## . . +
## CCM at lag:  3
## + . .
## . . +
## . . +
## CCM at lag:  4
## + . .
## . . .
## . . +
## CCM at lag:  5
## . . .
## . - .
## . . +
```





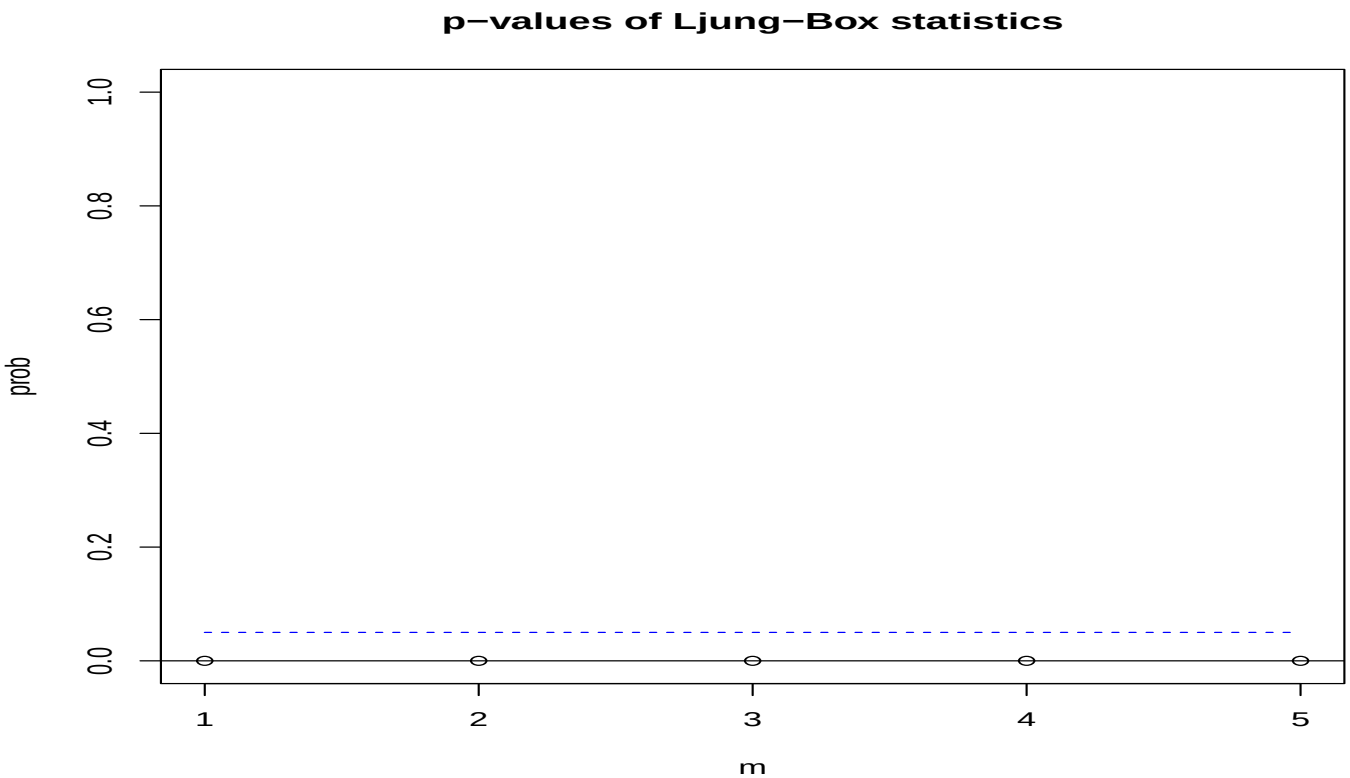
Série temporelle multivariée

Exercice : Solution (4)

Test de Ljung-Box

```
mq(as.matrix(diff_log_usdebt), lag=5)
```

```
## Ljung-Box Statistics:
##        m       Q(m)     df    p-value
## [1,]     1       112      9          0
## [2,]     2       181     18          0
## [3,]     3       242     27          0
## [4,]     4       330     36          0
## [5,]     5       352     45          0
```



11 Modèles multivariés pour l'analyse de séries temporelles stationnaires
- Introduction
- Série temporelle multivariée
- Processus VAR
- Test de causalité
- Estimation et sélection du modèle
- Diagnostics
- Fonction de réponse impulsionnelle
- Forecast Error Variance Decomposition
- Modèles VAR Bayesien (BVAR)
- Les modèles SVAR (Structural Vector Autoregressive)

Définition

- Les modèles VAR (Vector Autoregression) sont l'extension des modèles AR à des séries multivariées
- L'évolution d'une série est modélisée par ses valeurs passées et celles des autres séries
- La série multivariée $y_t = (y_{1t}, \ldots, y_{kt}, \ldots, y_{Kt}), \quad k = 1, \ldots, K$ contient K variables
- Un processus $VAR(p)$ est défini par :

$$y_t = A_1 y_{t-1} + \ldots + A_p y_{t-p} + \mu_t$$

avec $E(\mu_t) = 0$

- A_i est une matrice de $(K \times K)$ coefficients, $i = 1, \ldots, p$

Stationnarité

- La série multivariée y_t est stationnaire (covariance stationary) si :
  - Son espérance est constante : $E(y_t) = \mu$, $\mu = (\mu_1, \mu_2, \ldots, \mu_K)^T$
  - La covariance $cov(X_{t_1 i}, X_{t_2 j})$ est fonction uniquement de $t2 - t1$ pour chaque i et j

Taux de croissance du PIB (UK, Canada, US)

Données

- Exemple tiré de [29] : taux de croissances trimestriels du PIB (en pct.), au Royaume-Uni (uk), Canada (ca), et USA (us) du 2ème trimestre 1980 au 2ème trimestre 2011
- Données CVS (source Federal Reserve Bank, St. Louis)
- Le PIB est exprimé en millions (monnaie nationale)

```
varpib <- read.table("../data/q-gdp-ukcaus.txt", header=T)
varpib <- log(varpib[,3:5])
head(varpib)
```

```
##         uk       ca       us
## 1 12.05778 13.34518 15.59190
## 2 12.03978 13.34300 15.57119
## 3 12.03788 13.34265 15.56933
## 4 12.02683 13.35382 15.58766
## 5 12.02006 13.37849 15.60822
## 6 12.02211 13.38773 15.60021
```

- Les séries intégrées du log(PIB) représentent les taux de croissance

```
varpib_ts <- ts(varpib, start=c(1980,1), frequency=4)
pibgr <- diff(varpib_ts)*100
head(pibgr)
```

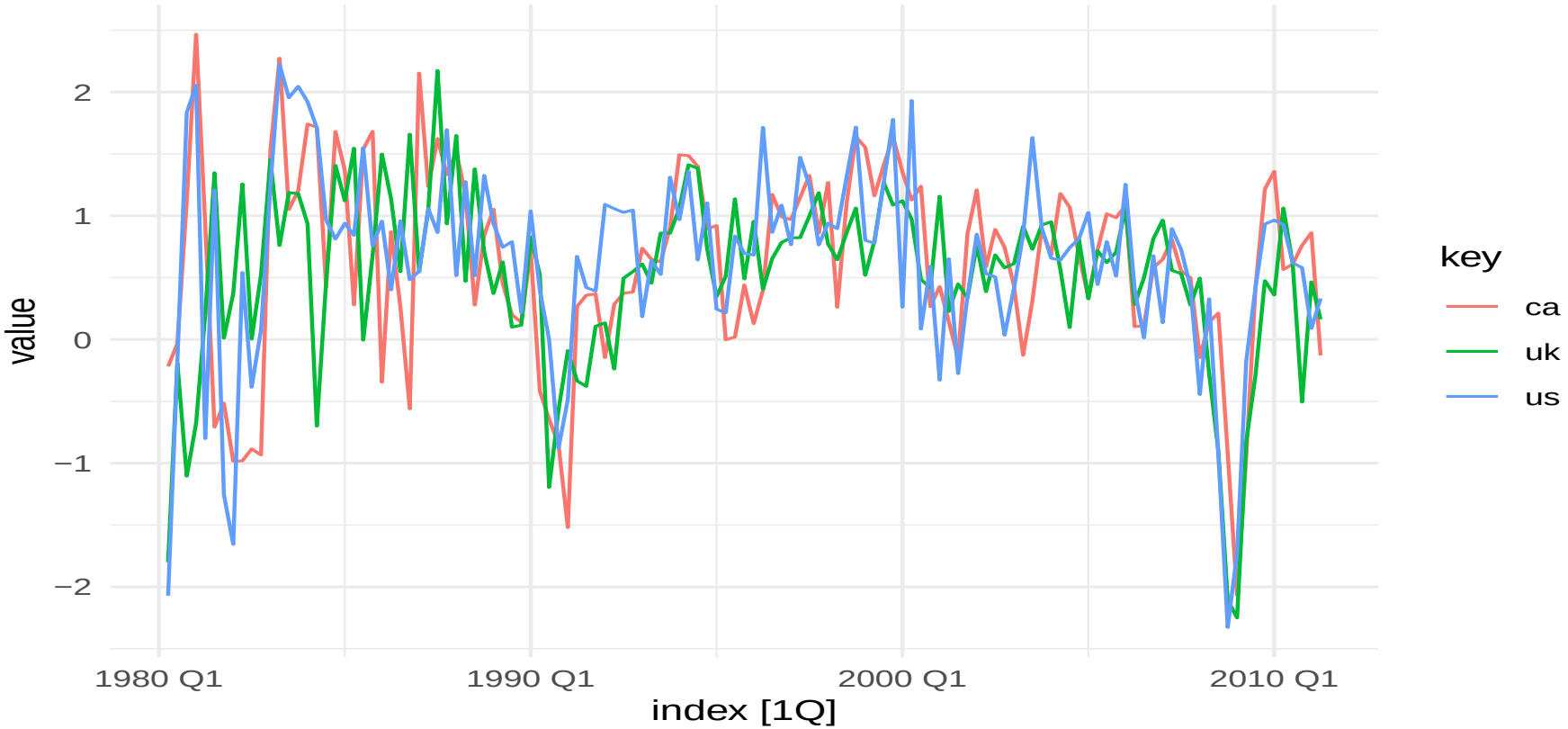
```
##                  uk          ca         us
## 1980 Q2 -1.8005431 -0.21806940 -2.0708851
## 1980 Q3 -0.1897181 -0.03497379 -0.1867866
## 1980 Q4 -1.1052465  1.11694674  1.8336549
## 1981 Q1 -0.6770084  2.46719071  2.0557195
## 1981 Q2  0.2051468  0.92410692 -0.8007733
## 1981 Q3  1.3479031 -0.70853560  1.2078892
```

Taux de croissance du PIB (UK, Canada, US)

Chronogramme

- Chronogramme des trois séries intégrées (taux de croissance)

```
pibgr %>% as_tsibble() %>% autoplot()
## Plot variable not specified, automatically selected `.vars = value`
```



Taux de croissance du PIB (UK, Canada, US)

Test de stationnarité sur les séries intégrée

- On vérifie que la série est stationnaire après différentiation
  - Hypothèse H_0 : la série présente une tendance stochastique (racine unitaire)
  - Hypothèse H_1 : la série est stationnaire

```
pibuk.df <- ur.df(pibgr[, "uk"], type = "drift", lags = 1)
```

- Valeur de la statistique du test

```
attr(pibuk.df, "teststat")
```

```
##                tau2     phi1
## statistic -4.588346 10.54152
```

- Valeurs critiques

```
attr(pibuk.df, "cval")
```

```
##       1pct  5pct 10pct
## tau2 -3.46 -2.88 -2.57
## phi1  6.52  4.63  3.81
```

Taux de croissance du PIB (UK, Canada, US)

Estimation du modèle VAR

- On utilise la fonction `VAR()` (librairie `vars` ([28], voir l'article ici)
- On estime ici un modèle $VAR(2)$

```
library("vars")
pibgr.VAR <- VAR(pibgr, p=2)
```

- Les estimations se trouvent dans `varresult`

```
names(pibgr.VAR)
```

```
## [1] "varresult"    "datamat"      "y"            "type"         "p"
## [6] "K"            "obs"          "totobs"       "restrictions" "call"
```

Taux de croissance du PIB (UK, Canada, US)

Résultats : Série UK

- La méthode `summary()` permet d'obtenir les estimations pour chaque série
- Pour la série `uk`, seul le paramètre associé à `uk.l1` (lag 1) est significatif

```
summary(pibgr.VAR)$varresult$uk
```

```
##
## Call:
## lm(formula = y ~ -1 + ., data = datamat)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.85491 -0.23752  0.05079  0.33566  1.31252
##
## Coefficients:
##       Estimate Std. Error t value Pr(>|t|)
## uk.l1  0.39307    0.09342   4.208 5.11e-05 ***
## ca.l1  0.10311    0.09838   1.048    0.297
## us.l1  0.05214    0.09113   0.572    0.568
## uk.l2  0.05660    0.09237   0.613    0.541
## ca.l2  0.10552    0.08756   1.205    0.231
## us.l2  0.01889    0.09382   0.201    0.841
## const  0.12582    0.07266   1.731    0.086 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.5473 on 116 degrees of freedom
## Multiple R-squared:  0.3829, Adjusted R-squared:  0.3509
## F-statistic: 11.99 on 6 and 116 DF,  p-value: 1.907e-10
```

Taux de croissance du PIB (UK, Canada, US)

Résultats : Série Canada

Estimations pour l'équation de la série ca

```
summary(pibgr.VAR)$varresult$ca
```

```
##
## Call:
## lm(formula = y ~ -1 + ., data = datamat)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.51554 -0.31867  0.04956  0.34149  1.57007
##
## Coefficients:
##        Estimate Std. Error t value Pr(>|t|)
## uk.l1  0.351314   0.094917   3.701 0.000330 ***
## ca.l1  0.338142   0.099963   3.383 0.000979 ***
## us.l1  0.469094   0.092589   5.066 1.55e-06 ***
## uk.l2 -0.191350   0.093856  -2.039 0.043747 *
## ca.l2 -0.174833   0.088964  -1.965 0.051780 .
## us.l2 -0.008678   0.095326  -0.091 0.927624
## const  0.123158   0.073829   1.668 0.097984 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.556 on 116 degrees of freedom
## Multiple R-squared:  0.5238, Adjusted R-squared:  0.4992
## F-statistic: 21.27 on 6 and 116 DF,  p-value: < 2.2e-16
```

Taux de croissance du PIB (UK, Canada, US)

Résultats : Série US

Estimations pour l'équation de la série us

```
summary(pibgr.VAR)$varresult$us
```

```
##
## Call:
## lm(formula = y ~ -1 + ., data = datamat)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -2.18937 -0.27457  0.03623  0.32495  1.56051
##
## Coefficients:
##       Estimate Std. Error t value Pr(>|t|)
## uk.l1  0.49070    0.10502   4.672 8.08e-06 ***
## ca.l1  0.24000    0.11060   2.170 0.032053 *
## us.l1  0.23564    0.10245   2.300 0.023226 *
## uk.l2 -0.31196    0.10385  -3.004 0.003265 **
## ca.l2 -0.13118    0.09843  -1.333 0.185259
## us.l2  0.08531    0.10547   0.809 0.420253
## const  0.28956    0.08169   3.545 0.000568 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.6152 on 116 degrees of freedom
## Multiple R-squared:  0.3606, Adjusted R-squared:  0.3276
## F-statistic:  10.9 on 6 and 116 DF,  p-value: 1.325e-09
```

Taux de croissance du PIB (UK, Canada, US)

Sélection du modèle (1)

- La fonction `VARselect` propose différents critères pour la sélection du modèle

```
VARselect(pibgr, lag.max = 10)
```

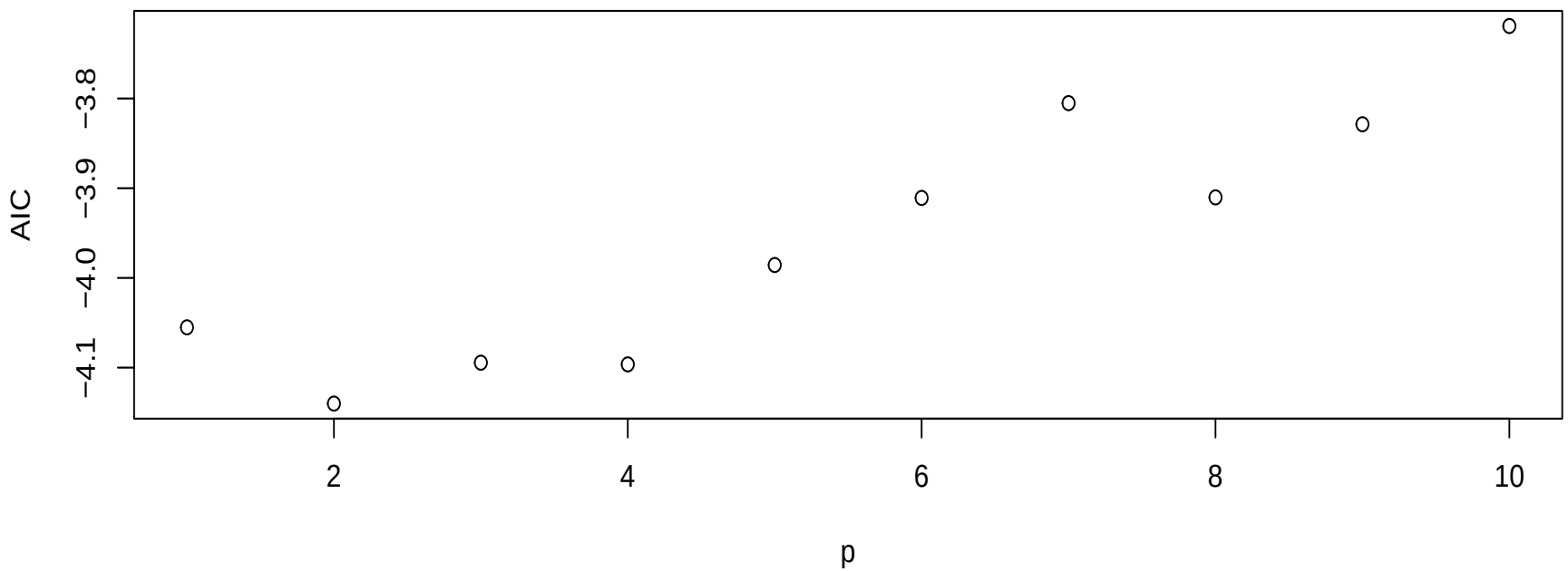
```
## $selection
## AIC(n)  HQ(n)  SC(n) FPE(n)
##      2      1      1      2
##
## $criteria
##                  1          2          3          4          5          6
## AIC(n) -4.05509087 -4.1400600 -4.09449192 -4.09635510 -3.98562143 -3.91082343
## HQ(n)  -3.93883130 -3.9366058 -3.80384298 -3.71851148 -3.52058313 -3.35859045
## SC(n)  -3.76866317 -3.6388115 -3.37842267 -3.16546507 -2.83991063 -2.55029185
## FPE(n)  0.01733536  0.0159291  0.01668624  0.01668169  0.01868243  0.02020848
##                  7          8          9         10
## AIC(n) -3.80517918 -3.91020870 -3.82872219 -3.71917784
## HQ(n)  -3.16575152 -3.18358636 -3.01490516 -2.81816614
## SC(n)  -2.22982683 -2.12003557 -1.82372828 -1.49936317
## FPE(n)  0.02257616  0.02046461  0.02239803  0.02526843
```

Taux de croissance du PIB (UK, Canada, US)

Sélection du modèle (2)

- Selon l'AIC, la valeur optimale est $p = 2$

```
plot(VARselect(pibgr, lag.max = 10)$criteria[1,], xlab='p', ylab='AIC')
```



Taux de croissance du PIB (UK, Canada, US)

Estimation avec la librairie MTS

- La librairie MTS [29] propose également une fonction VAR() pour l'estimation
- La syntaxe MTS::VAR() permet d'utiliser spécifiquement la fonction VAR() de MTS

```
library(MTS)
m1.mts <- MTS::VAR(pibgr,2)
```

```
## Constant term:
## Estimates:  0.1258163 0.1231581 0.2895581
## Std.Error:  0.07266338 0.07382941 0.0816888
## AR coefficient matrix
## AR( 1 )-matrix
##       [,1]  [,2]   [,3]
## [1,] 0.393 0.103 0.0521
## [2,] 0.351 0.338 0.4691
## [3,] 0.491 0.240 0.2356
## standard error
##        [,1]   [,2]   [,3]
## [1,] 0.0934 0.0984 0.0911
## [2,] 0.0949 0.1000 0.0926
## [3,] 0.1050 0.1106 0.1024
## AR( 2 )-matrix
##         [,1]   [,2]     [,3]
## [1,]  0.0566  0.106  0.01889
## [2,] -0.1914 -0.175 -0.00868
## [3,] -0.3120 -0.131  0.08531
## standard error
##        [,1]   [,2]   [,3]
## [1,] 0.0924 0.0876 0.0938
## [2,] 0.0939 0.0890 0.0953
## [3,] 0.1038 0.0984 0.1055
##
## Residuals cov-mtx:
##             [,1]       [,2]       [,3]
## [1,] 0.28244420 0.02654091 0.07435286
## [2,] 0.02654091 0.29158166 0.13948786
```

Taux de croissance du PIB (UK, Canada, US)

Simplification du modèle

- Les modèles $VAR(p)$ comprennent généralement un nombre élevé de paramètres (lorsque K et p augmentent), dont certains ne sont pas statistiquement significatifs
- On peut simplifier le modèle en éliminant les paramètres non-significatifs
- La librairie `MTS` propose des fonctions pour simplifier le modèle
- La fonction `refVAR()` force les paramètres non-significatifs d'un modèle VAR à 0, sur la base d'un test de Chi2 Tsay-2014

Taux de croissance du PIB (UK, Canada, US)

Simplification du modèle : la fonction `refVAR()`

- Le seuil pour le test est fixé ici à 5%

```
m2.mts <- refVAR(m1.mts, thres=1.96)
```

```
## Constant term:
## Estimates:  0.1628247 0 0.2827525
## Std.Error:  0.06814101 0 0.07972864
## AR coefficient matrix
## AR( 1 )-matrix
##       [,1]  [,2]  [,3]
## [1,] 0.467 0.207 0.000
## [2,] 0.334 0.270 0.496
## [3,] 0.468 0.225 0.232
## standard error
##       [,1]   [,2]   [,3]
## [1,] 0.0790 0.0686 0.0000
## [2,] 0.0921 0.0875 0.0913
## [3,] 0.1027 0.0963 0.1023
## AR( 2 )-matrix
##       [,1] [,2] [,3]
## [1,]  0.000    0    0
## [2,] -0.197    0    0
## [3,] -0.301    0    0
## standard error
##       [,1] [,2] [,3]
## [1,] 0.0000    0    0
## [2,] 0.0921    0    0
## [3,] 0.1008    0    0
##
## Residuals cov-mtx:
##            [,1]       [,2]       [,3]
## [1,] 0.29003669 0.01803456 0.07055856
## [2,] 0.01803456 0.30802503 0.14598345
## [3,] 0.07055856 0.14598345 0.36268779
##
## det(SSE) =  0.02494104
## AIC =  -3.531241
```

Taux de croissance du PIB (UK, Canada, US)

Simplification du modèle (2)

- Le modèle $VAR(2)$ contient 21 paramètres, le modèle simplifié contient 12 paramètres
- L'AIC du modèle simplifié est inférieur à celui du modèle de départ

```
m1.mts$aic
## [1] -3.502259
m2.mts$aic
## [1] -3.531241
```

Taux de croissance du PIB (UK, Canada, US)

Simplification du modèle (2)

- Equations du modèle simplifié

$$UK : \quad y_{1,t} = 0.16 + 0.47y_{1,t} + 0.21y_{2,t-1} + a_{1,t} \tag{2}$$

$$CA : \quad y_{2,t} = 0.33 + y_{1,t-1} + 0.27y_{2,t-1} + 0.5y_{3,t-1} - 0.2y_{1,t-2} + a_{2,t} \tag{3}$$

$$US : \quad y_{3,t} = 0.28 + 0.47y_{1,t-1} + 0.23y_{2,t-1} + 0.23y_{3,t-1} - 0.3y_{1,t-2} + a_{3,t} \tag{4}$$

Taux de croissance du PIB (UK, Canada, US)

Diagnostics du modèle VAR (1)

- La fonction `serial.test()` de la librairie `vars` calcule un test de Portmanteau multivarié

```
pibgr.VAR.serial <- serial.test(pibgr.VAR, type="PT.asymptotic")
pibgr.VAR.serial
```

```
##
##  Portmanteau Test (asymptotic)
##
## data:  Residuals of VAR object pibgr.VAR
## Chi-squared = 124.51, df = 126, p-value = 0.5207
```

Taux de croissance du PIB (UK, Canada, US)

Diagnostics du modèle VAR : Corrélations croisées

- La fonction `MTSdiag()` de la librairie `MTS` renvoie les matrices de corrélation croisées des résidus (ici jusqu'au lag 4)
- Etant donné qu'il s'agit des résidus d'une régression, il faut ajuster le nombre de degrés de liberté, ici le nombre de paramètres du modèle $VAR(2)$ simplifié

```
MTSdiag(m2.mts, gof=4, adj=12)
```

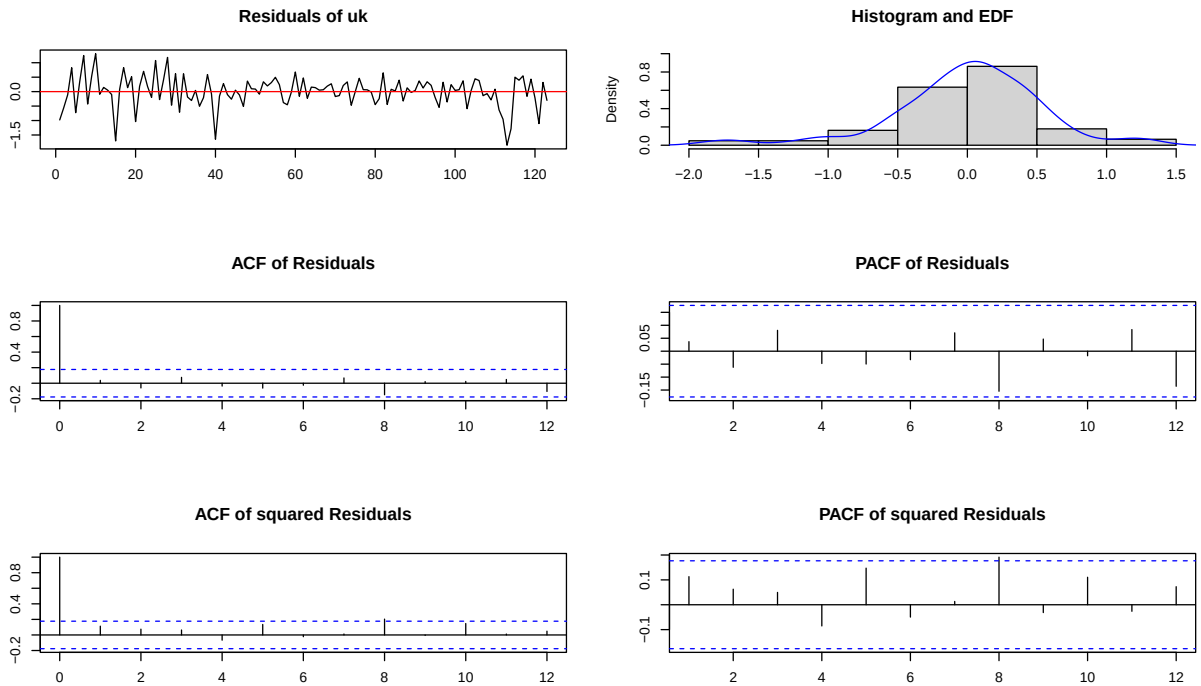
```
## [1] "Covariance matrix:"
##        uk     ca     us
## uk 0.2924 0.0182 0.0711
## ca 0.0182 0.3084 0.1472
## us 0.0711 0.1472 0.3657
## CCM at lag:  0
##        [,1]   [,2]  [,3]
## [1,] 1.0000 0.0605 0.218
## [2,] 0.0605 1.0000 0.438
## [3,] 0.2175 0.4382 1.000
## Simplified matrix:
## CCM at lag:  1
## . . .
## . . .
## . . .
## CCM at lag:  2
## . . .
## . . .
## . . .
## CCM at lag:  3
## . . .
## . . .
## . . .
## CCM at lag:  4
## . . -
## . . .
## . . .
```

Taux de croissance du PIB (UK, Canada, US)

Diagnostics : Analyse des résidus (série UK)

- La méthode générique `plot` pour les objets renvoyés par la fonction `serial.test` permet de visualiser les autocorrélations des résidus

```
plot(pibgr.VAR.serial, names="uk")
```

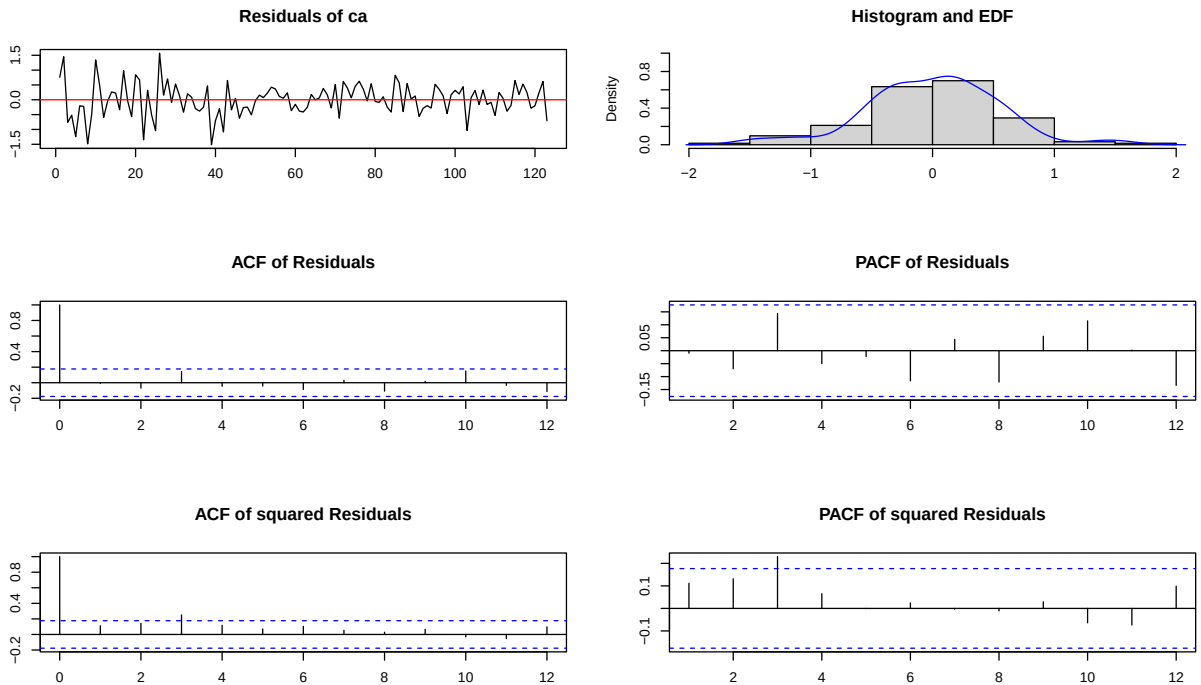


Taux de croissance du PIB (UK, Canada, US)

Diagnostics : Analyse des résidus (série Canada)

Résidus pour la série us

```
plot(pibgr.VAR.serial, names="ca")
```

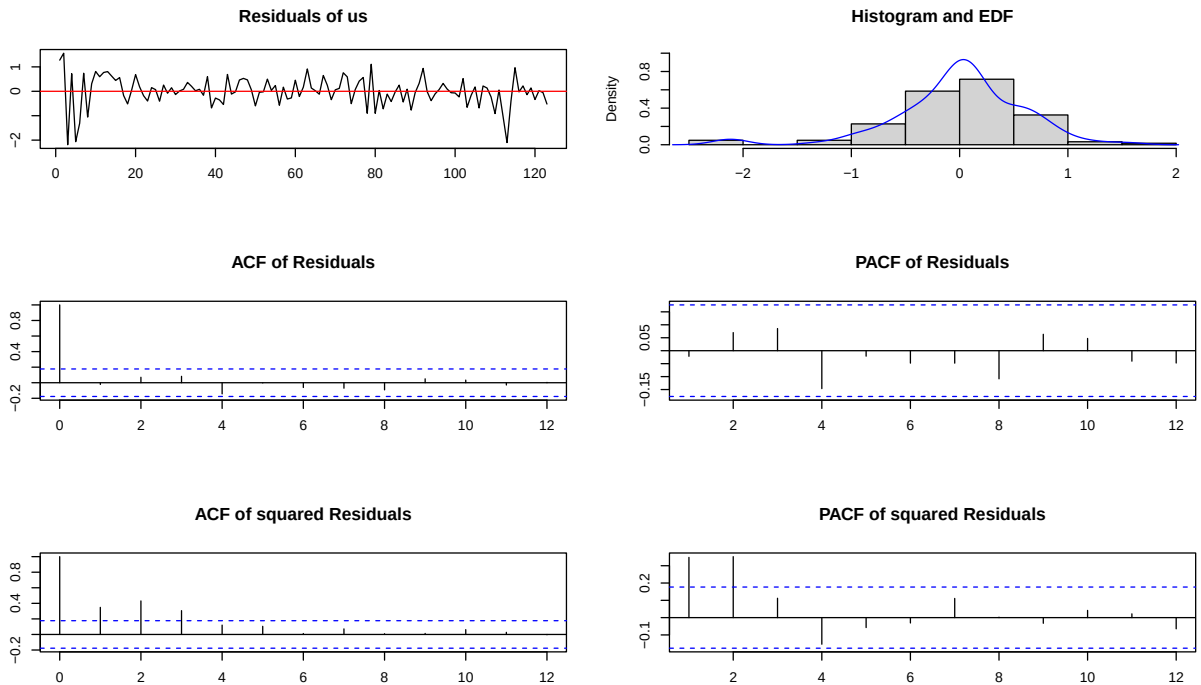


Taux de croissance du PIB (UK, Canada, US)

Diagnostics : Analyse des résidus (série US)

Résidus pour la série us

```
plot(pibgr.VAR.serial, names="us")
```



Taux de croissance du PIB (UK, Canada, US)

Diagnostics : Normalité des résidus

- Test de la normalité des résidus (on obtient un test univarié pour chaque série avec l'argument `multivariate.only = TRUE`)

```
pibgr.VAR.norm <- normality.test(pibgr.VAR, multivariate.only = TRUE)
pibgr.VAR.norm
```

```
## $JB
##
##  JB-Test (multivariate)
##
## data:  Residuals of VAR object pibgr.VAR
## Chi-squared = 44.251, df = 6, p-value = 6.592e-08
##
##
## $Skewness
##
##  Skewness only (multivariate)
##
## data:  Residuals of VAR object pibgr.VAR
## Chi-squared = 17.229, df = 3, p-value = 0.000634
##
##
## $Kurtosis
##
##  Kurtosis only (multivariate)
##
## data:  Residuals of VAR object pibgr.VAR
## Chi-squared = 27.021, df = 3, p-value = 5.827e-06
```

Taux de croissance du PIB (UK, Canada, US)

Diagnostics : Homoscédasticité des résidus

- Test de l'homoscédasticité des résidus (test ARCH, Hamilton [10] et Lütkepohl [22]) avec la fonction `arch.test()`
- L'hypothèse H_0 d'homoscédasticité des résidus est rejetée

```
pibgr.VAR.arch <- arch.test(pibgr.VAR, lags.multi = 5, multivariate.only = TRUE)
pibgr.VAR.arch
```

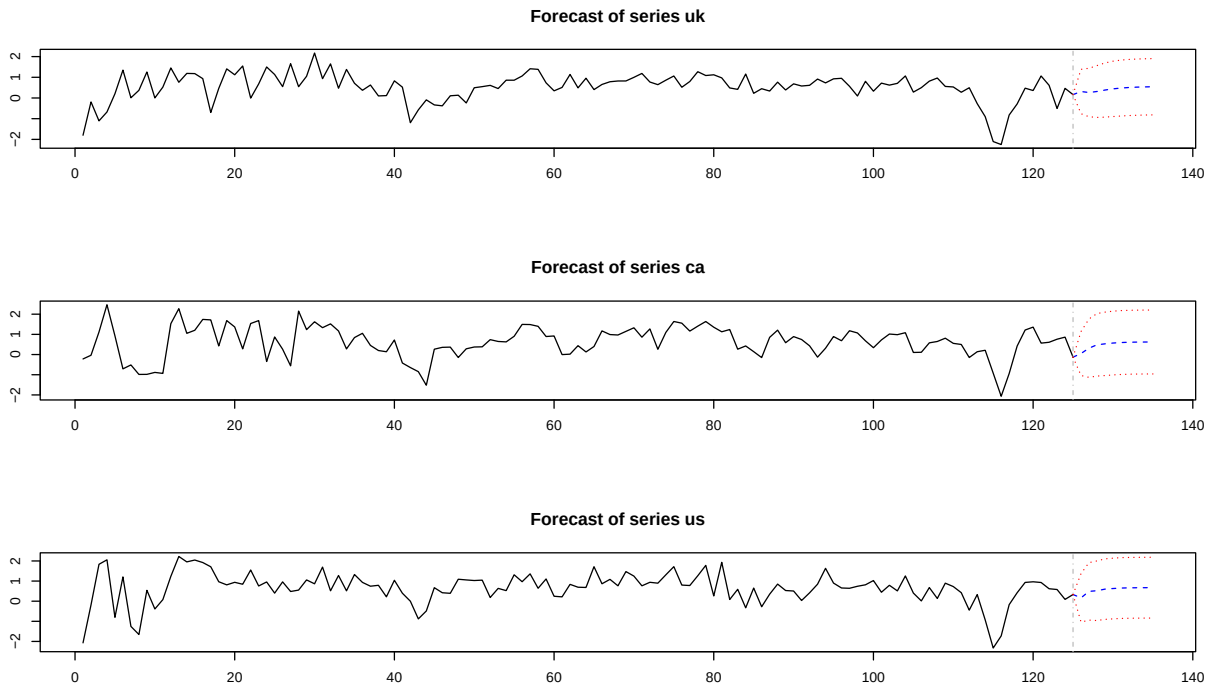
```
##
##  ARCH (multivariate)
##
## data:  Residuals of VAR object pibgr.VAR
## Chi-squared = 275.77, df = 180, p-value = 5.661e-06
```

Taux de croissance du PIB (UK, Canada, US)

Prévision

- On utilise la méthode générique `predict()` pour la prévision

```
plot(predict(pibgr.VAR))
```



Taux de croissance du PIB (UK, Canada, US)

Test structurel (1)

- La librairie `vars` propose également un test de stabilité structurelle basé sur la fonction `efp()` (empirical fluctuation process), librairie `strucchange`, voir Zeileis et al. [31])
- La fonction `stability()` permet d'appliquer les différents types de test à chacune des équations du modèle VAR

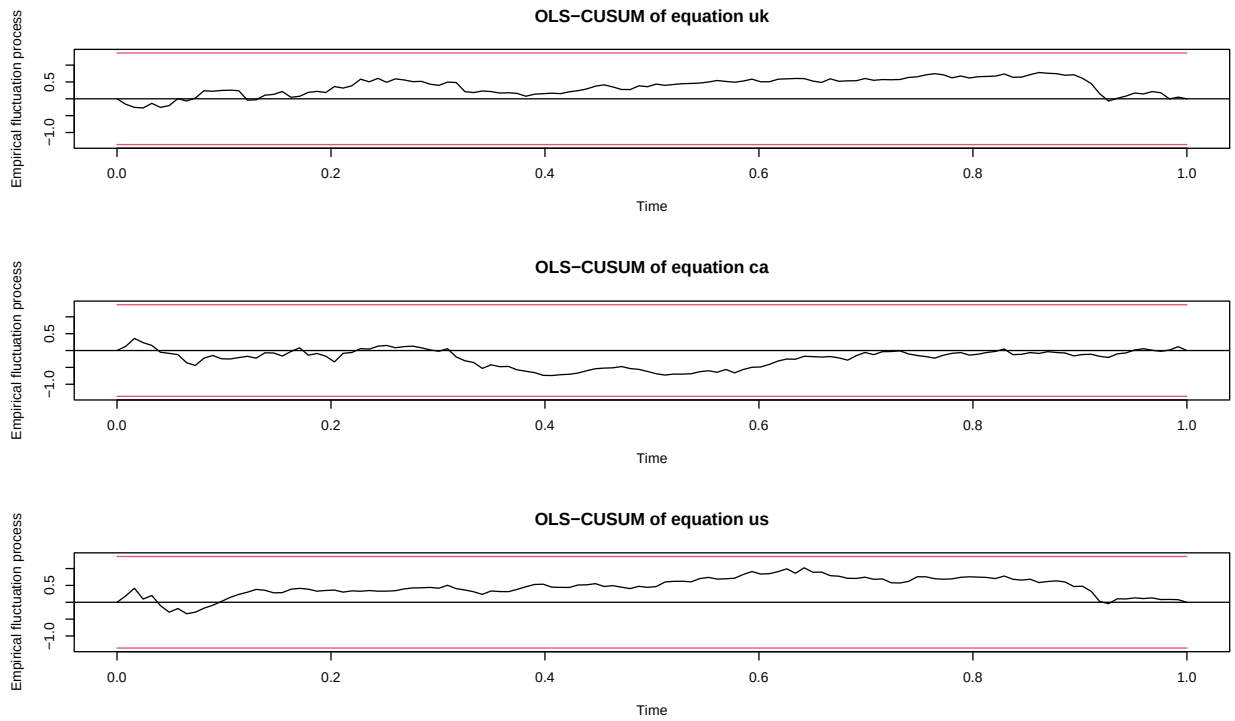
```
reccusum <- stability(pibgr.VAR, type = "OLS-CUSUM")
```

Taux de croissance du PIB (UK, Canada, US)

Test structurel (2)

- Le test ne détecte aucune instabilité structurelle

```
plot(reccusum)
```



11 Modèles multivariés pour l'analyse de séries temporelles stationnaires
- Introduction
- Série temporelle multivariée
- Processus VAR
- Test de causalité
- Estimation et sélection du modèle
- Diagnostics
- Fonction de réponse impulsionnelle
- Forecast Error Variance Decomposition
- Modèles VAR Bayesien (BVAR)
- Les modèles SVAR (Structural Vector Autoregressive)

- Le **test de Granger** est utilisé pour la détection de **causalités** entre variables
- Attention, le test de Granger ne permet pas de conclure à une causalité à proprement parler (voir par exemple [27])
- On parle de **causalité au sens de Granger** (*Granger causality*) si la connaissance du passé d'une série temporelle y_{1t} entraîne une prévision de y_{2t} distincte de celle fondée uniquement sur le passé de y_{2t}, i.e. y_{1t} aide à prédire y_{2t}
- On peut également réaliser un test de causalité instantannée (*instantaneous causality test*, voir Pfaff [27])

Taux de croissance du PIB (UK, Canada, US)

Test de causalité (1)

- La fonction `causality` de la librairie `vars` permet d'effectuer le test de Granger et le test de causalité immédiate
- On indique la liste des variables de la série dont on veut tester la causalité de Granger

```
causality(pibgr.VAR, cause=c("us", "ca"))
```

```
## $Granger
##
##  Granger causality H0: ca us do not Granger-cause uk
##
## data:  VAR object pibgr.VAR
## F-Test = 2.2372, df1 = 4, df2 = 348, p-value = 0.06467
##
##
## $Instant
##
##  H0: No instantaneous causality between: ca us and uk
##
## data:  VAR object pibgr.VAR
## Chi-squared = 6.4042, df = 2, p-value = 0.04068
```

Taux de croissance du PIB (UK, Canada, US)

Test de causalité (2)

- Ici on teste la causalité de Granger des séries `uk` et `ca`

```
causality(pibgr.VAR, cause=c("uk", "ca"))
```

```
## $Granger
##
##  Granger causality H0: uk ca do not Granger-cause us
##
## data:  VAR object pibgr.VAR
## F-Test = 6.8066, df1 = 4, df2 = 348, p-value = 2.768e-05
##
##
## $Instant
##
##  H0: No instantaneous causality between: uk ca and us
##
## data:  VAR object pibgr.VAR
## Chi-squared = 22.589, df = 2, p-value = 1.244e-05
```

Taux de croissance du PIB (UK, Canada, US)

Test de causalité (2)

- Causalité de Granger des séries `uk` et `us`

```
causality(pibgr.VAR, cause=c("uk", "us"))
```

```
## $Granger
##
##  Granger causality H0: uk us do not Granger-cause ca
##
## data:  VAR object pibgr.VAR
## F-Test = 12.21, df1 = 4, df2 = 348, p-value = 2.67e-09
##
##
## $Instant
##
##  H0: No instantaneous causality between: uk us and ca
##
## data:  VAR object pibgr.VAR
## Chi-squared = 19.379, df = 2, p-value = 6.194e-05
```

Données Canada (1)

- L'analyse suivante (voir Lütkepohl et Krätzig [23]) est reprise par Pfaff [27]
- Les données sont fournies avec la librairie `vars` (voir l'article ici)
- Les séries utilisées représentent des indicateurs macro-économiques du marché du travail au Canada (données OCDE) :
  - `e` : nombre d'actifs (employment)
  - `prod` : productivité du travail (log différence entre GDP et nombre d'actifs)
  - `rw` : log de l'index des salaires réels (real wage index)
  - `U` : taux de chômage (unemployment rate)
- Les séries s'étendent du premier trimestre 1980 aux quatrième trimestre 2004

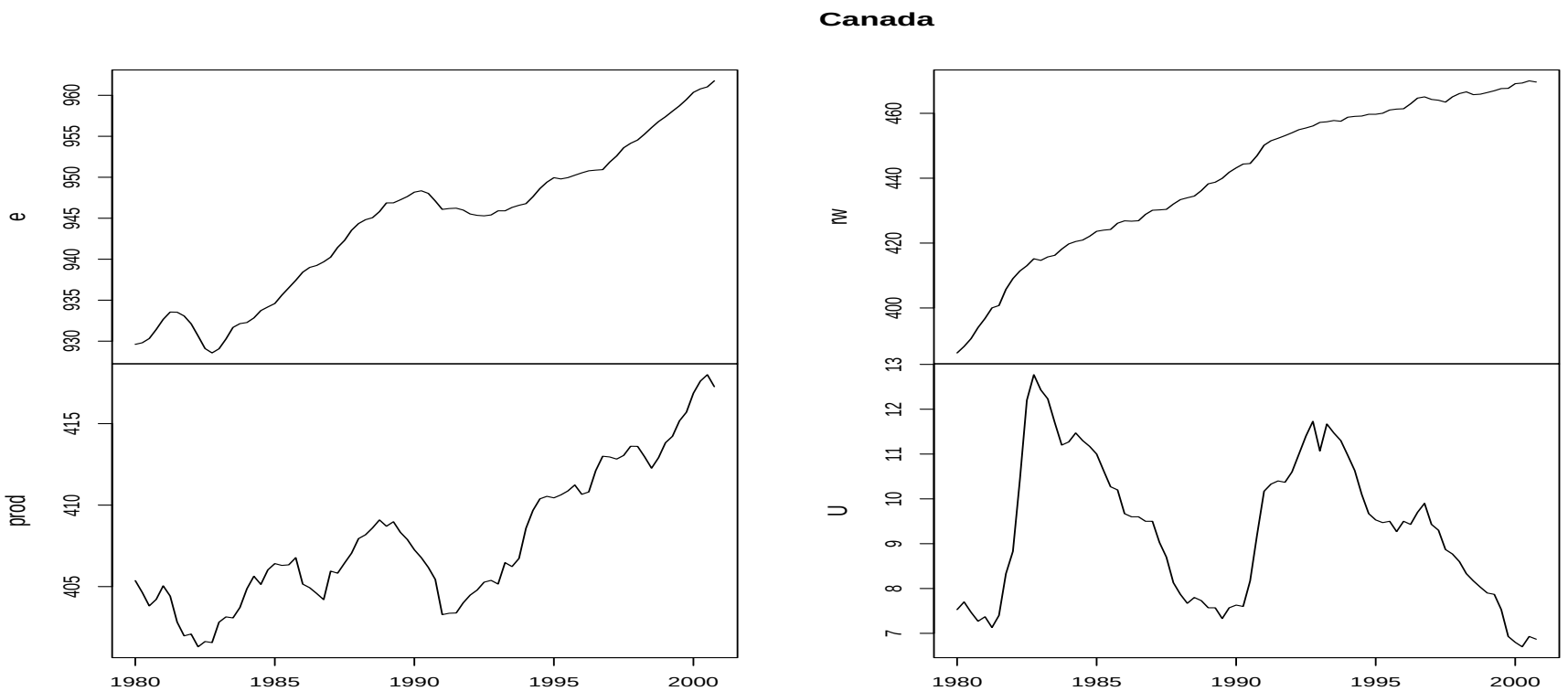
```
data("Canada")
window(Canada, start=c(1980,1), end=c(1981,4))
```

```
##                e     prod       rw    U
## 1980 Q1 929.6105 405.3665 386.1361 7.53
## 1980 Q2 929.8040 404.6398 388.1358 7.70
## 1980 Q3 930.3184 403.8149 390.5401 7.47
## 1980 Q4 931.4277 404.2158 393.9638 7.27
## 1981 Q1 932.6620 405.0467 396.7647 7.37
## 1981 Q2 933.5509 404.4167 400.0217 7.13
## 1981 Q3 933.5315 402.8191 400.7515 7.40
## 1981 Q4 933.0769 401.9773 405.7335 8.33
```

Données Canada (2)

- Chronogramme des séries `e`, `prod`, `rw`, `U` (données `Canada`)

```
plot(Canada, nc = 2, xlab = "")
```



Données Canada (3)

- La première étape de l'analyse consiste à estimer un modèle VAR sur les données
- Les séries `e`, `prod` et `rw` présentent une tendance haussière ; la série `U` ne paraît pas stationnaire
- On commence par réaliser un test de stationnarité ADF (Augmented Dickey-Fuller)sur les 4 séries avant l'esimation d'un modèle VAR

Données Canada

Test de stationnarité sur la série prod (1)

- La série prod présente une tendance, on utilise type='trend'
  - Hypothèse nulle H0 : la série est **non stationnaire avec dérive**
  - Hypothèse alternative H1 : la série est **stationnaire avec trend déterministe**

```
prod.adf1 <- ur.df(Canada[, "prod"], type = 'trend', lags = 2)
```

- Valeur de la statistique du test

```
attr(prod.adf1, "teststat")
```

```
##                tau3     phi2     phi3
## statistic -1.987512 2.300006 2.381689
```

- Valeurs critiques

```
attr(prod.adf1, "cval")
```

```
##       1pct  5pct 10pct
## tau3 -4.04 -3.45 -3.15
## phi2  6.50  4.88  4.16
## phi3  8.73  6.49  5.47
```

- La valeur tau3 est supérieure à la valeur critique à 10%, on accepte H0, la série peut être rendue stationnaire par différentiation

Données Canada

Test de stationnarité sur la série prod intégrée (1)

- On vérifie que la série est stationnaire après différentiation

```
prod.diff <- diff(Canada[, "prod"])
prod.diff.adf1 <- ur.df(prod.diff, type = "drift", lags = 1)
```

- Valeur de la statistique du test

```
attr(prod.diff.adf1, "teststat")
```

```
##                tau2     phi1
## statistic -5.160425 13.31839
```

- Valeurs critiques

```
attr(prod.diff.adf1, "cval")
```

```
##       1pct  5pct 10pct
## tau2 -3.51 -2.89 -2.58
## phi1  6.70  4.71  3.86
```

- La valeur de tau2 est inférieure à la valeur critique à 1%, on rejete H0 et on conclut à la stationnarité de la série

Données Canada

Test de stationnarité sur la série prod intégrée (2)

- Test de stationnarité avec la fonction `adf.test`

```
adf.test(diff(Canada[, "prod"]), k = 1)
```

```
## Warning in adf.test(diff(Canada[, "prod"]), k = 1): p-value smaller than printed p-value
```

```
##
##  Augmented Dickey-Fuller Test
##
## data:  diff(Canada[, "prod"])
## Dickey-Fuller = -5.1952, Lag order = 1, p-value = 0.01
## alternative hypothesis: stationary
```

Données Canada

Test de stationnarité sur les séries e, U et rw

- Les tests de stationarité sur les autres séries originales et différenciées sont également réalisés
- Etant donné la forme du chronogramme des séries, une constante et une tendance déterministe sont inclues dans les tests, sauf pour la série U pour laquelle seule une constante est inclue

```
summary(ur.df(Canada[, "e"], type = "trend", lags = 2))
summary(ur.df(Canada[, "U"], type = "drift", lags = 1))
summary(ur.df(Canada[, "rw"], type = "trend", lags = 4))
```

Données Canada

Test de stationnarité sur les séries e, U et rw différenciées

- Pour les tests de stationarité des séries différenciées, une constante est inclue dans la régression (`type = "drift"`) pour les séries `e` et `rw`, et aucun terme déterministe pour la série `U`
- Le nombre de retards est diminué de 1

```
summary(ur.df(diff(Canada[, "e"]), type = "drift", lags = 1))
summary(ur.df(diff(Canada[, "U"]), type = "none", lags = 0))
summary(ur.df(diff(Canada[, "rw"]), type = "drift", lags = 3))
summary(ur.df(diff(Canada[, "rw"]), type = "drift", lags = 0))
```

- Toutes les séries sont rendues stationnaires avec une intégration d'ordre 1, i.e. toutes les séries sont $I(1)$ (des relations de cointégration peuvent exister)

11 Modèles multivariés pour l'analyse de séries temporelles stationnaires
- Introduction
- Série temporelle multivariée
- Processus VAR
- Test de causalité
- Estimation et sélection du modèle
- Diagnostics
- Fonction de réponse impulsionnelle
- Forecast Error Variance Decomposition
- Modèles VAR Bayesien (BVAR)
- Les modèles SVAR (Structural Vector Autoregressive)

Données Canada

Sélection du modèle $VAR(p)$

- La longueur optimale du lag est déterminée avec la fonction `VARselect()`
- La valeur optimale selon le critère AIC est de 3

```
VARselect(Canada, lag.max = 8, type = "both")
```

```
## $selection
## AIC(n)  HQ(n)  SC(n) FPE(n)
##      3      2      1      3
##
## $criteria
##                  1            2            3            4            5
## AIC(n) -6.272579064 -6.636669705 -6.771176872 -6.634609210 -6.398132246
## HQ(n)  -5.978429449 -6.146420347 -6.084827770 -5.752160366 -5.319583658
## SC(n)  -5.536558009 -5.409967947 -5.053794411 -4.426546046 -3.699388378
## FPE(n)  0.001889842  0.001319462  0.001166019  0.001363175  0.001782055
##                  6            7           8
## AIC(n) -6.307704843 -6.070727259 -6.06159685
## HQ(n)  -5.033056512 -4.599979185 -4.39474903
## SC(n)  -3.118280272 -2.390621985 -1.89081087
## FPE(n)  0.002044202  0.002768551  0.00306012
```

Données Canada

Estimation du modèle $VAR(p)$

- On commence par un modèle d'ordre $p = 1$
- L'argument `type = 'both'` indique qu'on inclut la constante et la tendance dans le modèle

```
p1ct <- VAR(Canada, p = 1, type = 'both')
```

Données Canada

Equation pour la série e

Paramètres de l'équation pour la série e

```
summary(p1ct, equation = "e")$varresult
```

```
## $e
##
## Call:
## lm(formula = y ~ -1 + ., data = datamat)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.41558 -0.15408  0.04734  0.23819  1.14554
##
## Coefficients:
##           Estimate Std. Error t value Pr(>|t|)
## e.l1       1.23892    0.08632  14.353  < 2e-16 ***
## prod.l1    0.19465    0.03612   5.389 7.49e-07 ***
## rw.l1     -0.06776    0.02828  -2.396 0.018991 *
## U.l1       0.62301    0.16927   3.681 0.000430 ***
## const   -278.76121   75.18295  -3.708 0.000392 ***
## trend     -0.04066    0.01970  -2.064 0.042378 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.4701 on 77 degrees of freedom
## Multiple R-squared:  0.9975, Adjusted R-squared:  0.9973
## F-statistic:  6088 on 5 and 77 DF,  p-value: < 2.2e-16
```

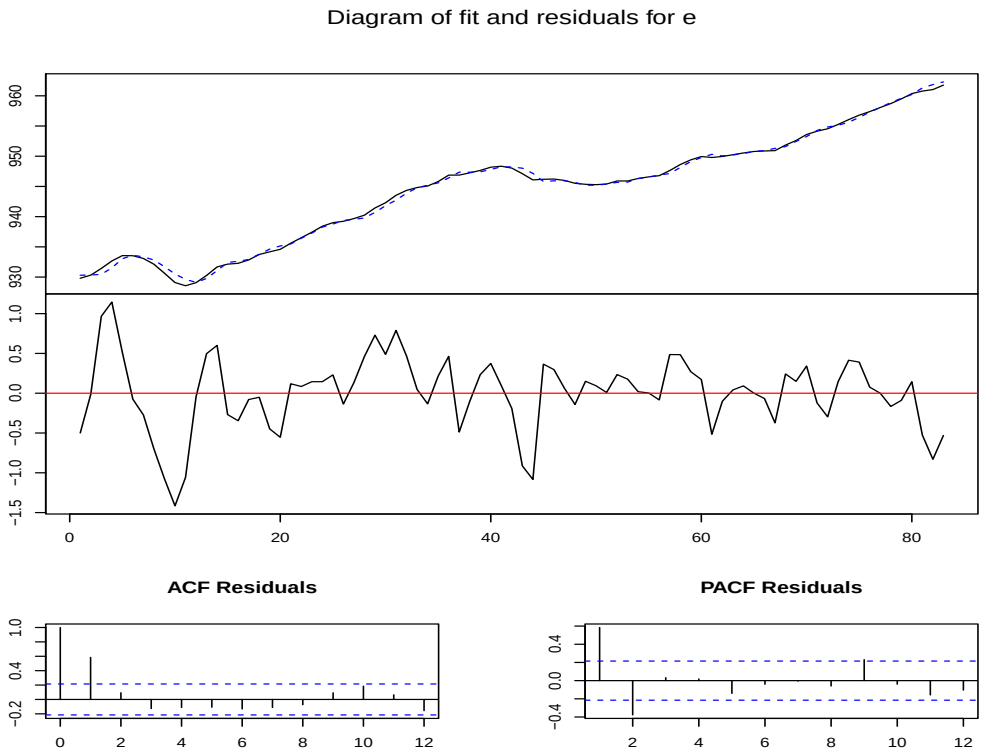
11 Modèles multivariés pour l'analyse de séries temporelles stationnaires
- Introduction
- Série temporelle multivariée
- Processus VAR
- Test de causalité
- Estimation et sélection du modèle
- Diagnostics
- Fonction de réponse impulsionnelle
- Forecast Error Variance Decomposition
- Modèles VAR Bayesien (BVAR)
- Les modèles SVAR (Structural Vector Autoregressive)

Données Canada

Série e : Valeurs prédites et résidus

- Valeurs prédites et résidus pour la série e

```
plot(p1ct, names = "e")
```



Données Canada

Diagnostics du modèle *VAR* (1)

Autocorrélation des résidus

```
ser11 <- serial.test(p1ct, lags.pt = 16, type = "PT.asymptotic")
ser11$serial
```

```
##
##  Portmanteau Test (asymptotic)
##
## data:  Residuals of VAR object p1ct
## Chi-squared = 233.5, df = 240, p-value = 0.606
```

Données Canada

Diagnostics du modèle *VAR* (2)

Matrice de corrélation croisée des résidus

```
ccm(residuals(p1ct))
```

```
## [1] "Covariance matrix:"
##            e     prod       rw       U
## e     0.2075  0.06354 -0.07774 -0.1240
## prod  0.0635  0.44089  0.00201 -0.0388
## rw   -0.0777  0.00201  0.55701  0.0599
## U    -0.1240 -0.03876  0.05985  0.1142
## CCM at lag:  0
##        [,1]     [,2]     [,3]   [,4]
## [1,]  1.000  0.21008 -0.22869 -0.805
## [2,]  0.210  1.00000  0.00406 -0.173
## [3,] -0.229  0.00406  1.00000  0.237
## [4,] -0.805 -0.17275  0.23731  1.000
## Simplified matrix:
## CCM at lag:  1
## + . . -
## . . . -
## . . . .
## - . . +
## CCM at lag:  2
## . . . .
## . . . .
## . . . .
## - . . .
## CCM at lag:  3
## . . . .
## . . . .
## . . . .
## . . . .
## CCM at lag:  4
## . . . .
## . . . .
## . . + .
## . . . .
## CCM at lag:  5
```

Tests de changement structurel

- La fonction `stability()` permet de réaliser des tests de changement structurel
- Cette fonction utilise la fonction `efp()` de la librairie `strucchange` ; elle retourne une liste de classe `varstabil` dont le premier élément contient les résultats des tests pour chacune des équations (i.e. des éléments de classe `efp`)

```
p1ct.stchange <- stability(p1ct)
names(p1ct.stchange$stability)
```

```
## [1] "e"    "prod" "rw"   "U"
```

-

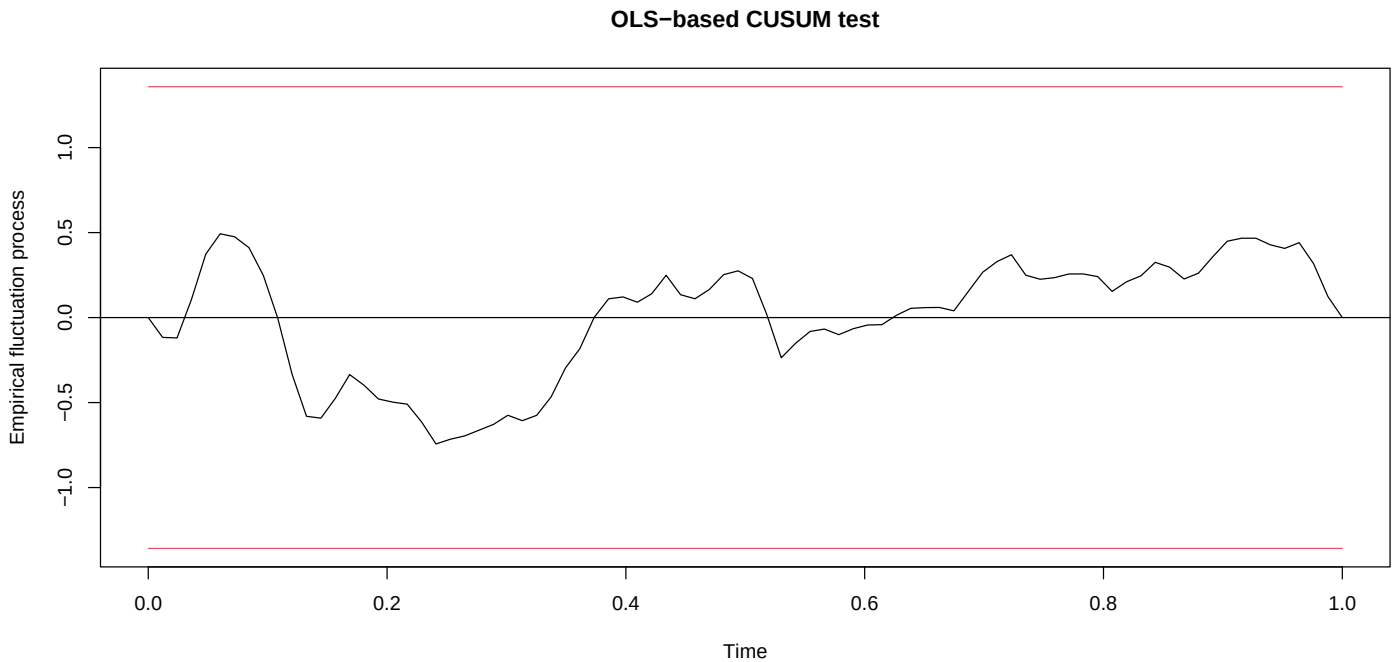
```
p1ct.stchange[["stability"]][["e"]]
```

```
##
## Empirical Fluctuation Process: OLS-based CUSUM test
##
## Call: efp(formula = formula, data = data, type = type, h = h, dynamic = dynamic,
##     rescale = rescale)
```

Tests de changement structurel

- On peut utiliser la méthode `plot` sur chacun des éléments de la liste

```
plot(p1ct.stchange[[1]][[1]], nc=3)
```



Modèle VAR

Exercice

- Modélisez l'évolution conjointe des PIB du Royaume-Uni, du Canada, des USA et de la France à l'aide d'un modèle VAR
- Faites le diagnostic du modèle

Modèle VAR

Exercice : Solutions (1)

- Préparation des données : taux de variation du PIB pour les séries UK, CA, US

```
# Données UK CA US
varpib <- read.table("../data/q-gdp-ukcaus.txt", header=T)
varpib <- log(varpib[,3:5])
varpib <- diff(ts(varpib, start=c(1980,1), frequency=4)) %>% as_tsibble()

# Passage du format long au format large
tmp1 <- varpib %>% spread(key=key, value=value)
tmp1
```

```
## # A tsibble: 125 x 4 [1Q]
##      index        ca          uk       us
##      <qtr>     <dbl>       <dbl>    <dbl>
##  1 1980 Q2 -0.00218  -0.0180    -0.0207
##  2 1980 Q3 -0.000350 -0.00190   -0.00187
##  3 1980 Q4  0.0112   -0.0111     0.0183
##  4 1981 Q1  0.0247   -0.00677    0.0206
##  5 1981 Q2  0.00924   0.00205   -0.00801
##  6 1981 Q3 -0.00709   0.0135     0.0121
##  7 1981 Q4 -0.00514   0.000107  -0.0125
##  8 1982 Q1 -0.00984   0.00371   -0.0166
##  9 1982 Q2 -0.00981   0.0126     0.00541
## 10 1982 Q3 -0.00885   0.0000292 -0.00386
## # i 115 more rows
```

Modèle VAR

Exercice : Solutions (2)

- Préparation des données : taux de variation du PIB pour la série FR

```
load("../data/fr_macro.RData")
frpib <- diff(log(ts(fr_macro$gnp, start=1949, frequency = 4))) %>%
  as_tsibble() %>% rename(fr=value)
```

- Fusion des deux jeux de données avec la fonction `inner_join()`

```
varpib2 <-  tmp1 %>% inner_join(frpib, by="index")
varpib2
```

```
## # A tsibble: 125 x 5 [1Q]
##      index        ca          uk       us        fr
##      <qtr>     <dbl>       <dbl>    <dbl>     <dbl>
##  1 1980 Q2 -0.00218  -0.0180    -0.0207  -0.00780
##  2 1980 Q3 -0.000350 -0.00190   -0.00187  0.00110
##  3 1980 Q4  0.0112   -0.0111     0.0183  -0.00172
##  4 1981 Q1  0.0247   -0.00677    0.0206   0.00339
##  5 1981 Q2  0.00924   0.00205   -0.00801  0.00683
##  6 1981 Q3 -0.00709   0.0135     0.0121   0.00664
##  7 1981 Q4 -0.00514   0.000107  -0.0125   0.00577
##  8 1982 Q1 -0.00984   0.00371   -0.0166   0.00831
##  9 1982 Q2 -0.00981   0.0126     0.00541  0.00720
## 10 1982 Q3 -0.00885   0.0000292 -0.00386 -0.000437
## # i 115 more rows
```

Modèle VAR

Exercice : Solutions (3)

- Sélection et estimation du modèle avec la fonction `VARselect()`

```
VARselect(varpib2[,2:5])
```

```
## $selection
## AIC(n)  HQ(n)  SC(n) FPE(n)
##      2      1      1      2
##
## $criteria
##                  1             2             3             4             5
## AIC(n) -4.265132e+01 -4.276568e+01 -4.274854e+01 -4.269153e+01 -4.248326e+01
## HQ(n)  -4.245756e+01 -4.241690e+01 -4.224475e+01 -4.203272e+01 -4.166944e+01
## SC(n)  -4.217394e+01 -4.190639e+01 -4.150736e+01 -4.106844e+01 -4.047826e+01
## FPE(n)  2.998206e-19  2.677071e-19  2.730419e-19  2.904664e-19  3.605396e-19
##                  6             7             8             9            10
## AIC(n) -4.236495e+01 -4.218361e+01 -4.220631e+01 -4.202615e+01 -4.203636e+01
## HQ(n)  -4.139612e+01 -4.105976e+01 -4.092746e+01 -4.059228e+01 -4.044748e+01
## SC(n)  -3.997805e+01 -3.941480e+01 -3.905561e+01 -3.849354e+01 -3.812185e+01
## FPE(n)  4.105658e-19  5.002728e-19  4.998426e-19  6.157932e-19  6.320158e-19
```

- Le critère AIC sélectionne un modèle $VAR(2)$

```
varpib2.VAR <- VAR(varpib2[,2:5], 2)
```

Modèle VAR

Exercice : Solutions (4)

Résultats pour la série FR

```
summary(varpib2.VAR)$varresult$fr
```

```
##
## Call:
## lm(formula = y ~ -1 + ., data = datamat)
##
## Residuals:
##        Min         1Q     Median         3Q        Max
## -0.0139449 -0.0025318 -0.0002342  0.0028730  0.0112062
##
## Coefficients:
##         Estimate Std. Error t value Pr(>|t|)
## ca.l1  0.1235667  0.0749780   1.648  0.10210
## uk.l1  0.0623811  0.0749369   0.832  0.40690
## us.l1  0.0581693  0.0701395   0.829  0.40865
## fr.l1  0.2906962  0.0975667   2.979  0.00353 **
## ca.l2 -0.0251569  0.0672496  -0.374  0.70904
## uk.l2  0.0787629  0.0757517   1.040  0.30066
## us.l2 -0.0567749  0.0721289  -0.787  0.43284
## fr.l2  0.1448222  0.0971135   1.491  0.13865
## const  0.0013654  0.0005972   2.286  0.02408 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.004168 on 114 degrees of freedom
## Multiple R-squared:  0.3652, Adjusted R-squared:  0.3207
## F-statistic: 8.198 on 8 and 114 DF,  p-value: 1.027e-08
```

Modèle VAR

Exercice : Solutions (5)

Résultats pour la série FR

```
summary(varpib2.VAR)$varresult$ca
```

```
##
## Call:
## lm(formula = y ~ -1 + ., data = datamat)
##
## Residuals:
##        Min         1Q     Median         3Q        Max
## -0.0134710 -0.0030687  0.0004647  0.0031347  0.0160507
##
## Coefficients:
##         Estimate Std. Error t value Pr(>|t|)
## ca.l1  0.3392500  0.0988767   3.431 0.000838 ***
## uk.l1  0.3544446  0.0988226   3.587 0.000495 ***
## us.l1  0.4445937  0.0924961   4.807 4.72e-06 ***
## fr.l1  0.1044684  0.1286656   0.812 0.418520
## ca.l2 -0.1741610  0.0886850  -1.964 0.051986 .
## uk.l2 -0.1142526  0.0998971  -1.144 0.255143
## us.l2  0.0150701  0.0951196   0.158 0.874396
## fr.l2 -0.2778514  0.1280679  -2.170 0.032116 *
## const  0.0016099  0.0007876   2.044 0.043249 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.005496 on 114 degrees of freedom
## Multiple R-squared:  0.5427, Adjusted R-squared:  0.5106
## F-statistic: 16.91 on 8 and 114 DF,  p-value: 2.441e-16
```

Modèle VAR

Exercice : Solutions (6)

- La fonction `serial.test()` de la librairie `vars` calcule un test de Portmanteau multivarié

```
varpib2.VAR.serial <- serial.test(varpib2.VAR, type="PT.asymptotic")
varpib2.VAR.serial
```

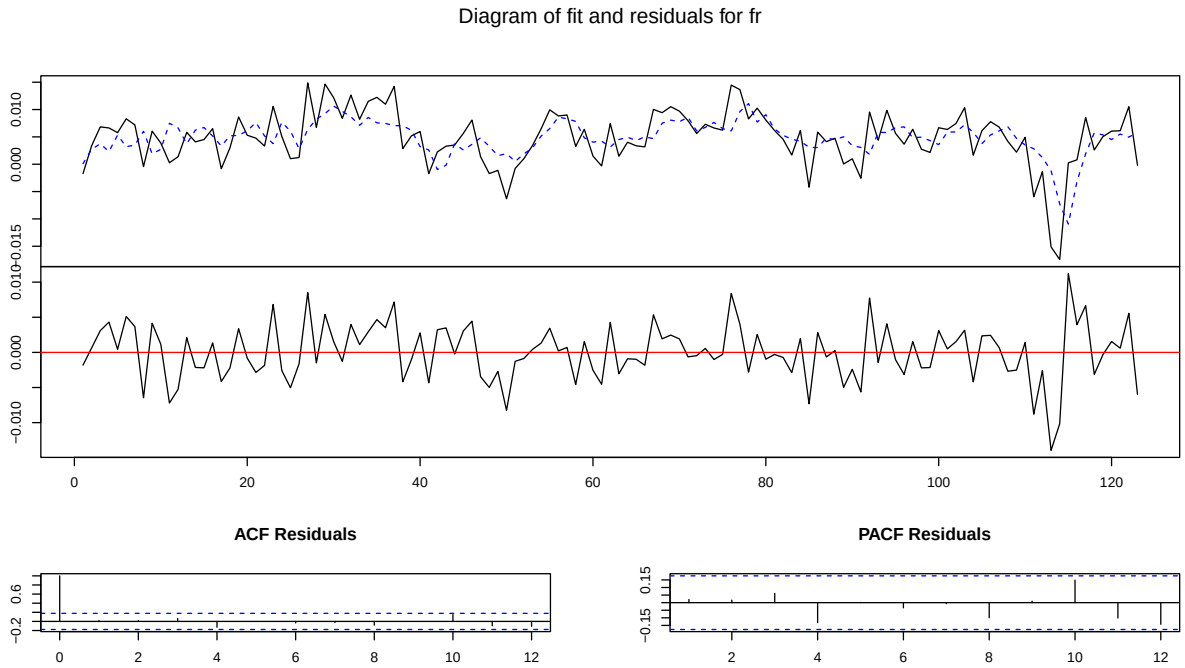
```
##
##  Portmanteau Test (asymptotic)
##
## data:  Residuals of VAR object varpib2.VAR
## Chi-squared = 206.19, df = 224, p-value = 0.7976
```

Modèle VAR

Exercice : Solutions (7)

- La fonction `serial.test()` de la librairie `vars` calcule un test de Portmanteau multivarié

```
plot(varpib2.VAR, names="fr")
```



11 Modèles multivariés pour l'analyse de séries temporelles stationnaires
- Introduction
- Série temporelle multivariée
- Processus VAR
- Test de causalité
- Estimation et sélection du modèle
- Diagnostics
- Fonction de réponse impulsionnelle
- Forecast Error Variance Decomposition
- Modèles VAR Bayesien (BVAR)
- Les modèles SVAR (Structural Vector Autoregressive)

Introduction

- Les tests de causalité permettent de savoir si une variable y_i améliore la prédiction d'une autre variable y_j de la série multivariée
- Cependant, cette analyse de causalité ne permet pas de quantifier l'impact sur le long terme de y_i (*impulse variable)* sur y_j (*response variable*)
- L'analyse de réponse impulsionnelle (*impulse response analysis*) permet d'étudier ces interactions dynamiques entre variables endogènes
- Elle est basée sur la représentation *MA* (*Moving Average*) d'un processus $VAR(p)$ (représentation de Wold)

Représentation de Wold d'un processus $VAR(p)$

- Comme pour les processus $AR(p)$, le processus $VAR(p)$

$$y_t = A_1 y_{t-1} + \ldots + A_p y_{t-p} + \mu_t$$

peut être représenté sous la forme d'un processus MA

$$y_t = \Phi_0 \mu_t + \Phi_1 \mu_{t-1} + \Phi_2 \mu_{t-2} + \ldots$$

- Les matrices Φ_s peuvent être calculées récursivement

$$\Phi_s = \sum_{j=1}^{s} \Phi_{s-j} A_j$$

$s = 1, 2, \ldots$, avec $\Phi_0 = I_K$ et $A_j = 0$ pour $j > p$

- Les coefficients (i, j) des matrices Φ_s représentent la réponse attendue de la variable $y_{j,t+s}$ à un changement d'une unité de la variable $y_{i,t}$
- En cumulant les effets dans le temps $s = 1, 2, \ldots$, on obtient l'impact cumulé d'un changement d'une unité de la variable y_i sur la variable y_j au temps s

Taux de croissance du PIB (UK, Canada, US)

Fonction de réponse impulsionnelle (librairie `vars`, 1)

- On étudie les réponses impulsionnelles à partir du modèle $VAR(2)$ estimé précédemment
- La fonction `irf()` (librairie) calcule les coefficients aux temps $s = 1, 2, \ldots, 10$ par défaut
- On précise ici la variable "impulse", on obtient les réponses impulsionnelles pour les trois variables de la série
- Par défaut la fonction produit des intervalles de confiance, ici on désactive cette option

```
p1ct.irf.uk <- irf(pibgr.VAR, impulse='uk', boot=FALSE)
p1ct.irf.uk
```

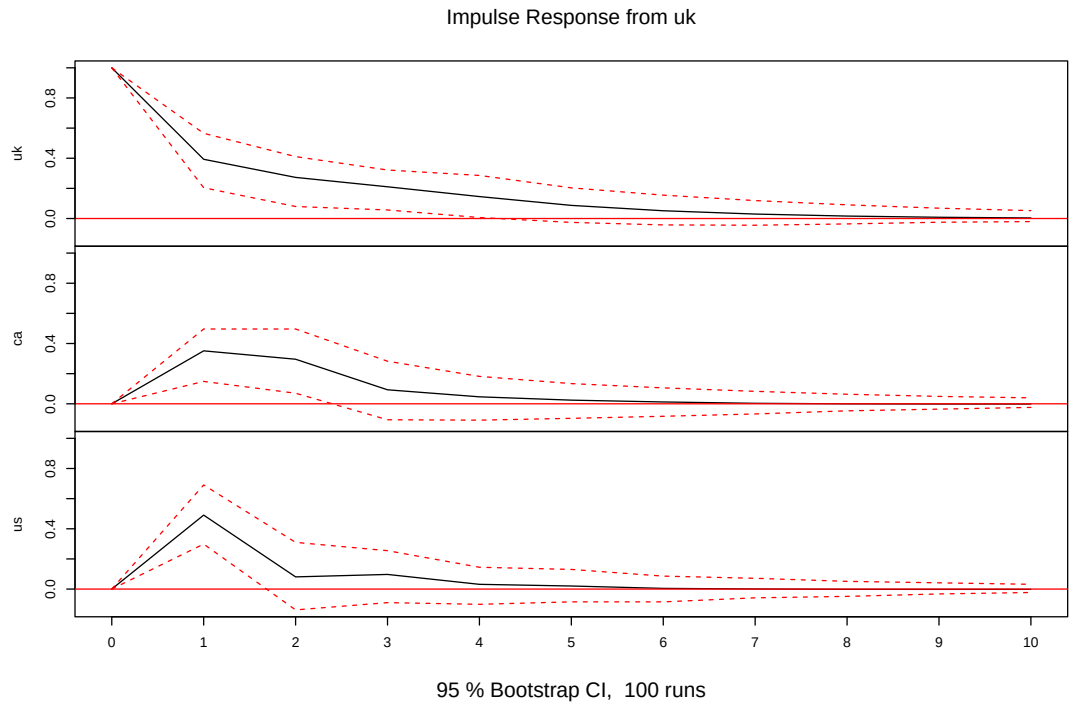
```
##
## Impulse response coefficients
## $uk
##                uk            ca            us
##  [1,] 0.547255196  0.0514248485  1.440638e-01
##  [2,] 0.227921104  0.2772264871  3.148264e-01
##  [3,] 0.173709665  0.2065383276  8.738689e-02
##  [4,] 0.142233601  0.0770451927  7.479194e-02
##  [5,] 0.101028260  0.0409976945  3.208096e-02
##  [6,] 0.063204307  0.0230690756  1.887699e-02
##  [7,] 0.038856911  0.0120822902  6.841629e-03
##  [8,] 0.023244236  0.0046546413  2.446301e-03
##  [9,] 0.013347572  0.0012804546 -2.341645e-05
## [10,] 0.007230333 -0.0001716288 -8.016378e-04
## [11,] 0.003672677 -0.0006716863 -1.015997e-03
```

Taux de croissance du PIB (UK, Canada, US)

Fonction de réponse impulsionnelle

- La méthode générique `plot()` produit un graphique des IRF
- Ici on a 3x3=9 FRIs, on représente ici uniquement les réponses de la variable impulse "uk"

```
plot(irf(pibgr.VAR, impulse="uk", ortho=FALSE), plot.type="multiple")
```

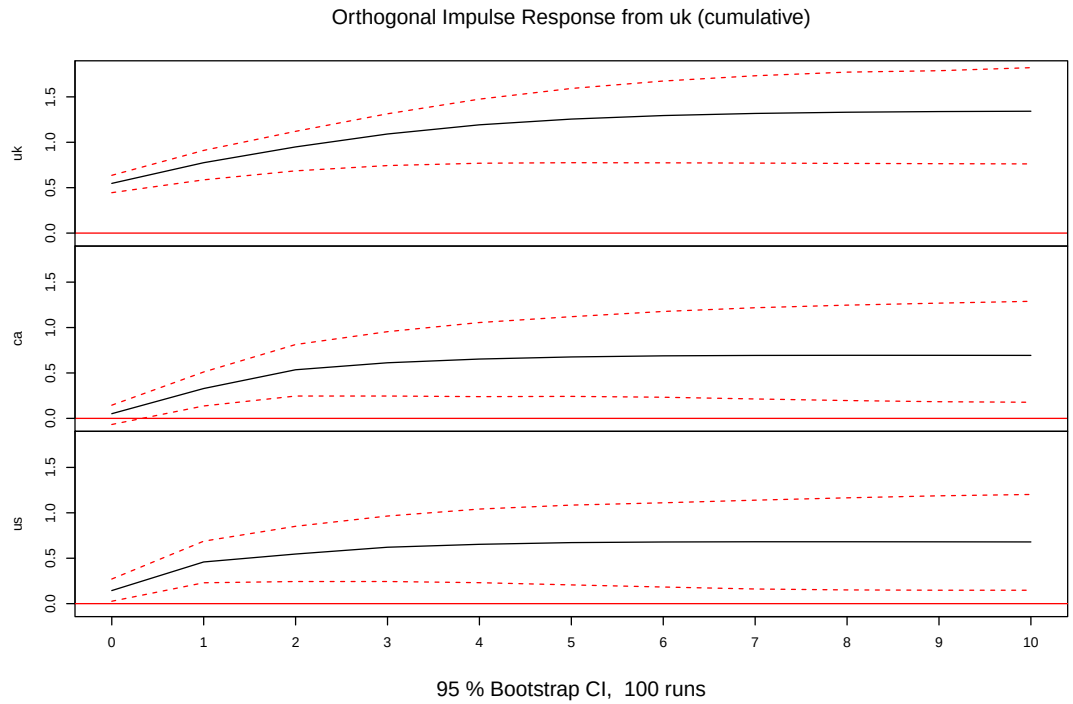


Taux de croissance du PIB (UK, Canada, US)

Fonction de réponse impulsionnelle : Réponses cumulées

- L'argument `cumulative = TRUE` de la fonction `irf()` produit les réponses cumulées

```
plot(irf(pibgr.VAR, impulse="uk", cumulative = TRUE), plot.type="multiple")
```



Taux de croissance du PIB (UK, Canada, US)

Réponse impulsionnelles orthogonales

- Si on suppose que les chocs n'arrivent pas de manière isolée, i.e. il y a une corrélation instantanée entre les éléments du processus

Taux de croissance du PIB (UK, Canada, US)

IRF avec la librairie MTS (1)

- La librairie MTS permet également l'analyse des réponses impulsionnelles
- On part du modèle *VAR*(2) simplifié proposé par Tsay-2014 (les paramètres non-significatifs sont nuls)

```
m1 <- MTS::VAR(pibgr, 2)
m2 <- refVAR(m1, thres=1.96)
```

- On extrait les coefficients Φ du modèle (notés A dans l'équation d'un modèle *VAR*)

```
Phi <- m2$Phi
Phi
```

```
##           [,1]      [,2]      [,3]       [,4] [,5] [,6]
## [1,] 0.4672294 0.2068333 0.0000000  0.0000000    0    0
## [2,] 0.3339973 0.2702527 0.4964759 -0.1967488    0    0
## [3,] 0.4683350 0.2247260 0.2320040 -0.3013031    0    0
```

- σ représente la matrice de covariance des innovations

```
Sig <- m2$Sigma
Sig
```

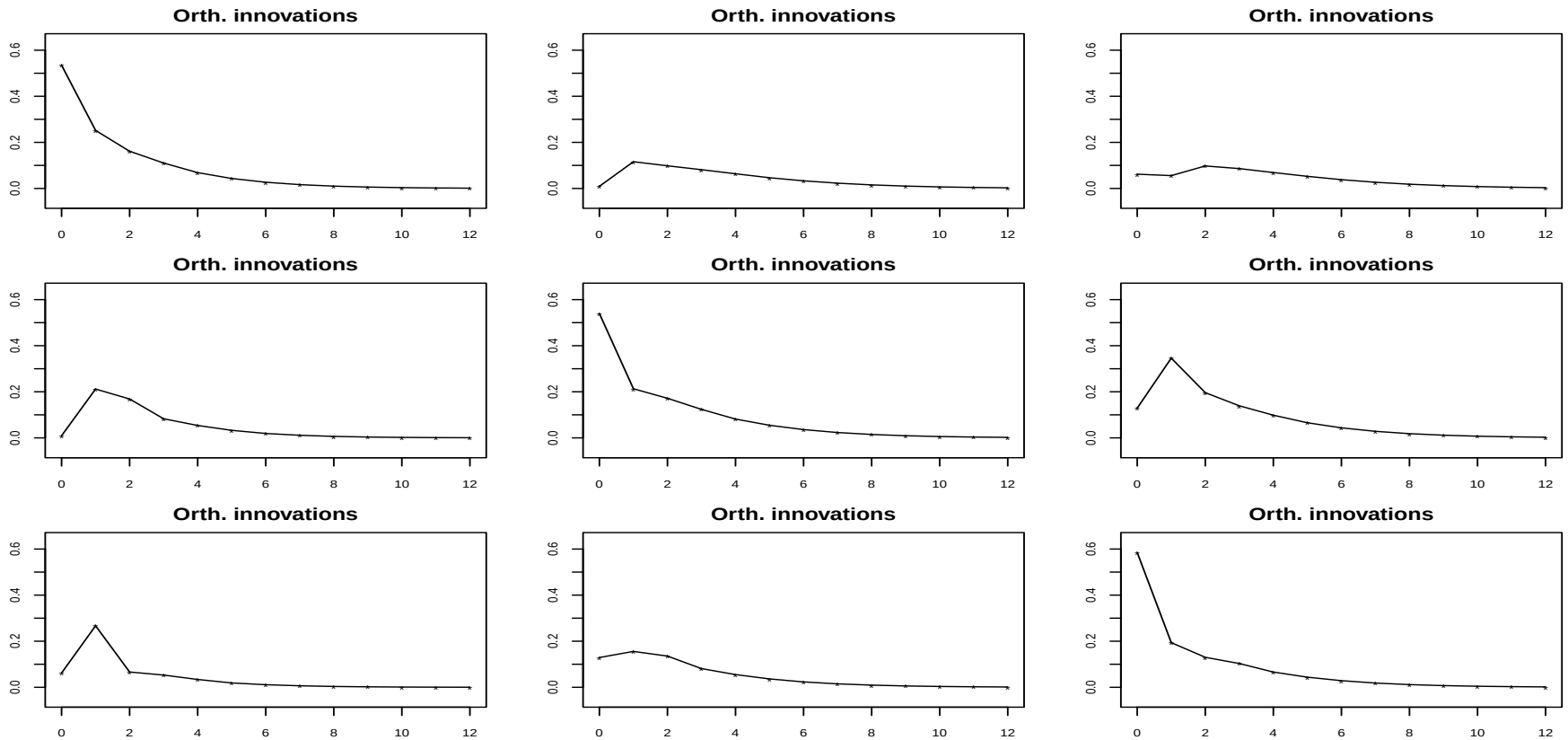
```
##            [,1]       [,2]       [,3]
## [1,] 0.29003669 0.01803456 0.07055856
## [2,] 0.01803456 0.30802503 0.14598345
## [3,] 0.07055856 0.14598345 0.36268779
```

Taux de croissance du PIB (UK, Canada, US)

IRF avec la librairie MTS (2)

- La fonction `VARMAirf()` de la librairie `MTS` produit deux graphiques des IRF (le premier produit la réponse impulsionnelle au temps $s = 1, 2, \ldots$, le second représente les réponses cumulées
- L'option `orth` produit les IRF originaux ou orthogonaux

```
m2.irf <- VARMAirf(Phi=Phi, Sigma=Sig)
```




```
## Press return to continue
```

11 Modèles multivariés pour l'analyse de séries temporelles stationnaires
- Introduction
- Série temporelle multivariée
- Processus VAR
- Test de causalité
- Estimation et sélection du modèle
- Diagnostics
- Fonction de réponse impulsionnelle
- Forecast Error Variance Decomposition
- Modèles VAR Bayesien (BVAR)
- Les modèles SVAR (Structural Vector Autoregressive)

Taux de croissance du PIB (UK, Canada, US)

Introduction

- L'analyse FEVD (Forecast Error Variance Decomposition) est basée sur les matrices de réponse impulsionnelle orthogonales
- Elle permet d'évaluer la contribution de la variable i à la variance de la prévision de la variable j à l'horizon $t + h$
- Si les éléments au carré de la matrice sont divisés par la variance de l'erreur de prévision, on obtient des pourcentages

Taux de croissance du PIB (UK, Canada, US)

FEVD (1)

- On utilise la fonction `fevd()` de la librairie `vars`
- Les résultats représentent des proportions (la somme de chaque ligne = 1)

```
pibgr.VAR.fevd <- fevd(pibgr.VAR)
pibgr.VAR.fevd$uk
```

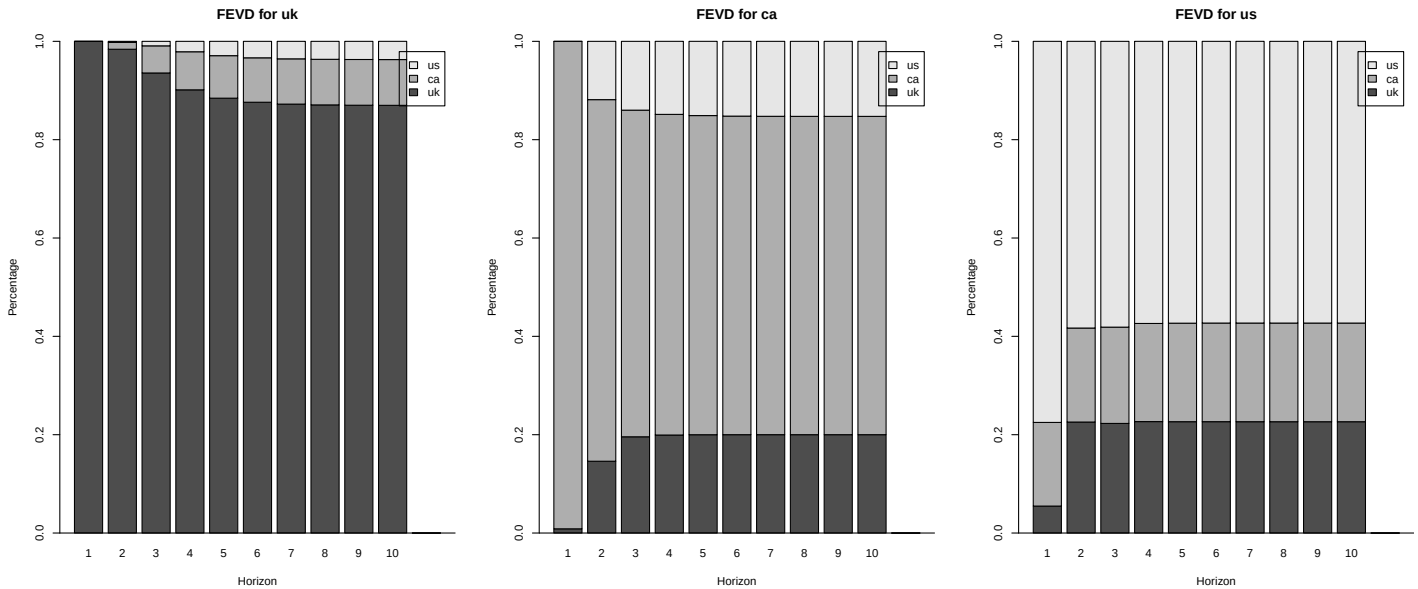
```
##              uk         ca          us
##  [1,] 1.0000000 0.00000000 0.000000000
##  [2,] 0.9839250 0.01384248 0.002232528
##  [3,] 0.9356905 0.05515564 0.009153855
##  [4,] 0.9013727 0.07744446 0.021182825
##  [5,] 0.8844127 0.08643078 0.029156537
##  [6,] 0.8761617 0.09034138 0.033496884
##  [7,] 0.8723541 0.09205878 0.035587114
##  [8,] 0.8707243 0.09273949 0.036536188
##  [9,] 0.8700871 0.09298464 0.036928267
## [10,] 0.8698566 0.09306599 0.037077400
```

Taux de croissance du PIB (UK, Canada, US)

FEVD (2)

- La méthode `plot()` produit une représentation graphique

```
plot(pibgr.VAR.fevd, nc=3)
```



11 Modèles multivariés pour l'analyse de séries temporelles stationnaires
- Introduction
- Série temporelle multivariée
- Processus VAR
- Test de causalité
- Estimation et sélection du modèle
- Diagnostics
- Fonction de réponse impulsionnelle
- Forecast Error Variance Decomposition
- Modèles VAR Bayesien (BVAR)
- Les modèles SVAR (Structural Vector Autoregressive)

L'estimation des modèles VAR

- Les modèles VAR sont couramment utilisés pour l'analyse de séries temporelles multivariées en macroéconomie
- En théorie, l'estimation de ces modèles est simple, elle peut être obtenue par MCO (ou maximum de vraissemblance)
- En pratique, le nombre de paramètres à estimer est un problème sérieux [1], et seuls des "petits" modèles peuvent être estimés correctement
- Au delà de 5 ou 6 séries maximum, le nombre de paramètres à estimer est trop grand par rapport aux tailles d'échantillons usuelles
- Dans ce cas, les estimations obtenues par OLS ne sont pas fiables, ce qui pose problème notamment pour les prévisions

L'approche Bayesienne pour l'estimation des modèles VARs

- Les solutions à ce problème de sur-paramétrisation (*over-parameterisation problem*) consistent à imposer des contraintes sur les paramètres [1]
- L'utilisation de méthodes bayésiennes est une des approches les plus intéressantes (exemples : [8, 14])
- Dans cette approche, les données ne sont pas la seule source d'information ; elles sont combinées avec des postulats à priori pour produire une fonction de densité de probabilité à posteriori (*posterior PDF, probability density function*) pour les paramètres du modèle
- Ces postulats, sous la forme d'une PDF à priori (*prior PDF*), imposent une structure aux coefficients du modèle et contraignent l'espace des paramètres libres [1, 19]

L'approche Bayesienne pour l'estimation des modèles VARs

- [1] Si on considère la $i^{ème}$ équation d'un modèle VAR :

$$Y_{it} = x'_t{}_i + \epsilon_{it}, \epsilon_{it} \sim N(0, \sigma_i^2)$$

où x_t inclu les premiers p retards de toutes les variables du VAR

- $\theta_i = ['_i, \sigma_i^2]'$ est le vecteur des paramètres apparaissant dans la $i^{ème}$ équation
- L'inférence classique consiste à maximiser la fonction de vraissemblance afin d'obtenir une estimation des paramètres
- On estime aussi l'incertitude autour de cette estimation, qui est utilisée pour construire des intervalles de confiance et réaliser des tests d'hypothèses

L'approche Bayesienne pour l'estimation des modèles VARs

- Avec l'approche bayesienne, l'idée d'une "vraie" valeur (inconnue) des paramètres n'existe pas
- Les paramètres sont au contraire considérés comme des variables aléatoires non-observées, sur lesquelles on dispose d'informations supplémentaires (hors échantillon) ; ces informations sont formalisées sous la forme d'une distribution à priori $p(\theta_i)$
- Les informations à priori et celles issues de l'échantillon sont combinées en utilisant le théorème de Bayes

$$p(\theta \mid y) \propto p(\theta)p(y \mid \theta)$$

  - θ : paramètres du modèle
  - $p(\theta)$: croyances initiales (prior)
  - $p(y \mid \theta)$: ce que disent les données (vraisemblance)
  - $p(\theta \mid Y)$: ce que l'on sait après avoir observé les données
- Une distribution a posteriori donne plusieurs valeurs plausibles, pas une seule estimation !

L'approche Bayesienne pour l'estimation des modèles VARs

- La distribution $p(\theta \mid y)$ est appelée distribution conjointe à posteriori (*joint posterior distribution*) du vecteur des paramètres
- Cette fonction de densité de probabilité (PDF) mesure l'incertitude sur les paramètres après avoir combiné toutes les sources d'information
- Cette densité à posteriori est utilisée ensuite pour obtenir une estimation ponctuelle (*point estimate*) du vecteur des paramètres ; on utilise généralement le mode ou l'espérance de la distribution à posteriori

La librairie BVAR

- La librairie BVAR utilise l'inférence bayésienne hiérarchique (voir [9] et ci-dessous) pour la sélection des distribution à priori
- La librairie BVAR inclu les bases données macroéconomiques FRED-MD et FRED-QD [25], contenant plus de 200 indicateurs mensuels et trimestriels
- Les données concernent l'économie des USA, à partir de 1959

Modèle BVAR : Données

- Nous suivons l'exemple figurant dans l'article de [19], qui utilise six variables des données FRED-QD (T1-1959 à T3-2019)
  - real gross domestic product (GDP, billions of 2012 dollars)
  - real personal consumption expenditures (billions of 2012 dollars)
  - real gross private domestic investment (billions of 2012 dollars)
  - number of total hours worked in the non-farm business sector
  - the GDP deflator index as a means to measure price inflation
  - the effective federal funds rate in percent per year
- Suivant [9], toutes les variables sauf le taux effectif des fonds fédéraux sont transformées en log

Modèle BVAR : Données

- La fonction `fred_transform()` permet d'effectuer des transformations sur les données selon la codification proposée par [25] (code 4=transformation logarithmique, 1=aucune transformation)

```
set.seed(42)
library("BVAR")
```

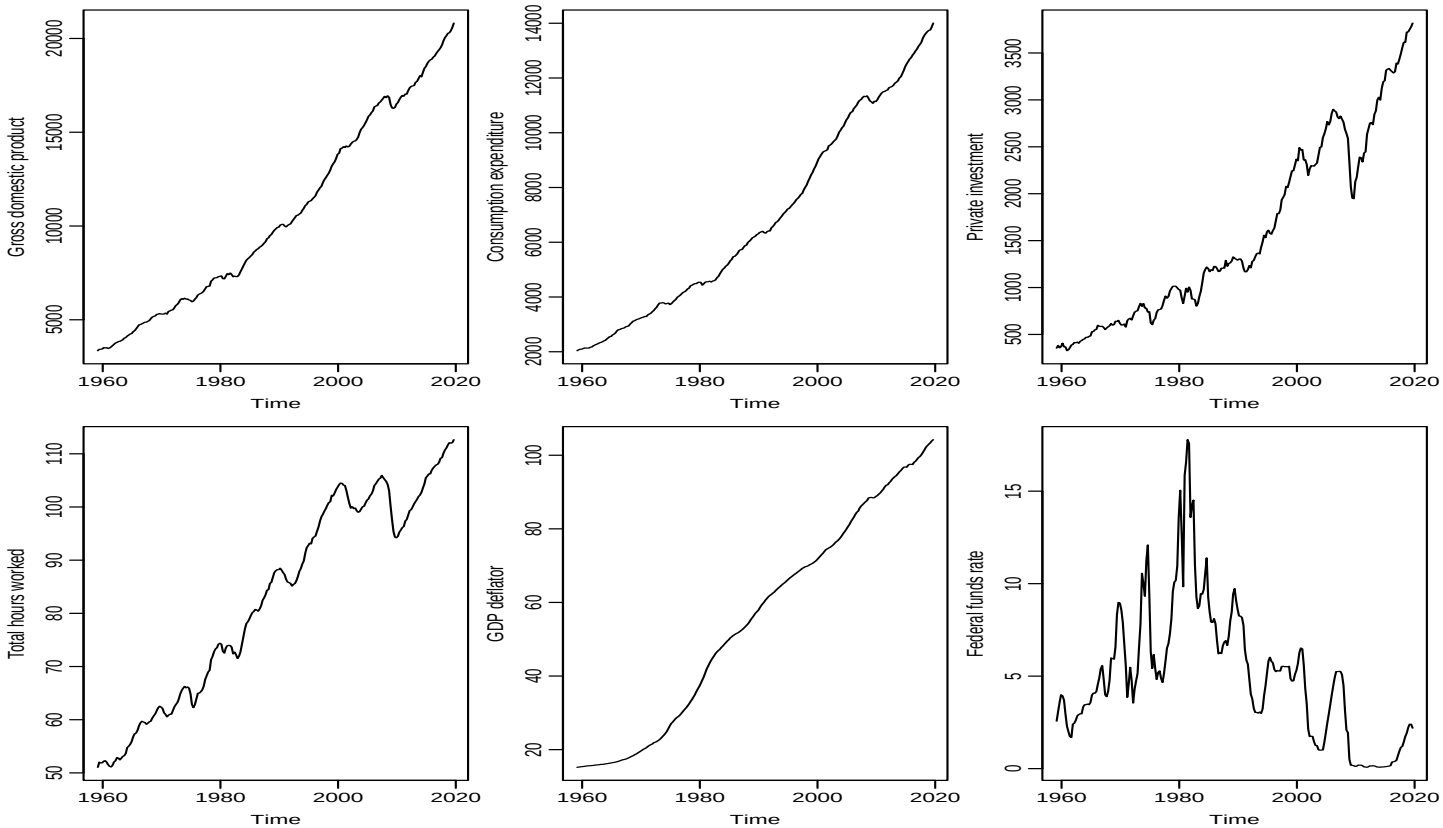
```
##
## Attachement du package : 'BVAR'
## Les objets suivants sont masqués depuis 'package:vars':
##
##     fevd, irf
```

```
fqddata <- fred_qd[1:243, c("GDPC1", "PCECC96", "GPDIC1", "HOANBS", "GDPCTPI", "FEDFUNDS")]
fdqdata <- fred_transform(fqddata, codes = c(4, 4, 4, 4, 4, 1))
head(fqddata)
```

```
##               GDPC1  PCECC96  GPDIC1 HOANBS GDPCTPI FEDFUNDS
## 1959-03-01 3352.129 2039.017 354.894 51.055  15.205   2.5700
## 1959-06-01 3427.667 2070.508 382.500 52.021  15.249   3.0833
## 1959-09-01 3430.057 2092.138 357.798 51.868  15.314   3.5767
## 1959-12-01 3439.832 2094.495 369.442 51.883  15.373   3.9900
## 1960-03-01 3517.181 2114.532 406.581 52.208  15.402   3.9333
## 1960-06-01 3498.246 2141.205 368.686 52.295  15.457   3.6967
```

Modèle BVAR : Données

- Représentation graphique des séries FRED-QD utitlisées



Spécification d'une distribution à priori

Minnesota prior

- L'approche bayésienne requiert la spécification d'une distribution à priori pour les paramètres du modèle
- Dans la littérature sur les modèles VAR bayésiens, la distribution à priori est spécifiée en tenant compte du fait que la plupart des séries temporelles économiques ont un comportement de long terme (*long run behaviour*) correspondant à une marche aléatoire (voir par exemple [21])
- Cette propriété est prise en compte dans la distribution à priori en spécifiant que dans chaque équation le paramètre du lag 1 de la variable dépendante ait une moyenne égale à 1, et tous les autres paramètres ont une moyenne de 0
- Cette spécification est devenue un standard dans la littérature et la distribution à priori qui en résulte est appelée *Minnesota prior*

Spécification d'une distribution à priori

Minnesota prior

- Si on considère le modèle $VAR(1)$

$$Y_t = AY_{t-1} + \mu_t.$$

- Le *Minnesota prior* privilégie

$$E(A_{ii}) = 1$$

et

$$E(A_{ij}) = 0, \qquad i \neq j$$

- On favorise donc initialement un comportement proche de

$$Y_t = Y_{t-1} + \mu_t$$

i.e., une marche aléatoire

- La non-stationnarité n'est pas nécessairement considérée comme un problème à éliminer : elle peut être intégrée directement dans les hypothèses a priori du modèle
- Les données permettent ensuite de déplacer les coefficients vers des valeurs différentes

Spécification d'une distribution à priori

Dummy-observation priors

- On utilise également des "a priori par pseudo-observations" (*dummy-observation priors*)
- Cette technique permet d'introduire des informations a priori sur les coefficients d'un modèle VAR en ajoutant des observations fictives (ou artificielles) aux données réellement observées
- Cette approche utilisée dans la littérature sur les BVAR permet d'imposer des restrictions souples sur les propriétés dynamiques des séries temporelles
- Les deux types de *dummy-observation priors* utilisés dans cet exemple sont :
  - *sum-of-coefficients* (SOC)
  - *single-unit-root priors* (SUR)

Distributions à priori

Sum-of-coefficients prior (SOC)

- Le *prior* SOC (*Sum Of Coefficients*) favorise une somme proche de 1 pour les coefficients autorégressifs propres à chaque variable et une somme proche de 0 pour les effets cumulés des autres variables
- Pour la variable i, il favorise donc :

$$\sum_{\ell=1}^{p} A_{\ell,ii} \approx 1$$

- Et pour $j \neq i$:

$$\sum_{\ell=1}^{p} A_{\ell,ij} \approx 0$$

- Cela correspond à l'idée qu'une variable persistante est principalement expliquée, à long terme, par ses propres retards

Distributions à priori

Single-unit-root prior (SUR)

- Le *single-unit-root prior* (à priori de racine unitaire commune, parfois appelé a priori de cointégration dans la littérature) favorise une dynamique dans laquelle les variables partagent une tendance stochastique commune, plutôt que de dériver indépendamment
- Cet a priori est notamment associé aux travaux de Sims (1993) et de Sims Zha (1998)
- Il ne faut toutefois pas l'interpréter comme une contrainte imposant exactement une seule racine unitaire : il s'agit d'une restriction probabiliste souple

Distributions à priori

- Les *dummy-observation priors* permettent d'introduire des hypothèses économiques sur la dynamique des séries temporelles, sans imposer de contraintes strictes sur les coefficients du VAR
- Dans les modèles VAR bayésiens, l'utilisation conjointe des a priori Minnesota, SOC et SUR peut être intéressante lorsque les variables macroéconomiques sont fortement persistantes, comme le PIB, les prix ou les taux d'intérêt
- Le Minnesota contrôle principalement la taille et la dispersion des coefficients individuels, tandis que SOC et SUR introduisent des restrictions sur des combinaisons de coefficients et sur les propriétés dynamiques du VAR

Distributions à priori

Hyperpriors

- Dans un modèle BVAR, on distingue trois niveaux :
  1. Les paramètres du modèle : les coefficients autorégressifs B et la matrice de covariance Σ
  2. Les hyperparamètres : chaque prior possède un hyperparamètre contrôlant sa force
     - λ contrôle le degré global de restriction du prior Minnesota (une petite valeur de λ renforce le Minnesota)
     - μ_{SOC} pour le prior SOC (une petite valeur de μ_{SOC} renforce les restrictions sur la somme des coefficients)
     - μ_{SUR} pour le prior SUR (une petite valeur de μ_{SUR} renforce les restrictions associées aux conditions initiales et aux tendances communes)
  3. Les hyperpriors (hyper-a priori) : les distributions de probabilité attribuées à ces hyperparamètres

Distributions à priori

Minnesota prior

- On commence par spécifier le *Minnesota prior* avec la fonction `bv_minnesota()`
- L'hyper-à priori de l'hyperparamètre λ de la distribution à priori est spécifié par une loi Gamma, avec des bornes inférieures et supérieures
- Dans cet exemple, le paramètre α, qui gouverne la décroissance des restrictions avec l'ordre du retard, reste fixé à sa valeur modale (il n'est pas estimé hiérarchiquement)
- The prior variance on the constant term of the model (var) is dealt a large value, for a diffuse prior
- We leave Ψ to be set automatically – i.e., to the square root of the innovations variance, after fitting AR(p) models to each of the variables

```
mn <- bv_minnesota(lambda = bv_lambda(mode = 0.2, sd = 0.4, min = 1e-04, max = 5),
  alpha = bv_alpha(mode = 2), var = 1e+07)
```

L'inférence bayésienne hiérarchique

- L'inférence bayésienne hiérarchique utilisée dans le package BVAR consiste à estimer simultanément les paramètres du modèle VAR et les hyperparamètres qui contrôlent la force des restrictions a priori
- L'idée fondamentale est de ne pas fixer arbitrairement le degré de restriction des coefficients du VAR, mais de laisser les données contribuer à sa détermination
- Giannone et al. (2015) propose setting prior hyperparameters in a data-based fashion, i.e., by treating them as additional parameters to be estimated. In their hierarchical approach, prior hyperparameters are assigned their own hyperpriors

L'inférence bayésienne hiérarchique

Les trois niveaux du modèle hiérarchique

- Niveau 3 : *Hyperpriors* : Distributions Gamma sur les hyperparamètres de restriction

$$\lambda, \mu_{\text{SOC}}, \mu_{\text{SUR}} \sim p(\gamma)$$

- Niveau 2 : Lois a priori des paramètres

$$p(B, \Sigma \mid \gamma)$$

  Minnesota et pseudo-observations SOC/SUR

- Niveau 1 - Modèle VAR et données observées

$$Y_t = c + \sum_{\ell=1}^{p} A_\ell Y_{t-\ell} + u_t u_t \sim \mathcal{N}(0, \Sigma)$$

Distributions à priori

A priori sur les hyperparamètres

- Les auteurs ne fixent pas les principaux hyperparamètres du BVAR à des valeurs déterministes
- Ils leur attribuent des distributions Gamma, qui sont ensuite actualisées par les données dans le cadre de l'inférence bayésienne hiérarchique
- Les lois Gamma sont utilisées pour modéliser leurs croyances a priori sur les hyperparamètres qui contrôlent les lois a priori des coefficients du modèle
- L'utilisation de la loi Gamma présente plusieurs avantages :
  - Elle garantit que les hyperparamètres sont positifs (nécessaire pour les hyperparamètres de restriction)
  - Elle est suffisamment flexible pour représenter différentes croyances a priori
  - Elle permet de contrôler la valeur centrale et la dispersion de l'hyperparamètre

```r
x <- seq(0.001, 5, length.out = 1000)

plot(x, dgamma(x, shape = 5, rate = 5),
     type = "l", col="blue", lwd = 2,
     xlab = expression(lambda),
     ylab = "Densité",
     main = "Exemples d'hyperpriors Gamma")

lines(x, dgamma(x, shape = 2, rate = 2),
      lty = 2, lwd = 2, col="green")

lines(x, dgamma(x, shape = 2, rate = 1),
      lty = 3, lwd = 2, col="purple")
```

Distributions à priori

- On assigne des distributions Gamma aux *hyperpriors* des hyper-paramètres pour les à priori SOC et SUR avec les fonctions `bv_soc` et `bv_sur`

```
soc <- bv_soc(mode = 1, sd = 1, min = 1e-04, max = 50)
sur <- bv_sur(mode = 1, sd = 1, min = 1e-04, max = 50)
```

Distributions à priori

- Une fois les *priors* spécifiés, on les passe comme arguments à la fonction `bv_priors()`
- L'argument `hyper` indique les hyperparamètres qui doivent être traités hiérarchiquement ; `hyper="auto"` (par défaut) inclus λ et les hyperparamètres des *dummy-observation priors* (ici SOC et SUR)
- Le paramètre α n'est pas traité hiérarchiquement ; sa valeur est fixée au mode

```
priors <- bv_priors(hyper = "auto", mn = mn, soc = soc, sur = sur)
```

Distributions à priori

- L'étape finale avant l'estimation consiste à ajuster l'algorithme Metropolis-Hastings
- La fonction `bv_metropolis()` permet de régler précisément l'exploration de l'espace des paramètres a posteriori, condition essentielle à la fiabilité de l'inférence bayésienne

```
mh <- bv_metropolis(scale_hess = c(0.05, 1e-04, 1e-04), adjust_acc = TRUE, acc_lower = 0.25,
  acc_upper = 0.45)
```

Estimation du modèle BVAR

- La fonction `bvar` permet l'estimation du modèle ; on précise l'ordre p du modèle VAR
- On passe également les objets `priors` et `mh` définis précédemment
- L'argument `n_draw` fixe le nombre total d'itérations ; l'argument `n_burn` défini le nombre initial d'itérations qui sont ignorées

```
run <- bvar(fqddata, lags = 5, n_draw = 50000, n_burn = 25000, n_thin = 1, priors = priors,
  mh = mh, verbose = FALSE)
```

```
## Caught an error while automatically setting psi. Column 1 appears to be integrated; caught error:
## Error in arima(x[, j], order = c(lags, 0, 0)): partie AR non stationnaire dans CSS
##
## Attempting to increase order of integration via an ARIMA(5, 1, 0) model.
## Caught an error while automatically setting psi. Column 2 appears to be integrated; caught error:
## Error in arima(x[, j], order = c(lags, 0, 0)): partie AR non stationnaire dans CSS
##
## Attempting to increase order of integration via an ARIMA(5, 1, 0) model.
## Caught an error while automatically setting psi. Column 3 appears to be integrated; caught error:
## Error in arima(x[, j], order = c(lags, 0, 0)): partie AR non stationnaire dans CSS
##
## Attempting to increase order of integration via an ARIMA(5, 1, 0) model.
## Caught a warning while setting psi automatically:
## simpleWarning in arima(x[, j], order = c(lags, 0, 0)): problème de convergence possible : optim renvoie un code = 1
```

Estimation du modèle BVAR

- La méthode `print()` fourni un résumé de l'estimation

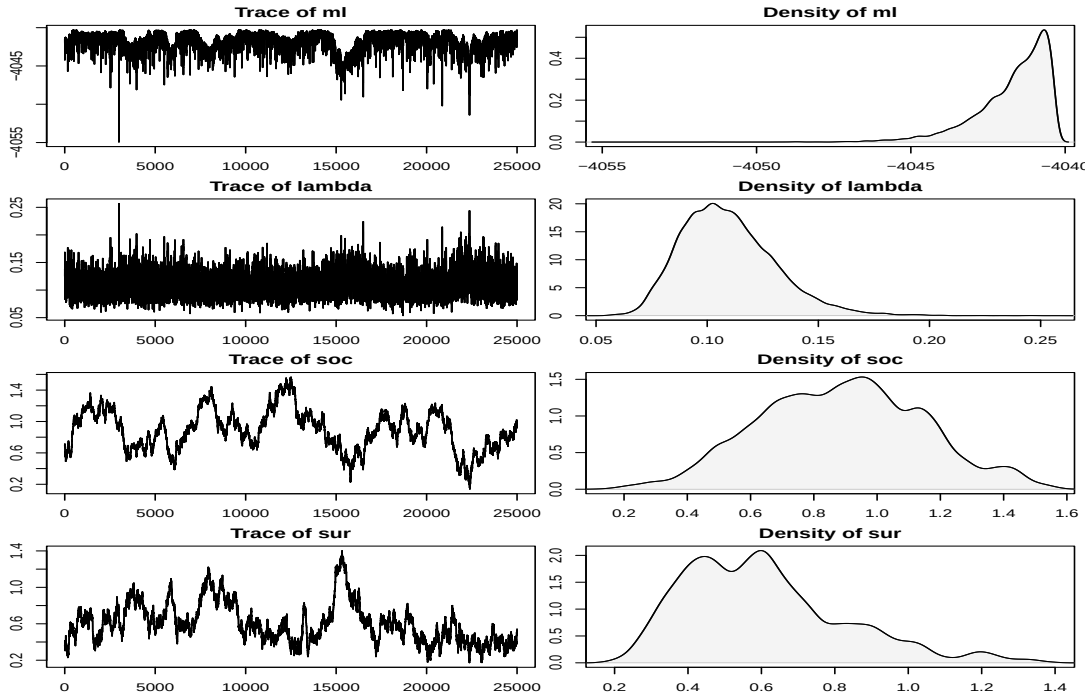
```
print(run)
```

```
## Bayesian VAR consisting of 238 observations, 6 variables and 5 lags.
## Time spent calculating: 48.98 secs
## Hyperparameters: lambda, soc, sur
## Hyperparameter values after optimisation: 0.10155, 0.93797, 0.52382
## Iterations (burnt / thinning): 50000 (25000 / 1)
## Accepted draws (rate): 11627 (0.465)
```

Estimation du modèle BVAR

- La méthode `plot()` permet de visualiser la convergence de l'algorithme MCMC
- Le graphique montre la trace et les distributions à posteriori des hyperparamètres

```
plot(run)
```

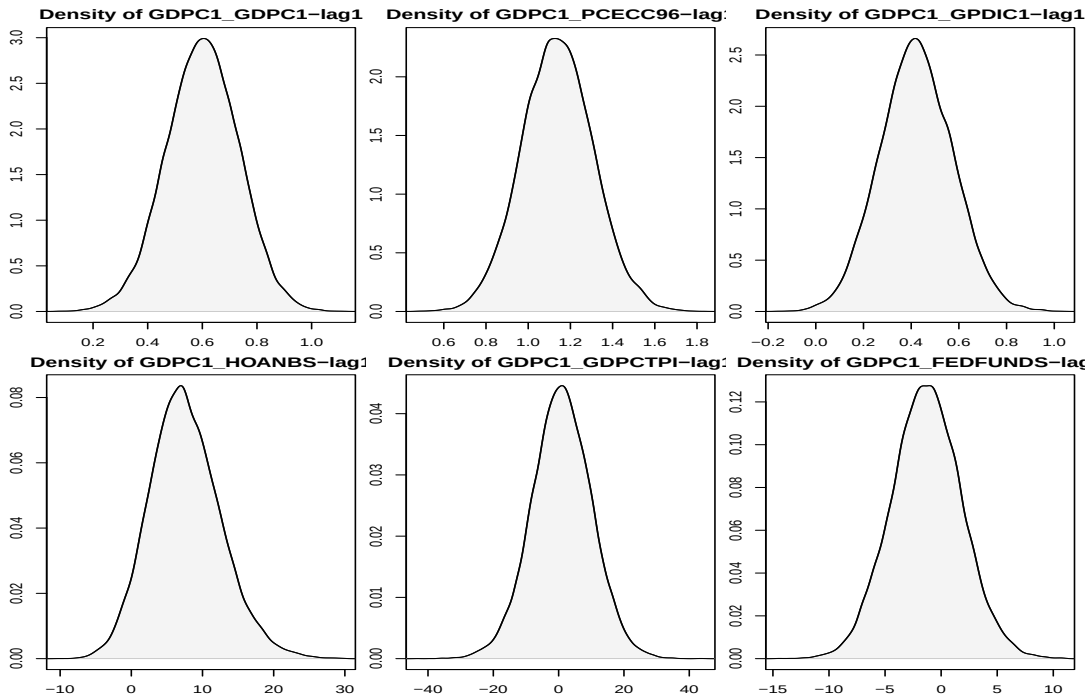


Distribution à posteriori des paramètres

- On peut également représenter la PDF des coefficients autorégressifs
- On utilise pour cela les arguments `var_response` et `var_impulse`
- Ici, le coefficient associé aux retards 1 des variables pour l'équation de GDPC1

```
clist <- paste(c("GDPC1", "PCECC96", "GPDIC1", "HOANBS", "GDPCTPI", "FEDFUNDS"), "-lag1", sep='')

plot(run, type = "dens", vars_response = "GDPC1", vars_impulse = clist, mfrow = c(2, 3))
```



Coefficients du modèle BVAR

- L'inspection visuelle des traces et des densités suggère une convergence satisfaisante des hyperparamètres
- Les chaînes MCMC explorent apparemment correctement leurs distributions a posteriori, sans anomalie majeure visible
- On peut maintenant examiner le modèle estimé

Coefficients du modèle BVAR

- On récupère ici les valeurs des coefficients du modèle (quantiles 10%, 50% et 90%), on a donc un objet de dimension [3,31,6]

```
coef(run, conf_bands = 0.10)
```

```
## Numeric array (dimensions 3, 31, 6) of coefficient values from a BVAR.
## Computed confidence bands: 10%, 50%, 90%
## Median values:
##                 GDPC1 PCECC96 GPDIC1 HOANBS GDPCTPI FEDFUNDS
## constant       39.803  17.481 12.652  0.728  -0.218   -0.102
## GDPC1-lag1      0.602  -0.091 -0.246 -0.001   0.000    0.002
## PCECC96-lag1    1.138   1.234  0.895  0.006   0.000    0.003
## GPDIC1-lag1     0.420   0.260  1.264  0.003   0.000   -0.001
## HOANBS-lag1     7.303  -0.799  5.873  1.075   0.006    0.133
## GDPCTPI-lag1    0.945  -1.748  3.023  0.068   1.087    0.154
## FEDFUNDS-lag1 -1.322  -4.229  1.021  0.010   0.034    0.997
## GDPC1-lag2      0.253   0.001  0.181  0.001   0.000    0.000
## PCECC96-lag2  -0.746  -0.056 -0.678 -0.004  -0.001   -0.007
## GPDIC1-lag2   -0.395  -0.101 -0.275 -0.003   0.000   -0.001
## HOANBS-lag2   -3.160   0.405 -3.375 -0.032  -0.004   -0.046
## GDPCTPI-lag2   1.270   1.736 -0.989 -0.006  -0.025   -0.030
## FEDFUNDS-lag2 -1.680   1.079 -1.424 -0.024   0.004   -0.064
## GDPC1-lag3    -0.008  -0.021 -0.053  0.000   0.000   -0.001
## PCECC96-lag3  -0.195   0.028 -0.178 -0.001   0.000    0.003
## GPDIC1-lag3   -0.055  -0.080 -0.003  0.000   0.000    0.000
## HOANBS-lag3   -1.964   0.395 -1.684 -0.025  -0.003   -0.035
## GDPCTPI-lag3  -0.014   0.365 -0.789 -0.019  -0.026   -0.063
## FEDFUNDS-lag3  0.838   1.117  0.121  0.000   0.002    0.015
## GDPC1-lag4     0.088   0.052  0.057  0.000   0.000   -0.001
## PCECC96-lag4  -0.224  -0.191 -0.004 -0.001   0.000    0.000
## GPDIC1-lag4    0.006   0.013 -0.084  0.000   0.000    0.000
## HOANBS-lag4   -1.018   0.241 -0.777 -0.016   0.001   -0.025
## GDPCTPI-lag4  -0.282   0.379 -0.422 -0.013  -0.023   -0.046
## FEDFUNDS-lag4 -0.704   0.334 -0.705 -0.003   0.001   -0.001
## GDPC1-lag5     0.053   0.055  0.063  0.000   0.000    0.000
## PCECC96-lag5   0.029  -0.030 -0.020 -0.001   0.000    0.000
## GPDIC1-lag5    0.021  -0.053  0.033  0.000   0.000    0.001
## HOANBS-lag5   -0.934   0.221 -0.544 -0.014   0.001   -0.020
```

Coefficients du modèle BVAR

- On obtient les valeurs médianes des estimations de la matrice de variance-covariance avec la fonction `vcov`

```
vcov(run, conf_bands = 0.5)
```

```
## Numeric array (dimensions 6, 6) of variance-covariance values from a BVAR.
## Median values:
##             GDPC1  PCECC96   GPDIC1 HOANBS GDPCTPI FEDFUNDS
## GDPC1    3607.937 1107.909 1761.704 14.639   0.667    9.462
## PCECC96  1107.909  978.587  158.048  4.468  -0.237    2.584
## GPDIC1   1761.704  158.048 1585.531  7.917   0.839    5.581
## HOANBS     14.639    4.468    7.917  0.189   0.010    0.125
## GDPCTPI     0.667   -0.237    0.839  0.010   0.020    0.023
## FEDFUNDS    9.462    2.584    5.581  0.125   0.023    0.632
```

Valeurs prédites

- Les valeurs prédites sont obtenues à l'aide de la fonction `fitted`
- On utilise l'argument `type='mean'` pour obtenir la moyenne des tirages des distributions postérieures

```
fitted(run, type = "mean")
```

```
## Numeric array (dimensions 238, 6) with fitted values from a BVAR.
## Median values:
##  GDPC1: 3559.29, 3550.78, 3533.21, [...], 20379.45, 20431.64, 20671.58
##  PCECC96: 2150.16, 2168.51, 2159.03, [...], 13811.48, 13812.13, 13937.55
##  GPDIC1: 406.41, 381.55, 344.41, [...], 3711.67, 3701.6, 3786.35
##  HOANBS: 52.5, 52.51, 52.06, [...], 111.89, 111.52, 112.07
##  GDPCTPI: 15.51, 15.52, 15.57, [...], 103.48, 103.87, 104.42
##  FEDFUNDS: 4.05, 3.69, 2.76, [...], 1.97, 2.08, 2.63
```

- On obtient des intervalles avec `conf_bands`

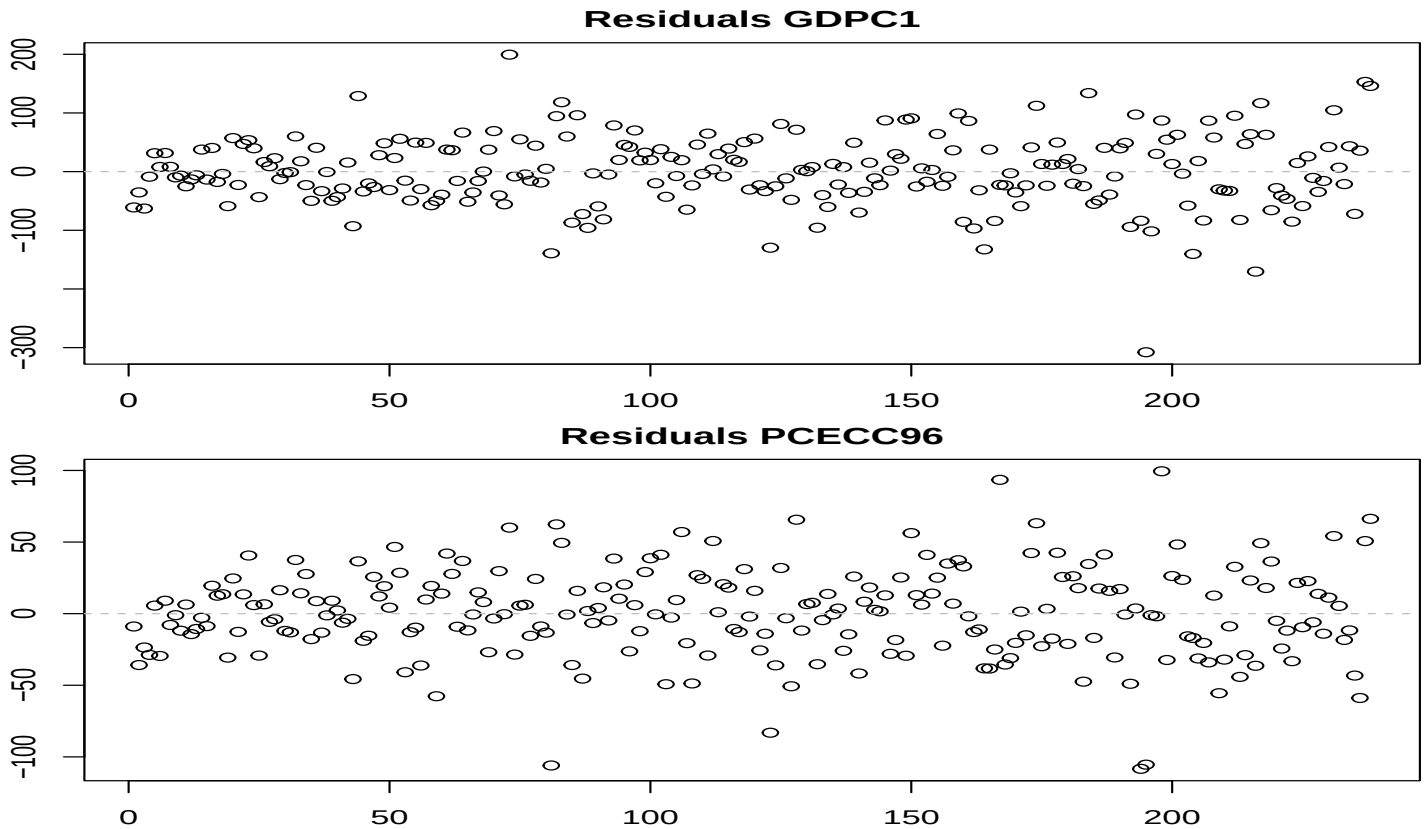
```
fitted(run, conf_bands=0.10)
```

```
## Numeric array (dimensions 3, 238, 6) with fitted values from a BVAR.
## Computed confidence bands: 10%, 50%, 90%
## Median values:
##  GDPC1: 3552.73, 3544.28, 3526.8, [...], 20344.07, 20396.24, 20636
##  PCECC96: 2144.05, 2162.38, 2152.89, [...], 13786.13, 13786.64, 13911.79
##  GPDIC1: 408.18, 383.33, 346.32, [...], 3719.67, 3709.68, 3794.55
##  HOANBS: 52.46, 52.48, 52.02, [...], 111.64, 111.27, 111.81
##  GDPCTPI: 15.51, 15.52, 15.57, [...], 103.44, 103.82, 104.37
##  FEDFUNDS: 4.15, 3.79, 2.86, [...], 2.06, 2.17, 2.72
```

Résidus

- Le code ci-dessous produit un graphique des résidus pour les variables GDPC1 et PCECC96

```
plot(residuals(run, type = "mean"), vars = c("GDPC1", "PCECC96"))
```



Fonctions de réponse impulsionelle

- Les fonctions de réponse impulsionnelle sont stockées dans un objet de classe `bvar_irf`
- Cet objet est stocké dans l'élément `irf` d'un objet `bvar` (un objet de classe `bvar` est une liste)

```
names(run)
```

```
##  [1] "beta"          "sigma"         "hyper"         "ml"
##  [5] "optim"         "priors"        "call"          "variables"
##  [9] "explanatories" "meta"
```

- La fonction `irf()` permet d'y accéder et également de calculer et de mettre à jour les IRFs

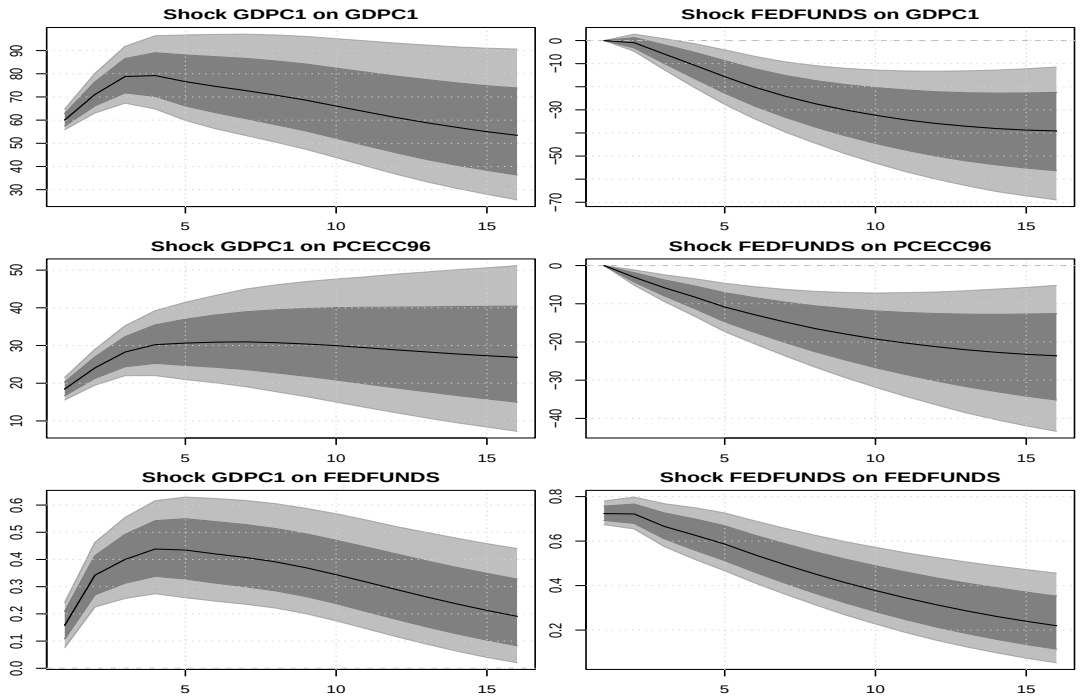
```
opt_irf <- bv_irf(horizon = 16, identification = TRUE)
irf(run) <- irf(run, opt_irf, conf_bands = c(0.05, 0.16))
irf(run)
```

```
## Impulse response object from `bvar()`.
## Horizon: 16
## Identification: Cholesky decomposition
## FEVD: FALSE
## Variables: 6
## Iterations: 25000
```

Impluse Response Functions

- La méthode générique `plot()` permet de visualiser les IRFs
- On peut spécifier les variables *impulses* et/ou les réponses avec le nom ou la position de la variable

```
plot(irf(run), area = TRUE, vars_impulse = c("GDPC1", "FEDFUNDS"),
     vars_response = c(1:2, 6))
```



11 Modèles multivariés pour l'analyse de séries temporelles stationnaires
- Introduction
- Série temporelle multivariée
- Processus VAR
- Test de causalité
- Estimation et sélection du modèle
- Diagnostics
- Fonction de réponse impulsionnelle
- Forecast Error Variance Decomposition
- Modèles VAR Bayesien (BVAR)
- Les modèles SVAR (Structural Vector Autoregressive)

Introduction

- L'idée générale dans la littérature sur les modèles SVAR structurels est de considérer les corrélations instantanées entre variable d'un point de vue théorique structurel [1]
- Lors de la modélisation VAR non-structurelle, ces corrélations sont 'cachées' dans la matrice de variance/covariance des innovations
- L'analyse basée sur les modèles SVAR tente d'isoler un ensemble de chocs indépendants au moyen de restrictions s'appuyant sur la théorie (macroéconomique) ou sur des propriétés des données [20]
- Ces chocs sont considérés comme la source ultime de variation stochastique des variables de la série multivariée

Modèle VAR structurel

- Un modèle VAR peut être interprété comme une forme réduite (*reduced-form model*)
- Le modèle SVAR est sa forme structurelle [27] :

$$Ay_t = A_1^* y_{t-1} + \ldots + A_p^* y_{t-p} + B\epsilon_t$$

- On considère que les erreurs structurelles ϵ_t sont un bruit blanc, et les matrices $A_i^*, i = 1, \ldots, p$ sont des coefficients structurels qui diffèrent de leur forme réduite si $A \neq I$
- Un modèle SVAR peut être utilisé pour identifier des chocs et analyser leur impact avec l'analyse de réponse impulsionnelle et la décomposition de variance (FEVD) en imposant des restrictions sur les matrices A

Restrictions

- Selon les restrictions imposées, on distingue trois types de modèles SVAR
  - Modèle A : $B = I_K$, le nombre minimum de restrictions est $K(K-1)/2)$
  - Modèle B : $A = I_K$, le nombre minimum de restrictions est $K(K-1)/2)$
  - Modèle AB : on peut définir des restrictions sur les deux matrices, le nombre minimum de restrictions est $K^2 + K(K-1)/2)$

La librairie SVAR

- La librairie `vars` permet l'estimation de modèles VAR structurels par l'utilisation de restrictions sur les effets à court et long terme basées sur la théorie (macro)économique
- La librairie `svars` [20] propose d'autres méthodes récentes pour l'estimation de ces modèles, avec plusieurs méthodes d'identification basées sur des propirétés statistiques des données (changements de covariance, de volatilité, ...)
- La librairie propose également des nouvelles méthodes pour l'analyse structurelle de séries multivariées

Modèle SVAR

- Le modèle VAR réduit (de dimension K et d'ordre p) et sa forme structurelle s'écrivent

$$y_t = \mu + A_1^* y_{t-1} + \ldots + A_p^* y_{t-p} + \upsilon_t \quad (5)$$

$$y_t = \mu + A_1^* y_{t-1} + \ldots + A_p^* y_{t-p} + B\epsilon_t \quad (6)$$

- Les constantes sont contenues dans μ
- Le vecteur υ_t représente les résidus de la forme réduite, qui sont sériellement non corrélés avec $E(\upsilon_t) = 0$ et $COV(\upsilon_t) = \Sigma_u$
- La matrice B capture les effets instantanés des chocs structurels sur les variables du système

Modèle SVAR

- La relation entre les résidus de la forme réduite et ceux de la forme structurelle est :

$$Cov(\upsilon_t) = \Sigma_u = B\Sigma_\epsilon B^T$$

où la covariance des chocs structurels $Cov(\epsilon_t) = \Sigma_\epsilon$ est une matrice diagonale (les chocs structurels sont non corrélés)

- Des restrictions sont nécessaires pour identifier la matrice B de manière unique
- Le point de départ est l'estimation de la forme réduite du modèle VAR, les estimations de $\hat{\upsilon}_t$ et $\hat{\Sigma}_\upsilon$ sont le point de départ pour les méthodes d'indentification proposées par la librairie `svars`

Modèle SVAR : Exemple

- Nous suivons l'exemple proposé dans Lange et al. [20], repris de Herwartz et Plödt [12]
- Les données trimestrielles contiennent trois séries :

x	output gap	log-deviation of real gross domestic product (GDP) from the
pi	inflation	quarter-on-quarter growth rates of the GDP deflator
i	federal funds rate	

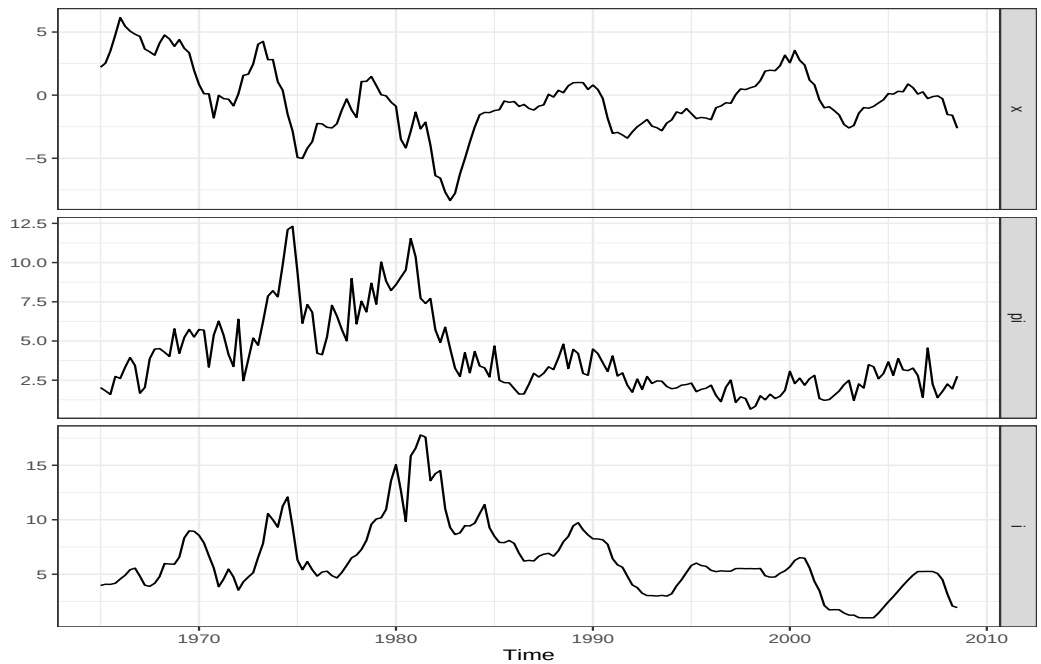
Modèle SVAR : Exemple

- Les données (textttUSA) sont fournies dans la librairie `svars`

```
library("svars")
```

```
## Registered S3 method overwritten by 'svars':
##   method           from
##   stability.varest vars
```

```
data("USA", package = "svars")
autoplot(USA, facets = T) + theme_bw() + ylab("")
```



- Herwartz et Plödt [12] utilisent l'identification du modèle SVAR par les changements de covariance *unconditional covariance shifts* pour étudier les effets d'un choc de politique monétaire sur l'économie des USA

Modèle SVAR : Exemple

- Pour l'identification, il faut fournir à l'avance le point de changement de la variance (*variance shift*)
- Ce point peut-être identifié par inspection visuelle des séries ou à l'aide de connaisances préalables
- Herwartz et Plödt [12] déterminent le point de bascule au T3 1979 (*due to a policy shift of the Federal Reserve Bank which caused a reduction of the volatility in US macroeconomic data*)

Modèle SVAR : Exemple

- La première étape est d'estimer un modèle VAR réduit, ici avec la fonction `VAR` de la librairie `vars`

```
plain.var <- vars::VAR(USA, p = 6, type = "const")
```

- On utilise ensuite la fonction `id.cv()` pour estimer le modèle structurel ; le point de rupture structurel est indiqué avec l'argument `SB = c(1979, 3)` (3ème trimestre 1979)

```
usa.cv <- id.cv(plain.var, SB = c(1979, 3))
summary(usa.cv)
```

```
##
## Identification Results
## ----------------------
##
## Method: Changes in Volatility
## Sample size: 169
## Log-Likelihood: -564.2994
## AIC: 1268.599
## Structural Break: At Observation Number 59 during 1979 Q3
## Number of GLS estimations: 21
## Number of Restrictions: 0
##
## Estimated unconditional Heteroscedasticity Matrix (Lambda):
##         [,1]     [,2]     [,3]
## x  0.3925906 0.000000 0.000000
## pi 0.0000000 0.191641 0.000000
## i  0.0000000 0.000000 1.244348
##
## Standard Errors of Lambda:
##          [,1]       [,2]      [,3]
## x  0.09265819 0.00000000 0.0000000
## pi 0.00000000 0.04527264 0.0000000
## i  0.00000000 0.00000000 0.2935572
##
## Estimated B Matrix (unique decomposition of the covariance matrix):
```

Modèle SVAR : Exemple

- On récupère la matrice B

```
usa.cv$B
```

```
##           [,1]       [,2]      [,3]
## x   0.61193300 -0.5931964 0.2241237
## pi  0.75559400  1.2987520 0.1131134
## i  -0.02899916  0.1572953 0.7084709
```

- Et les erreurs standard

```
usa.cv$B_SE
```

```
##         [,1]      [,2]       [,3]
## x  0.1330924 0.1955350 0.07101215
## pi 0.2498465 0.2600375 0.09960245
## i  0.1559671 0.1213446 0.07004431
```

Modèle SVAR : Exemple

- Pour l'interprétation, Herwartz et Plödt [12] ont réordonné les colonnes de la matrice (l'ordre est arbitraire, ici selon les variations du signe)

```
usa.cv$B <- usa.cv$B[, c(3, 2, 1)]
usa.cv$B[, 3] <- usa.cv$B[, 3] * (-1)

usa.cv$B_SE <- usa.cv$B_SE[, c(3, 2, 1)]

round(usa.cv$B, 3)
```

```
##      [,1]   [,2]   [,3]
## x   0.224 -0.593 -0.612
## pi  0.113  1.299 -0.756
## i   0.708  0.157  0.029
```

- L'interprétation de la matrice $\hat{\beta}$ par Herwartz et Plödt [12] :
  - colonne 1 : impacts d'un choc de demande
  - colonne 2 : impacts d'un choc d'offre
  - colonne 3 : impacts d'un choc de politique monétaire

Modèle SVAR : Interprétation

- La matrice $\hat{\lambda}$ représente les ruptures de covariance entre les deux périodes
- Les chocs d'offre et de politique monétaire ont une variance moins élevée dans la période 2 et les chocs de demande une variance plus élevée

```
usa.cv$Lambda <- diag(diag(usa.cv$Lambda)[c(3, 2, 1)])
usa.cv$Lambda_SE <- diag(diag(usa.cv$Lambda_SE)[c(3, 2, 1)])

round(usa.cv$Lambda, 3)
```

```
##       [,1]  [,2]  [,3]
## [1,] 1.244 0.000 0.000
## [2,] 0.000 0.192 0.000
## [3,] 0.000 0.000 0.393
```

- L'interprétation de la matrice $\hat{\beta}$ par Herwartz et Plödt [12] :
  - colonne 1 : impacts d'un choc de demande
  - colonne 2 : impacts d'un choc d'offre
  - colonne 3 : impacts d'un choc de politique monétaire

Modèle SVAR

Fonction de réponse impulsionnelle

- L'étape suivante est le calcul des IRF pour analyser les effets futurs des chocs structurels identifiés précédemment
- Des restrictions sur les signes sont imposées

```
signrest <- list(demand = c(1, 1, 1), supply = c(-1, 1, 1), monetary_policy = c(-1, -1, 1))
```

- Calcul (parallèle) des IRF et des bandes de confiance avec bootstrap

```
cores <- parallel::detectCores() - 1
set.seed(231)
usa.cv.boot <- wild.boot(usa.cv, design = "fixed", distr = "rademacher", nboot = 1000,
  n.ahead = 15, nc = cores, signrest = signrest)
```

Modèle SVAR

Fonction de réponse impulsionnelle

- La fonction générique `summary()` permet d'obtenir des résultats détaillés

```
summary(usa.cv.boot)
```

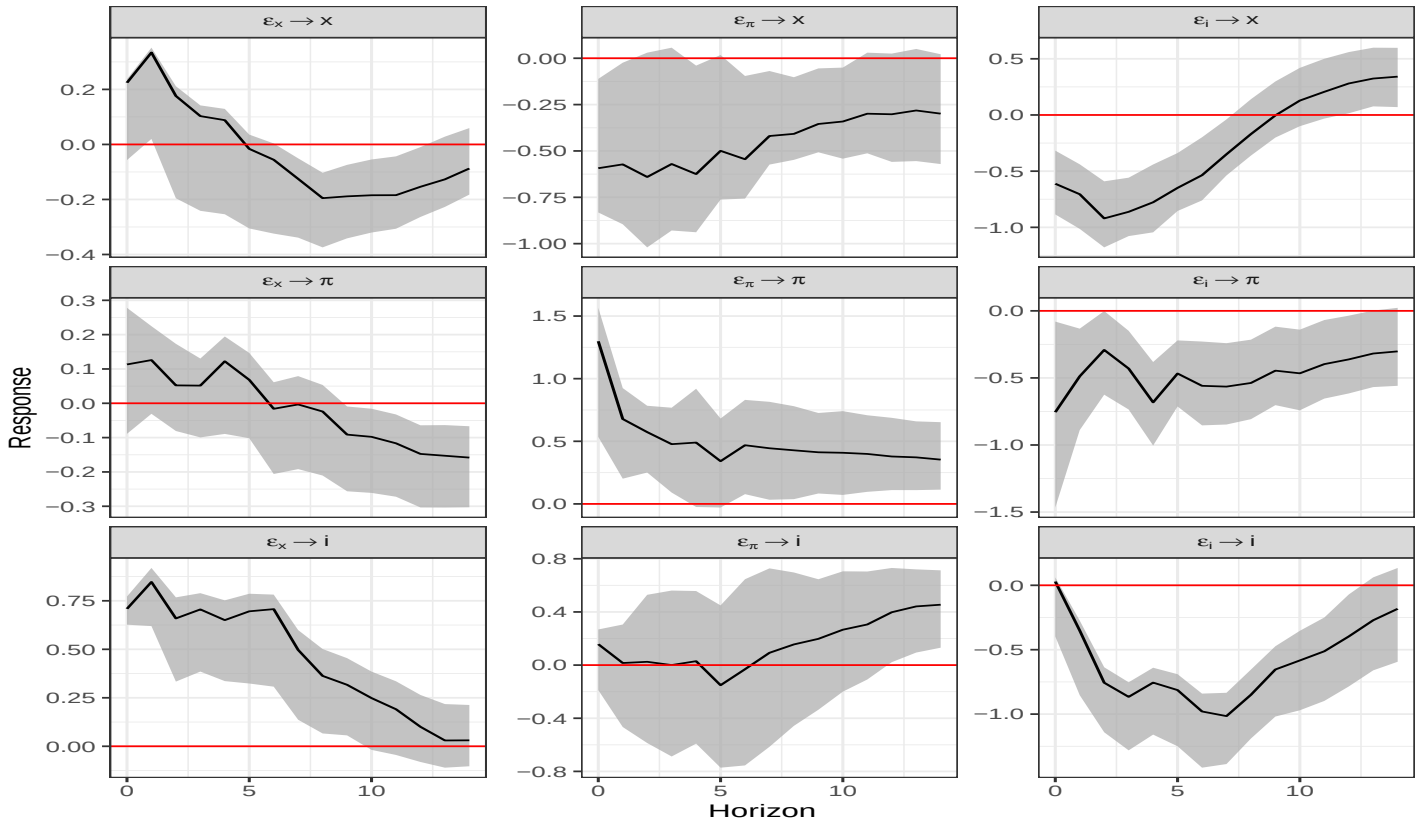
```
##
## Bootstrap Results
## -----------------
##
## Method: Wild bootstrap
## Bootstrap iterations: 1000
## Distribution used: rademacher
## Design: fixed
##
## Point estimates:
##         [,1]       [,2]        [,3]
## x  0.2241237 -0.5931964 -0.61193300
## pi 0.1131134  1.2987520 -0.75559400
## i  0.7084709  0.1572953  0.02899916
##
## Bootstrap means:
##          [,1]        [,2]       [,3]
## x  0.09580769 -0.50304855 -0.6303866
## pi 0.09639677  1.13108997 -0.7720160
## i  0.70100301  0.03648016 -0.1675659
##
## Bootstrap standard errors:
##          [,1]      [,2]      [,3]
## x  0.14143449 0.3077349 0.2494872
## pi 0.17500244 0.4556693 0.5925735
## i  0.07511819 0.2273603 0.2178135
##
## Identified sign patterns:
## =========================
## Specified sign pattern:
##
##    demand supply monetary_policy
## x       1     -1              -1
```

Modèle SVAR

Fonction de réponse impulsionnelle

- Les IRF représentent l'impact d'un choc unitaire sur les variables du système

```
plot(usa.cv.boot, lowerq = 0.16, upperq = 0.84)
```



Modèle SVAR

Décomposition de la variance de l'erreur de prévision

- La décomposition de la variance de l'erreur de prévision (FEVD, *Forecast Error Variance Decomposition*) permet d'étudier la contribution de chaque choc à l'erreur quadratique moyenne de prévision des différentes variables

```
fev.cv <- fevd(usa.cv, n.ahead = 30)
```

```
## Error in ‘fevd.default()‘:
## ! No methods for class svars found.
```

```
plot(fev.cv)
```

```
## Error in ‘h()‘:
## ! erreur d’ï¿½valuation de l’argument ’x’ lors de la sï¿½lection d’une mï¿½thode pour la fonction ’plot’ : objet ’fev.cv’ introuvable
```

12 Cointégration et modèle à correction d'erreur

Introduction

- Nous abordons maintenant l'analyse de séries temporelles multivariées lorsqu'une ou plusieurs des séries est intégrée d'ordre 1, i.e. non stationnaire
- Le concept de **cointégration** introduit par [7] permet l'étude des relations stables à long terme de variables non-stationnaires

12 Cointégration et modèle à correction d'erreur
- Régression fallacieuse
- Cointégration et modèle à correction d'erreur
- Modèle ECM : application

Régression fallacieuse

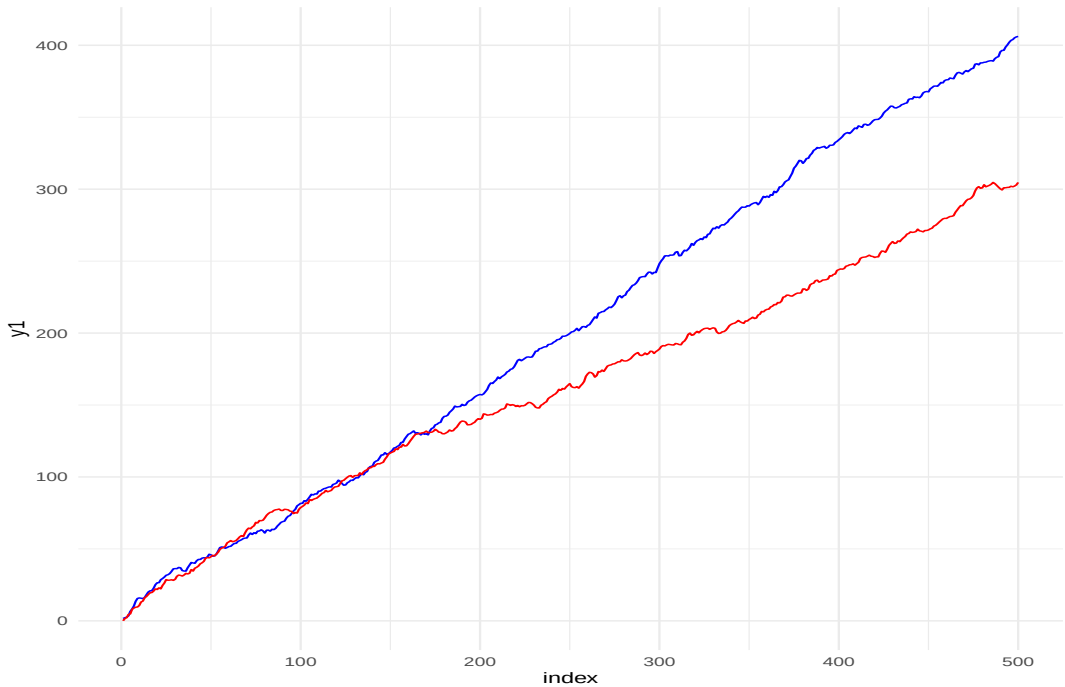
- Un modèle de régression linéaire impliquant des séries non-stationnaires de type stochastique (difference stationary data) produit des résultats erronés (voir par exemple [27])
- Le code suivant simule deux marches aléatoires avec dérive

```
set.seed(123456)
e1 <- rnorm(500)
e2 <- rnorm(500)
trd <- 1:500
y1 <- 0.8 * trd + cumsum(e1)
y2 <- 0.6 * trd + cumsum(e2)
spdata <- data.frame(index=1:500, y1,y2)
```

Régression fallacieuse

- Les deux séries paraissent avoir une tendance commune

```
ggplot(spdata) +
  geom_line(aes(x=index, y=y1), color="blue") +
  geom_line(aes(x=index, y=y2), color="red")
```



Régression fallacieuse (2)

- Le modèle de régression suivant indique que les coefficients sont significativement différents de 0, et le R^2 est proche de 1

```
sr.reg <- lm(y1 ~ y2)
sr.dw <- dwtest(sr.reg)$statistic
summary(sr.reg)
```

```
##
## Call:
## lm(formula = y1 ~ y2)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -30.6541 -11.5262   0.3589  11.1423  31.0058
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept) -29.326969   1.367155  -21.45   <2e-16 ***
## y2            1.440787   0.007519  191.62   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 13.71 on 498 degrees of freedom
## Multiple R-squared:  0.9866, Adjusted R-squared:  0.9866
## F-statistic: 3.672e+04 on 1 and 498 DF,  p-value: < 2.2e-16
```

- Il s'agit d'un exemple de régression fallacieuse (**spurious regression**) : une régression statistiquement trompeuse résultant notamment de la mise en relation de séries non stationnaires indépendantes et non cointégrées

12 Cointégration et modèle à correction d'erreur
- Régression fallacieuse
- Cointégration et modèle à correction d'erreur
- Modèle ECM : application

Cointégration

Introduction

- Pour éviter ce problème, on pourrait utiliser les séries intégrées dans la régression
- Cependant, de nouveaux problèmes apparaissent [27] :
  1. La différenciation atténue les fortes valeurs positives d'autocorrélation des résidus, affectant l'inférence sur les coefficients de la régression
  2. L'analyse des relations d'équilibre à long terme entre niveaux des variables est compliquée si on utilise les séries différenciées

Cointégration

Définition

- Le concept de **cointégration** introduit par [7] permet l'étude des relations stables à long terme de variables non-stationnaires
- Deux ou plusieurs séries chronologiques cointégrées partagent une dérive stochastique commune et leur combinaison linéaire est une série stationnaire
- Les composants du vecteur y_t sont cointégrés d'ordre d, b, noté $y_t \sim CI(d, b)$, si
  1. Tous les composants de y_t sont $I(d)$ (intégrés d'ordre d)
  2. Un vecteur $\alpha(\neq 0)$ existe tel que $z_t = \alpha\prime y_t \sim I(d - b), b > 0$
- Le vecteur α est appelé **vecteur de cointégration**
- Grace à cette approche, on peut étudier la relation de long terme entre variables non-stationnaires

Cointégration

Test de cointégration

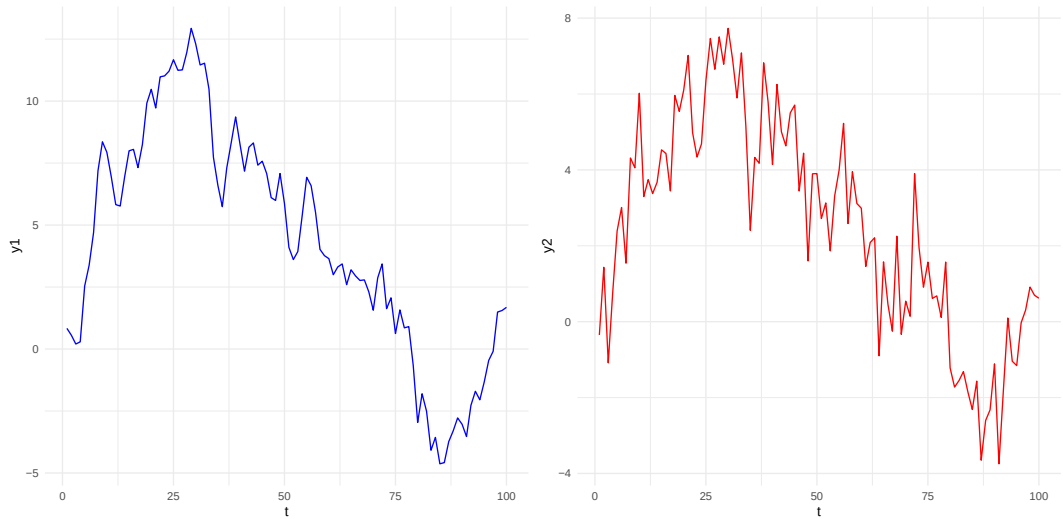
- Si deux séries non-stationnaires paraissent possèder des caractéristiques communes, on peut tester l'hypothèse d'une composante non-stationnaire commune au moyen d'un test de cointégration [17, cf.]
- Pour tester la cointégration, une première méthode en deux étapes a été proposée par [7] :
  1. Modèle de régression d'une série par l'autre
  2. Test de racine unitaire (ADF) sur les résidus

Simulation de séries cointégrées

- Pour illuster le concept de cointégration, on simule deux marches aléatoires `y1` et `y2` (exemple tiré de [27])
- La seconde marche aléatoire est définie par `y2=0.6 * y1 + e2`, `e2` est un bruit blanc

```
set.seed (123456)
e1 <- rnorm(100)
e2 <- rnorm(100)
y1 <- cumsum(e1)
y2 <- 0.6 * y1 + e2

g1 <- ggplot() + geom_line(aes(x=1:100, y=y1), color="blue") + xlab("t")
g2 <- ggplot() + geom_line(aes(x=1:100, y=y2), color='red') + xlab("t")
g1+g2
```



Cointégration

Test de cointégration : Etape 1

- Estimation du **vecteur de cointégration**

$$z_t = \alpha_1 y_{t,1} + \alpha_2 y_{t,2} + \ldots + \alpha_K y_{t,K} + \epsilon_t$$

pour $t = 1, \ldots, T$; ϵ_t représente les résidus

- Cette régression statique permet d'estimer α avec un modèle OLS, bien que, comme dans le cas de la régression fallacieuse, les statistiques t et F ne sont pas applicables [27]
- Si il y a cointégration, les résidus ϵ_t sont $I(0)$, i.e. stationnaires
- Ces résidus représentent les déviations de l'équilibre de long terme des variables cointégrées
- On réalise un test de stationnarité ADF sur les résidus
- On peut utiliser en première approche le test de Durbin-Watson (absence d'autocorrélation des résidus)

Cointégration : simulation

Equation de long terme

- On estime l'équation de long terme (long-run equation)

```
lr.reg <- lm(y2 ~ y1)
summary(lr.reg)
```

```
##
## Call:
## lm(formula = y2 ~ y1)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -2.44465 -0.66254  0.03931  0.83827  2.04555
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)  0.03785    0.13477   0.281    0.779
## y1           0.58112    0.02138  27.186   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.985 on 98 degrees of freedom
## Multiple R-squared:  0.8829, Adjusted R-squared:  0.8817
## F-statistic: 739.1 on 1 and 98 DF,  p-value: < 2.2e-16
```

- Le vecteur de cointégration est (1,-0.6), l'estimation sur les séries est (1,-0.58)

Cointégration : simulation

Stationarité de la série des résidus

- On réalise un test ADF sur les résidus :
  - Hypothèse H0 : la série présente une tendance stochastique (racine unitaire)
  - Hypothèse H1 : la série est stationnaire

```
errors <- residuals(lr.reg)
errors.df <- ur.df(y=errors, lags=1, type='drift')
```

- Valeur de la statistique du test

```
attr(errors.df, "teststat")
```

```
##                tau2     phi1
## statistic -6.592239 21.73528
```

- Valeurs critiques

```
attr(errors.df, "cval")
```

```
##       1pct  5pct 10pct
## tau2 -3.51 -2.89 -2.58
## phi1  6.70  4.71  3.86
```

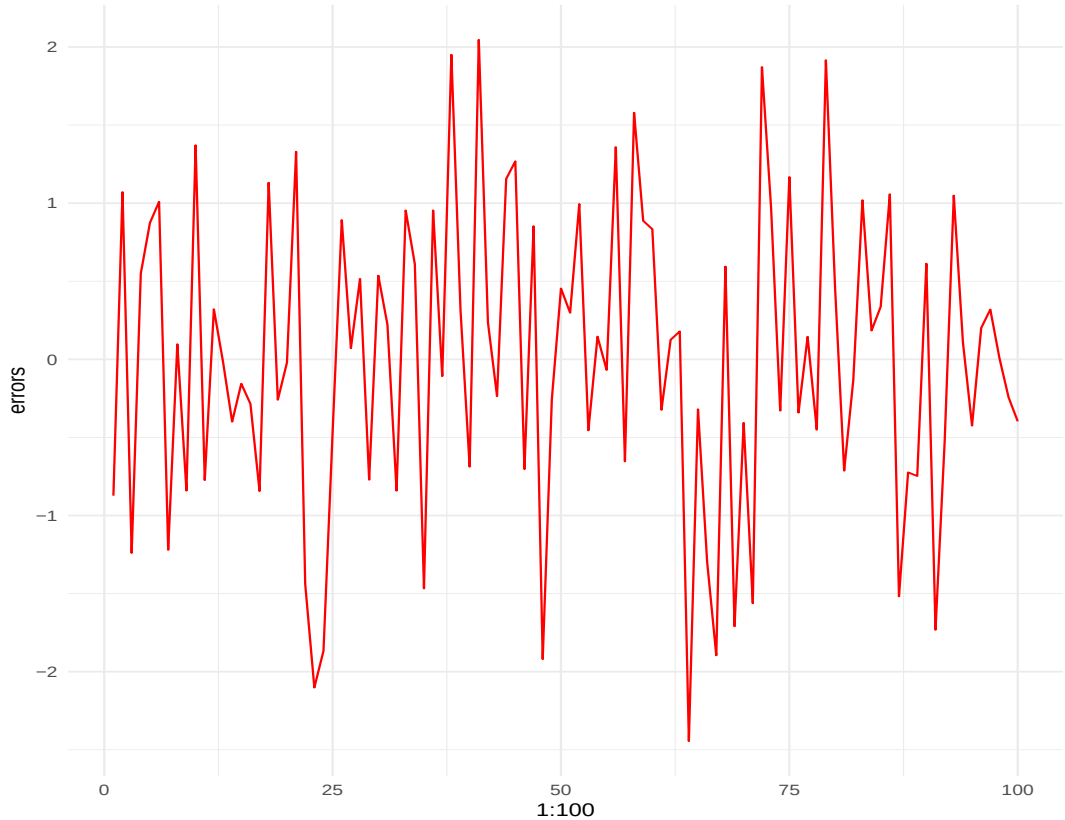
- La valeur de `tau2` est inférieure au seuil critique à 1%, la série des résidus est stationnaire

Cointégration : simulation

Série des résidus

- Série des résidus z_t

```
ggplot() + geom_line(aes(x=1:100, y=errors), color="red")
```



Cointégration

Test de cointégration : Etape 2 (1)

- Un fois l'hypothèse H0 d'une racine unitaire dans la série ϵ_t rejetée (.i.e. ϵ_t stationnaire), la deuxième étape consiste à estimer un modèle à correction d'erreur (Error-Correction Model, ECM)
- Soit y_t et x_t deux variables $I(1)$ (intégrées d'ordre 1), et $\hat{z}_t$ la série des résidus du modèle statique précédent

$$\Delta y_t = \psi_0 + \gamma_1 \hat{z}_{t-1} + \sum_{i=1}^{K} \psi_{1,i} \Delta x_{t-i} + \sum_{i=1}^{L} \psi_{2,i} \Delta y_{t-i} + \epsilon_{1,t} \quad (7)$$

$$\Delta x_t = \zeta_0 + \gamma_2 \hat{z}_{t-1} + \sum_{i=1}^{K} \zeta_{1,i} \Delta y_{t-i} + \sum_{i=1}^{L} \zeta_{2,i} \Delta x_{t-i} + \epsilon_{2,t} \quad (8)$$

Cointégration

Test de cointégration : Etape 2 (2)

- Les changements de y_t sont expliqués par
  - La série des changements précédents de y_t :

$$\sum_{i=1}^{L} \psi_{2,i} \Delta y_{t-i}$$

  - La série des changements précédents de x_t :

$$\sum_{i=1}^{L} \zeta_{2,i} \Delta x_{t-i}$$

  - La déviation de l'équilibre de long terme à la période précédente $t-1$:

$$\gamma_1 \hat{z}_{t-1}$$

- La valeur de γ_1 détermine la vitesse d'ajustement et son signe doit toujours être négatif (sinon le système diverge de son équilibre de long terme)
- Ces équations impliquent que la causalité au sens de Granger doit exister dans au moins une direction (au moins une variable doit aider à prédire l'autre)

Cointégration

Modèle ECM

- Une fois que l'hypothèse d'une racine unitaire d
- Préparation des données pour le modèle ECM, on utilise la fonction `diff`
- Pour obtenir la série des résidus z_{t-1} on supprime les deux dernières valeurs du vecteur `errors`

```
errors.lagged <- errors[- c(99, 100)]
dy1 <- diff(y1)
dy2 <- diff(y2)
diff.dat <- data.frame(embed(cbind(dy1,dy2) , 2))
colnames(diff.dat) <- c('dy1' , 'dy2' , 'dy1.1' , 'dy2.1')
head(diff.dat)
```

```
##           dy1        dy2       dy1.1      dy2.1
## 1 -0.35500184 -2.5179073 -0.27604777  1.7837118
## 2  0.08748742  1.8407098 -0.35500184 -2.5179073
## 3  2.25225573  1.6331811  0.08748742  1.8407098
## 4  0.83446013  0.6200382  2.25225573  1.6331811
## 5  1.31241551 -1.4668629  0.83446013  0.6200382
## 6  2.50264541  2.7717847  1.31241551 -1.4668629
```

Cointégration

Cointégration : Simulation (6)

- Résultats du modèle ECM : la déviation de la période précédente est annulée (le coefficient associé à z_{t-1} est proche de 1)

```
ecm.reg <- lm(dy2 ~ errors.lagged + dy1.1 + dy2.1,
  data= diff.dat)
summary(ecm.reg)
```

```
##
## Call:
## lm(formula = dy2 ~ errors.lagged + dy1.1 + dy2.1, data = diff.dat)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -2.9588 -0.5439  0.1370  0.7114  2.3065
##
## Coefficients:
##                Estimate Std. Error t value Pr(>|t|)
## (Intercept)    0.003398   0.103611   0.033    0.974
## errors.lagged -0.968796   0.158554  -6.110 2.24e-08 ***
## dy1.1          0.808633   0.112042   7.217 1.35e-10 ***
## dy2.1         -1.058913   0.108375  -9.771 5.64e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.026 on 94 degrees of freedom
## Multiple R-squared:  0.5464, Adjusted R-squared:  0.5319
## F-statistic: 37.74 on 3 and 94 DF,  p-value: 4.243e-16
```

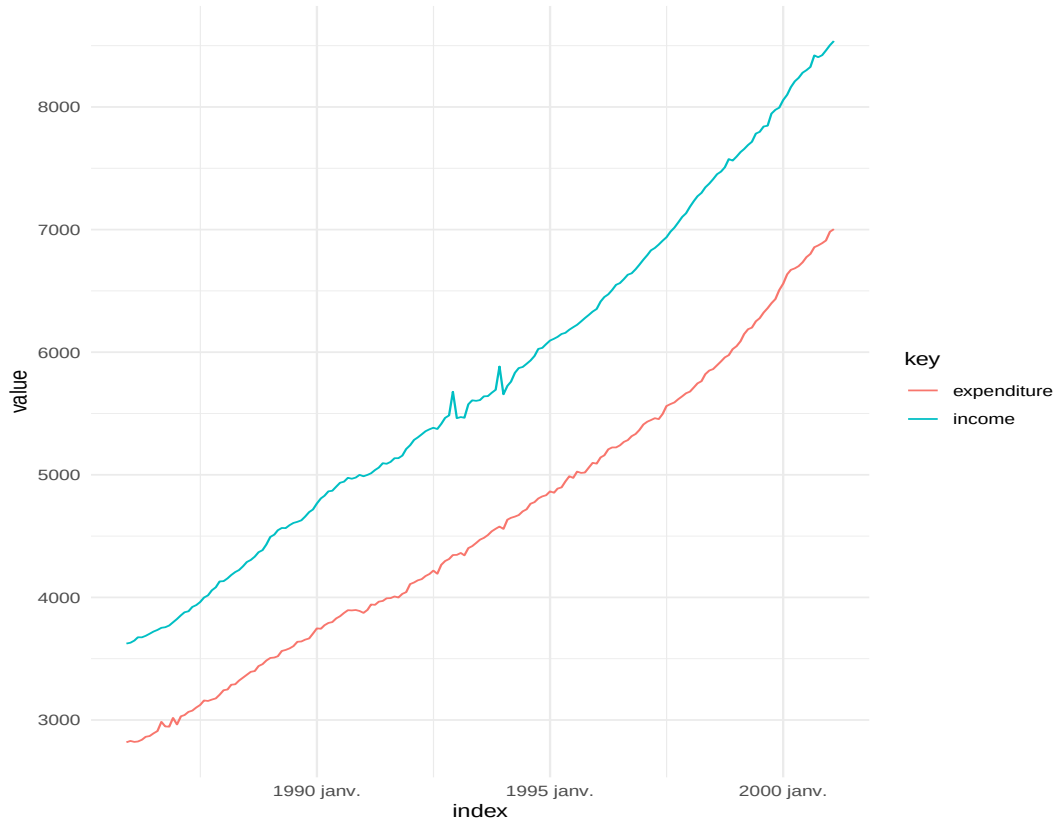
12 Cointégration et modèle à correction d'erreur
- Régression fallacieuse
- Cointégration et modèle à correction d'erreur
- Modèle ECM : application

Modèle ECM

Données USIncExp (1)

- Nous allons utiliser les données USIncExp et suivre l'exemple de l'article présentant la librairie strucchange [32]
- Les données représentent les séries mensuelles des revenus et des dépenses personnels aggrégés

```
USIncExp2 <- window(USIncExp, start = c(1985,12))
USIncExp2 %>% as_tsibble() %>% ggplot() + geom_line(aes(x=index, y=value, color=key))
```



Modèle ECM

Données `USIncExp` : Modèle ECM

- La fonction de consommation (consumption function) est modélisée par un modèle ECM (cf. [11]) :

$$\Delta c_t = \beta_1 + \beta_2 e_{t-1} + \beta_3 \Delta i_t + \mu_t \quad (9)$$

$$e_t = c_t - \alpha_1 - \alpha_2 i_t \quad (10)$$

où c_t représente les dépenses de consommation et i_t le revenu

- L'équation de cointégration

$$c_t = \alpha_1 + \alpha_2 i_t + e_t$$

est estimée à l'aide d'un modèle OLS

Modèle ECM

Données USIncExp : Equation de cointégration

- Estimation de l'équation d'intégration

```
USIncExp.mod1 <- lm(expenditure ~ income, data = USIncExp2)
summary(USIncExp.mod1)
```

```
##
## Call:
## lm(formula = expenditure ~ income, data = USIncExp2)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -160.536  -31.626    2.209   29.106  115.668
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept) -2.358e+02  1.511e+01   -15.6   <2e-16 ***
## income       8.352e-01  2.538e-03   329.1   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 47.74 on 181 degrees of freedom
## Multiple R-squared:  0.9983, Adjusted R-squared:  0.9983
## F-statistic: 1.083e+05 on 1 and 181 DF,  p-value: < 2.2e-16
```

- Les résidus $\hat{e}_t$ seront utilisés comme variable indépendante dans la modélisation de la fonction de consommation

Modèle ECM

Données USIncExp : Résidus de cointégration

- On vérifie que la série des résidus est stationnaire avec un test ADF

```
coint.res <- residuals(lm(expenditure ~ income, data = USIncExp2))
coint.res.df <- ur.df(y=coint.res, lags=1, type='drift')
```

- Valeur de la statistique du test

```
attr(coint.res.df, "teststat")
```

```
##                tau2     phi1
## statistic -2.910746 4.273995
```

- Valeurs critiques

```
attr(coint.res.df, "cval")
```

```
##       1pct  5pct 10pct
## tau2 -3.46 -2.88 -2.57
## phi1  6.52  4.63  3.81
```

- La valeur de tau2 est inférieure au seuil critique à 5%, on conclut que la série des résidus est stationnaire

Modèle ECM

Données USIncExp : Préparation des données (1)

- On utilise la fonction `lag` car les résidus dans l'équation (1) sont à $t-1$
- On utilise la fonction `diff` pour intégrer les séries (obtenir Δc_t et Δi_t)

```
coint.res <- stats::lag(ts(coint.res, start = c(1985,12), freq = 12), k = -1)
USIncExp2 <- cbind(USIncExp2, diff(USIncExp2), coint.res)
USIncExp2 <- window(USIncExp2, start = c(1986,1), end = c(2001,2))
colnames(USIncExp2) <- c("income", "expenditure", "diff.income", "diff.expenditure", "coint.res")
window(USIncExp2, start=c(1986,1), end=c(1986,12))
```

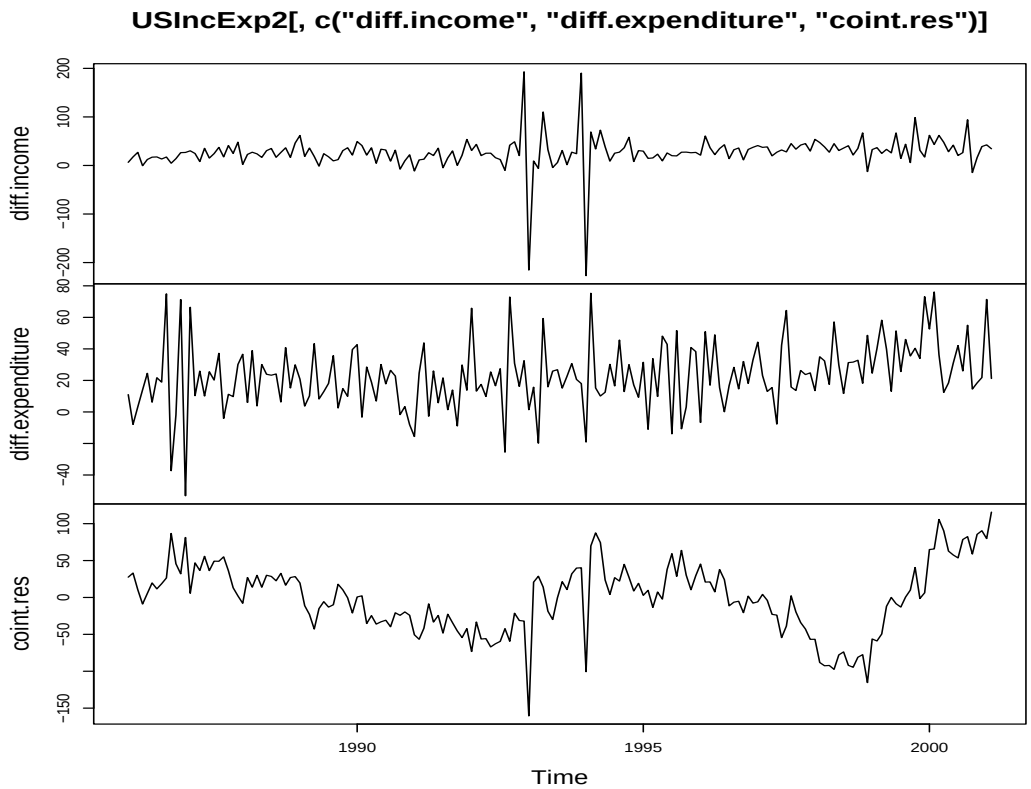
```
##          income expenditure diff.income diff.expenditure coint.res
## Jan 1986 3630.1      2829.3         6.5              11.0 27.510616
## Feb 1986 3647.7      2821.4        17.6              -7.9 33.081616
## Mar 1986 3674.8      2824.6        27.1               3.2 10.481554
## Apr 1986 3674.3      2838.6        -0.5              14.0 -8.953200
## May 1986 3686.6      2863.1        12.3              24.5  5.464415
## Jun 1986 3703.7      2869.3        17.1               6.2 19.691076
## Jul 1986 3721.3      2891.0        17.6              21.7 11.608630
## Aug 1986 3734.6      2909.9        13.3              18.9 18.608568
## Sep 1986 3752.0      2984.8        17.4              74.9 26.399998
## Oct 1986 3756.6      2947.5         4.6             -37.3 86.766982
## Nov 1986 3770.6      2945.6        14.0              -1.9 45.624921
## Dec 1986 3797.0      3016.9        26.4              71.3 32.031690
```

Modèle ECM

Données `USIncExp` : Préparation des données (2)

Séries utilisées dans le modèle ECM

```
plot(USIncExp2[,c("diff.income", "diff.expenditure", "coint.res")])
```



Modèle ECM

Données USIncExp : Estimation

- La variable dépendante est l'augmentation des dépenses et les variables indépendantes sont les résidus de cointegration et l'augmentation des revenus (plus une constante)

```
ecm.model <- diff.expenditure ~ coint.res + diff.income
summary(lm(ecm.model, data=USIncExp2))
```

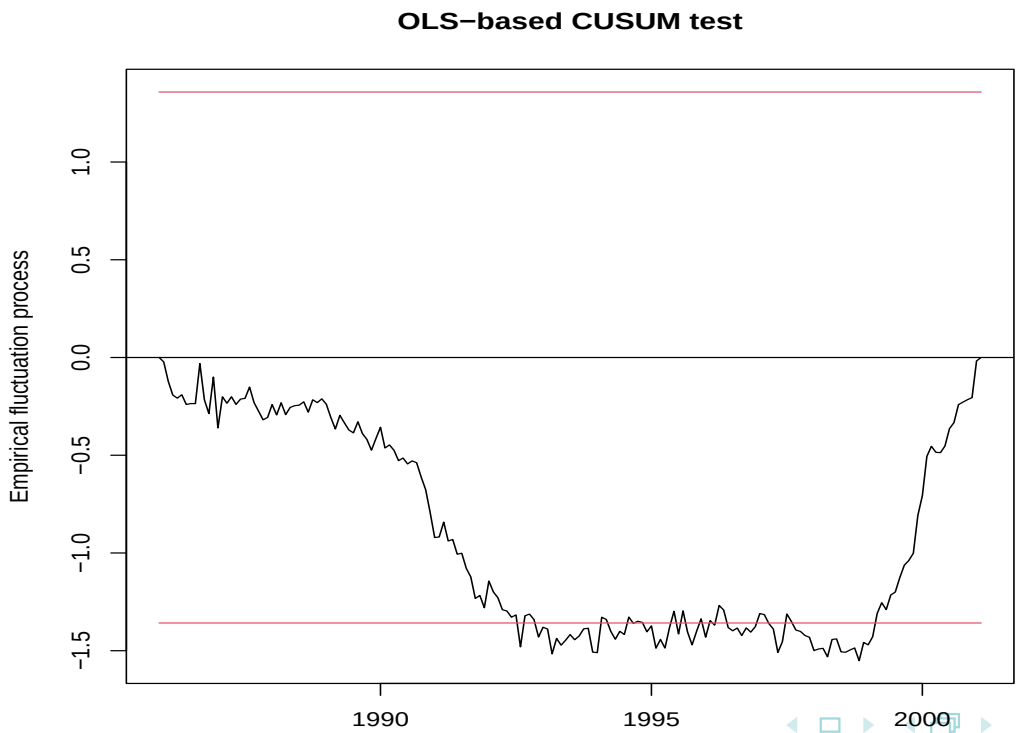
```
##
## Call:
## lm(formula = ecm.model, data = USIncExp2)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -70.555 -11.947  -0.528  10.452  55.576
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept) 17.82428    1.88918   9.435  < 2e-16 ***
## coint.res   -0.06843    0.03318  -2.063   0.0406 *
## diff.income  0.19001    0.04282   4.438 1.59e-05 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 20.09 on 179 degrees of freedom
## Multiple R-squared:  0.1021, Adjusted R-squared:  0.09203
## F-statistic: 10.17 on 2 and 179 DF,  p-value: 6.542e-05
```

Modèle ECM

Données USIncExp : Test de changement structurel (1)

- Test de changement structurel avec la fonction efp (empirical fluctuation process), basé sur la somme cumulée des résidus standardizés du modèle de régression (type="OLS-CUSUM")
- Le dépassement de la limite $b(t)$ (ligne rouge) indique la violation de l'hypothèse H0 'pas de changement structurel'

```
ocus <- efp(ecm.model, type="OLS-CUSUM", data=USIncExp2)
plot(ocus)
```

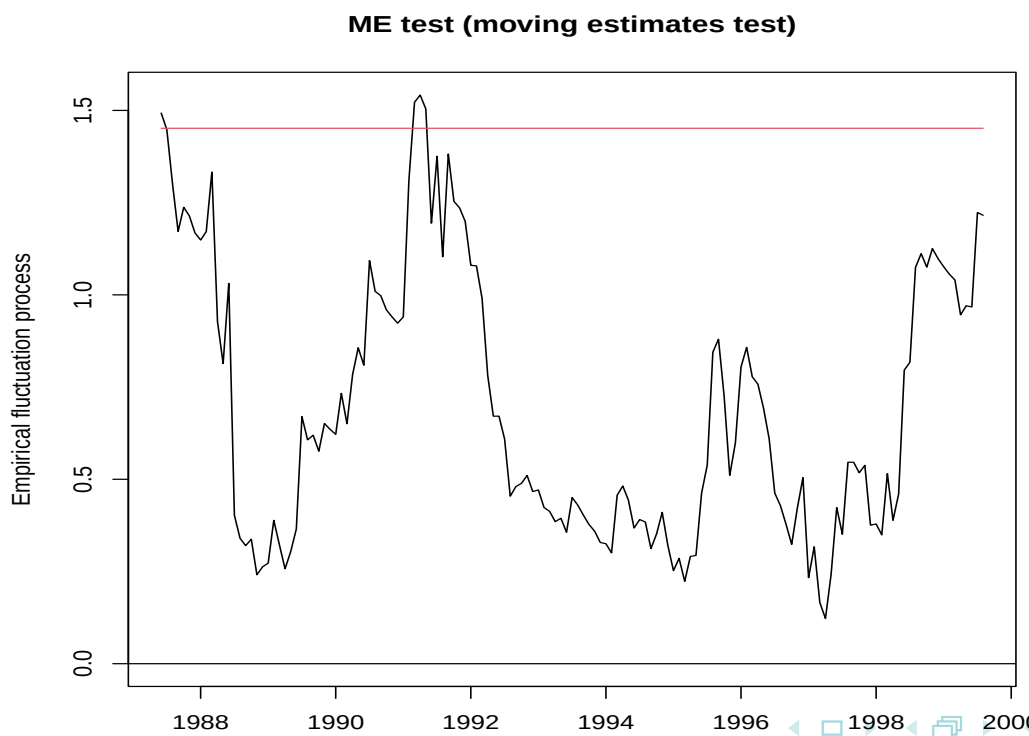


Modèle ECM

Données USIncExp : Test de changement structurel (2)

- Test de changement structurel avec la fonction efp (empirical fluctuation process), basé sur la somme mobile des résidus standardizés du modèle de régression (type="ME")
- Le dépassement de la limite $b(t)$ (ligne rouge) indique la violation de l'hypothèse H0 'pas de changement structurel'

```
me <- efp(ecm.model, type="ME", data=USIncExp2, h=0.2)
plot(me)
```



Modèle ECM

Données GermanM1 (1)

- Les données GermanM1 sont fournies avec la librairie strucchange
- Ces données sont utilisées par Lütkepohl, Teräsvirta et Wolters [24] dans leur article *Investigating stability and linearity of a German M1 money demand function*
- Le sujet de l'article est la stabilité de la fonction de demande de monnaie en Allemagne, avant et après la réunification
- Les données GermanM1 sont trimestrielles et non-corrigées des variations saisonnières

```
data("GermanM1")
head(GermanM1)
```

```
##          m        p        y     R          dm          dy2     dR    dR1
## 1 7.951792 3.449861 8.374404 0.060 -0.07895470  0.078125000 -0.002 -0.002
## 2 8.020040 3.427157 8.420854 0.057  0.06824780  0.011972427 -0.003 -0.002
## 3 8.002081 3.478868 8.482004 0.060 -0.01795864 -0.102929115  0.003 -0.003
## 4 8.090023 3.473332 8.488274 0.060  0.08794212  0.046449661  0.000  0.003
## 5 8.006645 3.494354 8.387732 0.058 -0.08337784  0.061150551 -0.002  0.000
## 6 8.071870 3.470630 8.461327 0.060  0.06522465  0.006269455  0.002 -0.002
##             dp       m1       y1    R1 season   ecm.res
## 1  0.029972315 8.030746 8.477333 0.062     Q1 0.1353536
## 2 -0.022703170 7.951792 8.374404 0.060     Q2 0.1322408
## 3  0.051710605 8.020040 8.420854 0.057     Q3 0.1321171
## 4 -0.005536079 8.002081 8.482004 0.060     Q4 0.1406767
## 5  0.021021843 8.090023 8.488274 0.060     Q1 0.1309088
## 6 -0.023723603 8.006645 8.387732 0.058     Q2 0.1286516
```

Modèle ECM

Données GermanM1 (2)

Données GermanM1

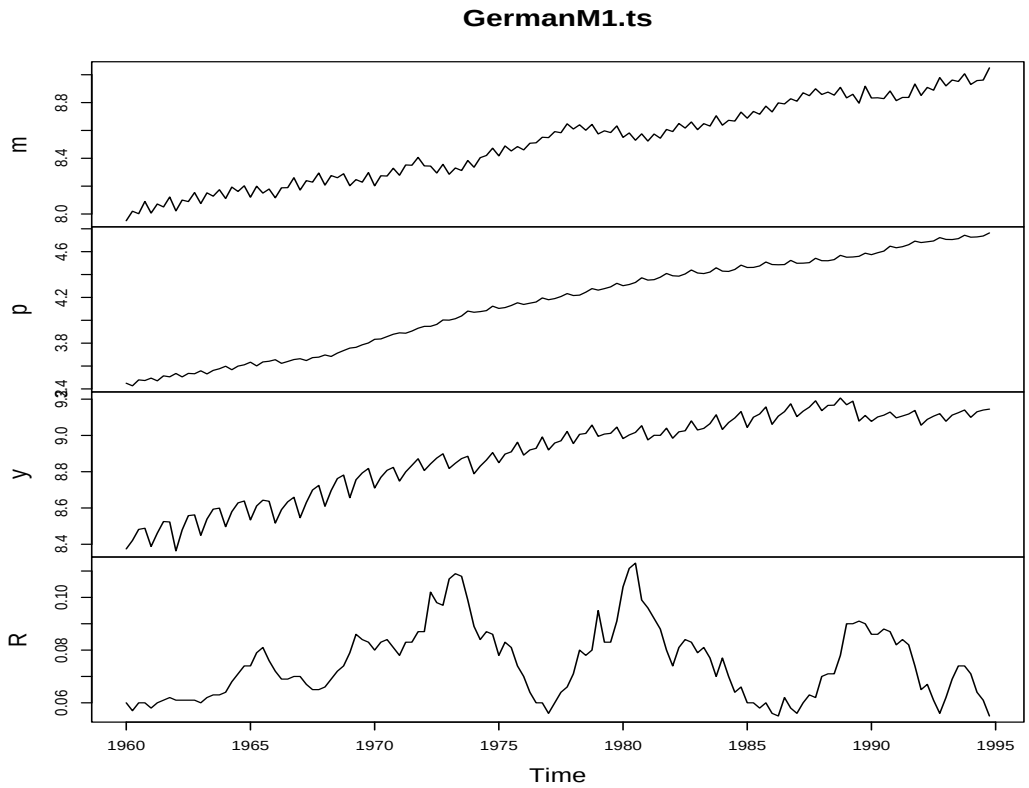
m	time series	Logarithm of real M1 per capita
p	time series	Logarithm of a price index
y	time series	Logarithm of real per capita gross national product
R	time series	Long-run interest rate
dm	time series	First differences of m
dy2	time series	First differences of lag 2 of y
dR	time series	First differences of R
dR1	time series	First differences of lag 1 of R
dp	time series	First differences of p
m1	time series	Lag 1 of m
y1	time series	Lag 1 of y
R1	time series	Lag 1 of R
season		factor coding the seasonality (quarter)
ecm.res		vector containing the OLS residuals of the Lütkepohl et al. (1999) mod fitted in the history period (up to 1990(2), i.e., the data before the Germa monetary unification on 1990-06-01)

Modèle ECM

Données GermanM1 (3)

- Séries `m` (demande de monnaie, log), `p` (indice des prix, log), `y` (PIB par habitant, log) et `R` (taux d'intérêt à long-terme)

```
GermanM1.ts <- ts(GermanM1[, c("m","p", "y", "R")], start=c(1960,1), frequency=4)
plot(GermanM1.ts)
```



Modèle ECM

Données GermanM1 : Modèle ECM

- Les données sont divisées en deux sous-ensembles :
  - `historyM1` : du 1er trimestre 1960 au 2ème trimestre 1990, avant la réunification monétaire du 01/06/1990
  - `monitorM1` : du 3ème trimestre 1990 au 4ème trimestre 1995, après la réunification
- Le modèle ECM estimé par Lütkepohl, Teräsvirta et Wolters [24] sur les données `historyM1` est le suivant :

```
LTW.model <- dm ~ dy2 + dR + dR1 + dp + m1 + y1 + R1 + season
```

Modèle ECM

Données GermanM1 : Modèle ECM original - Estimation

- Tous les coefficients (sauf l'intercept) sont significatifs :

```
LTW.model.res <- summary(lm(LTW.model, data=historyM1))
LTW.model.res$coefficients
```

```
##                Estimate  Std. Error     t value     Pr(>|t|)
## (Intercept) -0.05025441 0.067425080  -0.7453371 4.577021e-01
## dy2         -0.29729418 0.052162011  -5.6994386 1.072596e-07
## dR          -0.67278600 0.246249673  -2.7321295 7.362950e-03
## dR1         -0.99950033 0.265621540  -3.7628738 2.745855e-04
## dp          -0.52786586 0.077434377  -6.8169446 5.709247e-10
## m1          -0.12068219 0.034484939  -3.4995623 6.804996e-04
## y1           0.13480364 0.039541650   3.4091557 9.198022e-04
## R1          -0.61699509 0.143765400  -4.2916800 3.906345e-05
## seasonQ1    -0.13330148 0.003941372 -33.8210851 5.977754e-59
## seasonQ2    -0.01559447 0.004456409  -3.4993359 6.810179e-04
## seasonQ3    -0.10906796 0.007859376 -13.8774320 1.143802e-25
```

- R^2 ajusté :

```
LTW.model.res$adj.r.squared
```

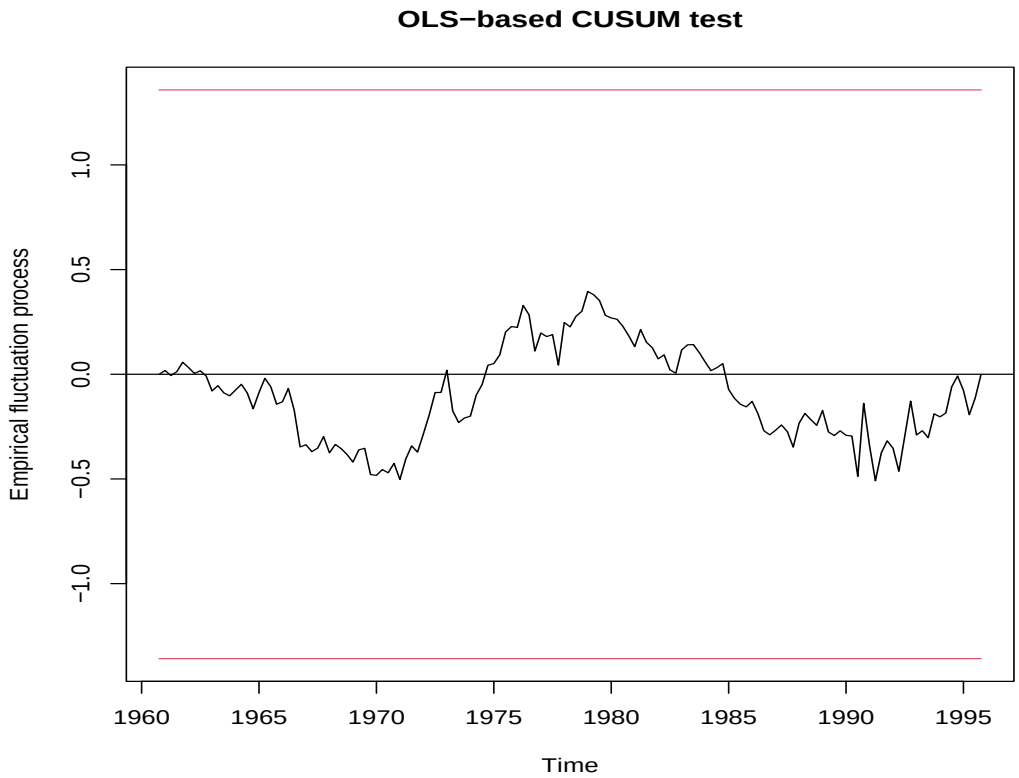
```
## [1] 0.9432524
```

Modèle ECM

Données GermanM1 : Modèle ECM - Test de changement structurel (1)

- Tests de changement structurel utilisant le modèle `LTW.model` sur les données `GermanM1`, basé sur les sommes cumulées des résidus de la régression MCO

```
ols <- efp(LTW.model, data = GermanM1, type = "OLS-CUSUM")
plot(ols)
```

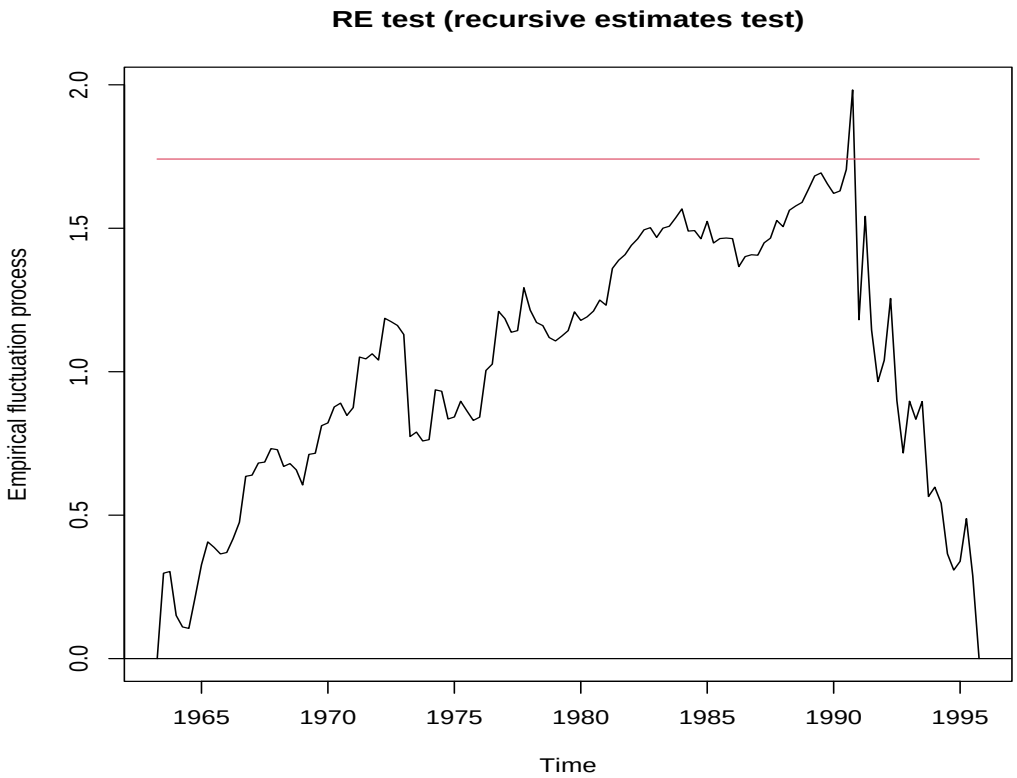


Modèle ECM

Données GermanM1 : Modèle ECM - Test de changement structurel (2)

- Test de changement structurel utilisant le modèle `LTW.model` sur les données `GermanM1`, basé sur l'estimation MCO récursive des coefficients de la régression

```
re <- efp(LTW.model, data = GermanM1, type = "RE")
plot(re)
```

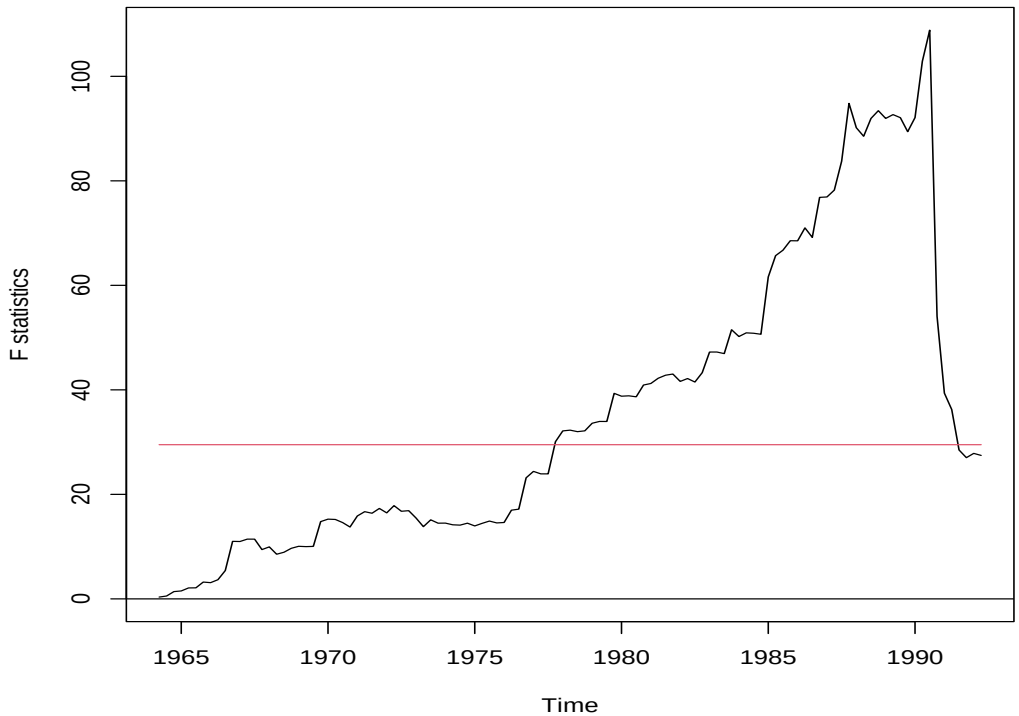


Modèle ECM

Données GermanM1 : Modèle ECM - Test de changement structurel (3)

- Test de Chow utilisant le modèle `LTW.model` sur les données `GermanM1`

```
fs <- Fstats(LTW.model, data = GermanM1, from = 0.1)
plot(fs)
```



Modèle ECM

Données GermanM1 : Modèle ECM

- Le modèle ECM suivant (différent du modèle original) avec `ecm.res` au lieu de `m1`, `y1` et `R1` est utilisé par Zeileis et al. [31]

```
M1.model <- dm ~ dy2 + dR + dR1 + dp + ecm.res + season
summary(lm(M1.model, data=historyM1))
```

```
##
## Call:
## lm(formula = M1.model, data = historyM1)
##
## Residuals:
##       Min        1Q    Median        3Q       Max
## -0.040770 -0.007048  0.000606  0.008347  0.029352
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) -0.050254   0.029492  -1.704 0.091230 .
## dy2         -0.297294   0.051576  -5.764 7.73e-08 ***
## dR          -0.672786   0.239145  -2.813 0.005818 **
## dR1         -0.999500   0.262608  -3.806 0.000234 ***
## dp          -0.527866   0.075938  -6.951 2.79e-10 ***
## ecm.res      1.000000   0.223619   4.472 1.91e-05 ***
## seasonQ1    -0.133301   0.003900 -34.181  < 2e-16 ***
## seasonQ2    -0.015594   0.004278  -3.646 0.000411 ***
## seasonQ3    -0.109068   0.007761 -14.053  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.01259 on 109 degrees of freedom
## Multiple R-squared:  0.9481, Adjusted R-squared:  0.9443
## F-statistic: 248.9 on 8 and 109 DF,  p-value: < 2.2e-16
```

- Les tests de changement structurel donnent des résultats similaires sur la période avant réunification, mais des résultats légèrement différents sur la période

Modèle ECM

Données GermanM1 : Monitoring (1)

- Ici on utilise la fonction `mefp` (Monitoring of Empirical Fluctuation Processes) sur la période 'historique' (avant réunification), qui sont supposées ne pas contenir de changements structurels

```
M1 <- historyM1
ols.efp <- efp(M1.model, type = "OLS-CUSUM", data = M1)
ols.mefp <- mefp(ols.efp)
```

- On utilise ensuite la fonction `monitor`, qui réalise un test séquentiel sur les nouvelles données avec H0=pas de changement structurel vs changement dans un ou plusieurs coefficients du modèle de régresion

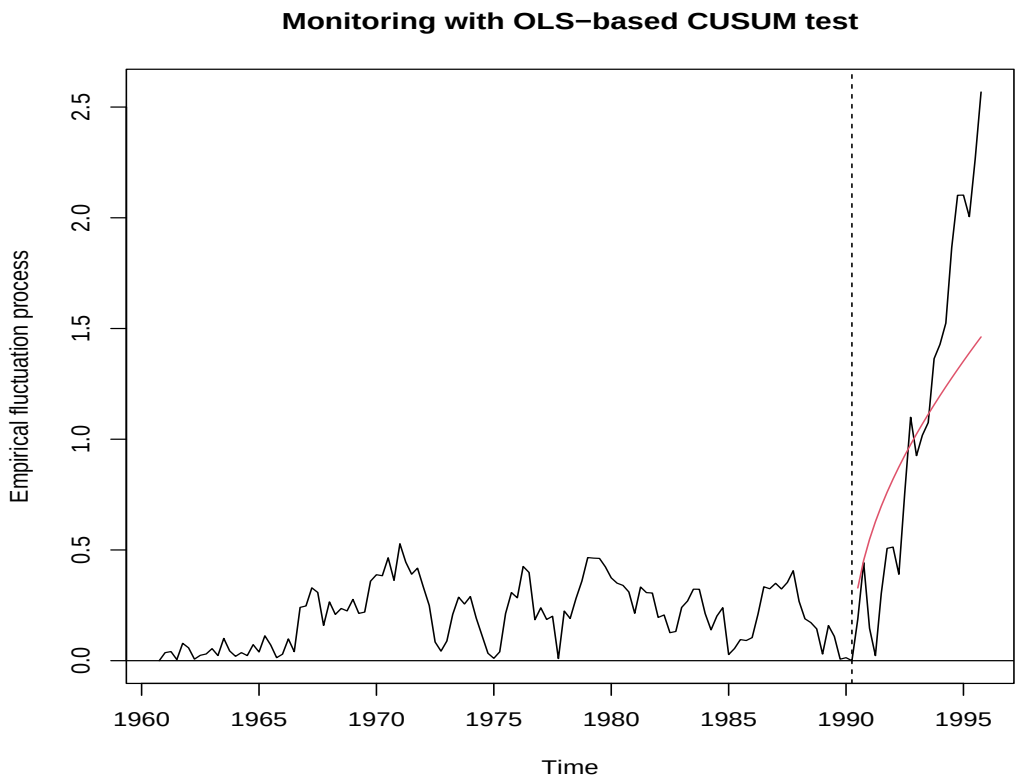
```
M1 <- GermanM1
ols.mon <- monitor(ols.mefp)
## Break detected at observation # 128
ols.mon
## Monitoring with OLS-based CUSUM test
##
## Initial call:
##   mefp.efp(obj = ols.efp)
##
## Last call:
##   monitor(obj = ols.mefp)
##
## Significance level   :  0.05
## Critical value       :  2.795483
## History size         :  118
## Last point evaluated :  140
## Structural break at  :  128
##
## Parameter estimate on history :
## (Intercept)         dy2          dR         dR1          dp     ecm.res
## -0.05025441 -0.29729418 -0.67278600 -0.99950033 -0.52786586  1.00000000
```

Modèle ECM

Données GermanM1 : Monitoring (2

- On utilise la méthode générique `plot` pour la représentation graphique de l'objet `mfep`

```
plot(ols.mon)
```



13 Modèles VECM et SVECM

Introduction

- Selon la définition initiale, un ensemble de variables $I(1)$ est **cointégré** si il existe une combinaison linéaire $I(0)$ de ces variables
- Il arrive cependant d'analyser des séries multivariées contenant à la fois des variables $I(1)$ et $I(0)$
- On étend pour cela le concept de cointégration : toute combinaison linéaire $I(0)$ est appelée relation de cointégration ([23, 27, voir par exemple]), y compris une combinaison de deux variables $I(0)$
- Le modèle VAR permet d'analyser des variables présentant une tendance stochastique, mais les relations de cointégration n'apparaissent pas explicitement
- Si l'intérêt porte sur l'analyse de ces relations de cointégration, la représentation VECM (*Vector Error Correction Model*) est plus appropriée

Cointégration

Définition étendue

- La définition étendue de la cointégration formulée par Campbell et Perron [4] est la suivante :
  Un vecteur de variables y_t de dimension ($nx1$) est cointégré si il existe au moins un élément non null dans le vecteur $_i$ de n éléments tel que β_i / y_t est stationnaire à une tendance près (trend-stationnary)

13 Modèles VECM et SVECM
- Modèle vectoriel à correction d'erreur VECM
- Test de cointégration
- Modèle SVECM (Structural Vector Error Correction Model)

Le modèle VECM

- Pour intégrer explicitement les relations de cointégration dans un modèle VAR, la représentation VECM (Vector Error Correction Model) est plus appropriée (voir [23])

$$\Delta y_t = \Pi y_{t-1} + \Gamma_1 \Delta y_{t-1} + \cdots + \Gamma_{p-1} \Delta y_{t-p+1} + u_t$$

- Ici $\Pi = -(I_K - A_1 - \cdots - A_p)$ et $\Gamma_i = -(A_{i+1} + \cdots + A_p)$ pour $i = 1, ..., p1$
- Ce modèle est obtenu à partir du modèle $VAR(p)$ en soustrayant y_{t-1} des deux côtés de l'équation
- Δy_t ne contient pas de tendance stochastique puisqu'on considère que toutes les variables sont au plus $I(1)$
- Le terme Πy_{t-1} est le seul qui contient des variables $I(1)$; donc Πy_{t-1} doit être $I(0)$ et contient les relations de cointégration
- Les termes $\Gamma_j (j = 1, \ldots, p-1)$ sont (souvent) appelés paramètres de court-terme (*short-run or short-term parameter*)
- Le terme Πy_{t-1} est (parfois) appelé composante de long terme (*long-run or long-term part*)

Formulations d'un VECM

- On distingue deux formulations d'un VECM [27]
- On considère le modèle $VAR(p)$

$$y_t = \Pi_1 y_{t-1} + \cdots + \Pi_K y_{t-p} + \mu + \Phi D_t + \varepsilon_t, \quad t = 1, \ldots, T \qquad \text{(Eq. 1)}$$

où y_t désigne le vecteur $(K \times 1)$ des séries temporelles à la période t, les matrices Π_i $(i = 1, \ldots, p)$ sont les matrices $(K \times K)$ des coefficients associés aux variables endogènes retardées, μ est un vecteur $(K \times 1)$ de constantes et D_t est un vecteur de variables non stochastiques, telles que des variables indicatrices saisonnières ou des variables indicatrices d'intervention

- Le terme d'erreur ε_t, de dimension $(K \times 1)$, est supposé indépendant et identiquement distribué (i.i.d.), selon une loi normale multivariée :

$$\varepsilon_t \sim \mathcal{N}(0, \Sigma).$$

- À partir de l'équation du modèle VAR précédente, deux formulations d'un modèle vectoriel à correction d'erreur (VECM, *Vector Error Correction Model*) peuvent être distinguées

Définition

Forme à long terme d'un VECM

- Dans la première formulation, les variables en niveaux y_t interviennent avec un retard de p périodes, c'est-à-dire sous la forme y_{t-p} :

$$\Delta y_t = \Gamma_1 \Delta y_{t-1} + \cdots + \Gamma_{p-1} \Delta y_{t-p+1} + \Pi y_{t-p} + \mu + \Phi D_t + \varepsilon_t, \tag{1}$$

$$\Gamma_i = -(I - \Pi_1 - \cdots - \Pi_i), \quad i = 1, \ldots, p-1, \tag{2}$$

$$\Pi = -(I - \Pi_1 - \cdots - \Pi_p) \tag{3}$$

où I est la matrice identité de dimension $(K \times K)$.

- Comme le montre l'équation (2), les matrices Γ_i $(i = 1, \ldots, p-1)$ contiennent les effets cumulés à long terme
- Cette spécification est appelée **forme à long terme** (*long-run form*)

Définition

Forme transitoire d'un VECM

- La forme transitoire (*transitory form*) d'un VECM est définie par :

$$\Delta y_t = \Gamma_1 \Delta y_{t-1} + \cdots + \Gamma_{p-1} \Delta y_{t-p+1} + \Pi y_{t-1} + \mu + \Phi D_t + \varepsilon_t, \tag{1}$$

$$\Gamma_i = -(\Pi_{i+1} + \cdots + \Pi_p), \quad i = 1, \ldots, p-1, \tag{2}$$

$$\Pi = -(I - \Pi_1 - \cdots - \Pi_p). \tag{3}$$

- La matrice Π est identique à celle de la première spécification
- En revanche, les matrices Γ_i diffèrent ; elles mesurent les effets transitoires
- Par ailleurs, les variables en niveaux constituant le vecteur y_t interviennent avec un retard d'une période, c'est-à-dire sous la forme y_{t-1}

13 Modèles VECM et SVECM

- Modèle vectoriel à correction d'erreur VECM
- Test de cointégration
- Modèle SVECM (Structural Vector Error Correction Model)

Test de cointégration

- On part de la représentation VECM d'un modèle VAR (forme simplifiée sans constantes ni variables déterministes)

$$\Delta y_t = \Pi y_{t-1} + \sum_{i=1}^{p-1} \Gamma_i \Delta y_{t-i} + u_t$$

avec

$$\Pi = \sum_{i=1}^{p} A_i - I_K$$

et

$$\Gamma_i = -\sum_{j=i+1}^{p} A_j$$

- La matrice Π contient l'information sur le relations de long terme entre les variables

Test de cointégration

Relations de cointégration

- Trois situations sont possibles :
  - $\text{rang}(\Pi) = 0$: aucune relation de cointégration
  - $0 < \text{rang}(\Pi) = r < K$: il existe r relations de cointégration linéairement indépendantes
  - $\text{rang}(\Pi) = K$: sous les hypothèses usuelles du modèle, le système est stationnaire en niveau
- Lorsque $0 < r < K$, on peut écrire :

$$\Pi = \alpha\beta'$$

où α et β sont deux matrices $K \times r$

- La matrice β contient les vecteurs de cointégration, tandis que α décrit les coefficients d'ajustement aux déséquilibres de long terme

Test de cointégration

La procédure de Johansen

- L'objectif de la procédure de Johansen [15] est de déterminer le rang r de la matrice Π
- Π est une matrice carrée de dimension $K \times K$, où K est le nombre de variables
- Lorsque $0 < r < K$, $\Pi = \alpha\beta'$, les r colonnes linéairement indépendantes de β définissent r combinaisons linéaires indépendantes des variables :

$$\beta' y_t = \begin{pmatrix} \beta_1' y_t \\ \vdots \\ \beta_r' y_t \end{pmatrix}$$

- Ces combinaisons sont stationnaires lorsque les variables sont $I(1)$

Test de cointégration

Les deux statistiques de test

- Contrairement à la méthode d'Engle-Granger, qui estime une relation de cointégration à partir d'une régression, la méthode de Johansen utilise une approche multivariée fondée sur le maximum de vraisemblance
- La procédure propose deux tests
  - Le test de la trace teste les hypothèses :

$$H_0 : \text{rang}(\Pi) \leq r$$

contre :

$$H_1 : \text{rang}(\Pi) > r$$

- Le test de la valeur propre maximale examine :

$$H_0 : \text{rang}(\Pi) = r$$

contre :

$$H_1 : \text{rang}(\Pi) = r + 1.$$

Test de cointégration

Données `Canada`

- On part du modèle VAR estimé sur les données `Canada`

`e`	nombre d'actifs (employment)
`prod`	productivité du travail (log différence entre GDP et nombre d'actifs)
`rw`	log de l'index des salaires réels (real wage index)
`U`	taux de chômage (unemployment rate)

- Toutes les séries sont $I(1)$, il existe potentiellement des relations de cointégration entre elles
- Après les diagnostics, les modèles $VAR(2)$ ou $VAR(3)$ (voir Pfaff [27]) sont retenus pour l'analyse de cointégration

Test de cointégration

Données Canada

- La fonction `ca.jo()` de la librairie `urca` permet de réaliser les tests de cointégration, suivant le procédure de Johansen
- Ici on utilise le test de la trace (`type = "trace"`)
- Le modèle VECM estimé est la forme transitoire, et on inclut une tendance déterministe pour la relation de cointégration (`ecdet = "trend"`)
- L'argument `K=3` représente l'ordre p du modèle VAR (i.e. le modèle VAR en niveaux)

```
coint.canada <- ca.jo(Canada, type = "trace", ecdet = "trend", K = 3, spec = "transitory")
coint.canada
```

```
##
## ######################################################
## # Johansen-Procedure Unit Root / Cointegration Test #
## ######################################################
##
## The value of the test statistic is: 3.8544 18.7197 36.4184 84.917
```

Test de cointégration

Données Canada

- La méthode `summary()` affiche les détails du test
- La première hypothèse qui n'est pas rejettée est $r = 1$, on conclut qu'il y a une relation de cointégration

```
summary(coint.canada)
```

```
##
## ######################
## # Johansen-Procedure #
## ######################
##
## Test type: trace statistic , with linear trend in cointegration
##
## Eigenvalues (lambda):
## [1]  4.505013e-01  1.962777e-01  1.676668e-01  4.647108e-02 -6.428036e-17
##
## Values of teststatistic and critical values of test:
##
##           test 10pct  5pct  1pct
## r <= 3 |  3.85 10.49 12.25 16.26
## r <= 2 | 18.72 22.76 25.32 30.45
## r <= 1 | 36.42 39.06 42.44 48.45
## r = 0  | 84.92 59.14 62.99 70.05
##
## Eigenvectors, normalised to first column:
## (These are the cointegration relations)
##
##                e.l1    prod.l1      rw.l1        U.l1    trend.l1
## e.l1        1.00000  1.0000000  1.0000000  1.00000000  1.00000000
## prod.l1   -41.92623  0.7723987 -0.3468422  0.23574849 -0.12062153
## rw.l1     -76.94643 -0.2572818 -0.4848433 -0.03286331 -0.29511866
## U.l1     -132.85354  2.6289928  2.5757023  1.61283935  1.51474627
## trend.l1   54.56954 -0.1778682  0.1766543 -0.37545112 -0.03414891
##
## Weights W:
```

Test de cointégration

Données Canada

- On peut récupérer chacun des éléments contenus dans l'objet `coint.canada` (de classe S4 `ca.jo`) avec la fonction `attr()` (la commande `?'ca.jo-class'` affiche la liste des éléments)
- Par exemple les valeurs des statistiques de test

```
attr(coint.canada, "teststat")
## [1]  3.854428 18.719749 36.418371 84.917023
```

- et les valeurs critiques

```
attr(coint.canada, "cval")
##          10pct  5pct  1pct
## r <= 3 | 10.49 12.25 16.26
## r <= 2 | 22.76 25.32 30.45
## r <= 1 | 39.06 42.44 48.45
## r = 0  | 59.14 62.99 70.05
```

Modèle VECM

Données Canada

- Le modèle VECM est réestimé avec la fonction `cajorls()` en spécifiant le rang de cointégration (`r=1`) (on impose donc une restriction sur la matrice β)
- Les équations du VECM sont estimées par MCO
- La fonction `cajorls()` normalise les vecteurs de cointégration. Elle normalise ici le coefficient de la première variable, `rw`, à 1 : la relation de long-terme est normalisée par rapport aux salaires réels

```
vecm <- ca.jo(Canada[, c("rw", "prod", "e", "U")], type = "trace", ecdet = "trend", K = 3, spec = "transitory")
vecm.r1 <- cajorls(vecm, r = 1)
```

Modèle VECM

Données Canada

- L'élément `rlm` contient les régressions

```
vecm.r1$rlm
```

```
##
## Call:
## lm(formula = substitute(form1), data = data.mat)
##
## Coefficients:
##             rw.d       prod.d     e.d        U.d
## ect1      -0.084815  -0.011994  -0.015606  -0.008660
## constant  55.469125   8.274808  10.331308   5.687832
## rw.dl1    -0.012082   0.004707  -0.078491   0.017263
## prod.dl1  -0.074493   0.234441   0.200953  -0.138916
## e.dl1     -0.634084  -0.246544   0.821558  -0.646846
## U.dl1      0.063137  -0.979868   0.003379  -0.191125
## rw.dl2    -0.157388  -0.190264  -0.095835   0.080354
## prod.dl2  -0.251940  -0.029520   0.048273  -0.002909
## e.dl2      0.081197  -0.580473  -0.459693  -0.019741
## U.dl2     -0.230009  -0.128101  -0.103415  -0.262685
```

Modèle VECM

Données Canada

- On peut obtenir des détails pour chaque équation, ici `rw.d` (Δrw)

```
summary(vecm.r1$rlm)[1]
```

```
## Response rw.d :
##
## Call:
## lm(formula = rw.d ~ ect1 + constant + rw.dl1 + prod.dl1 + e.dl1 +
##     U.dl1 + rw.dl2 + prod.dl2 + e.dl2 + U.dl2 - 1, data = data.mat)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -1.7546 -0.3233  0.0509  0.3092  1.8386
##
## Coefficients:
##          Estimate Std. Error t value Pr(>|t|)
## ect1     -0.08481    0.01586  -5.348 1.03e-06 ***
## constant 55.46912   10.27534   5.398 8.46e-07 ***
## rw.dl1   -0.01208    0.10705  -0.113   0.9105
## prod.dl1 -0.07449    0.12960  -0.575   0.5672
## e.dl1    -0.63408    0.32375  -1.959   0.0541 .
## U.dl1     0.06314    0.39824   0.159   0.8745
## rw.dl2   -0.15739    0.10658  -1.477   0.1442
## prod.dl2 -0.25194    0.13324  -1.891   0.0627 .
## e.dl2     0.08120    0.34086   0.238   0.8124
## U.dl2    -0.23001    0.40783  -0.564   0.5745
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.7435 on 71 degrees of freedom
## Multiple R-squared:  0.7601, Adjusted R-squared:  0.7263
## F-statistic:  22.5 on 10 and 71 DF,  p-value: < 2.2e-16
```

Modèle VECM

Données Canada

- La matrice β contient le vecteur de cointégration

```
vecm.r1$beta
```

```
##                 ect1
## rw.l1      1.00000000
## prod.l1    0.54487552
## e.l1      -0.01299605
## U.l1       1.72657186
## trend.l1 -0.70918872
```

- Avec quatre variables $I(1)$ et un rang de cointégration égal à 1, le système possède trois tendances stochastiques communes.
- Cela signifie que les quatre variables peuvent évoluer de façon non stationnaire, tout en étant liées par une seule relation d'équilibre de long terme

13 Modèles VECM et SVECM
- Modèle vectoriel à correction d'erreur VECM
- Test de cointégration
- Modèle SVECM (Structural Vector Error Correction Model)

Modèle SVECM

- L'étape suivante de l'analyse de Lütkepohl et Krätzig [23, p. 188], reprise par [27] est l'utilisation d'un modèle SVECM pour identifier les chocs macroéconomiques sur le marché du travail canadien
- L'objectif est de décomposer les innovations d'un VECM en chocs structurels économiquement interprétables, puis d'étudier leurs effets à court et à long terme
- Dans un modèle VECM, les innovations réduites u_t sont généralement corrélées entre elles :

$$E(u_t u_t') = \Sigma_u$$

- Pour leur donner une interprétation économique, on souhaite les relier à des chocs structurels ε_t, supposés orthogonaux et de variance unitaire :

$$E(\varepsilon_t \varepsilon_t') = I_K$$

Modèle SVECM

Données Canada

- Dans un modèle structurel de type B, on écrit :

$$u_t = B\varepsilon_t$$

où B est une matrice carrée $K \times K$

- On en déduit :

$$\Sigma_u = BB'$$

- Pour qu'un modèle SVEC de type B soit exactement identifié, il est nécessaire d'imposer $\frac{1}{2}K(K-1) = 6$ restrictions linéairement indépendantes

Modèle SVECM

Données Canada : chocs structurels

- Les variables du modèle

$prod_t$	Productivité du travail
e_t	Emploi
U_t	Taux de chômage
rw_t	Salaire réel

- L'ordre des variables est important puisqu'il détermine les lignes des matrices de restrictions structurelles
- Avec $r = 1$ il existe une relation de cointégration et trois tendances stochastiques communes
- Sous les hypothèses usuelles du modèle, cela conduit à distinguer trois directions de chocs permanents et une direction de choc transitoire

Modèle SVECM

Données Canada : chocs structurels

- Dans l'analyse [23], les quatre chocs sont interprétés comme suit :

Choc	Interprétation
ε_{1t}	Choc de production (output shock)
ε_{2t}	Choc de demande de travail (labor demand shock)
ε_{3t}	Choc d'offre de travail (labor supply shock)
ε_{4t}	Choc salarial (wage shock)

- On écrit :

$$\varepsilon_t = \begin{pmatrix} \varepsilon_{1t} \\ \varepsilon_{2t} \\ \varepsilon_{3t} \\ \varepsilon_{4t} \end{pmatrix}$$

- L'identification de ces chocs repose sur des hypothèses économiques concernant leurs effets sur les variables

Modèle SVECM

Données Canada : Restrictions de long terme

- La matrice des effets cumulés de long terme est :

$$L = \Xi B$$

où Ξ représente la matrice des multiplicateurs de long terme du VECM réduit

- Puisque $r = 1$:

$$\text{rang}(\Xi) = K - r = 3$$

- Pfaff impose les restrictions suivantes :

$$L = \begin{pmatrix} \ell_{11} & 0 & 0 & 0 \\ \ell_{21} & \ell_{22} & \ell_{23} & 0 \\ \ell_{31} & \ell_{32} & \ell_{33} & 0 \\ \ell_{41} & \ell_{42} & \ell_{43} & 0 \end{pmatrix}$$

- Les lignes correspondent respectivement à la productivité, à l'emploi, au chômage et au salaire réel. Les colonnes correspondent aux quatre chocs structurels

Modèle SVECM

Données Canada : Hypothèses (1)

- Première hypothèse : le choc salarial est transitoire
- La dernière colonne est nulle :

$$L_{\cdot,4} = 0$$

- Cela signifie que le choc salarial n'a aucun effet permanent sur les quatre variables
- Puisque Ξ est de rang 3, les quatre zéros ne représentent que trois restrictions indépendantes

Modèle SVECM

Données Canada : Hypothèses (2)

- Deuxième hypothèse : la productivité est déterminée à long terme par le choc de production
- La première ligne est :

$$L_{1,\cdot} = \begin{pmatrix} \ell_{11} & 0 & 0 & 0 \end{pmatrix}$$

- Ainsi, ni le choc de demande de travail, ni le choc d'offre de travail, ni le choc salarial ne peuvent modifier durablement la productivité
- Cette hypothèse repose sur l'hypothèse économique de rendements d'échelle constants retenue dans l'application
- Elle apporte deux restrictions indépendantes supplémentaires, puisque $L_{1,4} = 0$ était déjà imposé
- Nous disposons donc de cinq restrictions indépendantes sur les effets de long terme

Modèle SVECM

Données Canada : Hypothèses (3)

- Pour compléter l'identification, Pfaff impose une restriction sur les effets immédiats :

$$B_{4,2} = 0$$

- Cela signifie que le choc de demande de travail n'a aucun effet immédiat sur le salaire réel
- La matrice contemporaine est donc :

$$B = \begin{pmatrix} b_{11} & b_{12} & b_{13} & b_{14} \\ b_{21} & b_{22} & b_{23} & b_{24} \\ b_{31} & b_{32} & b_{33} & b_{34} \\ b_{41} & 0 & b_{43} & b_{44} \end{pmatrix}$$

- Il est important de noter que les autres coefficients de B ne sont pas nécessairement nuls.
- Le modèle n'est donc pas identifié par une décomposition de Cholesky, mais par une combinaison de restrictions contemporaines et de long terme.

Modèle SVECM

Données Canada : Restrictions

- On impose les restrictions sur la matrice SR des effets contemporains (0 indique une restriction, et NA indique un paramètre libre)

```
SR <- matrix(NA, nrow = 4, ncol = 4)
SR[4, 2] <- 0
SR
```

```
##      [,1] [,2] [,3] [,4]
## [1,]   NA   NA   NA   NA
## [2,]   NA   NA   NA   NA
## [3,]   NA   NA   NA   NA
## [4,]   NA    0   NA   NA
```

- On impose les restrictions sur les effets structurels de long terme

```
LR <- matrix(NA, nrow = 4, ncol = 4)
LR[1, 2:4] <- 0
LR[2:4, 4] <- 0
LR
```

```
##      [,1] [,2] [,3] [,4]
## [1,]   NA    0    0    0
## [2,]   NA   NA   NA    0
## [3,]   NA   NA   NA    0
## [4,]   NA   NA   NA    0
```

Modèle SVECM

Données Canada

- On utilise ensuite la fonction `SVEC()` pour estimer le modèle
- Avec l'argument `boot = TRUE`, les erreurs standard (et donc la statistique t) pour les coefficients LR et SR sont calculées par bootstrap

```
vecm <- ca.jo(Canada[, c("prod", "e", "U", "rw")], type = "trace",
              ecdet = "trend", K = 3, spec = "transitory")

svec <- SVEC(vecm, LR = LR, SR = SR, r = 1, lrtest = FALSE,
             boot = TRUE, runs = 100)
```

Modèle SVECM

Données Canada : Matrice SR

- L'élément `SR` de l'objet `svec` (une liste) contient les estimations des coefficients des impacts contemporains

```
round(svec$SR, 2)
```

```
##       prod     e     U   rw
## prod  0.58  0.07 -0.15 0.07
## e    -0.12  0.26 -0.16 0.09
## U     0.03 -0.27  0.01 0.05
## rw    0.11  0.00  0.48 0.49
```

- La statistique t est obtenue en divisant l'estimation par l'erreur standard (au seuil de 5%, on rejette l'hypothèse H0 : $\theta = 0$ si $|t| > 1,96$)

```
round(svec$SR / svec$SRse, 2)
```

```
##       prod     e     U   rw
## prod  6.17  0.65 -0.65 0.96
## e    -1.92  4.31 -0.82 2.36
## U     0.50 -6.35  0.10 1.67
## rw    0.71   NaN  0.71 5.73
```

Modèle SVECM

Données Canada : Matrice LR

- L'élément `LR` de l'objet `svec` contient les estimations des coefficients des impacts à long terme

```
round(svec$LR, 2)
```

```
##       prod     e     U rw
## prod  0.79  0.00  0.00  0
## e     0.20  0.58 -0.49  0
## U    -0.16 -0.34  0.14  0
## rw   -0.15  0.60 -0.25  0
```

- La statistique t est obtenue en divisant l'estimation par l'erreur standard (au seuil de 5%, on rejette l'hypothèse H0 : $\theta = 0$ si $|t| > 1,96$)

```
round(svec$LR / svec$LRse, 2)
```

```
##       prod     e     U  rw
## prod  4.91   NaN   NaN NaN
## e     1.00  3.09 -0.81 NaN
## U    -1.61 -3.73  0.89 NaN
## rw   -0.93  3.72 -0.89 NaN
```

Modèle SVECM

Données Canada : Hypothèse sur les effets de long terme

- Les auteurs ont ensuite cherché à déterminer si les chocs d'offre de travail n'avaient aucun effet à long terme sur le chômage
- Dans le contexte du modèle SVEC, les auteurs cherchent à vérifier l'hypothèse économique suivante :

$$H_0 : (\Xi B)_{U,\varepsilon_{LS}} = 0$$

où : - ΞB est la matrice des effets de long terme des chocs structurels ; - U désigne le taux de chômage ; - ε_{LS} désigne le choc d'offre de travail (labor supply shock)

- L'hypothèse se traduit par $\Xi B 3, 3 = 0$
- Avec l'ajout d'une restriction supplémentaire, le modèle SVEC est maintenant surindentifié (*over-identified*)

Modèle SVECM

Données Canada : Hypothèse sur les effets de long terme

- La validité de cette sur-identification peut être vérifiée avec un test de rapport de vraissemblance (LR, Likelihood-Ratio test)
- La restriction est ajoutée et le modèle est réestimé avec la fonction `update`
- Le résultat du test de rapport de vraissemblance est contenu dans l'élément `Rover`

```
LR[3, 3] <- 0
svec.oi <- update(svec, LR = LR, lrtest = TRUE, boot = FALSE)
svec.oi$LRover
```

```
##
##  LR overidentification
##
## data:  vecm
## Chi^2 = 6.0745, df = 1, p-value = 0.01371
```

- La statistique du test est distribuée selon une loi de 2 à 1 degré de liberté ; la p-valeur du test est inférieure au seuil de 5%
- L'hypothèse H0 : les chocs d'offre du travail n'ont pas d'impact à long terme sur le chômage est rejettée

14 Bibliographie

Bibliographie I

[1] Gianni Amisano et Carlo Giannini. *Topics in Structural VAR Econometrics*. by Gianni Amisano, Carlo Giannini. 2nd ed. 1997. Berlin, Heidelberg : Springer Berlin Heidelberg, 1997. ISBN : 9783642606236.

[2] Badi H. Baltagi, éd. *A Companion to Theoretical Econometrics*. Wiley, jan. 2003. ISBN : 9780470996249. DOI : 10.1002/9780470996249.

[3] Anindya Banerjee. *Co-integration, error correction, and the econometric analysis of non-stationary data*. Anindya Banerjee, Juan J. Dolado, John W. Galbraith, and David F. Hendry. Advanced texts in econometrics. Oxford : Clarendon Press. 329 p. ISBN : 0191521582.

[4] John Y. Campbell et Pierre Perron. *Pitfalls and Opportunities : What Macroeconomists Should Know about Unit Roots*. 1991. DOI : None.

[5] Fabio Canova. *Methods for Applied Macroeconomic Research*. Princeton University Press, 2007. ISBN : 9780691115047.

[6] Jonathan D. Cryer et Kung-Sik Chan. *Time Series Analysis : With Applications in R*. Springer New York, 2010. ISBN : 9780387759593. DOI : 10.1007/978-0-387-75959-3.

[7] Robert F. Engle et C. W. J. Granger. "Co-Integration and Error Correction : Representation, Estimation, and Testing". In : *Econometrica* 55.2 (mars 1987), p. 251. ISSN : 0012-9682. DOI : 10.2307/1913236.

[8] Gergely Ganics et Florens Odendahl. "Bayesian VAR forecasts, survey information, and structural change in the euro area". In : *International Journal of Forecasting* 37.2 (2021), p. 971-999. ISSN : 0169-2070. DOI : https://doi.org/10.1016/j.ijforecast.2020.11.001.

[9] Domenico Giannone, Michele Lenza et Giorgio E. Primiceri. "Prior Selection for Vector Autoregressions". In : *The Review of Economics and Statistics* 97.2 (mai 2015), p. 436-451. ISSN : 0034-6535. DOI : 10.1162/REST_a_00483.

[10] James D. Hamilton. *Time series analysis*. James D. Hamilton. [Nachdr.]09. Princeton, N. J. : Princeton Univ. Press, 1994. 799 p. ISBN : 0691042896.

[11] B. Hansen. "Testing for parameter instability in linear models". In : *Journal of Policy Modeling* 14 (1992), p. 517-533. DOI : 10.1016/0161-8938(92)90019-9.

Bibliographie II

[12] Helmut Herwartz et Martin Plödt. “Simulation Evidence on Theory-based and Statistical Identification under Volatility Breaks”. In : *Oxford Bulletin of Economics and Statistics* 78.1 (fév. 2015), p. 94-112. ISSN : 1468-0084. DOI : 10.1111/obes.12098.

[13] Robert L. Hetzel. “The Federal Reserve System and Control of the Money Supply in the 1970s”. In : *Journal of Money, Credit and Banking* 13.1 (fév. 1981), p. 31. ISSN : 0022-2879. DOI : 10.2307/1991806.

[14] Chenghan Hou, Bao Nguyen et Bo Zhang. “Real-time forecasting of the Australian macroeconomy using flexible Bayesian VARs”. In : *Journal of Forecasting* 42.2 (2023), p. 418-451. DOI : https://doi.org/10.1002/for.2913.

[15] Søren Johansen et Katarina Juselius. “MAXIMUM LIKELIHOOD ESTIMATION AND INFERENCE ON COINTEGRATION — WITH APPLICATIONS TO THE DEMAND FOR MONEY”. In : *Oxford Bulletin of Economics and Statistics* 52.2 (1990), p. 169-210. DOI : https://doi.org/10.1111/j.1468-0084.1990.mp52002003.x.

[16] Lutz Kilian et Helmut Lütkepohl. *Structural Vector Autoregressive Analysis*. Themes in Modern Econometrics. Cambridge University Press, 2017.

[17] Christian Kleiber et Achim Zeileis. *Applied Econometrics with R*. Use R ! Springer New York, 2008. ISBN : 9780387773186. DOI : 10.1007/978-0-387-77318-6.

[18] Walter Krämer et Harald Sonnberger. *The Linear Regression Model Under Test*. Physica-Verlag HD, 1986. ISBN : 9783642958762. DOI : 10.1007/978-3-642-95876-2.

[19] Nikolas Kuschnig et Lukas Vashold. “BVAR : Bayesian Vector Autoregressions with Hierarchical Prior Selection in R”. In : *Journal of Statistical Software* 100.14 (2021), 1–27. DOI : 10.18637/jss.v100.i14.

[20] Alexander Lange et al. “svars : An R Package for Data-Driven Identification in Multivariate Time Series Analysis”. In : *Journal of Statistical Software* 97.5 (2021), 1–34. DOI : 10.18637/jss.v097.i05.

[21] Robert B Litterman. “Forecasting with Bayesian vector autoregressions—five years of experience”. In : *Journal of Business & Economic Statistics* 4.1 (1986), p. 25-38.

[22] Helmut Lütkepohl. *New introduction to multiple time series analysis*. English. Berlin : Springer, 2005. ISBN : 3-540-40172-5.

Bibliographie III

[23] Helmut Lütkepohl et Markus Krätzig, éd. *Applied times series econometrics*. English. Themes Mod. Econom. Cambridge : Cambridge University Press, 2004. ISBN : 0-521-54787-3 ; 0-521-83919-X. DOI : 10.1017/CBO9780511606885.

[24] Helmut Lütkepohl, Timo Teräsvirta et Juergen Wolters. "Investigating Stability and Linearity of a German M1 Money Demand Function". In : *Journal of Applied Econometrics* 14.5 (1999), p. 511-25.

[25] Michael McCracken et Serena Ng. *FRED-QD : A Quarterly Database for Macroeconomic Research*. Working Paper 26872. National Bureau of Economic Research, 2020. DOI : 10.3386/w26872.

[26] Charles R. Nelson et Charles R. Plosser. "Trends and random walks in macroeconmic time series : Some evidence and implications". In : *Journal of Monetary Economics* 10.2 (1982), p. 139-162. ISSN : 0304-3932. DOI : https://doi.org/10.1016/0304-3932(82)90012-5.

[27] Bernhard Pfaff. *Analysis of Integrated and Cointegrated Time Series with R*. Springer New York, 2008. ISBN : 9780387759678. DOI : 10.1007/978-0-387-75967-8.

[28] Bernhard Pfaff. "VAR, SVAR and SVEC Models : Implementation Within R Package vars". In : *Journal of Statistical Software* 27.4 (2008), 1–32. DOI : 10.18637/jss.v027.i04.

[29] R. Tsay. *Multivariate Time Series Analysis : With R and Financial Applications*. Willey, 2014.

[30] Wayne A Woodward, Henry L Gray et Alan C Elliott. *Applied time series analysis with R*. CRC press, 2017.

[31] Achim Zeileis et al. "Monitoring structural change in dynamic econometric models". In : *Journal of Applied Econometrics* 20.1 (jan. 2005), p. 99-121. ISSN : 1099-1255. DOI : 10.1002/jae.776.

[32] Achim Zeileis et al. "strucchange : An R Package for Testing for Structural Change in Linear Regression Models". In : *Journal of Statistical Software* 7.2 (2002), 1–38. DOI : 10.18637/jss.v007.i02.